

考虑变物性与径向收缩的圆柱药材干燥模型

摘 要

针对圆柱药材热风干燥过程中的内部温湿分布预测与干燥时间判定问题, 本文依据题给环境记录、材料物性及观测半径, 构建由常热物性到变物性、由固定几何到径向收缩的递进式传热传质模型。在有效湿度边界、不显式计入潜热及所设后段环境条件下, 预测规定时刻与位置的温度和干基含水率, 并确定达到规定含水率阈值所需的模型干燥时长。

针对问题一, 将药材近似为轴对称圆柱体, 采用常热物性并考虑水分扩散系数随含水率变化, 建立一维径向温湿传递模型。利用圆环有限体积法共享相邻单元面通量, 并采用隐式 BDF 方法积分, 求得预热阶段的径向温湿分布。计算得到 1800 s 时轴心与表面温度分别为 33.5753°C 和 36.7856°C , 表面含水率降至 1.5102 kg/kg , 轴心仍接近初始值 2.55 kg/kg 。结果表明, 热量已传入内部而失水仍集中于外层, 传热与水分扩散具有不同的时间尺度。

针对问题二, 进一步考虑热容量、导热率和扩散率随局部状态变化的反馈, 从共同初态建立变物性热质耦合模型。采用 Chebyshev 配点与 Radau 隐式积分联立求解全过程温湿场, 并通过扩散系数的对数分解区分升温促进与失水抑制作用。计算得到 3 h 时轴心与表面温差仅 0.1169 K , 含水率差仍为 0.7581 kg/kg ; 2.5–3 h 内两处扩散率均已回落。结果表明, 温度场趋于均匀时, 含水率仍存在明显的径向梯度。

针对问题三, 在第二问一维径向有效模型的基础上, 以全径向最大含水率严格低于 0.15 kg/kg 为停止条件, 建立干燥终点判定模型。采用重构极值搜索追踪径向含水率的最大值, 计算发现临近干燥终点时轴心含水率仍为径向最大值, 因此以轴心达标确定干燥终点。结合网格加密和独立时间积分检验设置数值裕量, 在 4 h 后环境保持末小时均值的假设下, 得到达到规定含水率要求所需的模型干燥时长为 57.4741 h 。

针对问题四, 根据附件 2 的观测半径, 在恒长度、均匀径向收缩假设下, 由干物质守恒构建材料坐标模型, 将移动边界转化为固定参考域, 避免重复加入干基含水率的体积浓缩项。采用该问物性从初态求解, 在同一后段环境假设下得到一维径向模型的执行时长 51.0921 h 。进一步设置同物性恒半径对照, 其临界时间为 129.8484 h ; 施加观测收缩后, 模型临界时间缩短约 60.65% 。

关键词: 药材干燥; 变物性; 热质耦合; 径向收缩; 阈值事件

一、问题重述

1.1 问题背景

干燥是影响中药材成品品质的重要工序，热风烘干通常包括预热平衡与恒温干燥两个阶段。烘房环境通过表面换热和水分交换改变药材内部状态，热量向内传递、水分向外迁移，并可能伴随尺寸收缩。由于内部传输需要时间，表面的升温或失水不能直接代表内部各处的状态。因此，需要建立温度与含水率随时间、位置变化的模型，在给出内部状态预测的同时，确定满足干燥要求所需的时间。

题设药材近似为长 25 cm、初始半径 2 cm 的圆柱，初始温度为 28°C，干基含水率为 2.55 kg/kg。已知资料包括烘房温湿环境记录、药材半径记录和各问的物性参数。

1.2 问题提出

根据上述条件，建立模型完成以下四项任务。论文表格与完整过程文件分别按题设要求输出，结果统一保留四位小数。

问题一：采用附录 2 参数，建立 1800 s 预热阶段的温度与含水率模型。在论文中给出 100、300、600、900、1200、1500、1800 s 及到中心距离 0、0.5、1、1.5、2 cm 处的温湿结果；完整结果按 1 s 和 0.1 cm 采样，写入 `result1.xlsx`。

问题二：全程统一采用附录 3 经验公式，描述整个烘干过程的温度与含水率变化。在论文中给出前 3 h 每隔 0.5 h、上述五个位置的温湿结果；完整过程按 1 s 和 0.1 cm 采样，写入 `result2.xlsx`。

问题三：依据药材各处含水率低于 0.15 kg/kg 的要求，确定以小时计的烘干时长。论文表格列出每隔 6 h 及结束时刻、每隔 0.5 cm 的含水率；完整过程按 60 s 和 0.1 cm 采样，写入 `result3.xlsx`。

问题四：结合附件 2 半径记录与附录 4 物性，确定尺寸收缩下的烘干时长。论文表格列出每隔 6 h 及结束时刻、实际材料范围内每隔 0.5 cm 和当前表面的含水率；完整过程按 60 s 和 0.1 cm 采样，写入 `result4.xlsx`。

二、问题分析

对于问题一，重点是描述预热阶段内部与表面响应的空间差异。附录 2 的物性为常数，水分扩散系数随含水率变化，因而可分别建立导热方程与非线性扩散方程，以初始均匀状态、轴心对称条件和表面交换条件构成定解问题。采用圆环有限体积法共享相邻单元面通量，并通过隐式积分获得规定点值与完整过程 [1]。

对于问题二，物性随局部温度或含水率改变，温度与水分场之间形成反馈，需从共同初态联立求解变物性方程。前 3 h 规定表应从同一全过程轨迹提取，不接续第一问末态。采用 Chebyshev 配点与隐式积分处理空间梯度和长时演化，同时分析升温与失水对扩散系数的共同作用；观测时段之后的环境按明确的延拓假设处理。

对于问题三，难点在于把“各处低于阈值”转化为可计算的终止条件。沿用第二问轨迹，追踪全径向最大含水率，并检查重构曲线的内部极值，避免只凭表面值或平均值判断结束。先定位最大值等于阈值的事件根，再结合加密与独立积分检验设置数值余量，选取严格低于阈值的执行点；判定使用未舍入值，表格仅承担结果展示。

对于问题四，半径变化使计算区域和表面位置随时间移动，固定网格上的距离不再直接对应同一材料位置。以干物质守恒为基础建立材料坐标方程，将变动区域映射到固定参考域，并按附录 4 物性从共同初态独立积分。恒长度、均匀径向收缩是对内部运动的附加假设 [2, 3]，其他物料的经验参数不直接迁移。输出时区分实际域内位置与当前表面，并以同物性恒半径条件对照分析收缩的作用。

主模型的空间近似见假设1。有限圆柱全域终点还需结合相应二维证据判断，不能由一维阈值事件直接推出；统一适用边界见第11.3节。

三、模型假设

1. 假设药材为轴对称连续介质，初始温度、干基含水率及干密度在空间上均匀分布。主模型采用中截面径向近似，忽略端面影响。
2. 假设烘房环境在空间上均匀，题给空气水分量可等效作为药材表面传质的有效环境边界量。
3. 假设药材表面的传热、传质通量分别与温度差和有效含水率差成正比，各问沿用同一组恒定交换系数。
4. 假设烘干过程中药材干物质质量守恒，题给密度仅用于有效热容量。模型不显式计入水分蒸发潜热、水分迁移携焓及机械功。
5. 前三问假设药材尺寸保持不变；第四问假设药材长度不变，并随观测半径作均匀径向收缩。
6. 假设 4 h 后烘房环境保持稳定，其温度和空气水分量取 3–4 h 观测数据的算术平均值。按末小时 61 个采样点计算，相应环境平台为

$$T_a = 49.9989344262\text{ }^{\circ}\text{C}, \quad b = 0.04998754098\text{ kg/kg.} \quad (1)$$

四、符号说明

表 S1 主要符号及单位

符号	含义	单位
T	材料摄氏温度	$^{\circ}\text{C}$
T_K	材料绝对温度, $T_K = T + 273.15$	K
C	材料干基含水率, $C = m_w/m_d$	kg/kg
b	有效材料边界值	kg/kg
r	径向坐标	m
z	轴向坐标	m
R	当前外半径	m
L	药材总长度	m
ρ_{eff}	有效热容量采用的经验密度	kg/m^3
ρ_d	单位当前体积的干物质密度	kg/m^3
c_p	比热容	$\text{J}/(\text{kg} \cdot \text{K})$
k	导热系数	$\text{W}/(\text{m} \cdot \text{K})$
D	有效水分扩散系数	m^2/s
v_s	材料速度	m/s
J	材料映射的体积雅可比	1
ξ	参考坐标, $\xi = r/R$	1
x	配点坐标, $x = \xi^2$	1
t_*	最大含水率等于阈值的临界时刻	s
t_{exec}	正式执行时刻	s

五、统一传热传质模型与数值方法

本节给出四问共用的局部收支关系。第一问在固定圆柱内代入常热物性并作圆环积分，第二问将局部变物性代入并联立求解；第三问在第二问轨迹上求严格阈值的执行时刻，第四问再通过材料坐标处理观测收缩。相应的方程化简与求解步骤分别见第六–九节。依假设1，主模型的温度、含水率及局部物性仅随径向位置和时间变化；题表的“到药材中心距离”按轴向中截面内的径向位置解释。

5.1 通用方程与初边值条件

设 $D_s/dt = \partial_t + v_s \cdot \nabla$ 为材料导数。以 $\mathbf{j}_T = -k(C)\nabla T$ 表示导热通量，并用题给有效容量 $\rho_{\text{eff}}c_p$ 描述局部温升率；将单位体积的有效热储存率等同于导热净输入 $-\nabla \cdot \mathbf{j}_T$ ，得到

$$\rho_{\text{eff}}(C)c_p(C)\frac{D_s T}{dt} = \nabla \cdot (k(C)\nabla T). \quad (2)$$

式(2)采用上述有效容量闭合，没有蒸发潜热或组分携焓项，不是对真实总焓守恒的证明。对随干骨架运动的控制体 $V_s(t)$ ，干物质不穿过边界；内部不考虑压力驱动流动及流体相对干骨架的对流，水分仅以相对材料的扩散通量 $\mathbf{j}_w = -\rho_d D \nabla C$ 交换。记外法向为 $\mathbf{n}$ ，则

$$\begin{aligned} \frac{d}{dt} \int_{V_s(t)} \rho_d dV &= 0, \\ \frac{d}{dt} \int_{V_s(t)} \rho_d C dV &= - \int_{\partial V_s(t)} \mathbf{j}_w \cdot \mathbf{n} dA. \end{aligned} \quad (3)$$

将式(3)用输运定理展开，再从水分方程中减去 C 倍的干物质方程，得到

$$\partial_t \rho_d + \nabla \cdot (\rho_d \mathbf{v}_s) = 0, \quad \rho_d \frac{D_s C}{dt} = \nabla \cdot (\rho_d D(T, C) \nabla C). \quad (4)$$

由初始干密度均匀以及固定骨架或均匀径向收缩的假设， ρ_d 保持空间均匀，可以从水分方程约去；时间变化的体积不应再向干基变量额外添加浓缩源。将内部外通量与表面对流交换相等，水分条件按同一干质量标度约去 ρ_d ，得到

$$-k \partial_n T = h_T (T_s - T_a), \quad -D \partial_n C = h_m (C_s - b). \quad (5)$$

当 $C_s > b$ 时外向失水为正；若边界关系改变导致 $C_s < b$ ，不能继续强制只许失水。轴线和二维中面满足零法向梯度。初态统一为

$$T(r, z, 0) = 28^\circ\text{C}, \quad C(r, z, 0) = 2.55 \text{ kg/kg}. \quad (6)$$

在一维固定圆柱中，散度取 $r^{-1} \partial_r(r \cdot)$ ；轴对称二维再加 $\partial_z(\cdot)$ 。变系数保留在散度内部，不能将 $k(C)$ 或 $D(T, C)$ 简单提出算子； $\rho_{\text{eff}}c_p$ 乘温度变化率也不能直接改写为 $\partial_t(\rho_{\text{eff}}c_p T)$ 。

h_T 按单一系数处理，不另区分对流与辐射项。附件空气量记为 y_a ，有效边界值取 $b = g(y_a, T_s, \dots)$ ，现行计算取恒等映射 $g(y_a, \dots) = y_a$ 。观测区间内，环境与半径记录均采用分段线性插值；4 h 后的环境采用式(1)的平台。

5.2 分问物性与参数设置

表面换热、传质系数取附录 2 给定的 $h_T = 25 \text{ W}/(\text{m}^2\text{K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ 。表 S2 保留题给公式。所有指数中的温度使用 T_K ，材料水分变量均为干基 C ，不能与文献湿基量互换。

表 S2 题给物性与本文使用方式

	问题一	问题二、三	问题四
$\rho_{\text{eff}} (\text{kg}/\text{m}^3)$	820	$650 + 128C$	$760 + 90C$
$c_p (\text{J}/(\text{kg K}))$	2600	$1450 + \frac{2736C}{1+C}$	$1850 + \frac{2150C}{1+C}$
$k (\text{W}/(\text{m K}))$	0.36	$0.21 + \frac{0.38C}{1+C}$	$0.12 + \frac{0.20C}{1+C}$
$D (\text{m}^2/\text{s})$	$7 \times 10^{-9} e^{-0.89/C}$	$2.4 \times 10^{-3} e^{-0.45/C}$ $\times e^{-3850/T_K}$	$4.2 \times 10^{-4} e^{-0.30/C}$ $\times e^{-3850/T_K}$

5.3 求解方法的分问衔接

第一问采用圆环有限体积法：从控制体收支推导相邻单元共用的面通量，使内部交换在总量求和时抵消，具体推导见第六节。二维验证沿用这一通量收支思想，并增加轴向控制体和端面交换 [1]。

问题二至四的现行主轨迹采用 Chebyshev 配点和隐式 Radau 积分 [4]，并与有限体积、加密配点或另一时间方法交叉检验。为处理较陡水分剖面，对可分离的 $D = A(T)f(C)$ 定义单调原函数

$$U(C) = \int_0^C f(s) \text{d}s, \quad D \nabla C = A(T) \nabla U. \quad (7)$$

式(7)保持原扩散通量，允许先插值较平缓的 U 再反求 C 。第七节据此推导配点方程与表面 Robin 闭合，第九节进一步推导随半径改变的系数。

时间推进采用自适应隐式 BDF 或 Radau 方法，在输入折点分段积分。各问分别报告主离散、独立对照及结果误差；导出一致性、数值精度和物理模型正确性是不同层级的检验，不用单一求解成功标记替代。

5.4 事件、输出与可追溯性

定义指定空间区域上的最湿值 $C_{\text{max}}(t) = \max C(\cdot, t)$ 。等号事件和严格执行分别满足

$$C_{\text{max}}(t_*) = 0.15, \quad C_{\text{max}}(t_{\text{exec}}) < 0.15. \quad (8)$$

停止判断使用未舍入值；显示为 0.1500 的表格数不能区分阈值两侧。对配点解，除检查节点外，还利用式(7)的单调性查找重构多项式的全部实驻点，避免只看轴心而漏掉内部极值。连续二维严格界则需进一步控制空间、时间及重构误差，不能由节点值直接推出。

问题一和二按 1s 输出，问题三和四按 60s 输出；非整步的执行时刻单独追加。固定径向采样间隔为 0.1 cm。第四问在当前实际半径内取值，域外为空白，当前表面另外列出。每个情景保存未舍入数组、求解配置和图表数据，论文中的规定四位表值由对应数组生成，避免从图像或已舍入中间表反算。

六、问题一：预热阶段的径向分布

问题一以第五节的统一模型为基础，在固定几何、常热物性条件下，采用附录 2 物性预测前 1800s 的径向温度与含水率分布。水分扩散系数仍随含水率变化，两场分别求解；从参数代入到规定结果输出的过程如图1所示。

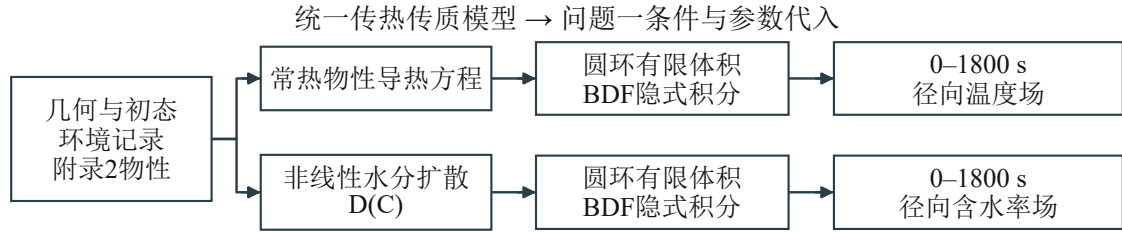


图 1 问题一由统一模型到径向预热模型的求解流程

6.1 常热物性方程与初边值条件

题设圆柱药材长 25 cm、半径 2 cm。本问取固定骨架 $v_s = 0$ ，式(2)、(4)中的材料导数退化为时间偏导数。初始干密度空间均匀且干物质守恒，故固定骨架下干密度保持均匀，可从水分方程约去。

依假设1取中截面径向近似并忽略端面影响，以 r, t 描述两场，计算区间为 $0 \leq r \leq R$ 。轴心取零径向梯度，表面通过线性换热、传质关系与环境相联系。几何与边界条件见图2。

代入附录 2 的常热物性参数与扩散系数关系，得到径向导热方程与非线性水分扩散方程：

$$\begin{aligned} T_t &= \alpha \frac{1}{r} \frac{\partial}{\partial r} (r T_r), & \alpha &= \frac{0.36}{820 \times 2600}, \\ C_t &= \frac{1}{r} \frac{\partial}{\partial r} (r D(C) C_r), & D(C) &= 7 \times 10^{-9} e^{-0.89/C}. \end{aligned} \quad (9)$$

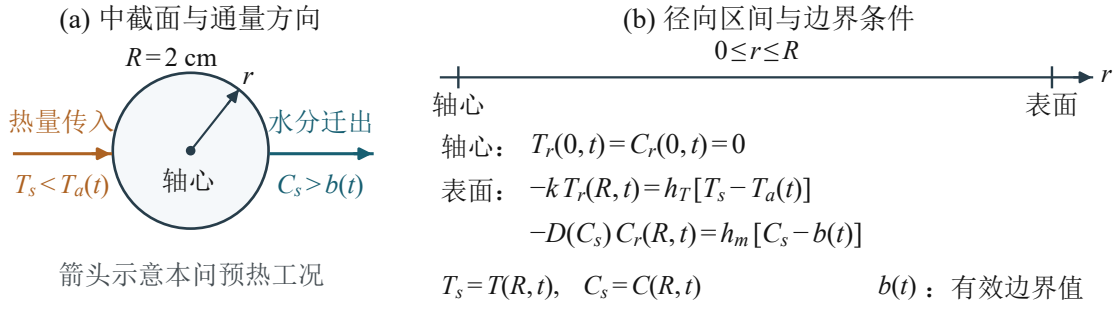


图 2 问题一中截面径向模型与边界条件示意图

式(9)定义在 $0 < r < R = 0.02 \text{ m}$, 初态为 $T(r, 0) = 28^\circ\text{C}$ 、 $C(r, 0) = 2.55 \text{ kg/kg}$ 。式(5) 在轴线对称性和圆柱表面外法向下具体写为

$$\begin{aligned} T_r(0, t) &= C_r(0, t) = 0, \\ -0.36 T_r(R, t) &= 25 [T(R, t) - T_a(t)], \\ -D(C_s) C_r(R, t) &= 8 \times 10^{-7} [C_s - b(t)], \quad C_s = C(R, t). \end{aligned} \quad (10)$$

其中 T_a, b 由环境记录线性插值得到。本问的 D 不依赖温度, 热方程也不依赖含水率, 因此两个场分别求解; 水分场仍是非线性的, 因为其散度展开后为 $D(C)(C_{rr} + C_r/r) + D'(C)C_r^2$, 不能按常扩散率处理。均匀初态与式(10)在初始表面角点不相容, 故保留题给初值, 允许 $t > 0$ 形成早期边界层, 不人为调整初始表面含水率。

6.2 圆环守恒离散与隐式求解

为同时说明两个场的计算, 记 $u = T$ 时 $a(u) = \alpha, \beta = 25/(820 \times 2600), u_a = T_a$; $u = C$ 时 $a(u) = D(u), \beta = 8 \times 10^{-7}, u_a = b$ 。取节点 $r_i = i\Delta r, i = 0, \dots, N, \Delta r = R/N$, 内部控制体面位于相邻节点中点, 两端面分别为 $r_{-1/2} = 0, r_{N+1/2} = R$ 。令外向加权通量 $G = -ra(u)u_r$, 将式(9)乘 r 并在圆环控制体内积分, 得

$$\frac{d}{dt} \int_{r_{i-1/2}}^{r_{i+1/2}} ru \, dr = G(r_{i-1/2}, t) - G(r_{i+1/2}, t). \quad (11)$$

真实体积中的公因子 $2\pi L$ 已约去。以节点值近似式(11) 的储存量, 权重为 $w_i = (r_{i+1/2}^2 - r_{i-1/2}^2)/2$, 因此轴心和表面节点均保留各自控制体容量。把相邻两半格的扩散阻力串联相加, 即 $\Delta r/\hat{a} = (\Delta r/2)/a(u_i) + (\Delta r/2)/a(u_{i+1})$, 内部面得到调和平均及通量:

$$\hat{a}_{i+1/2} = \frac{2a(u_i)a(u_{i+1})}{a(u_i) + a(u_{i+1})}, \quad G_{i+1/2} = r_{i+1/2} \hat{a}_{i+1/2} \frac{u_i - u_{i+1}}{\Delta r}. \quad (12)$$

同一式(12)同时用于相邻两格, 结合式(10) 即得到实际积分的常微分方程组

$$\begin{aligned} w_i \dot{u}_i &= G_{i-1/2} - G_{i+1/2}, \quad i = 0, \dots, N, \\ G_{-1/2} &= 0, \quad G_{N+1/2} = R\beta(u_N - u_a). \end{aligned} \quad (13)$$

轴心内面通量自然为零, 式(13)无须计算奇异的 $1/r$ 。逐格求和后内部面通量抵消, 由 $\sum_i w_i = R^2/2$ 可得

$$\frac{d}{dt} \left(\frac{2}{R^2} \sum_{i=0}^N w_i u_i \right) = -\frac{2\beta}{R} (u_N - u_a). \quad (14)$$

式(14)把圆柱加权平均的变化与表面交换直接对应, 也是数值程序累计边界通量、检查离散总量一致性的依据。

将式(13)写为 $\dot{\mathbf{u}} = \mathbf{F}(t, \mathbf{u})$, 使用自适应隐式 BDF 方法 [4]。每步的新状态由隐式残量及 Newton 修正确定:

$$\begin{aligned} \mathcal{R}(\mathbf{u}^{n+1}) &= \gamma_n \mathbf{u}^{n+1} - \mathbf{q}_n - \Delta t_n \mathbf{F}(t_{n+1}, \mathbf{u}^{n+1}) = \mathbf{0}, \\ [\gamma_n I - \Delta t_n J_F] \delta \mathbf{u} &= -\mathcal{R}. \end{aligned} \quad (15)$$

这里 $\gamma_n, \mathbf{q}_n$ 由积分阶数、步长和已知历史状态确定; 实际采用 SciPy 的变阶 BDF 实现及其精度修正, 式(15)表示共同的隐式求解结构。解析稀疏 Jacobian $J_F = \partial \mathbf{F} / \partial \mathbf{u}$ 保留 $D'(C) = 0.89D(C)/C^2$ 及调和平均的导数, 在非线性的迭代中更新扩散率。积分在每个环境输入折点重启并传递末态。正式径向区间数为 10240, 相对容差 2×10^{-12} , 最大步长 3s; 1s 是输出间隔, 不是积分固定步长。

6.3 规定结果与预热规律

表1和表2给出规定的七个时刻、五个径向位置。`result1.xlsx` 从第 1 秒至第 1800 秒, 每隔 0.1cm 输出 21 个位置的温度和干基含水率, 共 75600 个数值; 计算不提前舍入, 输出统一显示四位小数。

表 1 30 分钟内药材中截面温度 ($^{\circ}\text{C}$)

时间 / s	径向距离 / cm				
	0	0.5	1	1.5	2
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8489
600	28.4533	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1126	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7856

1800s 时轴心、表面温度分别为 33.5753°C 和 36.7856°C , 均低于该时刻环境温度 41.5130°C 。表面先响应, 轴心因传导而滞后。由未舍入值计算, 径向温差为 3.2102K 。同期表面含水率降至 1.5102kg/kg , 轴心实际含水率约为 2.5499924kg/kg , 四位显示仍为 2.5500 ; 因此不能把表中轴心数字未变解释为水分数学上严格不变。

表 2 30 分钟内药材中截面干基含水率 (kg/kg)

时间 / s	径向距离 / cm				
	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2470
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

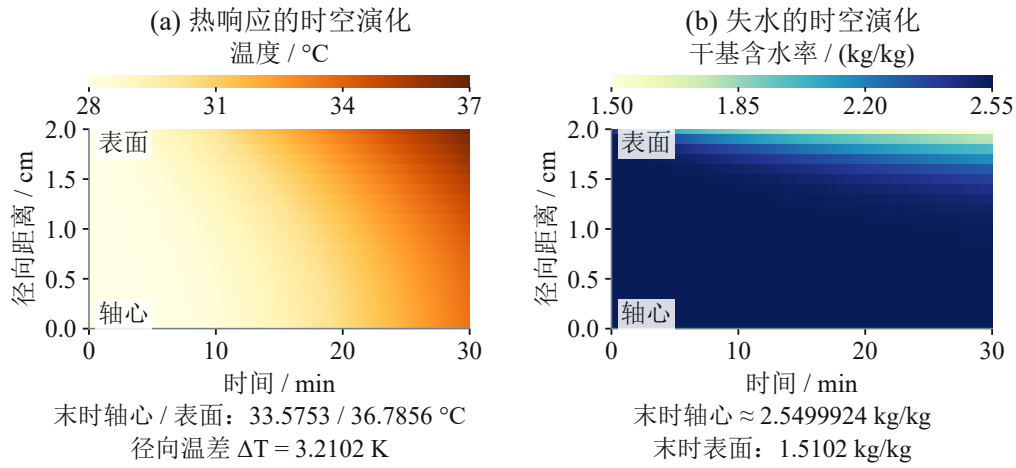


图 3 第一问的时间-径向温湿场。横轴为时间，纵轴为径向距离；左、右色标分别表示温度与干基含水率，均由已保存采样值着色。

沿图3横向可跟踪同一径向位置的变化，沿纵向可比较同一时刻的内部与表面。色阶显示，升温已遍及所考察的径向位置，而水分变化主要集中在表层。末时 $r = 1.5$ cm 处的含水率为 2.3755，较初值下降约 6.84%， $r = 1.0$ cm 处仍为 2.5383。这里不存在严格截断的扩散前沿；描述受影响层厚度时需要先规定含水率变化阈值。

以共同尺度 R 比较扩散时间：初始热扩散率 $\alpha = k/(\rho c_p) = 1.6886 \times 10^{-7} \text{ m}^2/\text{s}$ ， $D(C_0) = 4.9377 \times 10^{-9} \text{ m}^2/\text{s}$ ，故 $R^2/\alpha = 2368.89 \text{ s}$ 、 $R^2/D(C_0) = 81010.11 \text{ s}$ ，两者相差 34.20 倍。这解释了预热阶段热响应快于水分响应；因失水使 D 继续减小，初始扩散时间不是最终烘干时长。圆柱平均须按 $r \text{ d}r$ 加权，得到末时中截面平均温度 35.1679 °C、平均含水率 2.2936 kg/kg；它们不替代最湿点，也不能直接当作包含端部效应的整根平均量。

6.4 数值验证与适用范围

针对前述温湿分布，先以独立参考检验数值一致性。温度以贝塞尔特征函数展开作参考；水分经 $x = (r/R)^2$ 变换，以 Chebyshev 配点与非线性表面条件消元构造独立空间参考解。正式解与参考解在全部规定输出点上的最大温度差为 7.29×10^{-9} K，含水率差为 1.52×10^{-6} kg/kg；全部 75600 个正式输出四位一致。

再检查时间积分精度的影响。固定正式网格收紧时间控制后，温湿差分别约 5.03×10^{-10} K 和 8.15×10^{-11} kg/kg。接近舍入分界的数值必须逐项核对，不能仅凭最大误差小于半个末位单位推断整张表四位一致。

为进一步检验忽略端面影响后的中截面径向近似，采用同物理条件下的 80×128 有限圆柱二维模型进行配对比较。在前 1800 s 已检查采样点上，中截面的最大温湿差分别约为 5.65×10^{-8} K 与 4.09×10^{-9} kg/kg，支持本问中截面径向近似；端面局部差仍可较大，不能据此推广到整个圆柱。

独立核对与中截面配对为前述温湿分布提供了所选有效方程下的数值依据，不消除等效湿度边界、参考密度及无显式潜热假设引入的物理不确定性。本问预热段的相变能量缺口量级显著，相关诊断与解释范围见第11.3节。

七、问题二：变物性热质耦合与全过程输出

问题二在第五章统一模型基础上，采用附录 3 变物性，从共同初态联立求解固定半径下的径向温湿场。通过温度与含水率对物性的反馈，给出规定结果及所设环境延拓下的全程轨迹，为问题三判定终点提供基础。求解流程见图4。

统一传热传质模型 → 问题二条件与参数代入

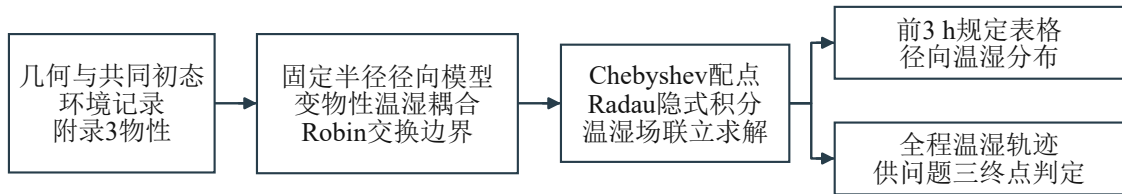


图 4 问题二由统一模型到变物性热质耦合模型的求解流程

7.1 从通用方程到变物性耦合系统

第二问从共同初态开始，全程使用附录 3 物性，不接续第一问末态，也不在 1800 s 处切换公式。有效热容量 $S(C) = \rho_{\text{eff}}(C)c_p(C)$ 和导热率随局部含水率改变，扩散率同时依赖局部绝对温度 $T_K = T + 273.15$ 和 C 。升温改变扩散，失水又通过热容量及导热率反馈温度，因此需联立推进两个场。

将表S2的第二问参数写成

$$S(C) = (650 + 128C) \left(1450 + \frac{2736C}{1+C} \right), \quad k(C) = 0.21 + \frac{0.38C}{1+C}, \quad (16)$$

$$D(T, C) = A(T)e^{-0.45/C}, \quad A(T) = 2.4 \times 10^{-3} e^{-3850/T_K}.$$

固定骨架给出 $v_s = 0$ ，空间均匀的参考干密度可从式(4) 约去。再将式(2)、(4)的散度取为圆柱径向形式，得

$$\begin{aligned} T_t &= \frac{1}{rS(C)} \partial_r [rk(C)T_r], \\ C_t &= \frac{1}{r} \partial_r [rA(T)e^{-0.45/C}C_r], \quad 0 < r < R_0. \end{aligned} \quad (17)$$

例如第一式展开为 $T_t = (k/S)(T_{rr} + T_r/r) + (k'(C)/S)C_rT_r$ ，显式给出含水率梯度对导热的反馈；不能将其改成 $\nabla \cdot [(k/S)\nabla T]$ ，否则会额外引入 S 的空间导数。这里经验 $\rho_{\text{eff}}(C)$ 仅用于有效热容量，不同时充当固定体积下满足真实组成守恒的湿体密度。初态为 $T = 28^\circ\text{C}$ 、 $C = 2.55$ ，轴线零梯度和表面交换沿用式(5)。

7.2 坐标变换、表面闭合与联合积分

为避免直接计算 $r = 0$ 处的 $1/r$ ，令 $x = (r/R_0)^2$ 。对任意光滑径向场 v 及系数 a ，由链式法则得到

$$\partial_r = \frac{2r}{R_0^2} \partial_x, \quad \frac{1}{r} \partial_r (ra \partial_r v) = \frac{4}{R_0^2} \partial_x (xa \partial_x v). \quad (18)$$

有限的 v_x 自动满足轴线 $v_r = 0$ ，且 $x = 0$ 处右端取 $4a(0)v_x(0)/R_0^2$ ，无需人为设置一个很小的替代半径。按式(7)取 $U(C) = \int_0^C e^{-0.45/s} ds$ ，则 $U'(C) = e^{-0.45/C} > 0$ ，水分通量中的 DC_x 恰好等于 $A(T)U_x$ 。这保留局部温度依赖，并使求导作用于较平缓的原函数场。

在 $x_j = [1 - \cos(j\pi/N)]/2$ ， $j = 0, \dots, N$ 上插值，记 $D_x = (\ell'_j(x_i))$ 为 Lagrange 插值求导矩阵， $X = \text{diag}(x_j)$ ； D_x 与物性扩散率 D 含义不同。对包含表面的节点向量，令 $\mathbf{q}_T = \text{diag}(k_j)D_x \mathbf{T}$ 、 $\mathbf{q}_C = \text{diag}(A_j)D_x \mathbf{U}$ 。由 $\partial_x(xq) = q + xq_x$ 逐项配点，得到实际采用的半离散系统

$$\begin{aligned} \dot{\mathbf{T}}_I &= \frac{4}{R_0^2} [\text{diag}(S_j^{-1})(\mathbf{I} + XD_x)\mathbf{q}_T]_I, \\ \dot{\mathbf{C}}_I &= \frac{4}{R_0^2} [(\mathbf{I} + XD_x)\mathbf{q}_C]_I, \quad I = \{0, \dots, N-1\}. \end{aligned} \quad (19)$$

其中 $\mathbf{I}$ 是单位矩阵，下标 I 表示取待积分节点；表面 $j = N$ 不另设演化方程。由式(5)及 $\partial_r = (2/R_0)\partial_x$ 在表面联立求解

$$\begin{aligned} \gamma k(C_s)(D_x \mathbf{T})_N + h_T(T_s - T_a) &= 0, \\ \gamma A(T_s)(D_x \mathbf{U})_N + h_m(C_s - b) &= 0, \quad \gamma = \frac{2}{R_0}. \end{aligned} \quad (20)$$

两式的正号来自将外向 Robin 通量移至同一侧。由于求导矩阵的行和为零，记 $g_T = \sum_{j < N} (D_x)_{Nj} (T_j - T_a)$ ，第一式可消去表面温度：

$$T_s(C_s) = T_a - \frac{\gamma k(C_s) g_T}{\gamma k(C_s) (D_x)_{NN} + h_T}. \quad (21)$$

代入第二式便得到关于 $C_s > 0$ 的一个非线性方程，用阻尼 Newton 法求根，再回代 T_s 。因此每次内部状态改变都会同步更新表面温湿，不能分别指定二者或用上一时刻的表面值替代；时间积分 Jacobian 也包含这一隐式响应。

将式(19)记为 $\dot{\mathbf{y}} = \mathbf{F}(t, \mathbf{y})$ ，其中 $\mathbf{y} = (T_0, C_0, \dots, T_{N-1}, C_{N-1})^\top$ 。采用三阶段 Radau 隐式积分 [4]，每步联立求解阶段状态并更新：

$$\begin{aligned} \mathbf{Y}_i &= \mathbf{y}_n + \Delta t \sum_{j=1}^3 a_{ij} \mathbf{F}(t_n + c_j \Delta t, \mathbf{Y}_j), \quad i = 1, 2, 3, \\ \mathbf{y}_{n+1} &= \mathbf{y}_n + \Delta t \sum_{i=1}^3 b_i \mathbf{F}(t_n + c_i \Delta t, \mathbf{Y}_i). \end{aligned} \quad (22)$$

a_{ij}, b_i, c_i 为该方法的固定系数；每个阶段均调用上述表面闭合，式(22)从而在同一步内处理温度与含水率反馈。

7.3 全过程输出与规定结果

前 4h 采用附件 1 分段线性环境，之后按统一假设保持末小时 61 点算术均值： $T_a = 49.9989344262^\circ\text{C}$ 、 $b = 0.04998754098$ 。第二问与第三问采用同一条温湿联合轨迹，积分至第三问执行端点 206906.76s，即 57.4741h。因此第二问的完整结果覆盖所选模型下的全过程，题设表 3、表 4 对应的表3、4仅展示前 3h。环境平台是超出观测范围的假设，不能将此完整时间覆盖称为数天实测验证。

全程轨迹采用 80 阶 Chebyshev 配点、原函数形式水分通量和 Radau 隐式时间积分，表面温湿由 Robin 条件联合确定；相对容差 2×10^{-13} ，最大步长 30s，前 4h 限制为 20s。**result2.xlsx** 包含两张表，每表从 1 秒起保存 206907 个时刻，即整数秒至 206906 秒并追加 206906.76 秒；21 个规定半径合计 8,690,094 个温湿数值。逐秒评价联合连续解，不将不同模型的温度、水分拼接。

7.4 规定表格及空间规律

至 3h，轴心、表面温度分别为 49.8495°C 、 49.9664°C ，相差 0.1169K；含水率分别为 1.7662、1.0081，相差 0.7581kg/kg。温度接近空间均匀时，水分仍有明显梯度，不能用均匀含水率的集中参数近似替代径向场。按 $r dr$ 加权，平均温度为 49.9043°C ，平均含水率为 1.3825kg/kg，相对初始含水率下降 45.7833%。该比例是固定参考干密度下的有效水分减少比例，不是实测总质量损失率。

图5左图显示温度总体随时间升高，并逐渐趋于径向均匀；右图显示含水率先在表层下降，低含水区域随后向内部扩展，径向差异仍较明显。

表 3 3 小时内药材中截面温度 ($^{\circ}\text{C}$)

时间 / h	径向距离 / cm				
	0	0.5	1	1.5	2
0.5000	32.1892	32.3821	32.9659	33.9608	35.4130
1.0000	40.3816	40.5536	41.0605	41.8782	42.9976
1.5000	45.8468	45.9348	46.1930	46.6051	47.1400
2.0000	48.4502	48.4881	48.5984	48.7736	49.0033
2.5000	49.4670	49.4792	49.5136	49.5653	49.6609
3.0000	49.8495	49.8553	49.8746	49.9101	49.9664

表 4 3 小时内药材中截面干基含水率 (kg/kg)

时间 / h	径向距离 / cm				
	0	0.5	1	1.5	2
0.5000	2.5499	2.5489	2.5256	2.3257	1.6486
1.0000	2.5257	2.4948	2.3578	2.0230	1.4711
1.5000	2.3861	2.3256	2.1344	1.8020	1.3476
2.0000	2.1709	2.1084	1.9236	1.6259	1.2311
2.5000	1.9566	1.9006	1.7360	1.4720	1.1166
3.0000	1.7662	1.7165	1.5702	1.3333	1.0081

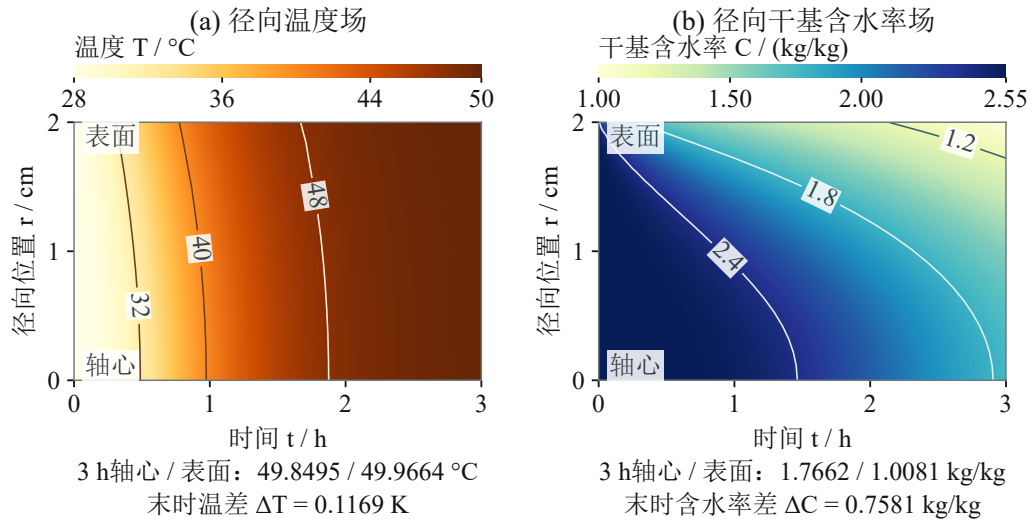


图 5 第二问前 3 小时径向温度场与干基含水率场的时空演化。横轴为时间，纵轴为径向位置，颜色表示场量大小；等值线辅助辨识时空分布，末时径向差值另作标注。

第二问 0.5 h 轴心温度 32.1892°C ，低于第一问同一时刻；这是从初态使用不同物性、尤其初始有效热容量更大的正常结果，不是问间接口错误。第一问和第二问是不同参数情形，不能共用一段预热轨迹。

7.5 升温促进与失水抑制的竞争

将附录 3 扩散系数相对初态取对数，可精确分解为

$$\ln \frac{D(t, r)}{D_0} = H(t, r) + M(t, r), \quad (23)$$

$$H = 3850 \left(\frac{1}{301.15} - \frac{1}{T_K} \right), \quad M = 0.45 \left(\frac{1}{2.55} - \frac{1}{C} \right).$$

其中 $D_0 = 5.64168037 \times 10^{-9} \text{ m}^2/\text{s}$ 。 H 度量沿当前轨迹的累计温度贡献， M 度量累计含水率贡献；这是代数分解，不能当作独立改变某一因素的因果贡献百分比。

图6展示上述分解；前 3 h 逐秒采样中，轴心与表面扩散率分别于约 2.46 h 和 2.02 h 达到该时段的采样峰值，随后回落。

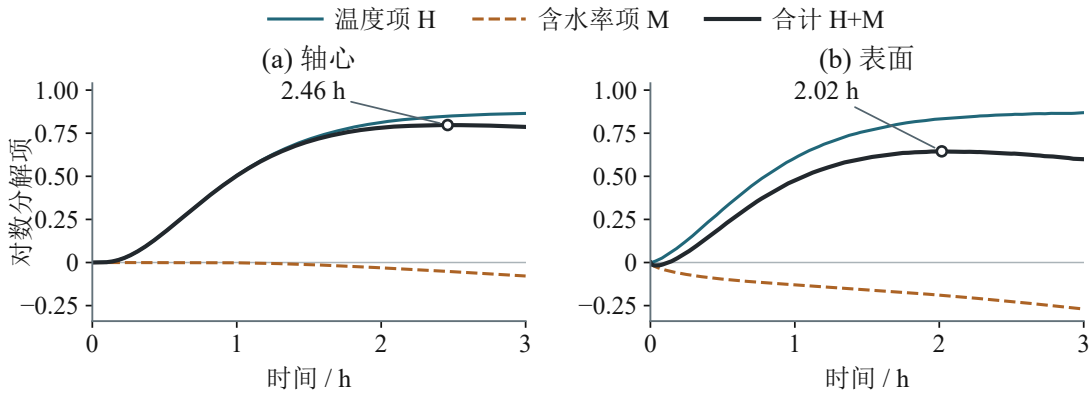


图 6 第二问前 3 小时轴心与表面的扩散率对数分解。 H 为温度贡献， M 为含水率贡献， $H + M = \ln(D/D_0)$ ；标记为逐秒采样峰值，不表示连续时间严格极值。

3 h 时，轴心与表面的 D/D_0 分别为 2.1957 与 1.8207，说明相对初态累计升温促进仍占优势；但最后半小时，两处扩散率分别下降约 1.06% 和 3.16%。因此“扩散系数全程单调增加”不成立。表面还曾在早期每秒采样的第 144 秒达到 $D/D_0 = 0.983282$ 的局部最低值。温度变化和失水抑制的竞争随阶段变化，前 3 小时的累计主导关系不能直接用于判定末期干燥瓶颈。

在同一 320 区间有限体积网格上单独扰动交换系数， h_m 降低、提高 20% 时，3 h 表面含水率分别为 1.1775、0.8737；相应轴心值为 1.8581、1.6921。对这一输出和扰动区间，水分交换系数的影响明显大于数值误差。该对照为人为情景而非参数置信区间（见第 10.2 节），不能替代气固水分基准识别 [1, 2]。

7.6 独立验证与适用范围

以下核查数值输出与模型账本的一致性，并检验降维适用范围。

前 3 小时已有节点有限体积逐级加密至 2560 区间及 Richardson 外推，另用独立空间离散与 Radau 积分核对。全程轨迹与该外推轨迹在前 3 小时共同采样点的最大温差约 $1.07 \times 10^{-9} \text{ K}$ 、含水率差约 $8.41 \times 10^{-8} \text{ kg/kg}$ ；前 3 小时温湿表及

规定 60 个论文表值四位一致。有限体积外推是这一前 3 小时交叉验证的来源，不是完整工作簿的生成方法。

长时段另与已完成的 160 阶 BDF 轨迹在全部 60 秒采样点及同一执行端点比较，最大温湿差约 3.27×10^{-9} K 与 9.33×10^{-9} kg/kg，四位输出一致。当前 869 万余格工作簿与同源未舍入轨迹的舍入核对全部通过，该检查支持导出一致性；高阶独立参考覆盖的是所列采样点，不能把导出一致性误写成全部逐秒状态均另作了独立求解。

原有限体积有效水分账本的最大残差约 6.51×10^{-12} kg/kg。热账本核对的是有效热容量积分与边界热流，即比较 $\iint S(C)T_t dVdt$ 与累计边界热量，相对残差约 3.11×10^{-11} ，并非 $\int ST dV$ 的末初差，更不构成潜热、携焓及真实总能量已经闭合的证明。同物理 80×256 二维配对在前 3 小时已检查采样点的中截面最大温湿差约为 1.692×10^{-3} K、 2.247×10^{-5} kg/kg。这支持中截面的工程近似，但不能保证降维后所有四位显示相同；长期全域达标须结合第三问终点证据，不能直接沿用前 3 小时的几何误差。本问早期大通量阶段亦受相变能量缺口约束，相关量级诊断与闭合限制见第 11.3 节。

综上，温湿场响应不同步，升温促进与失水抑制呈阶段性竞争；所得同一条轨迹为问题三追踪径向最大含水率、判定干燥终点提供计算基础。

八、问题三：由全域含水率约束确定干燥时长

问题三在问题二变物性热质耦合模型的基础上，进一步确定满足含水率要求的干燥时长。本问保持控制方程、物性关系与环境边界不变，沿用同一初态下的温湿联合轨迹。在一维径向模型中，以全径向最大含水率严格低于 0.15 kg/kg 为终止要求，不以表面含水率或截面平均值代替最大值。先通过下降穿越检测与括弧求根定位候选等号根，再以径向重构极值复核；结合网格加密和独立积分对照设置经验数值余量，选取并复核严格满足阈值要求的执行点，给出所设环境条件下的模型干燥时长与径向含水率分布。求解流程见图 7。

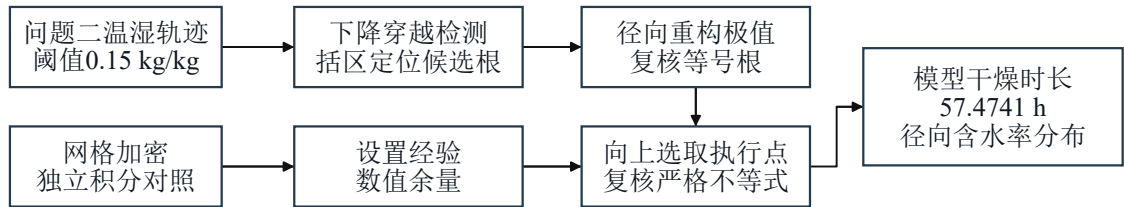


图 7 问题三的一维径向终点判定流程

8.1 终止判据与长时计算

问题三要求药材各处的干基含水率严格低于 $C_{\lim} = 0.15 \text{ kg/kg}$ 。本文沿用问题二的附录 3 物性、初态和环境条件，由同一温湿联立轨迹确定终点，而不以表面含水率或截面平均值代替最大值。因此，本问直接沿用式(17)的控制方程及式(19)、(20)的离散闭合，新增的求解任务是从时间轨迹中定位满足全径向约束的执行点。在一维径向近似中定义

$$M_1(t) = \max_{0 \leq r \leq R_0} C(r, t), \quad M_1(t_{*,3}) = C_{\lim}. \quad (24)$$

所取事件为结束附近的下降穿越。等号根是严格达标时间集合的边界，根本身不满足题设严格不等式；有限圆柱中“各处”的含义则由后述二维计算另行检验。

附件 1 仅覆盖前 4 h，长时计算需延拓环境。本文保持前述末小时 61 点算术均值平台，即环境温度 $49.9989344262^\circ\text{C}$ 、有效边界值 $b = 0.04998754098 \text{ kg/kg}$ 。该平台是假设的后续工况，不是 4 h 以后的观测记录。温度和含水率始终按式(2)–(5)联立推进，不在温度接近环境后直接将温度场替换为常数。

随着含水率降低，扩散系数中的 $\exp(-0.45/C)$ 迅速减小，长时低含水率阶段需要保留局部非线性通量。令

$$a_3(T) = 2.4 \times 10^{-3} \exp\left[-\frac{3850}{T + 273.15}\right], \quad (25)$$

$$F_3(C) = \int_0^C \exp(-0.45/s) \, ds, \quad D\nabla C = a_3(T)\nabla F_3(C).$$

采用 80 阶 Chebyshev 配点和隐式 Radau 方法推进，表面状态由 Robin 条件联立确定。式(25)只改写通量，不改变题给扩散系数。临界附近同时检查径向重构极值和边界值，确认最大含水率位于轴心。空间加密与不同隐式时间方法作为交叉核实验，规定表格和完整结果均由同一未舍入轨迹生成。

8.2 由重构空间极值到时间事件

令 $P_U(x, t) = \sum_{j=0}^N F_3(C_j(t))\ell_j(x)$ 为包含表面状态的原函数插值多项式， $x = (r/R_0)^2$ 。在 $P_U > 0$ 的物理解支上，含水率重构为 $C_N(x, t) = F_3^{-1}[P_U(x, t)]$ 。由于 $F_3'(C) > 0$ ，有 $\partial_x C_N = \partial_x P_U / F_3'(C_N)$ ，故两者的内部驻点相同。每一时刻的极值候选集为

$$\mathcal{X}(t) = \{0, 1\} \cup \{x \in (0, 1) : \partial_x P_U(x, t) = 0\}. \quad (26)$$

求出式(26)中的全部实根，并比较候选点值，得到

$$\widehat{M}_1(t) = \max_{x \in \mathcal{X}(t)} C_N(x, t) = F_3^{-1}\left[\max_{x \in \mathcal{X}(t)} P_U(x, t)\right]. \quad (27)$$

该式确定的是重构一维场的最大值；重构误差及与连续方程的差异仍需数值加密检验，不能把多项式极值的求全等同于连续模型严格误差为零。

实际程序先以内部节点事件 $g_{\text{node}}(t) = \max_{j < N} C_j(t) - C_{\text{lim}}$ 检测下降穿越。当相邻接受步满足 $g_{\text{node}}(t_a) > 0$ 、 $g_{\text{node}}(t_b) \leq 0$ 时，利用 Radau 的步内连续插值对 $g_{\text{node}}(t) = 0$ 作 Brent 括区求根，再在候选根和执行点按式(27)复核，并用根两侧节点值检查下降穿越。本算例确认根与执行点的重构最大值均位于轴心，节点候选根才可用于所述径向终点。若重构出现更湿的内部点，就必须重新定位事件，不能直接接受节点根。

为解释时间裕量的尺度，在根附近作一阶展开：

$$\widehat{M}_1(t_* + \delta t) - C_{\text{lim}} \approx \widehat{M}_1(t_*) \delta t, \quad |\delta t| \approx \frac{|\delta M|}{|\dot{\widehat{M}}_1(t_*)|}. \quad (28)$$

这里 δM 为含水率的局部数值差。已有事件记录给出末期斜率约 $-2.9128 \times 10^{-7} \text{ (kg/kg)/s}$ ；因此显示到四位小数的 0.1500 不能支持亚秒级的达标判定。式(28)仅作局部误差尺度解释，正式时间预算仍取自网格与时间方法对照，不能由该线性近似给出严格界。

8.3 结束时长与径向水分分布

一维模型的临界根为 $t_{*,3} = 206906.389932328 \text{ s}$ ，即约 57.4739972 h。为区分数值根、执行点和四位小时显示值，取显示时间量子 $\Delta t = 0.0001 \text{ h} = 0.36 \text{ s}$ ，并按空间、时间及独立方法差异设置 $\varepsilon_{t,3} = 0.03 \text{ s}$ 的经验数值余量，选择

$$t_{\text{exec},3} = \Delta t \left\lceil \frac{t_{*,3} + \varepsilon_{t,3}}{\Delta t} \right\rceil = 206906.76 \text{ s} = \boxed{57.4741 \text{ h}}. \quad (29)$$

在该实际时刻重新代入未舍入一维场，得到

$$M_1(t_{\text{exec},3}) = 0.149999892208 \text{ kg/kg} < 0.15 \text{ kg/kg}. \quad (30)$$

执行点比临界根晚约 0.3701 s。这里的经验余量用于当前方程的数值选择，不含环境、边界映射和物性假设的不确定性，也不是实验安全上界。

题设表 5 要求的每 6 h 及结束时刻的径向含水率见表5。18 h 时，表面已降至 0.0845 kg/kg，轴心仍为 0.2988 kg/kg；54 h 时轴心仍为 0.1539 kg/kg。因此，表面先达到要求并不能作为全体材料的终止依据。末期水分向低湿表面迁移，而局部扩散系数随含水率下降，共同形成中心缓慢趋近阈值的尾段。

完整的 **result3.xlsx** 以 60 s 为间隔、0.1 cm 为径向间隔输出，并在最后一个完整分钟 206880 s 之后补入执行点 206906.76 s。末行轴心显示为 0.1500，是式(30)的四位舍入值；达标判断使用未舍入场而不使用表格显示值。160 阶空间加密参考和另一隐式时间方法在全部 72429 个规定水分输出上四位一致，

表 5 题设表 5: 固定半径时的径向干基含水率 (kg/kg)

时间 / h	径向距离 / cm				
	0	0.5	1	1.5	2
6.0000	1.0170	0.9881	0.9015	0.7550	0.5328
12.0000	0.4561	0.4432	0.4031	0.3297	0.1642
18.0000	0.2988	0.2916	0.2687	0.2253	0.0845
24.0000	0.2378	0.2327	0.2167	0.1855	0.0665
30.0000	0.2058	0.2018	0.1892	0.1641	0.0598
36.0000	0.1859	0.1825	0.1718	0.1504	0.0566
42.0000	0.1721	0.1691	0.1597	0.1407	0.0548
48.0000	0.1618	0.1592	0.1507	0.1334	0.0537
54.0000	0.1539	0.1514	0.1436	0.1277	0.0530
57.4741	0.1500	0.1477	0.1402	0.1249	0.0526

其中空间加密参考与正式轨迹的最大未舍入差约 $9.33 \times 10^{-9} \text{ kg/kg}$ 。该结果支持已选方程的数值稳定性，不替代物理参数验证。

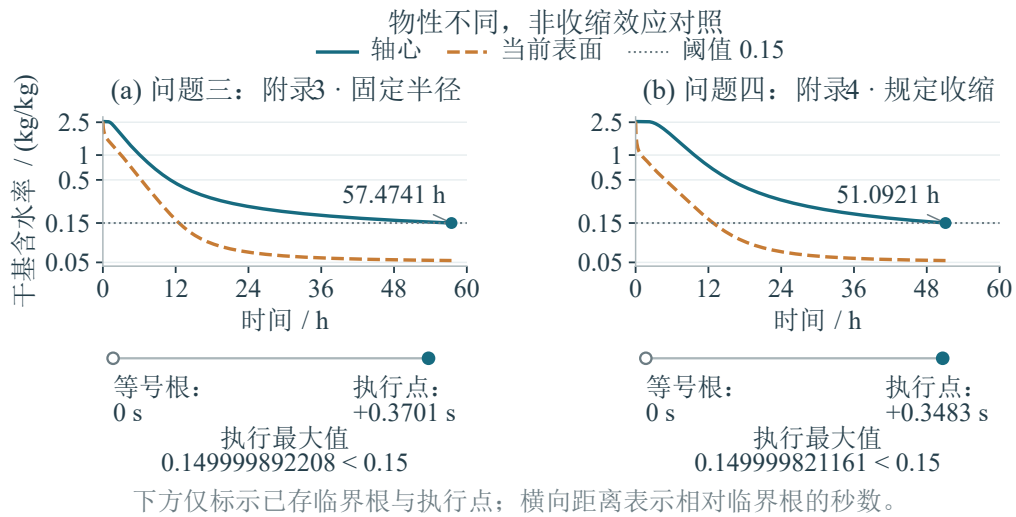


图 8 问题三(左)与问题四(右)的干燥过程与事件判定。上排为一维径向有效模型的轴心、当前表面含水率演化。纵轴为对数刻度，水平线为 0.15 kg/kg 阈值，终点标记对应各自正式执行时刻。下排以各自临界根为时间原点，仅标示已存临界根与执行点，并列出行执行最大值。两问采用各自附录物性，问题四另施加观测半径历程，因而曲线差异不能全部归因于收缩。

图8上排展示表面先失水、轴心控制尾段的过程；下排将等号临界根与已选执行点分开，时间差直接取自原事件记录。

8.4 有限圆柱检验与工况条件

为检验“各处”的空间含义，在长度 0.25 m 的有限圆柱上增加轴向传递，侧面与端面保持相同物性和交换系数。以 80 个径向区间、256 个轴向区间的同网格一维/二维配对计算比较事件根，二维比配对一维提前约 7.4740 s。这一差值用于识别端部效应，不能把粗网格二维绝对根直接替代精细一维根。

再从临界附近已保存的二维场推进至式(29)的执行时刻，得到全节点最大含水率 0.149998606290 kg/kg，最湿点位于中截面轴心，离散场已经达标。该节点结果与配对根差支持 57.4741 h 的工程保守方向；局部续算仍继承前史离散误差，尚不构成连续二维域的严格数学误差界，也不将该时长解释为精确的二维最短时间。上述证据层级集中见第11.3节。

后段环境对终时的影响大于末位数值差。例如，在相同 4 h 初始全场和其余参数下，将之后温度平台降低 1 K，事件根由约 57.4740 h 延至 59.3413 h；升高 1 K 则变为 55.6884 h。该单因素情景为人为设定而非观测误差范围（见第10.2节）。故 57.4741 h 应与环境平台和有效气固边界共同报告，不能仅凭小数位数给出真实工况下的确定性保证。

九、问题四：观测收缩下的干燥过程

问题四在第五章统一模型基础上，结合附件 2 的观测半径，在恒长度、均匀径向收缩假设下，用材料坐标与干物质守恒处理几何变化。采用附录 4 物性，从共同初态联立求解温湿场，沿用问题三的径向最大含水率判据确定所设环境条件下的模型干燥时长，并通过同物性恒半径对照分析收缩作用。求解流程见图9。

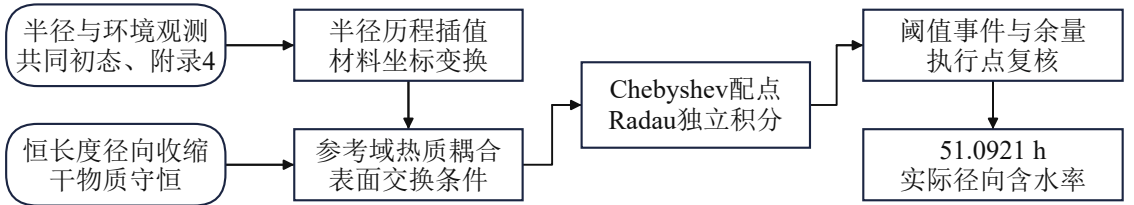


图 9 问题四的收缩模型求解流程

9.1 半径历程与材料运动

问题四将附录 4 物性代入式(2)–(5)，并根据附件 2 把固定几何改为运动材料域。温度和含水率从 28 °C、2.55 kg/kg 的共同初态重新求解，不接续问题三的干燥

末态。附件 2 含 0–72 h 的 145 个半径记录, 间隔 1800 s, 半径由 2 cm 降至 1.198 cm。本文将半径转换为米后分段线性插值为 $R(t)$, 保持其单调性和平台段; 所求主终点位于观测时域内。环境继续采用前 4 h 观测插值及之后的共同均值平台。

外半径只规定边界位置, 不能唯一确定内部材料运动。采用均匀径向收缩、长度 $L = L_0$ 不变的运动学闭合, 定义材料标记 ξ 及映射

$$r = R(t)\xi, \quad z = Z, \quad v_{s,r} = \frac{R'(t)}{R(t)}r = w_r, \quad v_{s,z} = 0, \quad J = \left(\frac{R(t)}{R_0}\right)^2. \quad (31)$$

其中 $w_r = \partial r / \partial t|_\xi$ 为网格速度, J 为体积 Jacobian。该材料映射是一项附加假设, 而不是半径记录直接测得的内部变形。移动边界模型须区分体积水浓度与材料干基含水率, 并以当前界面的相对通量写守恒条件 [2]。

9.2 干基守恒与参考域方程

当前体积水浓度为 $\rho_d C$ 。对干物质与水分别写守恒式, 并采用相对材料的扩散通量, 得

$$\begin{aligned} \partial_t \rho_d + \nabla \cdot (\rho_d \mathbf{v}_s) &= 0, \\ \partial_t (\rho_d C) + \nabla \cdot (\rho_d C \mathbf{v}_s + \mathbf{j}_w) &= 0, \quad \mathbf{j}_w = -\rho_d D \nabla C. \end{aligned} \quad (32)$$

第二式减去第一式的 C 倍, 即得到式(4)。由初始干密度均匀和式(31), $\rho_d J = \rho_{d,0}$, 所以 $\rho_d(t) = \rho_{d,0} [R_0/R(t)]^2$ 为空间常数, 可从干基水分方程中约去。对任一场量 $f(r, t) = \hat{f}(\xi, t)$, 链式法则给出

$$\partial_t f|_r = \partial_t \hat{f}|_\xi - \frac{R'}{R} \xi \partial_\xi \hat{f}, \quad v_{s,r} \partial_r f = \frac{R'}{R} \xi \partial_\xi \hat{f}. \quad (33)$$

两项中的材料运动恰好抵消, 故 $D_s f / dt = \partial_t \hat{f}|_\xi$ 。再将 $\partial_r = R^{-1} \partial_\xi$ 代入式(2)和(4), 省略场量帽号后, 一维参考域方程为

$$\begin{aligned} \partial_t C|_\xi &= \frac{1}{R(t)^2 \xi} \partial_\xi [\xi D(T, C) \partial_\xi C], \\ \rho_{\text{eff}}(C) c_p(C) \partial_t T|_\xi &= \frac{1}{R(t)^2 \xi} \partial_\xi [\xi k(C) \partial_\xi T]. \end{aligned} \quad (34)$$

轴线取正则对称条件; 将同一空间导数变换代入式(5), 当前侧表面的边界式为

$$-\frac{D(T_s, C_s)}{R} \partial_\xi C|_1 = h_m(C_s - b), \quad -\frac{k_s}{R} \partial_\xi T|_1 = h_T(T_s - T_a). \quad (35)$$

其中 $k_s = k(C_s)$, 表面系数均按当前温湿状态计算。

式(34)没有额外的几何浓缩源。无交换的纯收缩使体积水浓度 $\rho_d C$ 增加, 但水与干物质质量均不变, 因此干基 C 应保持不变。若再加入 $-2R'C/R$ 或另一个网格对流项, 就会重复计算已经包含的材料运动。附录 ρ_{eff} 用于有效热容量, ρ_d 用

于上述质量账本；二者不能同时通过 $\rho_d = \rho_{\text{eff}}/(1 + C)$ 强制等同为真实组分密度。热式仍无显式潜热，数值闭合不构成真实组分总焓守恒的证明（见第11.3节）。

附录 4 的局部扩散系数为

$$D(T, C) = 4.2 \times 10^{-4} \exp(-0.30/C) \exp\left[-\frac{3850}{T + 273.15}\right]. \quad (36)$$

全部热物性也在各空间位置按当前 C 更新，温湿两场继续联立求解。表面交换系数沿用 $h_T = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ ，式(35)中的质量通量相对于材料界面，不另将表面扫过的体积计为蒸发失水。

9.3 数值求解、账本与结束状态

令 $x = \xi^2$ ，对任意扩散系数 a 和场量 f 有

$$\frac{1}{R^2 \xi} \partial_\xi (a \xi \partial_\xi f) = \frac{4}{R^2} \partial_x (a x \partial_x f) = \frac{4}{R^2} [q + x \partial_x q], \quad q = a \partial_x f. \quad (37)$$

该式消去轴线显式奇点，在 $x = 0$ 处直接取 $4q(0)/R^2$ 。依式(7)定义 $F'_4(C) = \exp(-0.30/C)$ 、 $F_4(0) = 0$ ，并令 $A_4(T) = 4.2 \times 10^{-4} \exp[-3850/(T + 273.15)]$ 。沿用问题二的 Chebyshev 微分矩阵 D_x ，记 $q_{T,i} = k(C_i)(D_x \mathbf{T})_i$ 、 $q_{C,i} = A_4(T_i)[D_x F_4(\mathbf{C})]_i$ 及 $S_i = \rho_{\text{eff}}(C_i)c_p(C_i)$ ，则内部节点方程为

$$\begin{aligned} \dot{T}_i &= \frac{4}{R(t)^2 S_i} [q_{T,i} + x_i (D_x \mathbf{q}_T)_i], \\ \dot{C}_i &= \frac{4}{R(t)^2} [q_{C,i} + x_i (D_x \mathbf{q}_C)_i]. \end{aligned} \quad (38)$$

其中 $i = 0, \dots, N-1$ ，表面 $i = N$ 由式(35)确定：

$$\frac{2}{R} q_{T,N} + h_T (T_N - T_a) = 0, \quad \frac{2}{R} q_{C,N} + h_m (C_N - b) = 0. \quad (39)$$

这里 $\partial_\xi = 2\xi \partial_x$ 使表面因子成为 $2/R$ ，而内部演化因子为 $4/R^2$ ；二者均随观测半径更新。取 $N = 120$ ，联立求解表面温湿代数方程后用 Radau 推进内部未知量，在环境及半径输入节点处分段重启。

参考圆环的当前体积与固定干质量分别为 $V_i = \pi R^2 L_0 (\xi_e^2 - \xi_w^2)$ 、 $m_{d,i} = \rho_d V_i$ 。利用圆环权重与固定干质量构造水量账本 [1]，有

$$\overline{C}(t) = \int_0^1 C(x, t) dx, \quad \overline{C}(t) + \int_0^t \frac{2h_m}{R(\tau)} [C_s(\tau) - b(\tau)] d\tau = C_0. \quad (40)$$

边界通量积分与空间平均的全程段末最大差为 $4.96 \times 10^{-13} \text{ kg/kg}$ 。72 h 零交换纯收缩检验中，温度与干基含水率保持初态，干物质和水质量的相对变化小于 4.45×10^{-16} 。这两项检验针对已选干基守恒与材料运动，不把经验密度的物理解释也视为已验证。

将配点阶数由 120 增加至 160, 共同输出点最大含水率差为 8.56×10^{-12} kg/kg; 同阶 BDF 时间方法与 Radau 的临界根差约 5.98×10^{-4} s。另用 640 个径向区间的有限体积独立离散, 在相同材料位置比较, 最大含水率差约 1.58×10^{-5} kg/kg, 最后两级空间外推的根与谱法相差约 0.0129 s。前两类检验显示谱解已稳定, 有限体积外推提供不同离散方法的支持, 外推差仍是经验数值指标而非严格误差界。

终点检测先由内部节点最大值产生等号候选事件, 再将式(26)–(27) 中的原函数替换为 F_4 , 检查重构多项式的全部内部实驻点及两端点。由于 $F'_4(C) > 0$, 该检查同时给出重构含水率的最大值; 本算例在候选根和相邻时刻均确认最大值位于轴心。一维径向极值下降至阈值时, 临界根为 $t_{*,4} = 183931.211715489$ s。根前后 1 s 的最大值分别约为 0.150000513489 和 0.149999486518, 最大位置在轴心。采用 $\varepsilon_{t,4} = 0.129$ s 的经验数值预算, 按式(29)相同的 0.36 s 量子向上选择执行点, 得

$$t_{\text{exec},4} = 183931.56 \text{ s} = \boxed{51.0921 \text{ h}}, \quad (41)$$

$$\max_{0 \leq r \leq R(t_{\text{exec},4})} C(r, t_{\text{exec},4}) = 0.149999821161 \text{ kg/kg} < 0.15 \text{ kg/kg}.$$

该点晚于一维根约 0.3483 s, 当前半径为 1.2000 cm。题设表 6 的规定采样见表6。对规定的实际位置 r_j , 以 $P_{U,4}(x, t)$ 表示 $F_4(C)$ 的配点插值多项式, 导出值按下式由当前材料场重构:

$$C(r_j, t) = F_4^{-1} \left[P_{U,4} \left(\frac{r_j^2}{R(t)^2}, t \right) \right], \quad 0 \leq r_j \leq R(t). \quad (42)$$

当 $r_j > R(t)$ 时不作域外外推; 当前表面取 $x = 1$ 另列, 不能当作固定 2 cm 位置。

表 6 题设表 6: 收缩过程中的干基含水率 (kg/kg)

时间 / h	径向距离 / cm					当前表面
	0	0.5	1	1.5	2	
6.0000	1.7196	1.5378	1.0226	–	–	0.4207
12.0000	0.7377	0.6546	0.4083	–	–	0.1669
18.0000	0.4088	0.3687	0.2397	–	–	0.0892
24.0000	0.2851	0.2611	0.1790	–	–	0.0673
30.0000	0.2263	0.2094	0.1493	–	–	0.0595
36.0000	0.1928	0.1797	0.1317	–	–	0.0560
42.0000	0.1712	0.1604	0.1200	–	–	0.0541
48.0000	0.1561	0.1468	0.1116	–	–	0.0530
51.0921	0.1500	0.1413	0.1081	–	–	0.0526

表中域外位置以“—”表示不适用，不能解释成零含水率。**result4.xlsx** 按 60 s、0.1 cm 导出，并补入实际执行点；域外单元格留空，另保留表面值与当前半径。末行轴心四位显示 0.1500，严格不等式判断仍基于式(41)的未舍入值。

9.4 收缩作用与模型解释

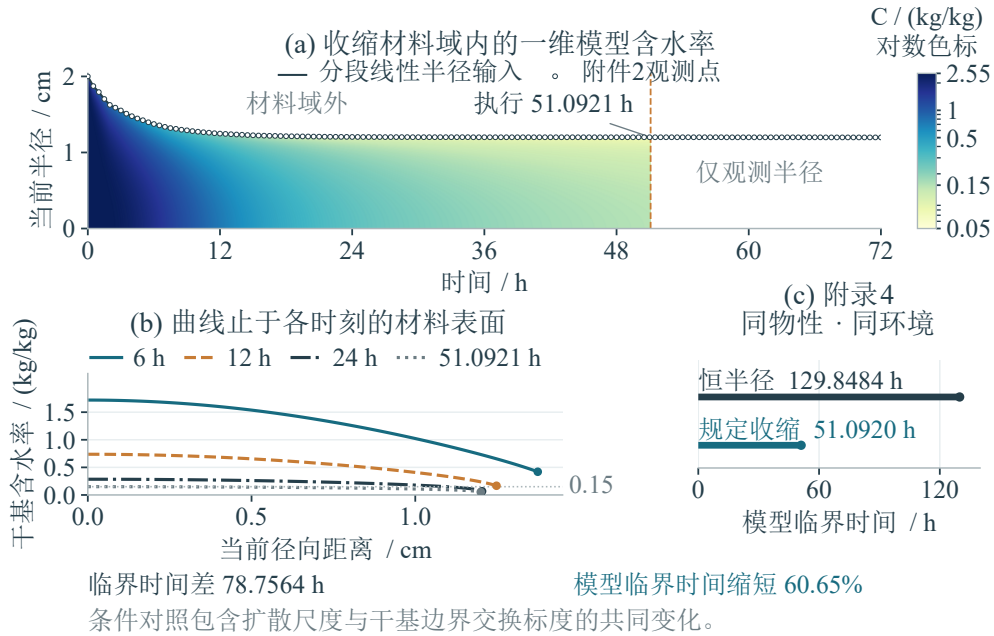


图 10 规定收缩下的径向有效模型场、观测输入与同物性条件对照。(a) 当前实际半径坐标中的含水率色图仅显示至正式执行时刻，采用对数色标；全部观测点及分段线性半径输入保留至 72h，执行线之后只展示半径。(b) 各时刻含水率剖面止于当时材料表面。(c) 附录 4 同物性、同环境下的恒半径与规定收缩临界根对照。临界根与执行时刻分别标示；时间差包含扩散尺度和干基边界交换标度变化。

图10在当前实际半径坐标中展示规定收缩与含水率演化；上方色图止于正式执行时刻并保留完整半径观测输入，左下剖面止于各时刻的材料表面，右下对照原恒半径与规定收缩的临界时间。为分离施加收缩历程的作用，另保持附录 4 和其余条件不变，将半径固定为 2 cm 作受控对照，临界时间为 129.848360839 h；施加观测收缩后为 51.092003254 h，模型临界时间缩短 78.7564 h，约 60.6526%。这反映当前模型中缩短扩散路径与改变边界交换标度的综合作用。第三、四问还更换了整套物性，故图8两问约 6.382 h 的净差不能全部归因于收缩。

固定 h_m 保持的是材料干基 Robin 系数。在所声明的干质量尺度下，表面质量通量为 $j_s = \rho_d(t)h_m(C_s - b)$ ，因而每面积、每单位干基差的通量系数为 $K_C = \rho_d h_m$ 。当 $R/R_0 = 0.6$ 时， K_C 为初始的 2.7778 倍，式(40)中的 $2h_m/R$ 为初始的 1.6667

倍。所以模型临界时间缩短 60.6526% 是同一有效方程内的条件对照结果，不等于固定真实气膜条件下仅由缩短路径得到的效应，也不能据此推断真实热量收支。

半径重建方式也影响终时。将线性插值改为保形 PCHIP，临界时间为 51.088501642 h，比主线提前约 12.6058 s，明显大于单一插值方案内部的数值预算。输入重建差不能当作半径测量误差的置信区间，适用边界见第11.3节。复现给定的观测 $R(t)$ 不能作为温湿预测的独立验证；改变环境或交换系数而沿用该半径历程时，结果仅为条件响应。

十、模型检验：降维有效性与后段环境灵敏度

10.1 中截面、端部与整根积分

中截面配对需要保持相同物性、环境与边界，才能识别额外轴向交换的影响。问题一 80×128 二维对照在 0–1800 s 中截面采样的最大温度差约为 5.65×10^{-8} K，含水率差约为 4.09×10^{-9} kg/kg。问题二采用最终 80×256 对照，在前 10800 s 的对应差为 1.69165×10^{-3} K 与 2.24706×10^{-5} kg/kg，支持中截面工程近似，但不能宣称所有四位单元均一致。

另一方面，端面交换会改变整根失水。按二维控制体权重与一维圆环权重积分得到表7。表中“失水增加”定义为 $(C_0 - \overline{C}_{2D}) / (C_0 - \overline{C}_{1D}) - 1$ ，不是平均含水率的相对误差。在空间均匀干密度的共同假设下，体积权重平均等于干质量权重平均。

表 7 同网格保存场的整根失水对照

情形与时刻	二维网格	$\overline{C}_{2D}$	$\overline{C}_{1D}$	失水增加/%
问题一，1800 s	80×128	2.274929	2.293532	7.2533
问题二，3 h	80×256	1.327482	1.382492	4.7118
问题四，6 h	80×128	1.024545	1.065198	2.7379

图11上排直接显示各时窗终点的中截面含水率差，下排并列对应的整根平均与失水增加，显示局部近似与总体失水应分别判断。上排各面板注明各自量级；下排各行时窗不同，只在同行的一维、二维之间作配对比较。中截面差小与整根总量差可见并不矛盾。若用于全长平均、总失水或端部状态，应采用二维积分或明确忽略端面的近似，不能把中截面表格的精度扩展到全部工程指标。

上述中截面与整根积分对照用于评价降维近似，不能据此直接判定第四问的

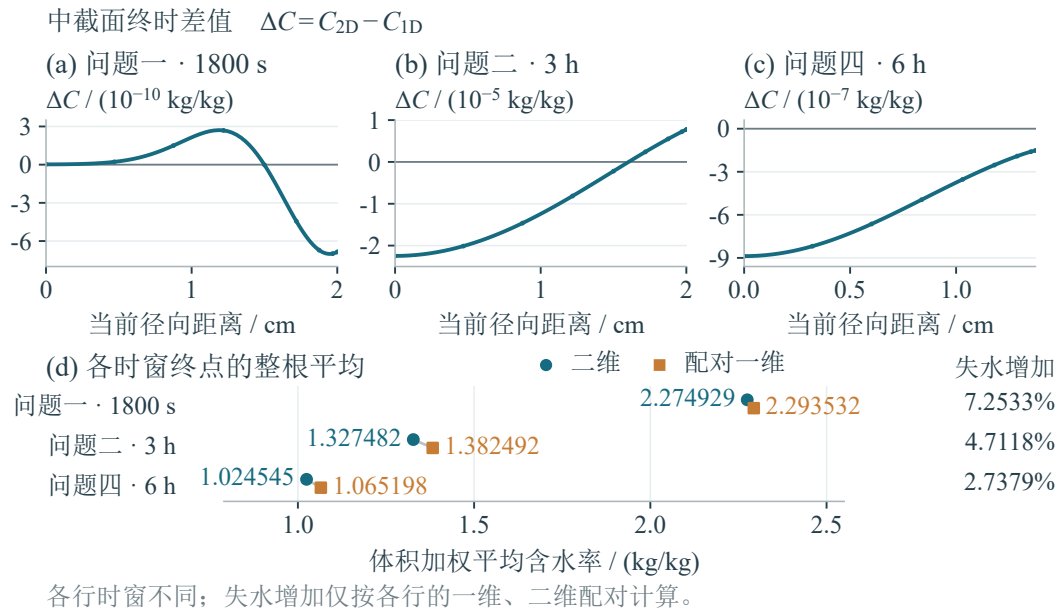


图 11 已保存一维、二维配对场的降维证据：上排为问题一 1800s、问题二 10800s 及问题四 21600s 的中截面径向含水率差 $\Delta C = C_{2D} - C_{1D}$ ；下排为各时窗终点的整根体积加权平均，圆点、方点分别表示二维和配对一维，终点失水增加与表7一致。问题一、二在已检时段中截面采样最大含水率差分别为 4.09×10^{-9} 、 $2.24706 \times 10^{-5} \text{ kg/kg}$ 。上述时段最大差与上排终点差值的统计范围不同。体积加权平均在空间均匀干密度假设下等于干质量加权平均。数据取自同网格一维、二维配对保存场，文件索引与哈希见支撑材料图表来源清单；二维半柱坐标的 $z = 0$ 为中截面对称面。

全域干燥终点。温度场与含水率相互耦合；即使 D 随温度增加，两个解之间的变系数通量散度也未必具有统一符号，因而不能将端面交换使干燥提前视为一般比较定理。

10.2 后段环境的受控情景

由于式(1)仅为 4h 后的延拓，保持短时实测过程不变，从保存的 4h 内部温湿场开展后段联立续算。每问设置原平台、温度 $\pm 1 \text{ K}$ 和有效边界 $b \pm 0.005$ 五种情景；第四问始终使用同一观测 $R(t)$ 。扰动幅度为人为选择的压力测试，没有作为测量不确定度或置信区间。表8给出 40 阶配点的等号事件根，差值相对于同阶基线。

表 8 4h 后环境改变的条件根与时长差

情景	问题三根/h	变化/h	问题四根/h	变化/h
原均值平台	57.473996	0	51.092003	0
$T_a - 1\text{ K}$	59.341275	+1.867279	52.729346	+1.637343
$T_a + 1\text{ K}$	55.688357	-1.785639	49.525343	-1.566660
$b - 0.005$	57.382748	-0.091249	50.993818	-0.098185
$b + 0.005$	57.584113	+0.110117	51.218753	+0.126750

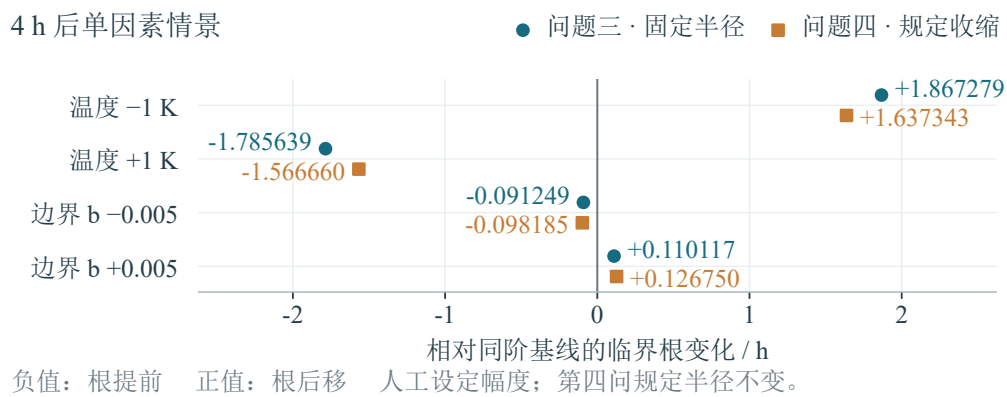


图 12 后段环境情景相对同阶基线的根变化；幅度为人工设定，第四问规定半径保持不变。圆点、方点分别表示问题三、四，共用零线；负值表示根提前，正值表示根后移。

对基线和降 1 K 情景进一步加密：问题三 40 阶到 80 阶、问题四 40 阶到 60 阶，单个根的最大变化分别约 0.002951 s 与 0.000174 s，降温净效应的变化更小。因此小时级差不是此次降阶造成的；其他扰动没有逐一加密，表中六位小数仅便于追踪数值，不代表真实预测精度。

长期降温在既定模型内减小扩散温度因子，推迟阈值；提高 b 减小失水驱动力，也使根后移。由于两类扰动尺度不同，不能据图 12 断定温度比所有湿度误差都重要。已有时间加权均值对照使问题三根变化约 20.04 s，问题四改用 PCHIP 半径插值使根提前约 12.61 s，二者也超过亚秒级一维经验数值预算。工程执行余量应建立在可用工况范围上，不能只依据四位小时的取整步长。

十一、模型评价、适用范围与推广

11.1 模型优点

本文的优势体现在模型、数值实现与结果证据之间的对应关系，适用范围限于已声明的有效方程与工况。

1. **四问采用共同收支框架。**第五节统一热量、水分与材料运动的关系，各问再代入物性、几何和停止条件，便于区分变物性、终点判定与观测收缩的作用。
2. **圆环离散可核查总量收支。**第一问相邻控制体共享面通量，内部交换在求和时抵消，式(14)将加权平均变化与表面交换对应，可用于检查离散总量一致性。
3. **原函数改写保持扩散通量。**式(7)把可分离扩散率写为温度因子与原函数梯度的乘积，允许插值较平缓的原函数后反求含水率，保留题给非线性关系。
4. **终点判断兼顾内部极值与时间裕量。**式(26)–(27)检查径向重构的全部实驻点及端点，式(8)区分等号临界根与严格执行时刻，避免用已舍入表值代替阈值判断。
5. **材料坐标避免重复计入收缩。**第九节由组分守恒推导固定参考域方程，干基含水率无额外浓缩项；无交换纯收缩检验保持初态温湿与干、水质量，与所选材料运动一致。
6. **验证与输出来源分层可追溯。**第六、七节以独立算法和加密结果交叉核验，规定表值由对应未舍入数组生成；二维节点、配对估计与连续严格界分开报告，便于辨认结论由哪一级证据支持。

11.2 模型推广

对具有同类有效物性与交换系数的圆柱形农产品，可将物性公式、环境记录和目标含水率作为替换接口，沿用控制体收支、离散组装与终点检测流程；迁移前须核对参数的单位、质量基准及适用温湿区间。本文未用其他物料作验证。

几何推广可从第10.1节的一维径向与轴对称二维配对出发，在共同收支方程中替换散度算子、控制体权重及边界面。其他几何仍须重新处理对称条件、表面交换和网格检验，不能沿用本题的降维精度。对于以空间最大值限制工艺终点的其他状态量，若有可求极值的空间重构和明确的时间穿越方向，可移植“空间极值搜索、等号根定位、执行裕量复核”的事件流程，并按该变量重新设置阈值与误差预算。

对于已知外半径历程的物料，可用材料坐标组织输入，并按干物质守恒建立随动权重；内部运动仍需独立依据，仅有外半径不能唯一确定内部变形。取得后段

工况记录后，第10.2节的单因素情景框架可用于比较允许工况范围内的终点变化，为工艺余量选择提供条件依据；本题人工扰动幅度不直接移作其他物料的余量。

11.3 适用范围与主要限制

题表距离按中截面径向解释并作二维校核，并非题面指定的位置。

气固边界与目标可达性。材料 C 与空气 y_a 即使同写 kg/kg，分母也未必相同；湿度比、相对湿度与蒸气浓度须区分 [5]。真实气膜候选边界为

$$j_s = k_g \{c_{v,s}(C_s, T_s) - c_{v,a}\}, \quad (43)$$

其中 $c_{v,s}$ 需材料水活度或解吸关系。仅当 $c_{v,s} - c_{v,a} \approx K(C_s - b)$ 时，才能建立 $\rho_d h_m = k_g K$ ，不能直接交换系数数值。相关模型采用平衡干基含水率或气相/解吸关系 [1, 2]，未提供本药材标定值。须先核实目标环境下 $C_{eq} < 0.15$ ；若 $C_{eq} \geq 0.15$ ，从较高含水率开始的扩散干燥可能无法实现全域严格阈值，加密网格不能消除不可达性，未经标定的边界情景也不能称为实测。

经验密度与材料质量的相容性。若将题给 $\rho = a + b_\rho C$ 同时解释为真实当前湿体密度，干基定义要求

$$\rho_d(C) = \frac{a + b_\rho C}{1 + C}, \quad \frac{d\rho_d}{dC} = \frac{b_\rho - a}{(1 + C)^2}. \quad (44)$$

这与前三问固定骨架的恒定干密度、第四问仿射收缩的空间均匀干密度不相容。本文仅将题给 ρ 用于有效容量，是建模处置而非题面规定。按第四问保存场强制采用真实湿密度解释，7h 和末态隐含干质量相对初值为 0.71952、0.89029；保持 C, R 场不变，凑足总干质量需使长度由 25 cm 增至 34.7454 cm 与 28.0807 cm。轴向收缩不能修复缺口，伸长凑平总量也未解决局部相容性；该反例不等于独立干质量账本数值泄漏，也不是可行伸长模型。

相变能量与可报告的指标。式(2)未显式计入相变能量，有限的 $\rho_{eff} c_p$ 不能在近等温持续失水时自动补充汽化功率。在“初始湿密度确定干质量、全部等效失水均蒸发”的附加条件下，取诊断常数 $\lambda = 2.45 \text{ MJ/kg}$ [6]，问题一 1800 s 的潜热需求约 45.59 kJ，原轨迹对流输入约 4.801 kJ，比例约 9.50。问题四 12 h 表面近于环境温度、原轨迹净对流输入接近零，条件潜热通量 $\lambda \rho_d h_m (C_s - b)$ 仍约 164.10 W/m^2 。这些量级表明相变能量缺口尚未闭合，并非实际热量测量；重新联立相变会改变表面温度、热输入和扩散，不能固定原热输入倍乘干燥时间。现稿可报告有效模型的温湿分布、阈值事件和条件时长差，不能据此推断真实热量收支或执行保证。

数值证据与工况的适用层级。第三问 57.4741 h 的工程保守方向由二维终点节点场和配对根差支持，局部续算继承的前史误差仍需区分，不能将该时长解释为精确的二维最短时间；第四问 51.0921 h 为所设条件下的一维径向模型执行时长，

已有粗网格二维核查在该时刻的最大含水率略高于阈值，有限圆柱全域达标尚待进一步验证，相关记录见支撑材料。两间均未取得连续二维严格误差界，也不能把条件时长作为实际工况保证。后段人为情景的幅度与用途见第10.2节，数值余量不覆盖环境、气固映射与物性假设的不确定性。半径重建方案间的差异属于输入处理敏感性，不给出测量置信区间；复现规定的观测半径也不验证温湿预测，改变工况而沿用该历程时仅能报告条件响应。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于求解代码编写、文献检索和语言润色，详细使用情况见支撑材料。

核心模型建立与论文初稿由参赛队完成。AI 给出的求解代码和文献检索结果均经队员检查、修改和核验后采用；语言润色不改变模型、公式与数值结论。

参考文献

- [1] da Silva W P, e Silva C M D P S, Gama F J A. Estimation of thermo-physical properties of products with cylindrical shape during drying: The coupling between mass and heat[J]. Journal of Food Engineering, 2014, 141: 65–73. DOI:10.1016/j.jfoodeng.2014.05.010.
- [2] Adrover A, Venditti C, Brasiello A. A Non-Isothermal Moving-Boundary Model for Continuous and Intermittent Drying of Pears[J]. Foods, 2020, 9(11): 1577. DOI:10.3390/foods9111577.
- [3] 吴孟秋, 雷登文, 朱广飞, 谢永康, 李星仪, 刘嫣红. 基于动网格的白萝卜热风干燥热质传递研究 [J]. 农业机械学报, 2022, 53(S2): 293–302. DOI:10.6041/j.issn.1000-1298.2022.S2.034.
- [4] SciPy developers. scipy.integrate.solve_ivp: SciPy v1.18.0 Manual[EB/OL]. [2026-09-12]. https://docs.scipy.org/doc/scipy-1.18.0/reference/generated/scipy.integrate.solve_ivp.html.
- [5] ASHRAE. 2021 ASHRAE Handbook—Fundamentals (SI Edition): Chapter 1, Psychrometrics[M/OL]. 2021[2026-09-12]. https://handbook.ashrae.org/Handbooks/F21/SI/F21_Ch01/F21_Ch01_si.aspx.
- [6] Allen R G, Pereira L S, Raes D, Smith M. Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements[M/OL]. FAO Irrigation and Drainage Paper 56. Rome: FAO, 1998. Chapter 3, Meteorological Data[2026-09-12]. <https://www.fao.org/4/X0490E/x0490e07.htm>.

A 支撑材料与完整采用代码

支撑包包含完整求解源码、题面附件、提交结果与数值复现入口。下列原求解代码保留；新增的完整复现控制、常规 Excel 导出、逐格核对及数值图生成代码一并列于本附录。代码附录不计入正文页数。

A.1 完整复现入口与验收范围

在新的 Python 环境中安装支撑包的 `requirements.txt`，运行 `python -X utf8 reproduce.py --out ../reproduced --jobs 2`。程序仅以包内源码和题目附件重新计算四问主解、数值与二维对照、交换系数及后段环境情景，再核对论文表格、终点和提交结果。包内已有结果仅供比较，不作为求解初态。

输出目录包含新算的未舍入场、实际命令及退出码、独立工作簿读回、比较记录 and 数值核查图。第二问新生成的 `result2.xlsx` 包含完整全过程；压缩提交包中的同名小工作簿保存前三小时，全过程另以 NPZ 提供。新导出入口使用 Python 标准库与 `openpyxl`，无需专有工作簿运行时。旧版导出代码保留以供追溯。

六张题设表按论文显示精度逐格比较；临界根和未舍入量使用记录在核验程序中的数值容差。复现结果不改变模型的条件性，也不将第四问一维执行值提升为有限圆柱连续全域的严格保证。运行方法及结果见支撑包的复现说明与验收记录。

A.2 原求解与诊断源码

A.3 匿名快照启动与冒烟检查

`support/launch.py`

```
1  #!/usr/bin/env python3
2  """Anonymous source snapshot: safe smoke by default, explicit numerical runs.
3
4  Run from the paper directory: python support/launch.py smoke
5  Use `list` to see numerical tasks. A `run` requires a new output directory.
6  This launcher never treats syntax checks as numerical validation.
7  """
8  from __future__ import annotations
9
10 import argparse
11 import ast
12 import hashlib
13 import importlib.metadata
14 import json
15 import os
16 from pathlib import Path
17 import platform
18 import subprocess
19 import sys
20
21 HERE = Path(__file__).resolve().parent
22 PROJECT = HERE / "project"
23
24 TASKS = {
25     "q1-main": "Q1 10240-cell main solution; includes independent flux quadrature",
26     "q1-full": "Q1 complete solver/validation/refinement/workbook/report chain",
27     "q2-sensitivity": "Seven 320-cell FV runs: baseline, hm/hT +/-20%, PCHIP, frozen
    ↪ properties",
```

```

28     "q23-main": "Current Q2/Q3 unified trajectory, 80-node Radau, one-second records",
29     "q3-main": "Current Q3 80-node trajectory, threshold budget and strict execution",
30     "q4-main": "Current Q4 120-node shrinking-radius main solution and balance audit",
31 }
32
33 SAFE_HELP = [
34     "q1_complete_delivery/q1_delivery/run_all.py",
35     *["q1_complete_delivery/q1_delivery/source/" + p for p in
36       ("q1_solver.py", "q1_validate.py", "q1_refine.py", "q1_excel.py", "q1_report.py")],
37     "q2_refinement_delivery/runtime/source/q2_core.py",
38     "q2_refinement_delivery/runtime/source/q2_spectral.py",
39     "q2_refinement_delivery/source/prepare_reused.py",
40     "q2_refinement_delivery/source/q2_mechanism.py",
41     "q4_complete_delivery/v1/q123_closeout/source/solve_unified_q23.py",
42     "q4_complete_delivery/v1/q123_closeout/source/audit_unified_q23.py",
43     *["q3_refinement_delivery/source/" + p for p in
44       ("q3_refined_reference.py", "future_scenarios_from_checkpoint.py",
45        "export_workbook.py", "validate_xlsx.py", "verify_frozen.py")],
46     *["q4_complete_delivery/v1/source/" + p for p in
47       ("run_q4.py", "geometry_solver.py", "geometry_solver_executed.py",
48        "geometry_checks.py", "check_numerics.py", "finalize_q4.py",
49        ↪ "package_workbooks.py")],
50     "q1234_overall_review_delivery/v1/source/environment_probe.py",
51 ]
52
53 def child_environment():
54     env = os.environ.copy()
55     env.update(PYTHONDONTWRITEBYTECODE="1", PYTHONUTF8="1",
56               OPENBLAS_NUM_THREADS="1", OMP_NUM_THREADS="1", MKL_NUM_THREADS="1")
57     return env
58
59
60 def sha(path):
61     return hashlib.sha256(path.read_bytes()).hexdigest()
62
63
64 def smoke(log=None, node=None):
65     """Parse every Python file and import only through reviewed --help paths."""
66     manifest = json.loads((HERE / "snapshot_manifest.json").read_text(encoding="utf-8"))
67     checks = []
68     for entry in manifest["files"]:
69         p = PROJECT / entry["path"]
70         assert p.is_file() and sha(p) == entry["snapshot_sha256"], entry["path"]
71         if p.suffix == ".py":
72             ast.parse(p.read_text(encoding="utf-8-sig"), filename=entry["path"])
73     ast.parse(Path(__file__).read_text(encoding="utf-8"), filename="support/launch.py")
74     for name in SAFE_HELP:
75         p = subprocess.run([sys.executable, str(PROJECT / name), "--help"],
76                           cwd=PROJECT, env=child_environment(), capture_output=True,
77                           text=True, encoding="utf-8", errors="replace", timeout=45)
78         # Store relative identity and status only; do not leak runtime home paths.
79         checks.append({"path": name, "check": "--help", "returncode": p.returncode})
80         if p.returncode:
81             raise RuntimeError(f"Safe entry check failed: {name}; return code
82                               ↪ {p.returncode}")
83     if node:
84         name = "q4_complete_delivery/v1/source/export_workbooks.mjs"
85         p = subprocess.run([node, "--check", str(PROJECT / name)], cwd=PROJECT,
86                           capture_output=True, text=True, timeout=30)
87         checks.append({"path": name, "check": "node --check", "returncode": p.returncode})
88         if p.returncode:
89             raise RuntimeError("JavaScript syntax check failed")
90     versions = {"python": platform.python_version()}

```

```

90     for package in ("numpy", "scipy", "matplotlib", "openpyxl"):
91         try:
92             versions[package] = importlib.metadata.version(package)
93         except importlib.metadata.PackageNotFoundError:
94             versions[package] = "not installed"
95     result = {"status": "PASS", "scope": "hash/AST/safe --help only; no PDE or workbook
↪ export",
96              "snapshot_files_checked": len(manifest["files"]),
97              "python_files_parsed": 1 + sum(e["path"].endswith(".py") for e in
↪ manifest["files"]),
98              "versions": versions, "entry_checks": checks}
99     if log:
100         log = Path(log)
101         log.parent.mkdir(parents=True, exist_ok=True)
102         log.write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
103     print(json.dumps(result, ensure_ascii=True, indent=2))
104     return result
105
106
107 def command(task, out):
108     q1 = PROJECT / "q1_complete_delivery/q1_delivery"
109     q23 = PROJECT / "q4_complete_delivery/v1/q123_closeout"
110     q3 = PROJECT / "q3_refinement_delivery"
111     q4 = PROJECT / "q4_complete_delivery/v1"
112     if task == "q1-full":
113         return [str(q1 / "run_all.py"), "--out", str(out)]
114     if task == "q1-main":
115         return [str(q1 / "source/q1_solver.py"), "--input", str(q1 / "input/附件 1.xlsx"),
116                 "--out", str(out), "--quadrature"]
117     if task == "q23-main":
118         return [str(q23 / "source/solve_unified_q23.py"), "--input", str(q23 /
↪ "inputs/attachment1.xlsx"),
119                 "--out", str(out / "q23_unified"), "--n", "80", "--method", "Radau",
120                 "--rtol", "2e-13", "--max-step", "30", "--step", "1", "--end", "206906.76"]
121     if task == "q3-main":
122         return [str(q3 / "source/q3_refined_reference.py"), "--input", str(q3 /
↪ "inputs/attachment1.xlsx"),
123                 "--out", str(out / "solution"), "--n", "80", "--scenario", "mean",
124                 "--method", "Radau", "--rtol", "2e-11", "--atol-c", "2e-13",
125                 "--max-step", "120", "--budget-s", "0.03", "--quantum-s", "0.36"]
126     if task == "q4-main":
127         return [str(q4 / "source/run_q4.py"), "--out", str(out / "main"), "--n", "120",
128                 "--kind", "linear", "--method", "Radau", "--rtol", "2e-11", "--max-step",
↪ "90", "--audit"]
129     raise ValueError(task)
130
131
132 def run(task, out):
133     out = Path(out).resolve()
134     if out.exists():
135         raise FileExistsError("Choose a new output directory; existing results are never
↪ overwritten.")
136     if out == PROJECT or PROJECT in out.parents:
137         raise ValueError("Numerical output must be outside the immutable project
↪ snapshot.")
138     out.mkdir(parents=True)
139     print(TASKS[task], flush=True)
140     if task == "q2-sensitivity":
141         for key, value in child_environment().items():
142             os.environ[key] = value
143         sys.path.insert(0, str(PROJECT / "q2_refinement_delivery/runtime/source"))
144         from q2_core import Environment, load_environment
145         from q2_tests import run_sensitivity

```

```

146     data, _ = load_environment(PROJECT /
147         ↪ "q2_refinement_delivery/runtime/inputs/附件1.xlsx")
148     run_sensitivity(Environment(data), out)
149     return
150     subprocess.run([sys.executable, *command(task, out)], cwd=PROJECT,
151         env=child_environment(), check=True)
152
153 def main():
154     parser = argparse.ArgumentParser(description=__doc__)
155     sub = parser.add_subparsers(dest="action")
156     s = sub.add_parser("smoke", help="safe default: no numerical solving")
157     s.add_argument("--log", type=Path)
158     s.add_argument("--node", help="optional Node executable for JavaScript syntax only")
159     sub.add_parser("list", help="list explicit, potentially expensive numerical tasks")
160     r = sub.add_parser("run", help="explicit numerical integration, never the default")
161     r.add_argument("task", choices=TASKS)
162     r.add_argument("--out", type=Path, required=True)
163     args = parser.parse_args()
164     if args.action == "run":
165         run(args.task, args.out)
166     elif args.action == "list":
167         for name, description in TASKS.items():
168             print(f"{name}: {description}")
169     else:
170         smoke(getattr(args, "log", None), getattr(args, "node", None))
171
172 if __name__ == "__main__":
173     main()

```

A.4 第一问求解及交付

A.5 run_all

support/project/q1_complete_delivery/q1_delivery/run_all.py

```

1  #!/usr/bin/env python3
2  """Reproduce Q1 from the original input in a separate output directory.
3  Usage: python run_all.py --out reproduced
4  The complete default run includes all finite-volume, independent-reference,
5  conservation, sensitivity and workbook audits, and writes figures/paper.
6  """
7  from __future__ import annotations
8  import argparse, hashlib, json, os, subprocess, sys, time
9  from pathlib import Path
10 # Set before NumPy/SciPy imports (also inherited by all child processes).
11 for key in ('OPENBLAS_NUM_THREADS', 'OMP_NUM_THREADS', 'MKL_NUM_THREADS'):
12     os.environ[key]='1'
13
14 def sha(p): return hashlib.sha256(Path(p).read_bytes()).hexdigest()
15
16 def main():
17     b=Path(__file__).resolve().parent; ap=argparse.ArgumentParser(description=__doc__)
18     ap.add_argument('--input', type=Path, default=b/'input'/附件 1.xlsx')
19     ap.add_argument('--template', type=Path, default=b/'input'/result1_template.xlsx')
20     ap.add_argument('--out', type=Path, default=b/'reproduced')
21     args=ap.parse_args(); args.out=args.out.resolve(); args.out.mkdir(parents=True, exist_ok=True)
22     ↪ if args.out.resolve()==(b/'input').resolve(): raise ValueError('Output must not be the
23     ↪ original-input folder')

```

```

23 originals={str(p.resolve()):sha(p) for p in (args.input,args.template)}
24 start=time.perf_counter();steps=[]
25 for module,params in [
26     ('q1_solver.py',[['--input',str(args.input),'--out',str(args.out),'--quadrature']],
27     ('q1_validate.py',[['--input',str(args.input),'--out',str(args.out/'validation')]]),
28     ('q1_refine.py',[['--input',str(args.input),'--out',str(args.out/'validation')]]),
29     ('q1_excel.py',[['--template',str(args.template),'--raw',str(args.out/'q1_unrounded.
    ↳ npz'),'--out',str(args.out/'result1.xlsx')]]),
30     ('q1_report.py',[['--input',str(args.input),'--out',str(args.out)]]):
31     cmd=[sys.executable,str(b/'source'/module)]+params
32     print('RUN',module,flush=True);tic=time.perf_counter()
33     with (args.out/(module.replace('.py','')+'.log')).open('w',encoding='utf-8') as
    ↳ log:
34         p=subprocess.run(cmd,stdout=log,stderr=subprocess.STDOUT,env=os.environ.copy())
35         steps.append({'module':module,'returncode':p.returncode,'elapsed_seconds':time.pe
    ↳ rf_counter()-tic})
36         if p.returncode:raise RuntimeError(f'{module} failed; see
    ↳ {args.out/module.replace(".py",".log")}')
37 for path,h in originals.items():assert sha(path)==h,'Original input changed'
38 # Core data reproducibility compared with delivered numerical outputs if available.
39 import numpy as np
40 report={'all_steps_successful':True,'steps':steps,'elapsed_seconds':time.perf_counter
    ↳ (-start,
41         'original_input_hashes_unchanged':True,'input_sha256':originals}
42 delivered=b/'output'/q1_unrounded.npz'
43 if delivered.exists() and delivered.resolve()!=(args.out/'q1_unrounded.npz').resolve():
44     a=np.load(delivered);c=np.load(args.out/'q1_unrounded.npz')
45     report['compared_with_delivered']={k:{'max_abs_difference':float(np.max(np.abs(a[
    ↳ k]-c[k]))),
46         'four_decimal_equal':bool(np.array_equal(np.round(a[k],4),np.round(c[k],4)))}
47     for k in ('temperature_degC','moisture_dry_basis')}
48 (args.out/'reproduction_audit.json').write_text(json.dumps(report,ensure_ascii=False,
    ↳ indent=2),encoding='utf-8')
49 print(json.dumps(report,ensure_ascii=False,indent=2),flush=True)
50 if __name__=='__main__':main()

```

A.6 q1_solver

support/project/q1_complete_delivery/q1_delivery/source/q1_solver.py

```

1  #!/usr/bin/env python3
2  """Q1 cylindrical effective transport model; no hidden notebook state.
3
4  Units: SI internally, temperature in degC (only temperature differences).
5  Run from the package root:
6      python source/q1_solver.py --input input/附件 1.xlsx --out output --n 10240
7  The validation driver calls the same spatial operator, but independent Bessel
8  and Chebyshev references are implemented in q1_validate.py and q1_refine.py.
9  """
10 from __future__ import annotations
11 import argparse, csv, hashlib, json, platform, time, zipfile
12 from dataclasses import dataclass, asdict
13 from pathlib import Path
14 import xml.etree.ElementTree as ET
15 import numpy as np
16 import scipy
17 from scipy.integrate import solve_ivp
18 from scipy.interpolate import PchipInterpolator
19 from scipy.sparse import diags, csr_matrix
20
21 @dataclass(frozen=True)
22 class Parameters:

```

```

23     R: float = 0.02
24     L: float = 0.25
25     rho: float = 820.0
26     cp: float = 2600.0
27     k: float = 0.36
28     hT: float = 25.0
29     hm: float = 8e-7
30     D0: float = 7e-9
31     a: float = 0.89
32     T0: float = 28.0
33     C0: float = 2.55
34     @property
35     def alpha(self): return self.k/(self.rho*self.cp)
36 P=Parameters()
37 NS={'s':'http://schemas.openxmlformats.org/spreadsheetml/2006/main'}
38
39 def read_xlsx(path: str|Path):
40     """Read numeric input via standard-library OOXML, without changing the file.
41     Supports shared strings / inline strings / numeric cells; rejects formulas
42     without cached numeric values and requires the expected three input columns.
43     This independent reader is cross-checked against artifact_tool in this run.
44     """
45     path=Path(path)
46     if not path.is_file(): raise FileNotFoundError(path)
47     with zipfile.ZipFile(path) as z:
48         bad=z.testzip()
49         if bad: raise ValueError(f'ZIP CRC failed: {bad}')
50         ss=[]
51         if 'xl/sharedStrings.xml' in z.namelist():
52             for si in ET.fromstring(z.read('xl/sharedStrings.xml')).findall('s:si',NS):
53                 ss.append(''.join(t.text or '' for t in si.iter('{'+NS['s']+'}t'))))
54         root=ET.fromstring(z.read('xl/workbook.xml'))
55         sheets=root.find('s:sheets',NS)
56         names=[s.attrib['name'] for s in sheets]
57         rid=sheets[0].attrib['{http://schemas.openxmlformats.org/officeDocument/2006/relationships}id']
58         rels=ET.fromstring(z.read('xl/_rels/workbook.xml.rels'))
59         target=next(e.attrib['Target'] for e in rels if e.attrib['Id']==rid)
60         target=target.lstrip('/') if target.startswith('/') else 'xl/'+target
61         rows=[]
62         for row in ET.fromstring(z.read(target)).findall('s:sheetData/s:row',NS):
63             vals={}
64             for c in row.findall('s:c',NS):
65                 col=''.join(ch for ch in c.attrib['r'] if ch.isalpha())
66                 v=c.find('s:v',NS); typ=c.attrib.get('t')
67                 if typ=='s': val=ss[int(v.text)] if v is not None else None
68                 elif typ=='inlineStr': val=''.join(e.text or '' for e in
69                 ↪ c.findall('.//s:t',NS))
69                 elif v is None: val=None
70                 else:
71                     try: val=float(v.text)
72                     except (ValueError,TypeError): val=v.text
73             vals[col]=val
74             if vals: rows.append([vals.get(c) for c in ['A','B','C']])
75     if rows[0]!='时间','温度','水分浓度':
76         raise ValueError(f'Unexpected headings: {rows[0]}')
77     data=np.asarray(rows[1:],dtype=float)
78     if data.ndim!=2 or data.shape[1]!=3 or not np.isfinite(data).all():
79         raise ValueError('Missing/nonfinite/non-numeric environment data')
80     dt=np.diff(data[:,0])
81     if not np.all(dt>0): raise ValueError('Input times must be strictly increasing;
82     ↪ duplicate or reversed rows')
83     if data[0,0]>0 or data[-1,0]<1800: raise ValueError('Input does not cover [0,1800] s')
84     if np.any(data[:,2]<0): raise ValueError('Negative environmental concentration')

```

```

84 raw=path.read_bytes()
85 audit={'input_name':path.name,'sha256':hashlib.sha256(raw).hexdigest(),
86        'git_blob_sha1':hashlib.sha1(b'blob
    ↪ '+str(len(raw)).encode()+b'\0'+raw).hexdigest(),
87        'sheet_names':names,'headers':rows[0],'data_rows':len(data),
88        'time_min_s':float(data[0,0]),'time_max_s':float(data[-1,0]),
89        'time_steps_s':np.unique(dt).tolist(),'missing':0,'duplicate_times':0,
90        'temperature_minmax_degC':[float(data[:,1].min()),float(data[:,1].max())],
91        'environment_C_minmax':[float(data[:,2].min()),float(data[:,2].max())],
92        'temperature_down_steps':int(np.sum(np.diff(data[:,1])<0)),
93        'environment_C_down_steps':int(np.sum(np.diff(data[:,2])<0)),
94        'interpretation':'External forcing only, not internal response observations',
95        'units_source':'Problem appendix 1; xlsx headings contain no unit strings'}
96 return data,audit
97
98 class Environment:
99     def __init__(self,data,kind='linear'):
100         self.data=np.asarray(data,dtype=float); self.kind=kind
101         self.knots=self.data[:,0]
102         if kind not in ('linear','pchip'): raise ValueError(kind)
103         self.pchip=PchipInterpolator(self.knots,self.data[:,1:],axis=0,extrapolate=False)
104         ↪ if kind=='pchip' else None
105     def __call__(self,t):
106         t=np.asarray(t)
107         if np.any(t<self.knots[0]-1e-10) or np.any(t>self.knots[-1]+1e-10):
108             raise ValueError('Environmental extrapolation is prohibited in Q1')
109         if self.kind=='pchip': return self.pchip(t)
110         return np.stack([np.interp(t,self.knots,self.data[:,i]) for i in (1,2)],axis=-1)
111
112 class ConstantEnvironment:
113     def __init__(self,T=28.,C=2.55,end=1800.):
114         self.T=T; self.C=C; self.knots=np.array([0.,end]); self.kind='constant'
115     def __call__(self,t):
116         t=np.asarray(t)
117         return np.stack([np.full_like(t,self.T,dtype=float),np.full_like(t,self.C,dtype=f
118             ↪ loat)],axis=-1)
119
120 class RadialFV:
121     """Vertex-centred finite volumes with nodes at r=0 and r=R.
122     w=integral r dr over each dual cell; actual volume=2*pi*L*w.
123     Boundary nodes have their proper half-cell storage, not ghost/centre output.
124     The final augmented state integrates outward flux divided by total volume.
125     """
126     def __init__(self,n,field,p=P,transfer_scale=1.,constant_D=None):
127         if n<4: raise ValueError('Need n>=4 radial intervals')
128         self.p=p; self.n=n; self.field=field; self.constant_D=constant_D
129         self.r=np.linspace(0,p.R,n+1); self.dr=p.R/n
130         self.faces=np.r_[0.,(self.r[:-1]+self.r[1:])/2,p.R]
131         self.w=np.diff(self.faces**2)/2; self.vol=self.w.sum()
132         self.s=self.faces[1:-1]/self.dr
133         if field=='T': self.beta=p.hT/(p.rho*p.cp)*transfer_scale; self.col=0
134         elif field=='C': self.beta=p.hm*transfer_scale; self.col=1
135         else: raise ValueError(field)
136     def diff(self,u):
137         if self.field=='T': return np.full_like(u,self.p.alpha),np.zeros_like(u)
138         if self.constant_D is not None: return
139         ↪ np.full_like(u,self.constant_D),np.zeros_like(u)
140         # No clipping of the solution: a nonpositive nonlinear iterate is a failure.
141         if not np.all(np.isfinite(u)) or np.any(u<=0):
142             raise FloatingPointError('Nonpositive/nonfinite C in nonlinear solver')
143         D=self.p.D0*np.exp(-self.p.a/u)
144         return D,D*self.p.a/u**2
145     def rhs(self,t,y,env):
146         u=y[:-1]; D,_=self.diff(u)

```

```

144     df=2*D[:-1]*D[1:]/(D[:-1]+D[1:])
145     g=self.s*df*(u[:-1]-u[1:]) # positive outward
146     f=np.zeros_like(y)
147     f[:-2]=g/self.w[:-1];f[1:-1]=g/self.w[1:]
148     go=self.p.R*self.beta*(u[-1]-env(t)[self.col])
149     f[-2]=go/self.w[-1];f[-1]=go/self.vol
150     return f
151 def jac(self,t,y,env):
152     u=y[:-1];D,Dp=self.diff(u);sumD=D[:-1]+D[1:]
153     df=2*D[:-1]*D[1:]/sumD
154     dleft=2*(D[1:]/sumD)**2*Dp[:-1]
155     dright=2*(D[:-1]/sumD)**2*Dp[1:]
156     jump=u[:-1]-u[1:]
157     a=self.s*(df+jump*dleft);b=self.s*(-df+jump*dright)
158     m=len(u);diag=np.zeros(m+1)
159     diag[m-1]=a/self.w[:-1];diag[1:m]=b/self.w[1:]
160     outder=self.p.R*self.beta
161     diag[m-1]=outder/self.w[-1]
162     lower=np.r_[a/self.w[1:],outder/self.vol]
163     upper=np.r_[-b/self.w[:-1],0.]
164     return diag([lower,diag,upper],[-1,0,1],format='csc')
165
166 def solve_fv(env,field,n=640,p=P,rtol=2e-10,atol=2e-12,max_step=15.,
167             end=1800.,times=None,initial=None,transfer_scale=1.,constant_D=None,
168             quadrature=False,verbose=False):
169     op=RadialFV(n,field,p,transfer_scale,constant_D)
170     times=np.arange(0.,end+0.5,1.) if times is None else np.asarray(times,dtype=float)
171     if times[0]!=0 or times[-1]!=end: raise ValueError('Requested times must include 0 and
172     ↪ end')
173     start=p.T0 if field=='T' else p.C0
174     if initial is None: u0=np.full(n+1,start)
175     elif callable(initial): u0=np.asarray(initial(op.r),dtype=float)
176     else: u0=np.broadcast_to(initial,(n+1,)).astype(float).copy()
177     y=np.r_[u0,0.]
178     rout=np.linspace(0,p.R,21)
179     # All production grids have intervals divisible by 20, so no spatial interpolation.
180     if n%20: raise ValueError('n must be divisible by 20 for exact output-node selection')
181     oi=np.arange(21)*(n//20)
182     out=np.empty((len(times),21));out[0]=u0[oi]
183     avg=np.empty(len(times));avg[0]=op.w@u0/op.vol
184     acc=np.zeros(len(times));int_error=[]
185     snapshots={0.:u0.copy()}; selected={100.,300.,600.,900.,1200.,1500.,1800.}
186     cuts=np.unique(np.r_[0.,env.knots[(env.knots>0)&(env.knots<end)],end])
187     stats={'field':field,'n_intervals':n,'n_nodes':n+1,'dr_m':op.dr,'rtol':rtol,'atol':atol,
188     ↪ 'ol',
189     'max_step_s':max_step,'method':'scipy.solve_ivp BDF, analytic sparse Jacobian',
190     'segments':len(cuts)-1,'nfev':0,'njev':0,'nlu':0,'accepted_steps':0,
191     'nonlinear_failure_policy':'raise; never silently accept or clip',
192     'nonlinear_iteration':'SciPy BDF Newton iteration; maximum 4 iterations;
193     ↪ unsuccessful convergence reduces step size',
194     'first_step_policy':'C only: min(segment length, 1e-3 s, 0.05*dr^2/D0); avoids
195     ↪ negative initial-step probe',
196     'newton_tolerance':max(10*np.finfo(float).eps/rtol,min(0.03,np.sqrt(rtol))),
197     'environment_interpolation':env.kind}
198     tic=time.perf_counter(); cumulative_quad=0.
199     gx,gw=np.polynomial.legendre.leggauss(4)
200     for left,right in zip(cuts[:-1],cuts[1:]):
201         sol=solve_ivp(lambda t,y:op.rhs(t,y,env),(left,right),y,
202                     method='BDF',jac=lambda t,y:op.jac(t,y,env),rtol=rtol,atol=atol,
203                     max_step=max_step,dense_output=True,
204                     first_step=min(right-left,1e-3,0.05*op.dr**2/p.D0) if field=='C'
205                     ↪ else None)
206         if not sol.success: raise RuntimeError(f'{field}, n={n}, [{left},{right}]:
207         ↪ {sol.message}')

```

```

202     for key in ('nfew', 'njev', 'nlu'): stats[key] += int(getattr(sol, key))
203     stats['accepted_steps'] += len(sol.t) - 1
204     mask = (times > left) & (times <= right + 1e-9)
205     ts = times[mask]; ys = sol.sol(ts)
206     out[mask] = ys[oi].T
207     avg[mask] = (op.w@ys[:, -1]) / op.vol; acc[mask] = ys[-1]
208     for t in selected:
209         if left < t <= right: snapshots[t] = sol.sol(t)[-1].copy()
210     if quadrature:
211         for k in range(0, len(sol.t) - 1, 32):
212             ta = sol.t[k: min(k + 32, len(sol.t) - 1)]; tb = sol.t[k + 1: min(k + 33, len(sol.t))]
213             tq = ((ta + tb)[:, None] / 2 + (tb - ta)[:, None] / 2 * gx).ravel()
214             vals = sol.sol(tq)[-2]
215             flow = op.p.R * op.beta * (vals - env(tq)[:, op.col]) / op.vol
216             cumulative_quad += float(np.sum(flow.reshape(-1, 4) * gw * (tb - ta)[:, None] / 2))
217             int_error.append([right, float(op.w@sol.y[:, -1, -1] / op.vol - avg[0] + cumulative_quad),
218                             ↪ d)])
219             y = sol.y[:, -1]
220             if verbose and int(right) % 300 == 0: print(f'{field} n={n} t={right:.0f}', flush=True)
221     stats['elapsed_seconds'] = time.perf_counter() - tic
222     stats['conservation_augmented_max_abs'] = float(np.max(np.abs(avg - avg[0] + acc)))
223     stats['independent_flux_quadrature_max_abs'] = float(np.max(np.abs(np.array(int_error)[
224     ↪ :, 1]))) if int_error else None
225     stats['min_value'] = float(out.min()); stats['max_value'] = float(out.max())
226     stats['initial_max_error'] = float(np.max(np.abs(out[0] - u0[oi])))
227     return {'time_s': times, 'radius_m': rout, 'values': out, 'average': avg, 'outward_flux_integ
228     ↪ ral_average_units': acc,
229             'internal_radius_m': op.r, 'internal_weights_rdr': op.w,
230             'snapshot_times_s': np.array(sorted(snapshots)),
231             'internal_snapshots': np.array([snapshots[t] for t in sorted(snapshots)]),
232             'quadrature_errors': np.array(int_error), 'stats': stats}
233
234 def save_field(path, result):
235     np.savez_compressed(path, **{k: v for k, v in result.items() if k != 'stats'})
236     Path(str(path) + '.json').write_text(json.dumps(result['stats'], indent=2, ensure_ascii=F
237     ↪ alse), encoding='utf-8')
238
239 def write_csvs(out, T, C):
240     out = Path(out); out.mkdir(parents=True, exist_ok=True)
241     for name, res in [('temperature', T), ('moisture', C)]:
242         np.savetxt(out / f'{name}_unrounded.csv', np.column_stack([res['time_s'], res['values
243         ↪ ']]),
244                 delimiter=',', fmt='%17g', header='time_s, '+', '.join(f'r_{r*100:.1f}_cm'
245         ↪ for r in res['radius_m'])), comments='')
246         ii = [np.flatnonzero(res['time_s'] == t)[0] for t in (100, 300, 600, 900, 1200, 1500, 1800)]
247         np.savetxt(out / f'table_{name}.csv', np.column_stack([res['time_s'][ii], res['values
248         ↪ '][ii][:, ::5]]),
249                 delimiter=',', fmt=[ '%.0f' ] + [ '%.4f' ] * 5, header='time_s, 0_cm, 0.5_cm, 1_cm,
250         ↪ 1.5_cm, 2_cm', comments='')
251
252 def main():
253     ap = argparse.ArgumentParser(description=__doc__)
254     ap.add_argument('--input', type=Path, default=Path(__file__).resolve().parents[1] / 'inpu
255     ↪ t' / '附件1.xlsx')
256     ap.add_argument('--out', type=Path, default=Path(__file__).resolve().parents[1] / 'output')
257     ap.add_argument('--n', type=int, default=10240)
258     ap.add_argument('--rtol', type=float, default=2e-12); ap.add_argument('--atol', type=floa
259     ↪ t, default=2e-14)
260     ap.add_argument('--max-step', type=float, default=3.)
261     ap.add_argument('--interp', choices=['linear', 'pchip'], default='linear')
262     ap.add_argument('--quadrature', action='store_true')
263     args = ap.parse_args(); args.out.mkdir(parents=True, exist_ok=True)
264     data, audit = read_xlsx(args.input); env = Environment(data, args.interp)

```

```

255 (args.out/'input_audit.json').write_text(json.dumps(audit,ensure_ascii=False,indent=2)
↪ ),encoding='utf-8')
256 results={}
257 for field in ('T','C'):
258     results[field]=solve_fv(env,field,n=args.n,rtol=args.rtol,atol=args.atol,
259                             max_step=args.max_step,quadrature=args.quadrature,verbose
↪ =True)
260     save_field(args.out/f'{field}_n{args.n}.npz',results[field])
261     print(json.dumps(results[field]['stats'],ensure_ascii=False),flush=True)
262 T,C=results['T'],results['C']
263 np.savez_compressed(args.out/'q1_unrounded.npz',time_s=T['time_s'],radius_cm=T['radiu
↪ s_m']*100,
264                     temperature_degC=T['values'],moisture_dry_basis=C['values'],
265                     average_temperature_degC=T['average'],average_C=C['average'],
266                     environment=env(T['time_s']))
267 write_csvs(args.out,T,C)
268 run={'parameters':asdict(P),'settings':vars(args)|{'input':str(args.input),'out':str(
↪ args.out)},
269     'versions':{'python':platform.python_version(),'numpy':np.__version__,'scipy':sc
↪ ipy.__version__},
270     'T':T['stats'],'C':C['stats']}
271 (args.out/'run.json').write_text(json.dumps(run,ensure_ascii=False,indent=2),encoding
↪ ='utf-8')
272 print('Tables:')
273 for field in ('T','C'):
274     ↪ print(field,results[field]['values'][[100,300,600,900,1200,1500,1800]][:,:5])
275 if __name__=='__main__': main()

```

A.7 q1_validate

support/project/q1_complete_delivery/q1_delivery/source/q1_validate.py

```

1  #!/usr/bin/env python3
2  """Actual numerical tests, independent Bessel series, and model sensitivities."""
3  from __future__ import annotations
4  import argparse,json,time
5  from pathlib import Path
6  import numpy as np
7  from scipy.special import j0,j1,jn_zeros
8  from scipy.optimize import brentq
9  from scipy.interpolate import PchipInterpolator
10 from q1_solver import
↪ P,read_xlsx,Environment,ConstantEnvironment,RadialFV,solve_fv,save_field
11 TABLE_TIMES=np.array([100,300,600,900,1200,1500,1800])
12
13 def differences(a,b):
14     x=a['values'] if isinstance(a,dict) else a
15     y=b['values'] if isinstance(b,dict) else b
16     d=np.abs(x-y)
17     return {'max_full':float(d[1:].max()),'rms_full':float(np.sqrt(np.mean(d[1:]**2))),
18           'max_table':float(d[TABLE_TIMES][:,:5].max()),
19           'four_decimal_changed_full':int(np.sum(np.round(x[1:],4)!=np.round(y[1:],4))),
20           'four_decimal_changed_table':int(np.sum(np.round(x[TABLE_TIMES][:,:5],4)!=np
↪ .round(y[TABLE_TIMES][:,:5],4))),
21           'max_location_t_rindex':list(map(int,np.unravel_index(np.argmax(d),d.shape)))}
22
23 def roots_cylinder(Bi,modes):
24     zj0=jn_zeros(0,modes);zj1=jn_zeros(1,modes)
25     f=lambda z:z*j1(z)-Bi*j0(z)
26     return np.array([brentq(f,1e-12 if i==0 else zj1[i-1]+1e-10,zj0[i]-1e-10,xtol=1e-13)
↪ for i in range(modes)])

```

```

27
28 def bessell_modes(diff,beta,modes=300):
29     mu=roots_cylinder(beta*P.R/diff,modes)
30     a=2*j1(mu)/(mu*(j0(mu)**2+j1(mu)**2))
31     return mu,a,diff*(mu/P.R)**2
32
33 def constant_cylinder(times,r,initial,bath,diff,beta,modes=300):
34     """Independently derived separation-of-variables Robin cylinder series."""
35     mu,a,decay=bessell_modes(diff,beta,modes)
36     phi=a[:,None]*j0(mu[:,None]*np.asarray(r)[None,:]/P.R)
37     ans=bath+(initial-bath)*(np.exp(-np.asarray(times)[:None]*decay)@phi)
38     ans[np.asarray(times)==0]=initial
39     return ans
40
41 def ramp_cylinder(data,times,r,modes=300):
42     """Exact convolution on piecewise-linear environmental segments; no FV code."""
43     mu,a,decay=bessell_modes(P.alpha,P.hT/(P.rho*P.cp),modes)
44     phi=a[:,None]*j0(mu[:,None]*np.asarray(r)[None,:]/P.R)
45     integ=np.zeros(modes);out=np.empty((len(times),len(r)));out[0]=P.T0
46     env=Environment(data)
47     for k,(ta,tb) in enumerate(zip(times[:-1],times[1:]),1):
48         cuts=np.r_[ta,data[(data[:,0]>ta)&(data[:,0]<tb),0],tb]
49         for l,h in zip(cuts[:-1],cuts[1:]):
50             dt=h-l;slope=(env(h)[0]-env(l)[0])/dt
51             ex=np.exp(-decay*dt)
52             integ=integ*ex+slope*(-np.expm1(-decay*dt))/decay
53         out[k]=env(tb)[0]-integ@phi
54     return out
55
56 def finite_cylinder_heat(data,times,r,z,mr=160,mz=160):
57     """2D axisymmetric r-z heat solution by product eigenfunction series.
58     Half-domain z in [0,L/2]; same Robin hT on side and both ends.
59     This is an actually evaluated finite-cylinder 2D solution, not 2D mass.
60     """
61     H=P.L/2;mu,A,dr=bessell_modes(P.alpha,P.hT/(P.rho*P.cp),mr)
62     biz=P.hT*H/P.k
63     lam=np.array([brentq(lambda
64         ↪ x:x*np.tan(x)-biz,i*np.pi+1e-10,i*np.pi+np.pi/2-1e-10,xtol=1e-13) for i in
65         ↪ range(mz)])
66     B=4*np.sin(lam)/(2*lam+np.sin(2*lam))
67     decay=dr[:,None]+P.alpha*(lam[None,:]/H)**2
68     pr=A[:,None]*j0(mu[:,None]*np.asarray(r)[None,:]/P.R)
69     pz=B[:,None]*np.cos(lam[:,None]*np.asarray(z)[None,:]/H)
70     rav=A*2*j1(mu)/mu;zav=B*np.sin(lam)/lam
71     I=np.zeros((mr,mz));out=[];means=[];env=Environment(data);last=0.
72     for t in times:
73         cuts=np.unique(np.r_[last,data[(data[:,0]>last)&(data[:,0]<t),0],t])
74         for a,b in zip(cuts[:-1],cuts[1:]):
75             dt=b-a;s=(env(b)[0]-env(a)[0])/dt
76             I=I*np.exp(-decay*dt)+s*(-np.expm1(-decay*dt))/decay
77         out.append(env(t)[0]-pr.T@I@pz)
78         means.append(env(t)[0]-rav@I@zav);last=t
79     return np.asarray(out),np.asarray(means)
80
81 class LatentEnvironment:
82     """CONDITIONAL boundary diagnostic: all outward water loss evaporates at side.
83     rho_d=initial_wet_density/(1+C0) is an EXTRA assumption, not appendix fact.
84     lambda=2.45e6 J/kg is FA056 Annex3 constant near 20degC.
85     Energy: -k*T_r=hT*(Ts-Tinf)+lambda*rho_d*hm*(Cs-Cinf).
86     """
87     def __init__(self,env,time_s,Csurface,latent=2.45e6):
88         self.env=env;self.cs=PchipInterpolator(time_s,Csurface);self.latent=latent
89         self.rho_d=P.rho/(1+P.C0);self.kind='conditional_surface_latent'

```

```

88     # Diagnostic only: boundary trace is interpolated at 1 s, not a startup-accurate
89     ↪ latent reference.
90     self.knots=np.unique(np.r_[env.knots, time_s[time_s<=10]])
91 def __call__(self,t):
92     a=self.env(t).copy();q=self.latent*self.rho_d*P.hm*(self.cs(t)-a[...,-1])
93     a[...,-1]=q/P.hT
94     return a
95
96 def get_or_run(out,env,field,n,**kwargs):
97     f=out/f'{field}_n{n}.npz'
98     if f.exists() and not kwargs and 'first_step_policy' in
99     ↪ json.loads(Path(str(f)+'.json').read_text()):
100         z=dict(np.load(f));z['stats']=json.loads(Path(str(f)+'.json').read_text());return z
101     z=solve_fv(env,field,n=n,**kwargs);save_field(f,z);return z
102
103 def main():
104     ap=argparse.ArgumentParser();base=Path(__file__).resolve().parents[1]
105     ap.add_argument('--input',type=Path,default=base/'input'/'附件 1.xlsx')
106     ap.add_argument('--out',type=Path,default=base/'output'/'validation')
107     ap.add_argument('--grids',type=int,nargs='+',default=[20,40,80,160,320,640,1280,2560,
108     ↪ 5120])
109     args=ap.parse_args();out=args.out;out.mkdir(parents=True,exist_ok=True)
110     data,audit=read_xlsx(args.input);env=Environment(data);report={'input_audit':audit,'t'
111     ↪ ests':{},'grid':{}}
112     cache={}
113     for n in args.grids:
114         cache[n]={}
115         for f in ['T','C']:
116             tic=time.perf_counter();cache[n][f]=get_or_run(out,env,f,n)
117             print('grid',n,f,'elapsed',round(time.perf_counter()-tic,2),flush=True)
118             report['grid'].append({'n':n,'dr_cm':100*P.R/n,**{f:cache[n][f]['stats'] for f in
119             ↪ ['T','C']}})
120     nf=args.grids[-1];ref=cache[nf]
121     for row in report['grid']:
122         row['difference_to_fineest']={f:differences(cache[row['n']][f],ref[f]) for f in
123         ↪ ['T','C']}
124     report['grid_pairwise']=[{'n_coarse':a,'n_fine':b,**{f:differences(cache[a][f],cache[
125     ↪ b][f]) for f in ['T','C']}]
126         for a,b in zip(args.grids[:-1],args.grids[1:])
127     (out/'progress.json').write_text(json.dumps(report,indent=2))
128     # Time error isolated on a fixed finest spatial grid.
129     tight={}
130     for f in ['T','C']:
131         tight[f]=solve_fv(env,f,n=nf,rtol=2e-12,atol=2e-14,max_step=3.,quadrature=True)
132         save_field(out/f'{f}_tight_n{nf}.npz',tight[f])
133         report['tests'][f+'time_']=differences(ref[f],tight[f])
134         print('tight',f,report['tests'][f+'time_'],flush=True)
135     # Maximum-principle and actual boundary flux signs.
136     for f in ['T','C']:
137         v=tight[f]['values'];amb=env(tight[f]['time_s'][:,-1])[0 if f=='T' else 1]
138         report['tests'][f+'physical_']={'min':float(v.min()),'max':float(v.max()),
139         'initial_error':float(np.max(np.abs(v[0]-(P.T0 if f=='T' else P.C0)))),
140         'outward_boundary_difference_min':float(np.min(v[:,-1]-amb)),
141         'outward_boundary_difference_max':float(np.max(v[:,-1]-amb)),
142         'radial_monotonicity_violation':float(max(0,-np.min(np.diff(v,axis=1)))) if
143         ↪ f=='T' else float(max(0,np.max(np.diff(v,axis=1))))}
144     # No external driving; and arbitrary nonuniform profile, zero flux.
145     for f in ['T','C']:
146         ini=P.T0 if f=='T' else P.C0
147         u=solve_fv(ConstantEnvironment(),f,n=80,end=600.,times=np.arange(601.))
148         q=solve_fv(ConstantEnvironment(),f,n=80,end=600.,times=np.arange(601.),transfer_s
149         ↪ cale=0,
150         initial=lambda r:ini+0.2*np.cos(np.pi*r/P.R))

```

```

142     report['tests']['uniform_'+f]={ 'max_error':float(np.max(np.abs(u['values']-ini))),
    ↪ 'pass':bool(np.max(np.abs(u['values']-ini))<1e-10)}
143     err=float(np.max(np.abs(q['average']-q['average'][0])))
144     report['tests']['no_flux_'+f]={ 'integral_average_drift':err, 'pass':bool(err<1e-10)}
145 # Constant-coefficient analytic check (new problem, same boundary implementation).
146 diff0=P.D0*np.exp(-P.a/P.C0);ncheck=1280
147 for f,diff,beta,ini,bath in
    ↪ [('T',P.alpha,P.hT/(P.rho*P.cp),P.T0,45.),('C',diff0,P.hm,P.C0,.025)]:
148     ce=ConstantEnvironment(T=45,C=.025)
149     u=solve_fv(ce,f,n=ncheck,constant_D=diff0 if f=='C' else None)
150     a=constant_cylinder(u['time_s'],u['radius_m'],ini,bath,diff,beta,modes=400)
151     b=constant_cylinder(u['time_s'],u['radius_m'],ini,bath,diff,beta,modes=800)
152     report['tests']['constant_analytic_'+f]={ 'fv_vs_series':differences(u,b), 'series_'
    ↪ 'truncation':differences(a,b)}
153     np.savez_compressed(out/f'constant_analytic_{f}.npz',time_s=u['time_s'],radius_m=
    ↪ u['radius_m'],fv=u['values'],series=b)
154 # Independent direct solution of the actual thermal task.
155 a=ramp_cylinder(data,tight['T']['time_s'],tight['T']['radius_m'],modes=300)
156 b=ramp_cylinder(data,tight['T']['time_s'],tight['T']['radius_m'],modes=600)
157 report['tests']['actual_thermal_analytic']={ 'fv_vs_series':differences(tight['T'],b),
    ↪ 'series_truncation':differences(a,b)}
158 np.savez_compressed(out/'actual_thermal_analytic.npz',temperature=b)
159 print('analytical done',flush=True)
160 # Finite-cylinder two-dimensional thermal comparison, no unperformed 2D claim.
161 r=tight['T']['radius_m'];z=np.array([0.,P.L/2-.02,P.L/2])
162 a,ma=finite_cylinder_heat(data,TABLE_TIMES,r,z,mr=100,mz=100)
163 b,mb=finite_cylinder_heat(data,TABLE_TIMES,r,z,mr=200,mz=200)
164 one=tight['T']['values'][TABLE_TIMES]
165 report['tests']['finite_cylinder_2d_heat']={ 'max_midplane_vs_1d_K':float(np.max(np.ab
    ↪ s(b[:, :,0]-one))),
166     'max_end_vs_midplane_K':float(np.max(np.abs(b[:, :, -1]-b[:, :, 0])),
167     'series_100_vs_200_max_K':float(np.max(np.abs(a-b))),
168     'average_2d_1800':float(mb[-1]), 'average_1d_1800':float(tight['T']['average'][-1]),
169     'end_center_T_1800':float(b[-1,0,-1]), 'end_corner_T_1800':float(b[-1,-1,-1]),
170     '2d_mass_executed':False}
171 np.savez_compressed(out/'finite_cylinder_2d_heat.npz',time_s=TABLE_TIMES,radius_m=r,z
    ↪ _m=z,T=b,average=mb)
172 # Interpolation and transfer coefficient sensitivities; separate from discretization.
173 nsens=1280
174 lin=cache.get(nsens,{})
175 for f in ['T','C']:
176     if f not in lin: lin[f]=solve_fv(env,f,n=nsens)
177     pp=solve_fv(Environment(data,'pchip'),f,n=nsens)
178     report['tests']['pchip_'+f]=differences(lin[f],pp)
179     save_field(out/f'pchip_{f}.npz',pp)
180 for factor in [.8,1.2]:
181     u=solve_fv(env,'C',n=nsens,transfer_scale=factor)
182     report['tests']['f_hm_factor_{factor}']={ 'difference':differences(lin['C'],u), 'C18
    ↪ 00':u['values'][-1,::5].tolist(), 'average1800':float(u['average'][-1])}
183     save_field(out/f'hm_{factor}.npz',u)
184 # Conditional latent-energy example: intentionally not asserted as calibrated truth.
185 le=LatentEnvironment(env,tight['C']['time_s'],tight['C']['values'][:, -1])
186 u=solve_fv(le,'T',n=1280,rtol=2e-10,atol=2e-12,max_step=1.,quadrature=True)
187 save_field(out/'conditional_latent_T.npz',u)
188 V=np.pi*P.R**2*P.L;rho_d=le.rho_d
189 water_loss=rho_d*V*(P.C0-tight['C']['average'][-1])
190 latent=le.latent*water_loss;sensible=P.rho*P.cp*V*(tight['T']['average'][-1]-P.T0)
191 report['tests']['latent_conditional']={ 'additional_assumptions':['820 interpreted as
    ↪ initial wet bulk density', 'constant dry density rho/(1+C0)', 'all effective outward
    ↪ loss evaporates at surface', 'lambda=2.45 MJ/kg constant (FA056 Annex3)'],
192     'rho_d_kg_dry_m3':rho_d, 'lambda_J_kg':le.latent, 'implied_water_loss_kg':float(wate
    ↪ r_loss),
193     'baseline_sensible_J':float(sensible), 'implied_latent_J':float(latent), 'ratio_late
    ↪ nt_to_sensible':float(latent/sensible),

```

```

194     'initial_latent_W_m2':float(1e.latent*rho_d*P.hm*(P.C0-data[0,2])),
195     'T1800_degC':u['values'][-1,:5].tolist(), 'min_T_degC':float(u['values'].min()),
196     'max_difference_vs_no_latent_K':float(np.max(np.abs(u['values']-lin['T']['values']
    ↪ )),
197     'interpretation':'Diagnostic exposes major closure uncertainty; not used in
    ↪ result1.xlsx; surface trace interpolated at 1s.}
198 report['scales']={'alpha_m2_s':P.alpha, 'D_initial_m2_s':diff0, 'BiT_R':P.hT*P.R/P.k,
199 'BiT_VoverAside':P.hT*P.R/(2*P.k), 'BiM_R':P.hm*P.R/diff0, 'BiM_VoverAside':P.hm*P.R/
    ↪ (2*diff0),
200 'thermal_radial_time_s':P.R**2/P.alpha, 'moisture_radial_time_s':P.R**2/diff0,
201 'thermal_penetration_m_1800':float(np.sqrt(P.alpha*1800)),
    ↪ 'moisture_penetration_m_1800':float(np.sqrt(diff0*1800)),
202 'aspect_ratio_length_diameter':P.L/(2*P.R), 'end_to_side_area':P.R/P.L,
203 'thermal_center_to_end_time_s':(P.L/2)**2/P.alpha, 'moisture_center_to_end_time_s':(
    ↪ P.L/2)**2/diff0}
204 report['production_recommendation']={'n':nf, 'rtol':2e-12, 'atol':2e-14, 'max_step_s':3.}
205 (out/'validation.json').write_text(json.dumps(report,indent=2,ensure_ascii=False),enc
    ↪ oding='utf-8')
206 print(json.dumps(report['tests'],indent=2,ensure_ascii=False),flush=True)
207 if __name__=='__main__':main()

```

A.8 q1_refine

support/project/q1_complete_delivery/q1_delivery/source/q1_refine.py

```

1  #!/usr/bin/env python3
2  """Further refinement and genuinely independent nonlinear collocation diagnostic."""
3  from pathlib import Path
4  import argparse, json, numpy as np
5  from scipy.integrate import solve_ivp
6  from scipy.interpolate import BarycentricInterpolator
7  from scipy.optimize import brentq
8  from q1_solver import *
9  from q1_validate import differences, finite_cylinder_heat, TABLE_TIMES, ramp_cylinder
10
11 def chebyshev_moisture(env,N=128,rtol=3e-11,atol=3e-13):
12     """Independent strong-form spectral differentiation in x=(r/R)^2.
13     C_t=4/R^2 [D(C)*C_x+x*d_x(D(C)*C_x)]. Surface C is algebraic.
14     Gauss-Lobatto nodes cluster near both axis and surface.
15     Prescribed uniform initial state is retained at t=0 in outputs; the
16     incompatible algebraic surface trace at 0+ is not substituted into it.
17     """
18     x=(1-np.cos(np.pi*np.arange(N+1)/N))/2
19     w=(-1.)*np.arange(N+1);w[[0,-1]]*=.5
20     dx=x[:,None]-x[None,:];np.fill_diagonal(dx,1.)
21     mat=(w[None,:]/w[:,None])/dx;np.fill_diagonal(mat,0.)
22     np.fill_diagonal(mat,-mat.sum(axis=1))
23     assert np.max(np.abs(mat@x-1))<1e-9
24     lastrow=mat[-1,:-1];dnn=mat[-1,-1]
25     def full(t,y):
26         cinf=env(t)[1];v=lastrow@y
27         def g(c):return 2*P.D0*np.exp(-P.a/c)/P.R*(v+dnn*c)+P.hm*(c-cinf)
28         cs=brentq(g,max(cinf+1e-8,0.5*float(np.min(y))),max(3.,2*cinf,-2*v/dnn),xtol=5e-15)
29         return np.r_[y,cs]
30     def rhs(t,y):
31         c=full(t,y);cx=mat@c;flux=P.D0*np.exp(-P.a/c)*cx
32         return (4/P.R**2*(flux+x*(mat@flux)))[-1]
33     y=np.full(N,P.C0);ts=np.arange(1801.);out=np.empty((1801,21));out[0]=P.C0
34     xo=np.linspace(0,1,21)**2
35     nf=0;steps=0
36     for a,b in zip(np.arange(0,1800,60.),np.arange(60,1801,60.)):

```

```

37     sol=solve_ivp(rhs,(a,b),y,method='BDF',rtol=rtol,atol=atol,max_step=6.,dense_outp
    ↪ ut=True,first_step=min(.0001,.0001*(96/N)**4))
38     if not sol.success: raise RuntimeError(sol.message)
39     for t in np.arange(a+1,b+1):
40         c=full(t,sol.sol(t));out[int(t)]=BarycentricInterpolator(x,c,wi=w)(xo)
41     y=sol.y[:, -1];nf+=sol.nfev;steps+=len(sol.t)-1
42     return out,{'N':N,'rtol':rtol,'atol':atol,'accepted_steps':steps,'nfev':nf,'method':'
    ↪ Independent Chebyshev collocation, algebraic surface Robin, BDF'}
43
44 def main():
45     base=Path(__file__).resolve().parents[1]
46     ap=argparse.ArgumentParser(description=__doc__)
47     ap.add_argument('--input',type=Path,default=base/'input'/'附件 1.xlsx')
48     ap.add_argument('--out',type=Path,default=base/'output'/'validation')
49     args=ap.parse_args();out=args.out
50     data,_=read_xlsx(args.input);env=Environment(data)
51     report=json.loads((out/'validation.json').read_text());R={}
52     for n in [5120,10240]:
53         R[n]={}
54         for f in ['T','C']:
55             if (out/f'{f}_tight_n{n}.npz').exists():
56                 z=dict(np.load(out/f'{f}_tight_n{n}.npz'));z['stats']=json.loads((out/f'{f}
    ↪ f}_tight_n{n}.npz.json').read_text())
57             else:
58                 z=solve_fv(env,f,n=n,rtol=2e-12,atol=2e-14,max_step=3.,quadrature=True)
59                 save_field(out/f'{f}_tight_n{n}.npz',z)
60                 R[n][f]=z
61                 print('refined',n,flush=True)
62     report['further_refinement']=[{'n_coarse':a,'n_fine':b,**{f:differences(R[a][f],R[b][
    ↪ f]) for f in ['T','C']}} for a,b in [(5120,10240)]]
63     # Production uses 10240; compare 5120/10240 and independent spectral references.
64     report['production_recommendation']={'n':10240,'rtol':2e-12,'atol':2e-14,'max_step_s'
    ↪ :3.}
65     tt={}
66     for f in ['T','C']:
67         z=solve_fv(env,f,n=10240,rtol=5e-13,atol=5e-15,max_step=1.)
68         save_field(out/f'{f}_ultratight_n10240.npz',z)
69         tt[f]=differences(R[10240][f],z)
70     report['production_time_precision']=tt
71     # Collocation cannot be mistaken for repeating the finite-volume routine.
72     coll={}
73     for N in [96,160,240]:
74         coll[N],st=chebyshev_moisture(env,N)
75         np.savez_compressed(out/f'independent_cheb_N{N}.npz',C=coll[N])
76         print('cheb',N,differences(R[10240]['C'],coll[N]),flush=True)
77     report['independent_nonlinear_collocation']={'N96_vs_N160':differences(coll[96],coll[
    ↪ 160]),
78         'N160_vs_N240':differences(coll[160],coll[240]),'fv10240_vs_N240':differences(R[102
    ↪ 40]['C'],coll[240]),
79         'settings':st}
80     a,ma=finite_cylinder_heat(data,TABLE_TIMES,R[10240]['T']['radius_m'],[0,P.L/2-.02,P.L
    ↪ /2],200,200)
81     b,mb=finite_cylinder_heat(data,TABLE_TIMES,R[10240]['T']['radius_m'],[0,P.L/2-.02,P.L
    ↪ /2],400,400)
82     report['2d_heat_refinement']={'200_vs_400_midplane_max_K':float(np.max(np.abs(a[:, :, 0
    ↪ ]-b[:, :, 0])),
83         '200_vs_400_all_max_K':float(np.max(np.abs(a-b))),
84         '400_midplane_vs_production_K':float(np.max(np.abs(b[:, :, 0]-R[10240]['T']['values'
    ↪ ][TABLE_TIMES]))),
85         'average_2d_1800':float(mb[-1]),'end_center_1800':float(b[-1,0,-1]),'end_corner_18
    ↪ 00':float(b[-1,-1,-1])}
86     np.savez_compressed(out/'finite_cylinder_2d_heat_refined.npz',time_s=TABLE_TIMES,radi
    ↪ us_m=R[10240]['T']['radius_m'],z_m=[0,P.L/2-.02,P.L/2],T=b,average=mb)
87     thermal=ramp_cylinder(data,R[10240]['T']['time_s'],R[10240]['T']['radius_m'],800)

```

```

88     report['production_actual_thermal_analytic']=differences(R[10240]['T'],thermal)
89     (out/'validation.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),enc
    ↪     oding='utf-8')
90     print(json.dumps({k:report[k] for k in ['further_refinement','production_time_precisi
    ↪     on','independent_nonlinear_collocation','2d_heat_refinement','production_actual_t
    ↪     hermal_analytic']}},indent=2),flush=True)
91 if __name__=='__main__':main()

```

A.9 q1_rounding_audit

support/project/q1_complete_delivery/q1_delivery/source/q1_rounding_audit.py

```

1  #!/usr/bin/env python3
2  """Extra receipt: compare the exact final decimal-formatting rule to independent
    ↪  references."""
3  from pathlib import Path
4  import argparse,json,numpy as np
5  from q1_solver import read_xlsx
6  from q1_validate import ramp_cylinder
7  from q1_excel import rounded,audit_excel
8
9  def main():
10     b=Path(__file__).resolve().parents[1];p=argparse.ArgumentParser(description=__doc__)
11     p.add_argument('--input',type=Path,default=b/'input'/'附件 1.xlsx')
12     p.add_argument('--out',type=Path,default=b/'output');a=p.parse_args()
13     data,_=read_xlsx(a.input);z=np.load(a.out/'q1_unrounded.npz')
14     audit_excel(a.out/'result1.xlsx',a.out/'q1_unrounded.npz',a.out)
15     T=ramp_cylinder(data,z['time_s'],z['radius_cm']/100,800)
16     C=np.load(a.out/'validation'/'independent_cheb_N240.npz')['C'];report={}
17     for key,ref in [('temperature_degC',T),('moisture_dry_basis',C)]:
18         x=np.array(rounded(z[key][1:]));y=np.array(rounded(ref[1:]))
19         report[key]={ 'cells':int(x.size),'different_four_decimal_cells':int(np.count_nonz
    ↪         ero(x!=y)),
20                     'max_unrounded_difference':float(np.max(np.abs(z[key][1:]-ref[1:])))}
21     assert all(x['different_four_decimal_cells']==0 for x in report.values())
22     np.savez_compressed(a.out/'validation'/'production_thermal_Bessel800.npz',time_s=z['t
    ↪     ime_s'],radius_cm=z['radius_cm'],temperature_degC=T)
23     (a.out/'rounding_reference_audit.json').write_text(json.dumps(report,ensure_ascii=Fa
    ↪     lse,indent=2),encoding='utf-8')
24     print(json.dumps(report,indent=2))
25 if __name__=='__main__':main()

```

A.10 q1_excel

support/project/q1_complete_delivery/q1_delivery/source/q1_excel.py

```

1  #!/usr/bin/env python3
2  """Fill the original result1 template and independently audit every result cell.
3  The delivered workbook was created with artifact_tool. A standard-library
4  OOXML fallback is provided for a local Python without that optional package;
5  that fallback was not used in the delivered workbook or runtime verification.
6  """
7  from __future__ import annotations
8  import argparse,hashlib,json,math,zipfile
9  from pathlib import Path
10 import xml.etree.ElementTree as ET
11 import numpy as np
12 NS='http://schemas.openxmlformats.org/spreadsheetml/2006/main'
13 Q=lambda name:('{'+NS+'}')+name

```

```

14 COLS=[chr(65+i) for i in range(22)]
15
16 def rounded(a):
17     return [[float(f'{v:.4f}') for v in row] for row in a]
18
19 def export_excel(template,raw,out,workbook=None):
20     template,raw,out=map(Path,(template,raw,out));out.parent.mkdir(parents=True,exist_ok=
    ↳ True)
21     if template.resolve()==out.resolve():raise ValueError('Never overwrite the original
    ↳ template')
22     z=np.load(raw);assert z['temperature_degC'].shape==(1801,21)
23     blocks=[('温度','temperature_degC','温度/°C'),('水分浓度','moisture_dry_basis','干基含水
    ↳ 率/(kg/kg)')]
24     try:
25         from artifact_tool import Blob,SpreadsheetFile
26     except ImportError:
27         return export_stdlib_fallback(template,z,out,blocks)
28     wb=workbook if workbook is not None else
    ↳ SpreadsheetFile.import_xlsx(Blob.load(str(template)))
29     for name,key,unit in blocks:
30         sh=wb.worksheets.get_item(name)
31         header=[f'时间/s \ 径向距离/cm\n'+unit]+[float(f'{r:.1f}') for r in z['radius_cm']]
32         sh.get_range('A1:V1').values=[header]
33         data=rounded(z[key][1:])
34         sh.get_range('A2:V1801').values=[[i+1]+row for i,row in enumerate(data)]
35         sh.get_range('B2:V1801').set_number_format('0.0000')
36         sh.get_range('B1:V1').set_number_format('0.0')
37         sh.get_range('A2:A1801').set_number_format('0')
38         sh.get_range('A1:V1801').format.horizontal_alignment='center'
39         sh.get_range('A1:V1801').format.vertical_alignment='center'
40         sh.get_range('A1:V1801').format.row_height=18
41         sh.get_range('B1:V1801').format.column_width=11
42         sh.get_range('A1:A1801').format.column_width=29
43         sh.get_range('A1:V1').format.row_height=38
44         sh.get_range('A1').format.wrap_text=True
45         sh.get_range('A1:V1').format.font.bold=True
46         sh.freeze_panes.freeze_rows(1);sh.freeze_panes.freeze_columns(1)
47     SpreadsheetFile.export_xlsx(wb).save(str(out))
48     return {'engine':'artifact_tool','path':str(out)}
49
50 def export_stdlib_fallback(template,z,out,blocks):
51     """Portable fallback, supplied but NOT executed in this delivery environment.
52     Retains template ZIP parts, replaces the two result sheets and appends one
53     numeric style. Uses public OOXML/ZIP formats, no spreadsheet dependencies.
54     """
55     ET.register_namespace('','NS')
56     with zipfile.ZipFile(template) as zz: parts={n:zz.read(n) for n in zz.namelist()}
57     styles=ET.fromstring(parts['xl/styles.xml']);nf=styles.find(Q('numFmts'))
58     if nf is None:nf=ET.Element(Q('numFmts'),{'count':'0'});styles.insert(0,nf)
59     ids=[int(e.attrib['numFmtId']) for e in nf];fmtid=max([163]+ids)+1
60     ET.SubElement(nf,Q('numFmt'),{'numFmtId':str(fmtid),'formatCode':'0.0000'});nf.set('c
    ↳ ount',str(len(nf)))
61     xfs=styles.find(Q('cellXfs'));sidx=len(xfs)
62     ET.SubElement(xfs,Q('xf'),{'numFmtId':str(fmtid),'fontId':'0','fillId':'0','borderId'
    ↳ ':0','xfId':'0','applyNumberFormat':'1'})
63     xfs.set('count',str(len(xfs)));parts['xl/styles.xml']=ET.tostring(styles,encoding='ut
    ↳ f-8',xml_declaration=True)
64     wb=ET.fromstring(parts['xl/workbook.xml']);rels=ET.fromstring(parts['xl/_rels/workboo
    ↳ k.xml.rels'])
65     targets={e.attrib['Id']:e.attrib['Target'] for e in rels}
66     sheets=wb.find(Q('sheets'))
67     for name,key,unit in blocks:
68         record=next(s for s in sheets if s.attrib['name']==name)

```

```

69     rid=record.attrib['{http://schemas.openxmlformats.org/officeDocument/2006/relatio
    ↪ nships}id']
70     p=targets[rid];p=p.lstrip('/') if p.startswith('/') else 'xl/'+p
71     root=ET.Element(Q('worksheet'));ET.SubElement(root,Q('dimension'),{'ref':'A1:V180
    ↪ 1'})
72     views=ET.SubElement(root,Q('sheetViews'));view=ET.SubElement(views,Q('sheetView')
    ↪ ,{'workbookViewId':'0'})
73     ET.SubElement(view,Q('pane'),{'xSplit':'1','ySplit':'1','topLeftCell':'B2','activ
    ↪ ePane':'bottomRight','state':'frozen'})
74     cols=ET.SubElement(root,Q('cols'))
75     ET.SubElement(cols,Q('col'),{'min':'1','max':'1','width':'29','customWidth':'1'})
76     ET.SubElement(cols,Q('col'),{'min':'2','max':'22','width':'11','customWidth':'1'})
77     sd=ET.SubElement(root,Q('sheetData'))
78     values=[['时间/s \ 径向距离/cm; '+unit]+z['radius_cm'].tolist()+[[i+1]+v for i,v
    ↪ in enumerate(rounded(z[key][1:]))]]
79     for i,row in enumerate(values,1):
80         rr=ET.SubElement(sd,Q('row'),{'r':str(i)})
81         for j,val in enumerate(row):
82             a={'r':COLS[j]+str(i)}
83             if i>1 and j>0:a['s']=str(sidx)
84             if isinstance(val,str):
85                 a['t']='inlineStr';cc=ET.SubElement(rr,Q('c'),a);ss=ET.SubElement(cc,
    ↪ Q('is'));ET.SubElement(ss,Q('t')).text=val
86             else:
87                 cc=ET.SubElement(rr,Q('c'),a);ET.SubElement(cc,Q('v')).text=format(fl
    ↪ oat(val),'.17g')
88     parts[p]=ET.tostring(root,encoding='utf-8',xml_declaration=True)
89     with zipfile.ZipFile(out,'w',zipfile.ZIP_DEFLATED) as zz:
90         for name,blob in parts.items():zz.writestr(name,blob)
91     return {'engine':'stdlib OOXML fallback (not tested in the delivery
    ↪ runtime)','path':str(out)}
92
93 def audit_excel(path,raw,table_dir=None):
94     """Independent ZIP/XML reader: does NOT trust the exporting library."""
95     path,raw=Path(path),Path(raw);z=np.load(raw);rep={'file':str(path),'checks':{},'sheet
    ↪ s':{}}
96     with zipfile.ZipFile(path) as zz:
97         assert zz.testzip() is None
98         strings=[]
99         if 'xl/sharedStrings.xml' in zz.namelist():
100             strings=[''.join(e.text or '' for e in si.iter(Q('t')) for si in
    ↪ ET.fromstring(zz.read('xl/sharedStrings.xml')))]
101         wb=ET.fromstring(zz.read('xl/workbook.xml'));sheets=wb.find(Q('sheets'))
102         names=[s.attrib['name'] for s in sheets];assert names==['温度','水分浓度'],names
103         rels=ET.fromstring(zz.read('xl/_rels/workbook.xml.rels'));targets={r.attrib['Id']
    ↪ :r.attrib['Target'] for r in rels}
104         st=ET.fromstring(zz.read('xl/styles.xml'));formats={int(e.attrib['numFmtId']):e.a
    ↪ ttrib['formatCode'] for e in st.find(Q('numFmts'))}
105         styles=list(st.find(Q('cellXfs')))
106         for s,key,tab in zip(sheets,['temperature_degC','moisture_dry_basis'],['temperatu
    ↪ re','moisture']):
107             target=targets[s.attrib['{http://schemas.openxmlformats.org/officeDocument/20
    ↪ 06/relationships}id']]
108             target=target.lstrip('/') if target.startswith('/') else 'xl/'+target
109             root=ET.fromstring(zz.read(target));values={};cell_styles={}
110             for c in root.findall('.//'+Q('sheetData')+'/'+Q('row')+'/'+Q('c')):
111                 v=c.find(Q('v'));typ=c.get('t','n');addr=c.attrib['r']
112                 if typ=='inlineStr':val=''.join(e.text or '' for e in c.iter(Q('t')))
113                 elif typ=='s':val=strings[int(v.text)]
114                 elif typ=='str':val=v.text if v is not None else ''
115                 elif v is None:val=None
116                 else:
117                     assert typ in ('n',''),(addr,typ)
118                     val=float(v.text)

```

```

119         values[addr]=val;cell_styles[addr]=int(c.get('s','0'))
120         assert c.find(Q('f')) is None,'No formulas expected in result arrays'
121     assert '时间/s' in values['A1'] and '距离/cm' in values['A1']
122     hdr=np.array([values[COLS[j]+'1'] for j in range(1,22)]);assert
123     ⇨ np.allclose(hdr,np.arange(21)/10,atol=1e-14)
124     tt=np.array([values[f'A{i}'] for i in range(2,1802)]);assert
125     ⇨ np.array_equal(tt,np.arange(1,1801))
126     a=np.array([[values[COLS[j]+str(i)] for j in range(1,22)] for i in
127     ⇨ range(2,1802)],float)
128     assert a.shape==(1800,21) and np.isfinite(a).all()
129     assert np.array_equal(a,np.array(rounded(z[key][1:]))),'Excel differs from
130     ⇨ declared four-decimal rounding'
131     for i in range(2,1802):
132         for j in range(1,22):
133             xf=styles[cell_styles[COLS[j]+str(i)]]
134             assert formats[int(xf.attrib['numFmtId'])]=='0.0000'
135     assert not any(isinstance(v,str) and ('...' in v or '...' in v) for v in
136     ⇨ values.values())
137     assert max(int(''.join(filter(str.isdigit,k))) for k in values)==1801
138     if table_dir is not None:
139         table=np.loadtxt(Path(table_dir)/f'table_{tab}.csv',delimiter=',',skiprow_
140         ⇨ s=1)
141         assert np.array_equal(table[:,1:],a[table[:,0].astype(int)-1][:,:5])
142     rep['sheets'][s.attrib['name']]={'time_count':1800,'radius_count':21,'result_
143     ⇨ count':37800,
144     't_first_last':[int(tt[0]),int(tt[-1])], 'r_first_last_cm':[float(hdr[0]),fl_
145     ⇨ oat(hdr[-1])],
146     'first_center_surface':a[0,[0,-1]].tolist(), 'last_center_surface':a[-1,[0,-_
147     ⇨ 1]].tolist(),
148     'max_absolute_rounding_difference':float(np.max(np.abs(a-z[key][1:]))),
149     'all_result_values_numeric_finite':True, 'all_result_number_formats':'0.0000',
150     'no_gaps_or_ellipses':True, 'paper_tables_match':table_dir is not None}
151     rep['checks']={'all_passed':True, 'total_result_values':75600, 'sheet_names_exact':True}
152     ⇨ , 'ZIP_CRC_passed':True}
153     rep['sha256']=hashlib.sha256(path.read_bytes()).hexdigest();return rep
154
155 def main():
156     ap=argparse.ArgumentParser();b=Path(__file__).resolve().parents[1]
157     ap.add_argument('--template',type=Path,default=b/'input'/'result1_template.xlsx')
158     ap.add_argument('--raw',type=Path,default=b/'output'/'q1_unrounded.npz')
159     ap.add_argument('--out',type=Path,default=b/'output'/'result1.xlsx')
160     args=ap.parse_args();info=export_excel(args.template,args.raw,args.out)
161     audit=audit_excel(args.out,args.raw,args.out.parent)
162     audit['export']=info;(args.out.parent/'excel_audit.json').write_text(json.dumps(audit_
163     ⇨ ,ensure_ascii=False,indent=2),encoding='utf-8');print(json.dumps(audit,ensure_asc_
164     ⇨ ii=False,indent=2))
165 if __name__=='__main__':main()

```

A.11 q1_report

support/project/q1_complete_delivery/q1_delivery/source/q1_report.py

```

1 #!/usr/bin/env python3
2 """Generate figures, numerical tables and reviewable Chinese paper from run files."""
3 from __future__ import annotations
4 import argparse, csv, hashlib, json
5 from pathlib import Path
6 import numpy as np
7 import matplotlib
8 matplotlib.use('Agg')
9 import matplotlib.pyplot as plt
10 from q1_solver import P, Environment, read_xlsx

```

```

11 from q1_excel import audit_excel
12 TIMES=[100,300,600,900,1200,1500,1800]
13
14 def md_table(headers,rows):
15     return '| '+'| '.join(headers)+'
    ↳ |\n| '+'| '.join(['---:']*len(headers))+'\n'+'\n'.join('| '+'|
    ↳ '.join(map(str,r))+ ' ' for r in rows)
16
17 def figures(out,data,row,v):
18     dest=out/'figures';dest.mkdir(exist_ok=True)
19     listing=[]
20     def save(name,title,xlabel,ylabel):
21         ax=plt.gca();ax.set_title(title,fontsize=12,pad=12)
22         ax.set_xlabel(xlabel,fontsize=11);ax.set_ylabel(ylabel,fontsize=11)
23         ax.tick_params(labelsize=10);ax.grid(True,alpha=.22);ax.legend(fontsize=9)
24         plt.tight_layout();plt.savefig(dest/(name+'.png'),dpi=300,bbox_inches='tight')
25         plt.savefig(dest/(name+'.svg'),bbox_inches='tight');plt.close()
26         listing.append(name)
27     t=raw['time_s'];r=raw['radius_cm'];e=Environment(data);p=Environment(data,'pchip')
28     sample=data[data[:,0]<=1800]
29     for i,(name,title,label) in enumerate([
30         ('01_environment_temperature','Drying-room temperature: measured boundary
    ↳ input','Ambient temperature (°C)'),
31         ('02_environment_moisture','Drying-room moisture: equivalent boundary
    ↳ input','Equivalent concentration (kg/kg)'))]:
32         plt.figure(figsize=(7.2,4.5));plt.plot(t,e(t)[:i],label='Piecewise linear (main)')
33         plt.plot(t,p(t)[:i], '--',label='PCHIP (sensitivity)')
34         plt.plot(sample[:,0],sample[:,i+1], 'o',ms=3,label='Attachment 1 measurements')
35         save(name,title,'Time (s)',label)
36     for key,field,label,name in [('temperature_degC','Temperature','Temperature
    ↳ (°C)','03_temperature_time'),
37         ('moisture_dry_basis','Dry-basis moisture','C (kg water /
    ↳ kg dry solid)','04_moisture_time')]:
38         plt.figure(figsize=(7.2,4.5))
39         for i in range(0,21,5):plt.plot(t,raw[key][:i],label=f'r = {r[i]:g} cm')
40         if key=='temperature_degC':plt.plot(t,raw['environment'][:0],':',label='Drying
    ↳ room')
41         save(name,field+' at the cylinder midplane','Time (s)',label)
42     for f,label,name in [('T','Temperature (°C)','05_temperature_profiles'),('C','C (kg
    ↳ water / kg dry solid)','06_moisture_profiles')]:
43         z=np.load(out/f'{f}_n10240.npz');rr=z['internal_radius_m']*100
44         plt.figure(figsize=(7.2,4.5))
45         for time,s in zip(z['snapshot_times_s'][1:],z['internal_snapshots'][1:]):
46             plt.plot(rr,s,label=f't = {time:.0f} s')
47         save(name,('Temperature' if f=='T' else 'Dry-basis moisture')+' profiles at the
    ↳ midplane','Radial distance from axis (cm)',label)
48     g=v['grid_pairwise']+v['further_refinement']
49     for f,unit,name in [('T','K','07_grid_temperature'),('C','kg/kg','08_grid_moisture')]:
50         dx=np.array([100*P.R/x['n_coarse'] for x in g])
51         full=np.array([x[f]['max_full'] for x in g]);tab=np.array([x[f]['max_table'] for x
    ↳ in g])
52         plt.figure(figsize=(7.2,4.5));plt.loglog(dx,full,'o-',label='Full 1800 × 21
    ↳ output: max |coarse - fine|')
53         plt.loglog(dx,tab,'s--',label='Required 7 × 5 table: max |coarse - fine|')
54         save(name,('Temperature' if f=='T' else 'Moisture')+' grid refinement','Coarse
    ↳ radial spacing (cm)',f'Successive-grid difference ({unit})')
55     (dest/'figure_manifest.json').write_text(json.dumps({'files':listing,'formats':['300
    ↳ dpi PNG','SVG'],'source':'actual unrounded production and validation
    ↳ outputs'},indent=2),encoding='utf-8')
56     return listing
57
58 def main():
59     b=Path(__file__).resolve().parents[1];ap=argparse.ArgumentParser(description=__doc__)
60     ap.add_argument('--input',type=Path,default=b/'input'/附件 1.xlsx')

```

```

61 ap.add_argument('--out',type=Path,default=b/'output');args=ap.parse_args();out=args.out
62 z=dict(np.load(out/'q1_unrounded.npz'));v=json.loads((out/'validation/'/'validation.js」
↳ on').read_text())
63 run=json.loads((out/'run.json').read_text());data,audit=read_xlsx(args.input)
64 ex=audit_excel(out/'result1.xlsx',out/'q1_unrounded.npz',out)
65 figures(out,data,z,v)
66 tokens={}
67 for key,title in
↳ [('temperature_degC','TEMP_TABLE'),('moisture_dry_basis','MOIST_TABLE')]:
68     tokens[title]=md_table(['时间/s','0 cm','0.5 cm','1 cm','1.5 cm','2
↳ cm'],[f'{x:.4f}' for x in z[key][t,::5]] for t in TIMES])
69 g=v['grid_pairwise']+v['further_refinement']
70 tokens['GRID_TABLE']=md_table(['区间数
↳ N→2N','粗网格间距/cm','全场温差/K','全场含水率差/(kg/kg)','两张论文表最大含水率差'],
71     [f'{x['n_coarse']}'→{x['n_fine']}',f'{100*P.R/x['n_coarse']:.7g}',f'{x['T']]['max_」
↳ full']:.3e}',f'{x['C']]['max_full']:.3e}',f'{x['C']]['max_table']:.3e}'] for x
↳ in g])
72 indT=v['production_actual_thermal_analytic'];indC=v['independent_nonlinear_collocatio」
↳ n']['fv10240_vs_N240']
73 tests=v['tests'];lt=tests['latent_conditional'];sc=v['scales'];two=v['2d_heat_refinem」
↳ ent']
74 vals={
75     'AVG_T':z['average_temperature_degC'][-1],'AVG_C':z['average_C'][-1],
76     'CENTER_T':z['temperature_degC'][-1,0],'SURFACE_T':z['temperature_degC'][-1,-1],
77     'CENTER_C':z['moisture_dry_basis'][-1,0],'SURFACE_C':z['moisture_dry_basis'][-1,-1],
78     'LOSS_PERCENT':100*(P.C0-z['average_C'][-1])/P.C0,'TEMP_DIFF':z['temperature_degC']」
↳ [-1,-1]-z['temperature_degC'][-1,0],
79     'MEAN_2D_T':two['average_2d_1800'],'END_T':two['end_center_1800'],
80     'LATENT_RATIO':lt['ratio_latent_to_sensible'],'LATENT_LOSS_KG':lt['implied_water_lo」
↳ ss_kg'],
81     'LATENT_J':lt['implied_latent_J'],'SENSIBLE_J':lt['baseline_sensible_J'],
82     'LATENT_INITIAL_FLUX':lt['initial_latent_W_m2'],'LATENT_T_CENTER':lt['T1800_degC」]」
↳ [0],
83     'LATENT_T_SURFACE':lt['T1800_degC'][-1],
84     'HM_LOW':tests['hm_factor_0.8']['C1800'][-1],'HM_HIGH':tests['hm_factor_1.2']['C180」
↳ 0'][-1],
85 }
86 tokens.update({k:f'{float(x):.4f}' for k,x in vals.items()})
87 for key,val in {
88     'TIME_T':v['production_time_precision']['T']['max_full'],'TIME_C':v['production_tim」
↳ e_precision']['C']['max_full'],
89     'INDEP_T':indT['max_full'],'INDEP_C':indC['max_full'],'INDEP_C_TABLE':indC['max_tab」
↳ le'],
90     'CONSERV_T':run['T']['conservation_augmented_max_abs'],'CONSERV_C':run['C']['conser」
↳ vation_augmented_max_abs'],
91     'QUAD_T':run['T']['independent_flux_quadrature_max_abs'],'QUAD_C':run['C']['indepem」
↳ dent_flux_quadrature_max_abs'],
92     'NOFLUX_T':tests['no_flux_T']['integral_average_drift'],'NOFLUX_C':tests['no_flux_C」
↳ ['integral_average_drift'],
93     'PCHIP_T':tests['pchip_T']['max_full'],'PCHIP_C':tests['pchip_C']['max_full'],
94     'TWO_D_DIFF':two['400_midplane_vs_production_K'],
95     'TWO_D_TRUNC':two['200_vs_400_all_max_K'],
96     'CONST_T':tests['constant_analytic_T']['fv_vs_series']['max_full'],
97     'CONST_C':tests['constant_analytic_C']['fv_vs_series']['max_full'],
98 }.items():tokens[key]=f'{val:.4e}'
99 tokens['TIMESTEPS_T']=str(run['T']['accepted_steps']);tokens['TIMESTEPS_C']=str(run['」
↳ C']['accepted_steps'])
100 tokens['VERSIONS']=''.join(k+' '+s for k,s in run['versions'].items())
101 template=(Path(__file__).parent/'paper_template.md').read_text(encoding='utf-8')
102 for k,val in tokens.items():template=template.replace('@@'+k+'@@',val)
103 if '@@' in template:raise ValueError('Unresolved report token')
104 paper=out/'第一问论文正文.md';paper.write_text(template,encoding='utf-8')
105 # Audit the two Markdown tables against all corresponding workbook cells.

```

```

106     for key,name in
107         ↪ [ ('temperature_degC','TEMP_TABLE'), ('moisture_dry_basis','MOIST_TABLE') ]:
108         assert tokens[name] in template
109     summary={'main_model':'Midplane 1D effective Robin, independent sensible-heat and
110         ↪ nonlinear dry-basis diffusion',
111         'production_grid_intervals':10240, 'values_1800':{k:float(x) for k,x in vals.items()},
112         'independent_comparison':{'T':indT, 'C':indC}, 'excel':ex['checks'],
113         'paper_tables_generated_from_unrounded_output':True, 'paper_tables_same_as_Excel':Tr
114         ↪ ue,
115         'limitations':['Original problem PDF formula image not read; repository extracted
116         ↪ text and explicit supplied formula used',
117         'Foods11 LFS full PDF not obtained', 'No internal-response experiment', 'No 2D
118         ↪ nonlinear moisture computation',
119         'Latent omission not small under literal evaporating-mass interpretation; baseline
120         ↪ is conditional' ]}
121     (out/'delivery_summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent
122     ↪ =2),encoding='utf-8')
123     print('Generated:',paper,'and 8 PNG/SVG figure pairs',flush=True)
124 if __name__=='__main__':main()

```

A.12 第二问机制与参数情景

A.13 q2_core

support/project/q2_refinement_delivery/runtime/source/q2_core.py

```

1  """Q2 effective coupled radial transport. All dimensional quantities are SI.
2
3  The temperature storage is S(C)*dT/dt, NOT d(S*T)/dt. The supplied density
4  is used only inside the empirical effective heat capacity. It is not asserted
5  to be the actual mass density of a fixed, motionless two-component mixture.
6  """
7  from __future__ import annotations
8  import argparse, csv, hashlib, json, platform, time, zipfile
9  from dataclasses import dataclass, asdict
10 from pathlib import Path
11 import xml.etree.ElementTree as ET
12 import numpy as np
13 import scipy
14 from scipy.integrate import solve_ivp
15 from scipy.interpolate import PchipInterpolator
16 from scipy.sparse import coo_matrix
17
18 NS={'s':'http://schemas.openxmlformats.org/spreadsheetml/2006/main'}
19
20 def read_workbook_values(path):
21     """Read actual OOXML bytes, including shared strings, without editing input."""
22     sheets={}
23     with zipfile.ZipFile(path) as z:
24         if z.testzip() is not None: raise ValueError('Damaged XLSX ZIP')
25         shared=[]
26         if 'xl/sharedStrings.xml' in z.namelist():
27             for si in ET.fromstring(z.read('xl/sharedStrings.xml')).findall('s:si',NS):
28                 shared.append(''.join(t.text or '' for t in si.iter('{'+NS['s']+'}t')))
29         rels=ET.fromstring(z.read('xl/_rels/workbook.xml.rels'))
30         rel={r.attrib['Id']:r.attrib['Target'] for r in rels}
31         for sh in ET.fromstring(z.read('xl/workbook.xml')).find('s:sheets',NS):
32             rid=sh.attrib['{http://schemas.openxmlformats.org/officeDocument/2006/relatio
33             ↪ nships}id']
34             target=rel[rid];target=target.lstrip('/') if target.startswith('/') else
35             ↪ 'xl/'+target

```

```

34     vals={}; styles={}; types={}
35     for c in ET.fromstring(z.read(target)).findall('.//s:sheetData/s:row/s:c',NS):
36         addr=c.attrib['r']; typ=c.attrib.get('t','n'); v=c.find('s:v',NS)
37         if c.find('s:f',NS) is not None: raise ValueError('Formula in numeric
↪ source/result')
38         if typ=='s': value=shared[int(v.text)]
39         elif typ=='inlineStr': value=''.join(t.text or '' for t in
↪ c.findall('.//s:t',NS))
40         elif v is None: value=None
41         elif typ in ('str','e'): value=v.text
42         else: value=float(v.text)
43         vals[addr]=value; styles[addr]=int(c.attrib.get('s',0)); types[addr]=typ
44         sheets[sh.attrib['name']]={'values':vals, 'styles':styles, 'types':types}
45     return sheets
46
47 def file_hashes(path):
48     b=Path(path).read_bytes()
49     return {'sha256':hashlib.sha256(b).hexdigest(),
50             'git_blob_sha1':hashlib.sha1(b'\blob
↪ '+str(len(b)).encode()+b'\0'+b).hexdigest(),
51             'bytes':len(b)}
52
53 def load_environment(path):
54     sheets=read_workbook_values(path)
55     if len(sheets)!=1: raise ValueError('Unexpected environmental sheets')
56     sh=next(iter(sheets.values()))['values']
57     if [sh.get(f'{c}1') for c in 'ABC']!=['时间','温度','水分浓度']:
58         raise ValueError('Unrecognized attachment 1 headings')
59     last=max(int(a[1:]) for a in sh if a.startswith('A'))
60     a=np.array([[sh.get(f'{c}{i}',np.nan) for c in 'ABC'] for i in range(2,last+1)],float)
61     if not np.isfinite(a).all() or not np.all(np.diff(a[:],0)>0):
62         raise ValueError('Missing, nonnumeric, duplicate, or reversed input')
63     if np.any(a[:,2]<0) or a[0,0]!=0: raise ValueError('Invalid initial time/concentration')
64     audit={'file':Path(path).name, **file_hashes(path), 'sheets':list(sheets), 'count':len(a),
65           'headings':['时间','温度','水分浓度'], 'units':['s','degC','kg/kg (environmental
↪ basis unspecified)'],
66           'time_range_s':[float(a[0,0]),float(a[-1,0])],
67           'time_steps_s':np.unique(np.diff(a[:],0)).tolist(), 'missing_values':0, 'duplica
↪ te_times':0,
68           'first_3h_records':int(np.sum(a[:],0)<=10800)),
69           'temperature_minmax': [float(a[:,1].min()),float(a[:,1].max())],
70           'environmental_moisture_minmax':[float(a[:,2].min()),float(a[:,2].max())],
71           'internal_response_observations':False}
72     return a,audit
73
74 class Environment:
75     def __init__(self,data,kind='linear',extension='error'):
76         self.data=np.asarray(data,float); self.knots=self.data[:,0]
77         if kind not in ('linear','pchip'): raise ValueError(kind)
78         if extension not in ('error','hold_last'): raise ValueError(extension)
79         self.kind,self.extension=kind,extension
80         self.interp=PchipInterpolator(self.knots,self.data[:,1:],axis=0,extrapolate=False)
↪         if kind=='pchip' else None
81     def __call__(self,t):
82         t=np.asarray(t,float)
83         if np.any(t<0) or (self.extension=='error' and np.any(t>self.knots[-1]+1e-9)):
84             raise ValueError('No implicit environmental extrapolation; explicitly select
↪ hold_last for scenarios')
85         if self.kind=='pchip': return self.interp(np.minimum(t,self.knots[-1]))
86         return np.stack([np.interp(t,self.knots,self.data[:,j]) for j in (1,2)],axis=-1)
87     def derivative(self,t):
88         t=np.asarray(t,float)
89         if self.kind=='pchip': return np.where((t<=self.knots[-1])[...],None,self.interp.d
↪ erivative()(np.minimum(t,self.knots[-1])),0.)

```

```

90         ix=np.clip(np.searchsorted(self.knots,t,side='right')-1,0,len(self.knots)-2)
91         d=(self.data[ix+1,1:]-self.data[ix,1:])/(self.knots[ix+1]-self.knots[ix])[...,None]
92         return np.where((t<self.knots[-1])[...,None],d,0.)
93
94 @dataclass(frozen=True)
95 class Parameters:
96     R:float=0.02
97     L:float=0.25
98     T0:float=28.
99     C0:float=2.55
100     hT:float=25.
101     hm:float=8e-7
102 P=Parameters()
103
104 def properties(T,C,derivatives=False,freeze=False):
105     """T in degC, C in kg water/kg dry solid. No silent clipping."""
106     T,C=np.broadcast_arrays(np.asarray(T),np.asarray(C))
107     if np.any(np.real(C)<=0) or np.any(np.real(T+273.15)<=0):raise
108     ↪ FloatingPointError('Nonpositive C or absolute temperature')
109     if freeze:T=T*0+28.;C=C*0+2.55
110     rho=650.+128.*C;cp=1450.+2736.*(1.+C)
111     k=.21+.38*C/(1.+C);S=rho*cp
112     D=.0024*np.exp(-.45/C-3850./(T+273.15))
113     if not derivatives:return S,k,D
114     Sc=128.*cp+rho*2736./(1.+C)**2
115     kc=.38/(1.+C)**2;Dt=D*3850./(T+273.15)**2;Dc=D*.45/C**2
116     if freeze:Sc=Sc*0;kc=kc*0;Dt=Dt*0;Dc=Dc*0
117     return S,k,D,Sc,kc,Dt,Dc
118
119 def harmonic_with_derivatives(a,b):
120     s=a+b
121     return 2*a*b/s,2*(b/s)**2,2*(a/s)**2
122
123 class CoupledFV:
124     """Interleaved [T_0,C_0,T_1,C_1,...], vertex/dual-cell conservative fluxes.
125     Temperature state offset supports a genuine degC/K invariance test.
126     """
127     def __init__(self,n,p=P,temperature_offset=0.,freeze=False,fault=None):
128         self.n=int(n);self.m=n+1;self.p=p;self.offset=temperature_offset
129         self.freeze=freeze;self.fault=fault
130         self.r=np.linspace(0,p.R,n+1);self.dr=p.R/n
131         self.faces=np.r_[0.,(self.r[1:]+self.r[:-1])/2,p.R]
132         self.w=np.diff(self.faces**2)/2;self.v=self.w.sum();self.s=self.faces[1:-1]/self.dr
133     def evaluate(self,t,y,env,with_jac=False):
134         u=y.reshape(self.m,2);T=u[:,0]-self.offset;C=u[:,1]
135         S,k,D,Sc,kc,Dt,Dc=properties(T,C,True,self.freeze)
136         if self.fault=='celsius_exponent':
137             D=.0024*np.exp(-.45/C-3850./T);Dt=D*3850./T**2;Dc=D*.45/C**2
138         if self.fault=='frozen_D':
139             D=np.full_like(C,properties(np.array([28.]),np.array([2.55]))[2][0]);Dt*=0;Dc
140             ↪ *=0
141         af,akl,akr=harmonic_with_derivatives(k[:-1],k[1:])
142         if self.fault=='face_alpha':
143             af,_,_=harmonic_with_derivatives((k/S)[-1],(k/S)[1:])
144         df,dL,dr=harmonic_with_derivatives(D[:-1],D[1:])
145         dT=T[:-1]-T[1:];dC=C[:-1]-C[1:]
146         q=self.s*af*dT;j=self.s*df*dC
147         f=np.zeros_like(u);f[:-1,0]=-q;f[1:,0]=q;f[:-1,1]=-j;f[1:,1]=j
148         ambient=env(t)
149         qo=self.p.R*self.p.hT*(T[-1]-ambient[0]);jo=self.p.R*self.p.hm*(C[-1]-ambient[1])
150         if self.fault=='face_alpha':qo/=S[-1]
151         f[-1,-]=[qo,jo];f[:,0]/=self.w*(1 if self.fault=='face_alpha' else S);f[:,1]/=self.w
152         if self.fault=='product_storage':f[:,0]-=Sc/S*T*f[:,1]
153         if not with_jac:return f.ravel()

```

```

152         if self.fault is not None:raise ValueError('Fault paths deliberately use numerical
            ↪ Jacobian')
153         # 2x2 derivative of each outward edge flux w.r.t. left/right state.
154         l=np.empty((self.n,2,2));r=np.empty_like(l)
155         l[:,0,0]=self.s*af;r[:,0,0]=-self.s*af
156         l[:,0,1]=self.s*akl*kc[:-1]*dT;r[:,0,1]=self.s*akr*kc[1:]*dT
157         l[:,1,0]=self.s*d1*Dc[:-1]*dC;r[:,1,0]=self.s*dr*Dc[1:]*dC
158         l[:,1,1]=self.s*(df+d1*Dc[:-1]*dC);r[:,1,1]=self.s*(-df+dr*Dc[1:]*dC)
159         storage=np.stack([self.w*S,self.w],axis=1)
160         ix=np.arange(self.n);rows=[];cols=[];vals=[]
161         for ro,sign in ((0,-1.),(1,1.)):
162             for co,edge in ((0,1),(1,r)):
163                 for a in range(2):
164                     for b in range(2):
165                         rows.append(2*(ix+ro)+a);cols.append(2*(ix+co)+b)
166                         vals.append(sign*edge[:,a,b]/storage[ix+ro,a])
167         idx=np.arange(self.m)
168         rows += [2*idx,np.array([2*self.n,2*self.n+1])]
169         cols += [2*idx+1,np.array([2*self.n,2*self.n+1])]
170         vals += [-f[:,0]*Sc/S,np.array([-self.p.R*self.p.hT/storage[-1,0],-self.p.R*self.p.
            ↪ p.hm/storage[-1,1]])]
171         jac=coo_matrix((np.concatenate(vals),(np.concatenate(rows),np.concatenate(cols))))
            ↪ ,shape=(2*self.m,2*self.m)).tocsc()
172         return jac
173     def sparsity(self):
174         from scipy.sparse import diags,kron
175         return kron(diags([np.ones(self.n),np.ones(self.m),np.ones(self.n)],[-1,0,1]),np.
            ↪ ones((2,2)),format='csc')
176
177     def dense_derivative(poly,t):
178         """Differentiate the accepted BDF dense polynomial, not the ODE RHS."""
179         t=np.atleast_1d(t)
180         x=(t[None,:]-poly.t_shift[:,None])/poly.denom[:,None]
181         prod=np.ones(t.size);der=np.zeros(t.size)
182         result=np.zeros((poly.D.shape[1],t.size))
183         for j in range(poly.order):
184             der=der*x[j]+prod/poly.denom[j];prod*=x[j]
185             result+=poly.D[j+1,:,None]*der
186         return result
187
188     def solve_fv(env,n=320,p=P,end=10800.,rtol=2e-11,atol_T=2e-12,atol_C=2e-13,max_step=20.,
189         temperature_offset=0.,freeze=False,initial=None,quadrature=False,fault=None,
            ↪ output_step=1.,verbose=False):
190         if n%20:raise ValueError('FV n must be divisible by 20 to sample requested radii
            ↪ without interpolation')
191         if end>env.knots[-1] and env.extension=='error':raise ValueError('End outside data
            ↪ range')
192         op=CoupledFV(n,p,temperature_offset,freeze,fault)
193         y=np.empty((n+1,2));y[:,0]=p.T0+temperature_offset;y[:,1]=p.C0
194         if initial is not None:y=np.array(initial(op.r),float);y[:,0]+=temperature_offset
195         times=np.unique(np.r_[np.arange(0,end+1e-9,output_step),end]);oi=np.arange(21)*(n//20)
196         out=np.empty((len(times),21,2));out[0]=y[oi];out[0,:,-1]=temperature_offset
197         average=np.empty((len(times),2));average[0]=op.w@y/op.v;average[0,0]=-temperature_off
            ↪ set
198         snaps=[y.copy()];st=[0.];stats={'n':n,'nodes':n+1,'dr_m':op.dr,'end_s':end,'rtol':rtol,
199             'atol_T':atol_T,'atol_C':atol_C,'max_step_s':max_step,'method':'BDF + exact coupled
            ↪ block sparse Jacobian',
200             'environment':env.kind,'extension':env.extension,'temperature_offset':temperature_o
            ↪ ffset,
201             'freeze':freeze,'fault':fault,'nfev':0,'njev':0,'nlu':0,'accepted_steps':0,'paramet
            ↪ ers':asdict(p)}
202         cuts=np.unique(np.r_[0.,env.knots[(env.knots>0)&(env.knots<end)],end])
203         tic=time.perf_counter();qint=jint=storeint=0.;balance=[];field_min=np.array([np.inf,n
            ↪ p.inf]);field_max=-field_min

```

```

204 gx,gw=np.polynomial.legendre.leggauss(6)
205 atol=np.tile([atol_T,atol_C],n+1)
206 for a,b in zip(cuts[:-1],cuts[1:]):
207     fun=lambda t,z:op.evaluate(t,z,env)
208     jac=(lambda t,z:op.evaluate(t,z,env,True)) if fault is None else None
209     sol=solve_ivp(fun,(a,b),y.ravel(),method='BDF',jac=jac,jac_sparsity=op.sparsity())
    ↪ if fault else None,
210     rtol=rtol,atol=atol,max_step=max_step,dense_output=True,first_step=min(1e-3,.05
    ↪ *op.dr**2/2e-8,b-a))
211 if not sol.success:raise RuntimeError(f'FV failed n={n}, [{a},{b}]: {sol.message}')
212 if not np.isfinite(sol.y).all() or np.any(sol.y[1::2]<=0):raise
    ↪ FloatingPointError('Invalid accepted state')
213 for key in ('nfev','njev','nlu'):stats[key]+=int(getattr(sol,key))
214 stats['accepted_steps']+=len(sol.t)-1
215 states=sol.y.reshape(n+1,2,-1)
216 field_min=np.minimum(field_min,states.min(axis=(0,2)));field_max=np.maximum(field
    ↪ _max,states.max(axis=(0,2)))
217 mask=(times>a)&(times<=b+1e-8)
218 v=sol.sol(times[mask]).reshape(n+1,2,-1)
219 out[mask]=v[oi].transpose(2,0,1);out[mask,:,-1]-=temperature_offset
220 average[mask]=np.einsum('i,ijt->tj',op.w,v)/op.v;average[mask,0]-=temperature_off
    ↪ set
221 for t in (1800.,3600.,5400.,7200.,9000.,10800.):
222     if a<t<=b:snaps.append(sol.sol(t).reshape(n+1,2));st.append(t)
223 if quadrature:
224     for lo,hi,poly in zip(sol.t[:-1],sol.t[1:],sol.sol.interpolants):
225         tq=(lo+hi)/2+(hi-lo)/2*gx;ys=poly(tq).reshape(n+1,2,-1)
226         yd=dense_derivative(poly,tq).reshape(n+1,2,-1)
227         S=properties(ys[:,0]-temperature_offset,ys[:,1],freeze=freeze)[0]
228         boundary=env(tq)
229         qdot=p.R*p.ht*(ys[-1,0]-temperature_offset-boundary[:,0])/op.v
230         jdot=p.R*p.hm*(ys[-1,1]-boundary[:,1])/op.v
231         # Independently differentiate accepted interpolation; no flux cancellation
    ↪ RHS reuse.
232         storedot=np.einsum('i,it,it->t',op.w,S,yd[:,0])/op.v
233         weights=gw*(hi-lo)/2
234         qint+=float(qdot@weights);jint+=float(jdot@weights);storeint+=float(store
    ↪ _int+dot@weights)
235         currentC=float(op.w@sol.y[1::2,-1])/op.v
236         balance.append([b,currentC-average[0,1]+jint,storeint+qint,qint,storeint])
237     y=sol.y[:,1:].reshape(n+1,2)
238     if verbose and int(b)%1800==0:print(f'FV n={n}, t={b:g} s',flush=True)
239 stats['elapsed_s']=time.perf_counter()-tic
240 field_min[0]-=temperature_offset;field_max[0]-=temperature_offset
241 stats['accepted_internal_min']=field_min.tolist();stats['accepted_internal_max']=fiel
    ↪ _d_max.tolist()
242 stats['environment_knots_restarted']=len(cuts)-1
243 stats['quadrature_performed']=quadrature
244 if balance:
245     ba=np.array(balance);stats['water_balance_max_kgkg']=float(np.max(abs(ba[:,1])))
246     stats['effective_heat_balance_max_J_m3']=float(np.max(abs(ba[:,2])))
247     stats['net_heat_input_J_m3']=-qint;stats['effective_heat_balance_relative']=float
    ↪ (np.max(abs(ba[:,2]))/max(abs(qint),1))
248 return {'time_s':times,'radius_cm':np.linspace(0,2,21),'temperature_degC':out[:,0],
249         'moisture_dry_basis':out[:,1],'average_temperature_degC':average[:,0],
250         'average_moisture_dry_basis':average[:,1],'environment':env(times),'internal_radiu
    ↪ s_m':op.r,
251         'internal_weights_rdr':op.w,'snapshot_times_s':np.array(st),
252         'snapshots_temperature_degC':np.array(snaps)[:,-1]-temperature_offset,
253         'snapshots_moisture_dry_basis':np.array(snaps)[:,-1],'balance_records':np.array(b
    ↪ _alance),'stats':stats}
254
255 def save_solution(path,sol):
256     path=Path(path);path.parent.mkdir(parents=True,exist_ok=True)

```

```

257     np.savez_compressed(path,**{k:v for k,v in sol.items() if k!='stats'})
258     path.with_suffix('.json').write_text(json.dumps(sol['stats'],ensure_ascii=False,inden
    ↪ t=2),encoding='utf-8')
259
260 def main():
261     base=Path(__file__).resolve().parents[1];ap=argparse.ArgumentParser()
262     ap.add_argument('--input',type=Path,default=base/'inputs/附件1.xlsx');ap.add_argument(
    ↪ '--out',type=Path,required=True)
263     ap.add_argument('--n',type=int,default=1280);ap.add_argument('--end',type=float,defau
    ↪ lt=10800.)
264     ap.add_argument('--rtol',type=float,default=2e-11);ap.add_argument('--max-step',type=
    ↪ float,default=20.)
265     ap.add_argument('--quadrature',action='store_true');ap.add_argument('--extension',cho
    ↪ ices=['error','hold_last'],default='error')
266     args=ap.parse_args();a,audit=load_environment(args.input);args.out.mkdir(parents=True
    ↪ ,exist_ok=True)
267     (args.out/'input_audit.json').write_text(json.dumps(audit,ensure_ascii=False,indent=2
    ↪ ),encoding='utf-8')
268     sol=solve_fv(Environment(a,extension=args.extension),n=args.n,end=args.end,rtol=args.
    ↪ rtol,max_step=args.max_step,quadrature=args.quadrature,verbose=True)
269     save_solution(args.out/f'fv_n{args.n}.npz',sol);print(json.dumps(sol['stats'],indent=
    ↪ 2))
270 if __name__=='__main__':main()

```

A.14 q2_spectral

support/project/q2_refinement_delivery/runtime/source/q2_spectral.py

```

1  """Independent Chebyshev collocation in  $x=(r/R)^2$ .
2  No FV operator or face averaging is reused. Robin conditions are algebraically
3  eliminated. All nonlinear material derivatives and boundary derivatives are
4  included in the dense Jacobian. BDF/Radau integrate the interior system.
5  """
6  from __future__ import annotations
7  import argparse,json,time
8  from pathlib import Path
9  import numpy as np
10 from scipy.integrate import solve_ivp
11 from scipy.optimize import brentq
12 from q2_core import P,Environment,load_environment,save_solution
13
14 def spectral_properties(T,C,freeze=False):
15     if np.any(np.real(C)<=0) or np.any(np.real(T+273.15)<=0):raise
    ↪ FloatingPointError('Invalid spectral state')
16     if freeze:T=T*0+28.;C=C*0+2.55
17     rho=650.+128.*C
18     cp=(1450.+4186.*C)/(1.+C)
19     k=(.21+.59*C)/(1.+C)
20     S=rho*cp;D=.0024*np.exp(-.45/C)*np.exp(-3850./(T+273.15))
21     Sc=128.*cp+rho*2736./(1.+C)**2
22     kc=.38/(1.+C)**2;Dt=3850.*D/(T+273.15)**2;Dc=.45*D/C**2
23     if freeze:Sc*=0;kc*=0;Dt*=0;Dc*=0
24     return S,k,D,Sc,kc,Dt,Dc
25
26 def cheb_grid(n):
27     j=np.arange(n+1);x=(1-np.cos(np.pi*j/n))/2
28     bw=(-1.)*j;bw[[0,-1]]*=.5
29     dx=x[:,None]-x[None,: ]
30     G=np.divide(bw[None,:]/bw[:,None],dx,out=np.zeros_like(dx),where=dx!=0)
31     G[np.diag_indices(n+1)]=-G.sum(axis=1)
32     # Derivative implementation anchors at the surface to preserve constants.
33     G[:,-1]=-G[:,-1].sum(axis=1)

```

```

34 theta=np.pi*j/n;weights=np.zeros(n+1);ii=np.arange(1,n);v=np.ones(n-1)
35 if n%2==0:
36     weights[[0,n]]=1/(n*n-1)
37     for k in range(1,n//2):v-=2*np.cos(2*k*theta[ii])/(4*k*k-1)
38     v-=np.cos(n*theta[ii])/(n*n-1)
39 else:
40     weights[[0,n]]=1/(n*n)
41     for k in range(1,(n-1)//2+1):v-=2*np.cos(2*k*theta[ii])/(4*k*k-1)
42 weights[ii]=2*v/n
43 return x,G,bw,weights/2
44
45 def interpolation_matrix(x,bw,target):
46     M=np.empty((len(target),len(x)))
47     for i,t in enumerate(target):
48         d=t-x;j=np.argmin(abs(d))
49         if abs(d[j])<1e-14:M[i]=0.;M[i,j]=1.
50         else:
51             v=bw/d;M[i]=v/v.sum()
52     return M
53
54 class Spectral:
55     def __init__(self,n,p=P,freeze=False):
56         self.n=n;self.m=n+1;self.p=p;self.freeze=freeze
57         self.x,self.G,self.bw,self.weights=cheb_grid(n)
58         self.row=self.G[-1,:-1];self.diag=-self.row.sum();self.fac=4/p.R**2;self.A=2/p.R
59         self.interp=interpolation_matrix(self.x,self.bw,np.linspace(0,1,21)**2)
60         self.max_newton=0
61     def surface(self,t,u,env,derivative=False):
62         Ti=u[:,0];Ci=u[:,1];et,ec=env(t)
63         gT=self.row@(Ti-et);anchor=Ci[-1];gC=self.row@(Ci-anchor)
64         near=anchor-gC/self.diag
65         def calc(c,need=False):
66             _,k,_,kc,_,_=spectral_properties(et+0*c,c,self.freeze)
67             den=self.A*k*self.diag+self.p.hT
68             T=et-self.A*k*gT/den
69             Tp=-self.A*kc*gT*self.p.hT/den**2
70             S,k,D,Sc,kc,Dt,Dc=spectral_properties(T,c,self.freeze)
71             cx=gC+self.diag*(c-anchor)
72             F=self.A*D*cx+self.p.hm*(c-ec)
73             Fp=self.A*((Dc+Dt*Tp)*cx+D*self.diag)+self.p.hm
74             return F,Fp,T,cx,(S,k,D,Sc,kc,Dt,Dc)
75         c=near
76         for it in range(20):
77             F,Fp,T,cx,props=calc(c)
78             step=F/Fp;trial=c-step
79             while np.real(trial)<=0:step*=.5;trial=c-step
80             c=trial
81             if abs(step)<3e-14*(1+abs(c)):break
82         else:
83             # On an unexpected Newton failure choose the largest positive root
84             # continuously connected to the Neumann-side value, never a tiny spurious root.
85             if np.iscomplexobj(u):raise RuntimeError('Complex-step surface solve failed')
86             high=float(near);fh=calc(high)[0];found=False
87             for low in np.geomspace(high*.99,max(1e-12,high*1e-9),160):
88                 fl=calc(low)[0]
89                 if fl*fh<=0:
90                     c=brentq(lambda a:calc(a)[0],low,high,xtol=5e-15);found=True;break
91                 high=low;fh=fl
92             if not found:raise RuntimeError('Positive surface root not found')
93         self.max_newton=max(self.max_newton,it+1)
94         F,Fp,T,cx,props=calc(c)
95         full=np.vstack([u,np.array([T,c])])
96         if not derivative:return full
97         _,k,D,_,kc,Dt,Dc=props

```

```

98     tx=self.row@(Ti-T)
99     M=np.array([[self.A*k*self.diag+self.p.hT,self.A*k*tx],
100                [self.A*Dt*cx,self.A*(D*self.diag+Dc*cx)+self.p.hm]])
101     part=np.zeros((2,2*self.n));part[0,0:2]=self.A*k*self.row;part[1,1:2]=self.A*D*
↪ self.row
102     bd=-np.linalg.solve(M,part)
103     BT=np.zeros((self.m,2*self.n));BC=np.zeros_like(BT)
104     BT[np.arange(self.n),2*np.arange(self.n)]=1.
105     BC[np.arange(self.n),2*np.arange(self.n)+1]=1.
106     BT[-1]=bd[0];BC[-1]=bd[1]
107     return full,BT,BC
108 def evaluate(self,t,y,env,jac=False):
109     u=y.reshape(self.n,2)
110     if jac:full,BT,BC=self.surface(t,u,env,True)
111     else:full=self.surface(t,u,env)
112     T,C=full.T;S,k,D,Sc,kc,Dt,Dc=spectral_properties(T,C,self.freeze)
113     gt=self.G@(T-T[-1]);gc=self.G@(C-C[-1])
114     qt=k*gt;q=D*gc
115     ht=qt+self.x*(self.G@(qt-qt[-1]));hc=qc+self.x*(self.G@(qc-qc[-1]))
116     fT=self.fac*ht/S;fC=self.fac*hc
117     if not jac:return np.column_stack([fT[:-1],fC[:-1]]).ravel()
118     gtj=self.G@(BT-BT[-1]);gcj=self.G@(BC-BC[-1])
119     tqj=k[:,None]*gtj+(kc*gt)[:,None]*BC
120     cqj=D[:,None]*gcj+gc[:,None]*(Dt[:,None]*BT+Dc[:,None]*BC)
121     htj=tqj+self.x[:,None]*(self.G@(tqj-tqj[-1]))
122     hcj=cqj+self.x[:,None]*(self.G@(cqj-cqj[-1]))
123     ftj=self.fac*htj/S[:,None]-(fT*Sc/S)[:,None]*BC
124     fcj=self.fac*hcj
125     J=np.empty((2*self.n,2*self.n));J[0:2]=ftj[:-1];J[1:2]=fcj[:-1]
126     return J
127
128 def solve_spectral(env,n=160,p=P,end=10800.,rtol=2e-12,atol_T=2e-13,atol_C=2e-14,max_step
↪ =15.,method='BDF',freeze=False,verbose=False):
129     op=Spectral(n,p,freeze);y=np.tile([p.T0,p.C0],n)
130     times=np.arange(int(end)+1, dtype=float)
131     if times[-1]!=end:raise ValueError('Spectral test end must be integer seconds')
132     out=np.empty((len(times),21,2));out[0,:,0]=p.T0;out[0,:,1]=p.C0
133     average=np.empty((len(times),2));average[0]=[p.T0,p.C0]
134     snapshot_times=[0.];snaps=[np.tile([p.T0,p.C0],(n+1,1))]
135     cuts=np.unique(np.r_[0.,env.knots[(env.knots>0)&(env.knots<end)],end])
136     stats={'n':n,'spatial_method':'Chebyshev-Lobatto collocation in x=(r/R)^2; coupled
↪ Robin elimination',
137           'time_method':method,'rtol':rtol,'atol_T':atol_T,'atol_C':atol_C,'max_step_s':max_s
↪ tep,
138           'nfev':0,'njev':0,'nlu':0,'accepted_steps':0,'environment':env.kind,'extension':env
↪ .extension,'end_s':end}
139     tic=time.perf_counter()
140     for a,b in zip(cuts[:-1],cuts[1:]):
141         sol=solve_ivp(lambda t,y:op.evaluate(t,y,env),(a,b),y,method=method,
142                       jac=lambda
↪ t,y:op.evaluate(t,y,env,True),rtol=rtol,atol=np.tile([atol_T,atol_C],n),
143                       max_step=max_step,dense_output=True,first_step=min(1e-4,b-a))
144         if not sol.success:raise RuntimeError(f'Spectral failed on [{a},{b}]:
↪ {sol.message}')
145         for k in ('nfev','njev','nlu'):stats[k]+=int(getattr(sol,k))
146         stats['accepted_steps']+=len(sol.t)-1
147         mask=(times>a)&(times<=b+1e-8);ts=times[mask];ys=sol.sol(ts).reshape(n,2,-1).tran
↪ spose(2,0,1)
148         for ix,t,u in zip(np.flatnonzero(mask),ts,ys):
149             full=op.surface(t,u,env);out[ix]=op.interp@full;average[ix]=op.weights@full
150             if t in (1800.,3600.,5400.,7200.,9000.,10800.):snapshot_times.append(t);snaps
↪ .append(full.copy())
151         if not np.isfinite(out[mask]).all() or np.any(out[mask,:,1]<=0):raise
↪ FloatingPointError('Spectral output invalid')

```

```

152     y=sol.y[:, -1]
153     if verbose and int(b)%1800==0:print(f'Cheb n={n}, t={b:g}', flush=True)
154     stats['elapsed_s']=time.perf_counter()-tic;stats['max_boundary_newton_iterations']=op ↵
↵     .max_newton
155     return {'time_s':times, 'radius_cm':np.linspace(0,2,21), 'temperature_degC':out[:, :, 0],
156           'moisture_dry_basis':out[:, :, 1], 'average_temperature_degC':average[:, 0],
157           'average_moisture_dry_basis':average[:, 1], 'environment':env(times),
158           'internal_radius_m':p.R*np.sqrt(op.x), 'internal_weights_normalized':op.weights,
159           'snapshot_times_s':np.array(snapshot_times), 'snapshots_temperature_degC':np.array(s ↵
↵     naps)[:, :, 0],
160           'snapshots_moisture_dry_basis':np.array(snaps)[:, :, 1], 'stats':stats}
161
162 def main():
163     b=Path(__file__).resolve().parents[1];ap=argparse.ArgumentParser()
164     ap.add_argument('--input', type=Path, default=b'/inputs/附件1.xlsx');ap.add_argument('-- ↵
↵     out', type=Path, required=True)
165     ap.add_argument('--n', type=int, default=160);ap.add_argument('--rtol', type=float, defau ↵
↵     lt=2e-12)
166     ap.add_argument('--end', type=float, default=10800.);ap.add_argument('--method', choices ↵
↵     =['BDF', 'Radau'], default='BDF')
167     args=ap.parse_args();a, _=load_environment(args.input)
168     sol=solve_spectral(Environment(a), n=args.n, end=args.end, rtol=args.rtol, method=args.me ↵
↵     thod, verbose=True)
169     save_solution(args.out/f'cheb_n{args.n}_{args.method}.npz', sol);print(json.dumps(sol[ ↵
↵     'stats'], indent=2))
170 if __name__=='__main__':main()

```

A.15 q2_tests

support/project/q2_refinement_delivery/runtime/source/q2_tests.py

```

1  """Executable physics-contract, invariance, analytical-reference and sensitivity tests.
2  Tests are isolated from production inputs/outputs; deliberately faulty operators
3  are never used to create result2.xlsx.
4  """
5  from __future__ import annotations
6  import json, time
7  from dataclasses import replace
8  from pathlib import Path
9  import numpy as np
10 from scipy.optimize import brentq
11 from scipy.special import j0, j1, jn_zeros
12 from q2_core import *
13 from q2_spectral import Spectral, solve_spectral, spectral_properties
14
15 FIELDS=('temperature_degC', 'moisture_dry_basis')
16
17 def difference(a, b, times=None, radii=None):
18     d=np.asarray(a)-np.asarray(b);ii=np.unravel_index(np.argmax(abs(d)), d.shape)
19     ans={'max_abs':float(np.max(abs(d))), 'rms':float(np.sqrt(np.mean(d*d))), 'index':list( ↵
↵     map(int, ii)),
20         'four_decimal_mismatches':int(np.count_nonzero(np.round(a, 4)!=np.round(b, 4)))}
21     if times is not None:ans['time_s']=float(times[ii[0]])
22     if radii is not None:ans['radius_cm']=float(radii[ii[1]])
23     return ans
24
25 def eigen_roots(Bi, n=600):
26     hi=jn_zeros(0, n);lo=np.r_[0., jn_zeros(1, n-1)]
27     return np.array([brentq(lambda x:x*j1(x)-Bi*j0(x), a+1e-12, b, xtol=1e-14) for a, b in ↵
↵     zip(lo, hi)])
28
29 def constant_reference(times, radii, p=P, Te=50., Ce=.03, nmodes=600):

```

```

30 S,k,D=properties(np.array([28.]),np.array([2.55]));S,k,D=float(S[0]),float(k[0]),float(D[0])
31 ↪ t(D[0])
32 ans={}
33 for field,diffusivity,Bi,initial,ambient in ((FIELDS[0],k/S,p.hT*p.R/k,p.T0,Te),
34                                                (FIELDS[1],D,p.hm*p.R/D,p.C0,Ce)):
35     roots=eigen_roots(Bi,nmodes)
36     coeff=2*j1(roots)/(roots*(j0(roots)**2+j1(roots)**2))
37     shape=j0(roots[:,None]*radii[None,:])/(p.R*100)
38     decay=np.exp(-diffusivity*np.asarray(times[:,None]*roots[None,:])**2/p.R**2)
39     vals=ambient+(initial-ambient)*(decay*coeff)@shape
40     vals[np.asarray(times)==0]=initial;ans[field]=vals
41 return ans
42
43 def run_unit_tests(env,out):
44     out=Path(out);out.mkdir(parents=True,exist_ok=True);report={}
45     # Dimensional scales from the actual constitutive routines.
46     states=[(28,2.55),(50,2.55),(50,.15),(50,.05)]
47     report['parameter_states']=[]
48     for T,C in states:
49         S,k,D=map(float,properties(np.array(T),np.array(C)))
50         rho=650+128*C;cp=1450+2736*C/(1+C)
51         report['parameter_states'].append({'T_degC':T,'C':C,'rho':rho,'cp':cp,'k':k,'D':D}
52         ↪ , 'S':S,
53         ↪ 'alpha':k/S,'BiT_R':P.hT*P.R/k,'BiC_R':P.hm*P.R/D,'heat_time_R2_alpha_s':P.R**2
54         ↪ /k/S,
55         ↪ 'moisture_time_R2_D_s':P.R**2/D))
56     s=report['parameter_states'][0];report['scales']={
57         'alpha0':s['alpha'],'D0':s['D'],'BiT_R':s['BiT_R'],'BiC_R':s['BiC_R'],
58         'thermal_diffusion_length_3h_m':float(np.sqrt(s['alpha']*10800)),
59         'moisture_diffusion_length_3h_m':float(np.sqrt(s['D']*10800)),
60         'L_over_diameter':P.L/(2*P.R),'ends_over_side_area':P.R/P.L}
61     # Independent 60-digit mpmath evaluation of the printed Appendix-3 formula.
62     assert abs(s['D']/5.6416803730256675e-09-1)<2e-14
63     # Exact analytical Jacobians checked by complex-step directional differentiation.
64     rng=np.random.default_rng(8624)
65     op=CoupledFV(40);r=op.r/P.R
66     u=np.column_stack([35+3*r*r,2.5-.8*r*r]).ravel();v=rng.normal(size=u.size)
67     num=np.imag(op.evaluate(1000,u+1e-28j*v,env))/1e-28
68     rel=float(np.max(abs(op.evaluate(1000,u,env,True)@v-num))/np.max(abs(num)))
69     sp=Spectral(48);x=sp.x[:-1]
70     us=np.column_stack([35+3*x,2.5-.8*x]).ravel();vs=rng.normal(size=us.size)
71     nums=np.imag(sp.evaluate(1000,us+1e-28j*vs,env))/1e-28
72     rels=float(np.max(abs(sp.evaluate(1000,us,env,True)@vs-nums))/np.max(abs(nums)))
73     report['jacobian_complex_step']={'fv_relative':rel,'spectral_relative':rels,'passed':
74     ↪ rel<1e-10 and rels<1e-10}
75     assert report['jacobian_complex_step']['passed']
76     # No driving, actual integration.
77     constant=Environment(np.array([[0,28,2.55],[10800,28,2.55]]),float))
78     z=solve_fv(constant,n=80,end=600,quadrature=True)
79     report['uniform_no_drive']={'T_max_change':float(np.max(abs(z[FIELDS[0]]-28))),
80     ↪ 'C_max_change':float(np.max(abs(z[FIELDS[1]]-2.55)))}
81     assert max(report['uniform_no_drive'].values())<1e-10
82     # Nonuniform water redistribution with no exchange and exactly uniform temperature.
83     closedp=replace(P,hT=0.,hm=0.)
84     init=lambda r:np.column_stack([np.full_like(r,35.),2+.4*np.cos(np.pi*(r/P.R)**2)])
85     closed=solve_fv(constant,n=160,p=closedp,end=600,initial=init,quadrature=True)
86     report['closed_redistribution']={'T_max_change':float(np.max(abs(closed[FIELDS[0]]-35)
87     ↪ )),
88     ↪ 'C_weighted_average_drift':float(np.max(abs(closed['average_moisture_dry_basis']-cl
89     ↪ osed['average_moisture_dry_basis']*[0]))),
90     ↪ 'water_profile_changed':float(np.max(abs(closed[FIELDS[1]][-1]-closed[FIELDS[1]][0]
91     ↪ )),
92     ↪ 'scope':'Fixed-reference-dry-density effective mass integral; not rho(C)*C'}
93     assert report['closed_redistribution']['T_max_change']<1e-8

```

```

87     assert report['closed_redistribution']['C_weighted_average_drift']<1e-9
88     # Frozen coefficients versus cylindrical Bessel solution.
89     bench=Environment(np.array([[0,50,.03],[600,50,.03]],float))
90     analytical_sol=solve_spectral(bench,n=128,end=600,freeze=True,rtol=1e-12)
91     analytic=constant_reference(analytical_sol['time_s'],analytical_sol['radius_cm'])
92     report['constant_coefficient_bessel']={k:difference(analytical_sol[k][1:],analytic[k]
93     ↪ [1:],analytical_sol['time_s'][1:],analytical_sol['radius_cm']) for k in FIELDS}
94     assert report['constant_coefficient_bessel'][FIELDS[0]]['max_abs']<2e-7
95     assert report['constant_coefficient_bessel'][FIELDS[1]]['max_abs']<2e-8
96     save_solution(out/'constant_coefficient_spectral.npz',analytical_sol)
97     np.savez_compressed(out/'constant_coefficient_bessel.npz',**analytic,time_s=analytica
98     ↪ l_sol['time_s'])
99     # Fault injection: these tests must identify incorrect model implementations.
100     faults={}
101     correctD=s['D'];wrongD=.0024*np.exp(-.45/2.55-3850/28)
102     faults['Celsius_in_exponent']={'wrong_over_correct_D':float(wrongD/correctD),'detecte
103     ↪ d':abs(wrongD/correctD-1)>.99}
104     mid=(r>.1)&(r<.9);expected=4/P.R**2/properties(u[0::2],u[1::2])[0]*3*(
105     ↪ properties(u[0::2],u[1::2])[1]+r*r*.38/(1+u[1::2])**2*(-.8))
106     correct=op.evaluate(1000,u,env)[0::2]
107     faulty=CoupledFV(40,fault='face_alpha').evaluate(1000,u,env)[0::2]
108     ec=float(np.max(abs(correct[mid]-expected[mid])));ef=float(np.max(abs(faulty[mid]-exp
109     ↪ ected[mid])))
110     faults['face_alpha_instead_of_k']={'correct_interior_error_K_s':ec,'faulty_error_K_s'
111     ↪ :ef,'detected':ef>100*ec}
112     heated=u.copy();heated[0::2]+=10
113     reaction=np.max(abs(op.evaluate(1000,heated,env)[1::2]-op.evaluate(1000,u,env)[1::2]))
114     fr=CoupledFV(40,fault='frozen_D')
115     reaction_bad=np.max(abs(fr.evaluate(1000,heated,env)[1::2]-fr.evaluate(1000,u,env)[1:
116     ↪ :2]))
117     faults['frozen_RHS_D_coupling']={'correct_response_C_s':float(reaction),'faulty_respo
118     ↪ nse':float(reaction_bad),'detected':reaction>1e-6 and reaction_bad<1e-20}
119     uniform=init(op.r).ravel()
120     true_rhs=CoupledFV(40,p=closedp).evaluate(0,uniform,constant)[0::2]
121     bad_rhs=CoupledFV(40,p=closedp,fault='product_storage').evaluate(0,uniform,constant)[
122     ↪ 0::2]
123     faults['unclosed_product_storage']={'correct_T_rate_max':float(np.max(abs(true_rhs))),
124     ↪ 'faulty_T_rate_max':float(np.max(abs(bad_rhs))),'detected':np.max(abs(true_rhs))<1
125     ↪ e-14 and np.max(abs(bad_rhs))>1e-5}
126     report['fault_injection']=faults
127     assert all(v['detected'] for v in faults.values())
128     # Unit change, separate integrations on the same mesh.
129     celsius=solve_fv(env,n=320,rtol=3e-13,atol_T=2e-14,atol_C=2e-15,max_step=10.)
130     kelvin=solve_fv(env,n=320,rtol=3e-13,atol_T=2e-14,atol_C=2e-15,max_step=10.,temperatu
131     ↪ re_offset=273.15)
132     report['temperature_unit_invariance']={k:difference(celsius[k],kelvin[k],celsius['tim
133     ↪ e_s'],celsius['radius_cm']) for k in FIELDS}
134     assert report['temperature_unit_invariance'][FIELDS[0]]['max_abs']<2e-8
135     assert report['temperature_unit_invariance'][FIELDS[1]]['max_abs']<2e-9
136     save_solution(out/'kelvin_invariance.npz',kelvin)
137     report['passed']=True
138     (out/'unit_tests.json').write_text(json.dumps(report,ensure_ascii=False,indent=2,defa
139     ↪ ult=lambda x:x.item()),encoding='utf-8')
140     return json.loads(json.dumps(report,default=lambda x:x.item()))
141
142 def run_sensitivity(env,out):
143     out=Path(out);out.mkdir(parents=True,exist_ok=True)
144     baseline=solve_fv(env,n=320,rtol=1e-10,max_step=20.)
145     save_solution(out/'sensitivity_baseline.npz',baseline);report={}
146     cases=[('hT_0.8',replace(P,hT=.8*P.hT),env,False),('hT_1.2',replace(P,hT=1.2*P.hT),en
147     ↪ v,False),
148     ↪ ('hm_0.8',replace(P,hm=.8*P.hm),env,False),('hm_1.2',replace(P,hm=1.2*P.hm),en
149     ↪ v,False),

```

```

136         ('pchip_all241', P, Environment(env.data, kind='pchip'), False), ('freeze_all_initi
    ↪ al', P, env, True)]
137     for name, p, e, freeze in cases:
138         sol=solve_fv(e, n=320, p=p, freeze=freeze, rtol=1e-10, max_step=20.)
139         report[name]={ 'differences': {k: difference(sol[k], baseline[k], sol[ 'time_s'], sol[ 'r
    ↪ adius_cm']) for k in FIELDS},
140             'T_center_end': float(sol[FIELDS[0]][-1, 0]), 'T_surface_end': float(sol[FIELDS[0]]
    ↪ [-1, -1]),
141             'C_center_end': float(sol[FIELDS[1]][-1, 0]), 'C_surface_end': float(sol[FIELDS[1]]
    ↪ [-1, -1]),
142             'mean_C_end': float(sol['average_moisture_dry_basis'][-1]), 'stats': sol['stats']}
143         save_solution(out/f'sensitivity_{name}.npz', sol)
144         report['scope']='Deterministic scenarios, not confidence intervals; all compared with
    ↪ identical-mesh baseline'
145         (out/'sensitivity.json').write_text(json.dumps(report, ensure_ascii=False, indent=2, def
    ↪ ault=lambda x:x.item()), encoding='utf-8')
146         return json.loads(json.dumps(report, default=lambda x:x.item()))

```

A.16 q2_core_original

support/project/q2_refinement_delivery/reused/q2_core_original.py

```

1  """Q2 effective coupled radial transport. All dimensional quantities are SI.
2
3  The temperature storage is S(C)*dT/dt, NOT d(S*T)/dt. The supplied density
4  is used only inside the empirical effective heat capacity. It is not asserted
5  to be the actual mass density of a fixed, motionless two-component mixture.
6  """
7  from __future__ import annotations
8  import argparse, csv, hashlib, json, platform, time, zipfile
9  from dataclasses import dataclass, asdict
10 from pathlib import Path
11 import xml.etree.ElementTree as ET
12 import numpy as np
13 import scipy
14 from scipy.integrate import solve_ivp
15 from scipy.interpolate import PchipInterpolator
16 from scipy.sparse import coo_matrix
17
18 NS={'s':'http://schemas.openxmlformats.org/spreadsheetml/2006/main'}
19
20 def read_workbook_values(path):
21     """Read actual OOXML bytes, including shared strings, without editing input."""
22     sheets={}
23     with zipfile.ZipFile(path) as z:
24         if z.testzip() is not None: raise ValueError('Damaged XLSX ZIP')
25         shared=[]
26         if 'xl/sharedStrings.xml' in z.namelist():
27             for si in ET.fromstring(z.read('xl/sharedStrings.xml')).findall('s:si', NS):
28                 shared.append(''.join(t.text or '' for t in si.iter('{'+NS['s']+'}t')))
29         rels=ET.fromstring(z.read('xl/_rels/workbook.xml.rels'))
30         rel={r.attrib['Id']:r.attrib['Target'] for r in rels}
31         for sh in ET.fromstring(z.read('xl/workbook.xml')).find('s:sheets', NS):
32             rid=sh.attrib['{http://schemas.openxmlformats.org/officeDocument/2006/relatio
    ↪ nships}id']
33             target=rel[rid];target=target.lstrip('/') if target.startswith('/') else
    ↪ 'xl/'+target
34             vals={}; styles={}; types={}
35             for c in ET.fromstring(z.read(target)).findall('..s:sheetData/s:row/s:c', NS):
36                 addr=c.attrib['r']; typ=c.attrib.get('t', 'n'); v=c.find('s:v', NS)
37                 if c.find('s:f', NS) is not None: raise ValueError('Formula in numeric
    ↪ source/result')

```

```

38         if typ=='s': value=shared[int(v.text)]
39         elif typ=='inlineStr': value=''.join(t.text or '' for t in
    ↪ c.findall('.//s:t',NS))
40         elif v is None:value=None
41         elif typ in ('str','e'):value=v.text
42         else:value=float(v.text)
43         vals[addr]=value;styles[addr]=int(c.attrib.get('s',0));types[addr]=typ
44         sheets[sh.attrib['name']]={'values':vals,'styles':styles,'types':types}
45     return sheets
46
47 def file_hashes(path):
48     b=Path(path).read_bytes()
49     return {'sha256':hashlib.sha256(b).hexdigest(),
50             'git_blob_sha1':hashlib.sha1(b'blob
    ↪ '+str(len(b)).encode()+b'\0'+b).hexdigest(),
51             'bytes':len(b)}
52
53 def load_environment(path):
54     sheets=read_workbook_values(path)
55     if len(sheets)!=1: raise ValueError('Unexpected environmental sheets')
56     sh=next(iter(sheets.values()))['values']
57     if [sh.get(f'{c}1') for c in 'ABC']!=['时间','温度','水分浓度']:
58         raise ValueError('Unrecognized attachment 1 headings')
59     last=max(int(a[1:]) for a in sh if a.startswith('A'))
60     a=np.array([[sh.get(f'{c}{i}',np.nan) for c in 'ABC'] for i in range(2,last+1)],float)
61     if not np.isfinite(a).all() or not np.all(np.diff(a[:,0])>0):
62         raise ValueError('Missing, nonnumeric, duplicate, or reversed input')
63     if np.any(a[:,2]<0) or a[0,0]!=0:raise ValueError('Invalid initial time/concentration')
64     audit={'file':Path(path).name,**file_hashes(path),'sheets':list(sheets),'count':len(a),
65           'headings':['时间','温度','水分浓度'],'units':['s','degC','kg/kg (environmental
    ↪ basis unspecified)'],
66           'time_range_s':[float(a[0,0]),float(a[-1,0])],
67           'time_steps_s':np.unique(np.diff(a[:,0])).tolist(),'missing_values':0,'duplica
    ↪ te_times':0,
68           'first_3h_records':int(np.sum(a[:,0]<=10800)),
69           'temperature_minmax': [float(a[:,1].min()),float(a[:,1].max())],
70           'environmental_moisture_minmax':[float(a[:,2].min()),float(a[:,2].max())],
71           'internal_response_observations':False}
72     return a,audit
73
74 class Environment:
75     def __init__(self,data,kind='linear',extension='error'):
76         self.data=np.asarray(data,float);self.knots=self.data[:,0]
77         if kind not in ('linear','pchip'):raise ValueError(kind)
78         if extension not in ('error','hold_last'):raise ValueError(extension)
79         self.kind,self.extension=kind,extension
80         self.interp=PchipInterpolator(self.knots,self.data[:,1:],axis=0,extrapolate=False)
    ↪ if kind=='pchip' else None
81     def __call__(self,t):
82         t=np.asarray(t,float)
83         if np.any(t<0) or (self.extension=='error' and np.any(t>self.knots[-1]+1e-9)):
84             raise ValueError('No implicit environmental extrapolation; explicitly select
    ↪ hold_last for scenarios')
85         if self.kind=='pchip':return self.interp(np.minimum(t,self.knots[-1]))
86         return np.stack([np.interp(t,self.knots,self.data[:,j]) for j in (1,2)],axis=-1)
87     def derivative(self,t):
88         t=np.asarray(t,float)
89         if self.kind=='pchip':return np.where((t<=self.knots[-1])[...None],self.interp.d
    ↪ erivative()(np.minimum(t,self.knots[-1])),0.)
90         ix=np.clip(np.searchsorted(self.knots,t,side='right')-1,0,len(self.knots)-2)
91         d=(self.data[ix+1,1:]-self.data[ix,1:])/(self.knots[ix+1]-self.knots[ix])[...None]
92         return np.where((t<self.knots[-1])[...None],d,0.)
93
94 @dataclass(frozen=True)

```

```

95 class Parameters:
96     R:float=0.02
97     L:float=0.25
98     T0:float=28.
99     C0:float=2.55
100     hT:float=25.
101     hm:float=8e-7
102 P=Parameters()
103
104 def properties(T,C,derivatives=False,freeze=False):
105     """T in degC, C in kg water/kg dry solid. No silent clipping."""
106     T,C=np.broadcast_arrays(np.asarray(T),np.asarray(C))
107     if np.any(np.real(C)<=0) or np.any(np.real(T+273.15)<=0):raise
108     ↪ FloatingPointError('Nonpositive C or absolute temperature')
109     if freeze:T=T*0+28.;C=C*0+2.55
110     rho=650.+128.*C;cp=1450.+2736.*C/(1.+C)
111     k=.21+.38*C/(1.+C);S=rho*cp
112     D=.0024*np.exp(-.45/C-3850./(T+273.15))
113     if not derivatives:return S,k,D
114     Sc=128.*cp+rho*2736./(1.+C)**2
115     kc=.38/(1.+C)**2;Dt=D*3850./(T+273.15)**2;Dc=D*.45/C**2
116     if freeze:Sc=Sc*0;kc=kc*0;Dt=Dt*0;Dc=Dc*0
117     return S,k,D,Sc,kc,Dt,Dc
118
119 def harmonic_with_derivatives(a,b):
120     s=a+b
121     return 2*a*b/s,2*(b/s)**2,2*(a/s)**2
122
123 class CoupledFV:
124     """Interleaved [T_0,C_0,T_1,C_1,...], vertex/dual-cell conservative fluxes.
125     Temperature state offset supports a genuine degC/K invariance test.
126     """
127     def __init__(self,n,p=P,temperature_offset=0.,freeze=False,fault=None):
128         self.n=int(n);self.m=n+1;self.p=p;self.offset=temperature_offset
129         self.freeze=freeze;self.fault=fault
130         self.r=np.linspace(0,p.R,n+1);self.dr=p.R/n
131         self.faces=np.r_[0.,(self.r[1:]+self.r[:-1])/2,p.R]
132         self.w=np.diff(self.faces**2)/2;self.v=self.w.sum();self.s=self.faces[1:-1]/self.dr
133     def evaluate(self,t,y,env,with_jac=False):
134         u=y.reshape(self.m,2);T=u[:,0]-self.offset;C=u[:,1]
135         S,k,D,Sc,kc,Dt,Dc=properties(T,C,True,self.freeze)
136         if self.fault=='celsius_exponent':
137             D=.0024*np.exp(-.45/C-3850./T);Dt=D*3850./T**2;Dc=D*.45/C**2
138         if self.fault=='frozen_D':
139             D=np.full_like(C,properties(np.array([28.]),np.array([2.55]))[2][0]);Dt*=0;Dc
140             ↪ *=0
141         af,akl,akr=harmonic_with_derivatives(k[:-1],k[1:])
142         if self.fault=='face_alpha':
143             af,_,_=harmonic_with_derivatives((k/S)[:-1],(k/S)[1:])
144         df,dl,dr=harmonic_with_derivatives(D[:-1],D[1:])
145         dT=T[:-1]-T[1:];dC=C[:-1]-C[1:]
146         q=self.s*af*dT;j=self.s*df*dC
147         f=np.zeros_like(u);f[:-1,0]=-q;f[1:,0]=q;f[:-1,1]=-j;f[1:,1]=j
148         ambient=env(t)
149         qo=self.p.R*self.p.hT*(T[-1]-ambient[0]);jo=self.p.R*self.p.hm*(C[-1]-ambient[1])
150         if self.fault=='face_alpha':qo/=S[-1]
151         f[-1,-]=qo;f[:,0]/=self.w*(1 if self.fault=='face_alpha' else S);f[:,1]/=self.w
152         if self.fault=='product_storage':f[:,0]=-Sc/S*T*f[:,1]
153         if not with_jac:return f.ravel()
154         if self.fault is not None:raise ValueError('Fault paths deliberately use numerical
155         ↪ Jacobian')
156         # 2x2 derivative of each outward edge flux w.r.t. left/right state.
157         l=np.empty((self.n,2,2));r=np.empty_like(l)
158         l[:,0,0]=self.s*af;r[:,0,0]=-self.s*af

```

```

156     l[:,0,1]=self.s*akl*kc[:-1]*dT;r[:,0,1]=self.s*akr*kc[1:]*dT
157     l[:,1,0]=self.s*d1*Dc[:-1]*dC;r[:,1,0]=self.s*dr*Dc[1:]*dC
158     l[:,1,1]=self.s*(df+d1*Dc[:-1]*dC);r[:,1,1]=self.s*(-df+dr*Dc[1:]*dC)
159     storage=np.stack([self.w*S,self.w],axis=1)
160     ix=np.arange(self.n);rows=[];cols=[];vals=[]
161     for ro,sign in ((0,-1.),(1,1.)):
162         for co,edge in ((0,1),(1,r)):
163             for a in range(2):
164                 for b in range(2):
165                     rows.append(2*(ix+ro)+a);cols.append(2*(ix+co)+b)
166                     vals.append(sign*edge[:,a,b]/storage[ix+ro,a])
167     idx=np.arange(self.m)
168     rows += [2*idx,np.array([2*self.n,2*self.n+1])]
169     cols += [2*idx+1,np.array([2*self.n,2*self.n+1])]
170     vals += [-f[:,0]*Sc/S,np.array([-self.p.R*self.p.hT/storage[-1,0],-self.p.R*self.p.
↪ p.hm/storage[-1,1]])]
171     jac=coo_matrix((np.concatenate(vals),(np.concatenate(rows),np.concatenate(cols)))
↪ ,shape=(2*self.m,2*self.m)).tocsc()
172     return jac
173 def sparsity(self):
174     from scipy.sparse import diags,kron
175     return kron(diags([np.ones(self.n),np.ones(self.m),np.ones(self.n)],[-1,0,1]),np.
↪ ones((2,2)),format='csc')
176
177 def dense_derivative(poly,t):
178     """Differentiate the accepted BDF dense polynomial, not the ODE RHS."""
179     t=np.atleast_1d(t)
180     x=(t[None,:]-poly.t_shift[:,None])/poly.denom[:,None]
181     prod=np.ones(t.size);der=np.zeros(t.size)
182     result=np.zeros((poly.D.shape[1],t.size))
183     for j in range(poly.order):
184         der=der*x[j]+prod/poly.denom[j];prod*=x[j]
185         result+=poly.D[j+1,,:None]*der
186     return result
187
188 def solve_fv(env,n=320,p=P,end=10800.,rtol=2e-11,atol_T=2e-12,atol_C=2e-13,max_step=20.,
189             temperature_offset=0.,freeze=False,initial=None,quadrature=False,fault=None,
↪ output_step=1.,verbose=False):
190     if n%20:raise ValueError('FV n must be divisible by 20 to sample requested radii
↪ without interpolation')
191     if end>env.knots[-1] and env.extension=='error':raise ValueError('End outside data
↪ range')
192     op=CoupledFV(n,p,temperature_offset,freeze,fault)
193     y=np.empty((n+1,2));y[:,0]=p.T0+temperature_offset;y[:,1]=p.C0
194     if initial is not None:y=np.array(initial(op.r),float);y[:,0]+=temperature_offset
195     times=np.unique(np.r_[np.arange(0,end+1e-9,output_step),end]);oi=np.arange(21)*(n//20)
196     out=np.empty((len(times),21,2));out[0]=y[oi];out[0, :,0]-=temperature_offset
197     average=np.empty((len(times),2));average[0]=op.w@y/op.v;average[0,0]-=temperature_off
↪ set
198     snaps=[y.copy()];st=[0.];stats={'n':n,'nodes':n+1,'dr_m':op.dr,'end_s':end,'rtol':rtol,
199             'atol_T':atol_T,'atol_C':atol_C,'max_step_s':max_step,'method':'BDF + exact coupled
↪ block sparse Jacobian',
200             'environment':env.kind,'extension':env.extension,'temperature_offset':temperature_o
↪ ffset,
201             'freeze':freeze,'fault':fault,'nfev':0,'njev':0,'nlu':0,'accepted_steps':0,'paramet
↪ ers':asdict(p)}
202     cuts=np.unique(np.r_[0.,env.knots[(env.knots>0)&(env.knots<end)],end])
203     tic=time.perf_counter();qint=jint=storeint=0.;balance=[];field_min=np.array([np.inf,n]
↪ p.inf]);field_max=-field_min
204     gx,gw=np.polynomial.legendre.leggauss(6)
205     atol=np.tile([atol_T,atol_C],n+1)
206     for a,b in zip(cuts[:-1],cuts[1:]):
207         fun=lambda t,z:op.evaluate(t,z,env)
208         jac=(lambda t,z:op.evaluate(t,z,env,True)) if fault is None else None

```

```

209 sol=solve_ivp(fun,(a,b),y.ravel(),method='BDF',jac=jac,jac_sparsity=op.sparsity()
    ↪ if fault else None,
210 rtol=rtol,atol=atol,max_step=max_step,dense_output=True,first_step=min(1e-3,.05
    ↪ *op.dr**2/2e-8,b-a))
211 if not sol.success:raise RuntimeError(f'FV failed n={n}, [{a},{b}]: {sol.message}')
212 if not np.isfinite(sol.y).all() or np.any(sol.y[1::2]<=0):raise
    ↪ FloatingPointError('Invalid accepted state')
213 for key in ('nfev','njev','nlu'):stats[key]+=int(getattr(sol,key))
214 stats['accepted_steps']+=len(sol.t)-1
215 states=sol.y.reshape(n+1,2,-1)
216 field_min=np.minimum(field_min,states.min(axis=(0,2)));field_max=np.maximum(field
    ↪ _max,states.max(axis=(0,2)))
217 mask=(times>a)&(times<=b+1e-8)
218 v=sol.sol(times[mask]).reshape(n+1,2,-1)
219 out[mask]=v[oi].transpose(2,0,1);out[mask,:,-1]-=temperature_offset
220 average[mask]=np.einsum('i,ijt->tj',op.w,v)/op.v;average[mask,0]-=temperature_off
    ↪ set
221 for t in (1800.,3600.,5400.,7200.,9000.,10800.):
222     if a<t<=b:snaps.append(sol.sol(t).reshape(n+1,2));st.append(t)
223 if quadrature:
224     for lo,hi,poly in zip(sol.t[:-1],sol.t[1:],sol.sol.interpolants):
225         tq=(lo+hi)/2+(hi-lo)/2*gx;ys=poly(tq).reshape(n+1,2,-1)
226         yd=dense_derivative(poly,tq).reshape(n+1,2,-1)
227         S=properties(ys[:,0]-temperature_offset,ys[:,1],freeze=freeze)[0]
228         boundary=env(tq)
229         qdot=p.R*p.hT*(ys[-1,0]-temperature_offset-boundary[:,0])/op.v
230         jdot=p.R*p.hm*(ys[-1,1]-boundary[:,1])/op.v
231         # Independently differentiate accepted interpolation; no flux cancellation
    ↪ RHS reuse.
232         storedot=np.einsum('i,it,it->t',op.w,S,yd[:,0])/op.v
233         weights=gw*(hi-lo)/2
234         qint+=float(qdot@weights);jint+=float(jdot@weights);storeint+=float(store
    ↪ dot@weights)
235         currentC=float(op.w@sol.y[1::2,-1]/op.v)
236         balance.append([b,currentC-average[0,1]+jint,storeint+qint,qint,storeint])
237     y=sol.y[:, -1].reshape(n+1,2)
238     if verbose and int(b)%1800==0:print(f'FV n={n}, t={b:g} s',flush=True)
239 stats['elapsed_s']=time.perf_counter()-tic
240 field_min[0]-=temperature_offset;field_max[0]-=temperature_offset
241 stats['accepted_internal_min']=field_min.tolist();stats['accepted_internal_max']=fiel
    ↪ d_max.tolist()
242 stats['environment_knots_restarted']=len(cuts)-1
243 stats['quadrature_performed']=quadrature
244 if balance:
245     ba=np.array(balance);stats['water_balance_max_kgkg']=float(np.max(abs(ba[:,1])))
246     stats['effective_heat_balance_max_J_m3']=float(np.max(abs(ba[:,2])))
247     stats['net_heat_input_J_m3']=-qint;stats['effective_heat_balance_relative']=float
    ↪ (np.max(abs(ba[:,2]))/max(abs(qint),1.))
248 return {'time_s':times,'radius_cm':np.linspace(0,2,21),'temperature_degC':out[:,0],
249         'moisture_dry_basis':out[:,1],'average_temperature_degC':average[:,0],
250         'average_moisture_dry_basis':average[:,1],'environment':env(times),'internal_radiu
    ↪ s_m':op.r,
251         'internal_weights_rdr':op.w,'snapshot_times_s':np.array(st),
252         'snapshots_temperature_degC':np.array(snaps)[:,-1,0]-temperature_offset,
253         'snapshots_moisture_dry_basis':np.array(snaps)[:,-1,1],'balance_records':np.array(b
    ↪ alance),'stats':stats}
254
255 def save_solution(path,sol):
256     path=Path(path);path.parent.mkdir(parents=True,exist_ok=True)
257     np.savez_compressed(path,**{k:v for k,v in sol.items() if k!='stats'})
258     path.with_suffix('.json').write_text(json.dumps(sol['stats'],ensure_ascii=False,inden
    ↪ t=2),encoding='utf-8')
259
260 def main():

```

```

261 base=Path(__file__).resolve().parents[1];ap=argparse.ArgumentParser()
262 ap.add_argument('--input',type=Path,default=base/'inputs/附件1.xlsx');ap.add_argument(
↳ '--out',type=Path,required=True)
263 ap.add_argument('--n',type=int,default=1280);ap.add_argument('--end',type=float,defau
↳ lt=10800.)
264 ap.add_argument('--rtol',type=float,default=2e-11);ap.add_argument('--max-step',type=
↳ float,default=20.)
265 ap.add_argument('--quadrature',action='store_true');ap.add_argument('--extension',cho
↳ ices=['error','hold_last'],default='error')
266 args=ap.parse_args();a,audit=load_environment(args.input);args.out.mkdir(parents=True
↳ ,exist_ok=True)
267 (args.out/'input_audit.json').write_text(json.dumps(audit,ensure_ascii=False,indent=2
↳ ),encoding='utf-8')
268 sol=solve_fv(Environment(a,extension=args.extension),n=args.n,end=args.end,rtol=args.
↳ rtol,max_step=args.max_step,quadrature=args.quadrature,verbose=True)
269 save_solution(args.out/'fv_n{args.n}.npz',sol);print(json.dumps(sol['stats'],indent=
↳ 2))
270 if __name__=='__main__':main()

```

A.17 prepare_reused

support/project/q2_refinement_delivery/source/prepare_reused.py

```

1 """Select unchanged inputs from the accepted handoff; never recompute a PDE."""
2 from pathlib import Path
3 import argparse,hashlib,json,shutil,zipfile
4 import numpy as np
5
6 def digest(p):
7     b=Path(p).read_bytes()
8     return {'bytes':len(b),'sha256':hashlib.sha256(b).hexdigest(),'git_blob_sha1':hashlib
↳ .sha1(b'blob '+str(len(b)).encode()+b'\0'+b).hexdigest()}
9
10 def main():
11     ap=argparse.ArgumentParser();ap.add_argument('--handoff',type=Path,required=True);ap.
↳ add_argument('--arrays_zip',type=Path,required=True);ap.add_argument('--out',type
↳ =Path,required=True);a=ap.parse_args()
12     a.out.mkdir(parents=True,exist_ok=True)
13     old=a.handoff/'q2_final_delivery';manifest=json.loads((old/'evidence/delivery_manifes
↳ t.json').read_text('utf-8-sig'))
14     entries={d['path']:d for d in manifest['files']}
15     reduced={};sources=[]
16     names=['baseline','hm_0.8','hm_1.2','hT_0.8','hT_1.2','pchip_all241','freeze_all_init
↳ ial']
17     with zipfile.ZipFile(a.arrays_zip) as z:
18         for name in names:
19             suffix=f'output/validation/sensitivity_{name}.npz'
20             candidate=[x for x in z.namelist() if x.endswith(suffix)]
21             if len(candidate)!=1:raise ValueError(f'Missing or ambiguous original array:
↳ {name}')
22             b=z.read(candidate[0]);sha=hashlib.sha256(b).hexdigest()
23             assert sha==entries[suffix]['sha256'],f'Original Pro array hash differs:
↳ {name}'
24             import io
25             with np.load(io.BytesIO(b),allow_pickle=False) as ar:
26                 assert np.array_equal(ar['time_s'],np.arange(10801))
27                 reduced['time_s']=ar['time_s'];reduced['radius_cm']=ar['radius_cm'][:,0,10
↳ ,20]]
28                 for f in ['temperature_degC','moisture_dry_basis']:
29                     reduced[f'{name}__{f}']=ar[f][:,[0,10,20]]
30                 for f in ['average_temperature_degC','average_moisture_dry_basis']:

```

```

31         reduced[f'{name}__{f}']=ar[f]
32         reduced[f'{name}__environment']=ar['environment']
33         sources.append({'name':name,'original_zip_member':candidate[0],'sha256':sha,'
    ↪ bytes':len(b),'original_Pro_manifest_match':True,'kind':'original Pro, not
    ↪ local review recomputation'})
34     np.savez_compressed(a.out/'sensitivity_tracks.npz',**reduced)
35     report={'source_archive':a.arrays_zip.name,**digest(a.arrays_zip),'projection':'Exact
    ↪ selection: r=0,1,2 cm; stored rdr averages; environment; all seconds. No
    ↪ interpolation, rounding or PDE recomputation.','source_arrays':sources,'projected'
    ↪ _file':digest(a.out/'sensitivity_tracks.npz')}}
36     (a.out/'sensitivity_provenance.json').write_text(json.dumps(report,ensure_ascii=False)
    ↪ ,indent=2),encoding='utf-8')
37     print('Selected and hash-verified',len(sources),'original Pro sensitivity arrays')
38 if __name__=='__main__':main()

```

A.18 q2_mechanism

support/project/q2_refinement_delivery/source/q2_mechanism.py

```

1  """Q2 post-acceptance diagnostics, using existing unrounded trajectories only.
2
3  No PDE solver is called. Array identities and 1/2/5-second derivative stencils
4  are checked. Figures use no manually selected colors and each has its own axes.
5  """
6  from __future__ import annotations
7  import argparse, csv, hashlib, json, platform, sys, zipfile
8  from pathlib import Path
9  import xml.etree.ElementTree as ET
10 import numpy as np
11
12 FIELDS=('temperature_degC','moisture_dry_basis')
13 POSITIONS=(0,10,20)
14 LABELS=('Axis, r = 0 cm','Interior, r = 1 cm','Surface, r = 2 cm')
15
16 def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
17
18 def json_write(p,obj):p.write_text(json.dumps(obj,ensure_ascii=False,indent=2,allow_nan=F
    ↪ alse),encoding='utf-8')
19
20 def environment_read(p):
21     """Read raw worksheet values with stdlib ZIP/XML; no workbook modifications."""
22     ns={'s':'http://schemas.openxmlformats.org/spreadsheetml/2006/main'}
23     with zipfile.ZipFile(p) as z:
24         if z.testzip():raise ValueError('XLSX CRC failure')
25         ss=[]
26         if 'xl/sharedStrings.xml' in z.namelist():
27             for e in ET.fromstring(z.read('xl/sharedStrings.xml')).findall('s:si',ns):
28                 ss.append(''.join(i.text or '' for i in e.iter({'+ns['s']+'}t'))))
29         wb=ET.fromstring(z.read('xl/workbook.xml'));sh=wb.find('s:sheets',ns)
30         if len(sh)!=1:raise ValueError('Unexpected sheets')
31         rid=sh[0].attrib['{http://schemas.openxmlformats.org/officeDocument/2006/relation
    ↪ ships}id']
32         target=next(x.attrib['Target'] for x in
    ↪ ET.fromstring(z.read('xl/_rels/workbook.xml.rels')) if x.attrib['Id']==rid)
33         target=target.lstrip('/') if target.startswith('/') else 'xl/'+target
34         rows=[]
35         for row in ET.fromstring(z.read(target)).findall('./s:sheetData/s:row',ns):
36             vals={}
37             for c in row.findall('s:c',ns):
38                 if c.find('s:f',ns) is not None:raise ValueError('Formula in raw input')
39                 v=c.find('s:v',ns);typ=c.attrib.get('t')
40                 if typ=='s': val=ss[int(v.text)]

```

```

41         elif typ=='inlineStr':val=''.join(e.text or '' for e in
42             ↪ c.findall('.//s:t',ns))
43         else:val=float(v.text) if v is not None else None
44         vals[''.join(x for x in c.attrib['r'] if x.isalpha())]=val
45         rows.append([vals.get(c) for c in 'ABC'])
46     assert rows[0]==['时间','温度','水分浓度']
47     a=np.asarray(rows[1:],float)
48     assert a.shape==(241,3) and np.isfinite(a).all() and
49     ↪ np.array_equal(a[:,0],np.arange(0,14401,60))
50     return a
51
52 def d_piecewise(y,h=1):
53     """Fourth-order finite differences, not crossing the 60-s input knots.
54     Central [-2h,-h,0,h,2h] away from knots, otherwise 5-point forward/backward.
55     At a knot select the right segment; at the final endpoint select the left.
56     Early 0..9 s values are not trusted as instantaneous-rate estimates.
57     """
58     n=y.shape[0];t=np.arange(n);last=n-1
59     start=(t//60)*60;end=np.minimum(start+60,last)
60     start[-1]=max(0,last-60)
61     central=(t-2*h>=start)&(t+2*h<=end)
62     forward=(~central)&(t+4*h<=end)
63     backward=~central&~forward
64     out=np.zeros_like(y)
65     stencils=[(central,np.array([-2,-1,0,1,2]),np.array([1,-8,0,8,-1])/12),
66               (forward,np.arange(5),np.array([-25,48,-36,16,-3])/12),
67               (backward,-np.arange(5),-np.array([-25,48,-36,16,-3])/12)]
68     for mask,steps,weights in stencils:
69         idx=t[mask,None]+steps[None,:]*h
70         # Subtract anchor before summation to reduce cancellation of constants.
71         out[mask]=np.einsum('ijk,j->ik',y[idx]-y[t[mask],None,:],weights)/h
72     return out
73
74 def run(out,raw,sens_path,env_path,validation_path,figures=True):
75     out=Path(out);(out/'data').mkdir(parents=True,exist_ok=True);(out/'figures').mkdir(ex
76     ↪ ist_ok=True)
77     with np.load(raw,allow_pickle=False) as z:a={k:z[k] for k in z.files}
78     t=a['time_s'];T=a[FIELDS[0]][:,:POSITIONS];C=a[FIELDS[1]][:,:POSITIONS];Tk=T+273.15
79     assert np.array_equal(t,np.arange(10801)) and np.isfinite(T).all() and np.all(C>0)
80     env=environment_read(env_path)
81     np.testing.assert_allclose(a['environment'],np.column_stack([np.interp(t,env[:,0],env
82     ↪[:,j]) for j in (1,2)]),rtol=0,atol=0)
83     np.savetxt(out/'data/environment_original.csv',env,delimiter=',',fmt='%0.17g',header='
84     ↪ time_s,temperature_degC,environmental_quantity_kgkg_basis_unspecified',comments='
85     ↪ ')
86     D=.0024*np.exp(-.45/C)*np.exp(-3850/Tk);D0=float(.0024*np.exp(-.45/2.55)*np.exp(-3850
87     ↪ /301.15))
88     H=3850*(1/301.15-1/Tk);M=.45*(1/2.55-1/C);lnratio=np.log(D/D0)
89     resid=float(np.max(abs(lnratio-H-M)))
90     assert resid<2e-13
91     derived={};ratecheck={}
92     for h in [1,2,5]:
93         td=d_piecewise(T,h);cd=d_piecewise(C,h)
94         hd=3850/Tk**2*td;md=.45/C**2*cd
95         ld=d_piecewise(lnratio,h)
96         derived[h]=(td,cd,hd,md,hd+md,ld)
97     trustworthy=(t>=10)&(t<=10790)
98     # Additional conservative uncertainty band: all three step widths must agree.
99     r1=derived[1][4];err=np.maximum(abs(r1-derived[2][4]),abs(r1-derived[5][4]))
100     eps=np.maximum(5*err,1e-10)
101     sign=np.where(r1>eps,1,np.where(r1<-eps,-1,0));sign[~trustworthy]=0
102     ratecheck={'method':'piecewise fourth-order finite differences, 1/2/5 s stencils, no
103     ↪ crossing 60 s knots; right derivative at knots; not a PDE RHS.'},

```

```

96     'early_rate_not_interpreted_s':[0,9], 't0_surface_rate_note':'Initial Robin corner
    ↳ produces a boundary layer; no finite instantaneous derivative is inferred from
    ↳ t=0.',
97     'step_1_vs_2_max_abs_s-1_after10':float(np.max(abs(r1-derived[2][4])[trustworthy])),
98     'step_1_vs_5_max_abs_s-1_after10':float(np.max(abs(r1-derived[5][4])[trustworthy])),
99     'step_1_vs_5_max_abs_s-1_after60':float(np.max(abs(r1-derived[5][4])[t>=60])),
100     'chain_rule_vs_direct_log_derivative_after10_s-1':float(np.max(abs(r1-derived[1][5]
    ↳ )[trustworthy])),
101     'sign_certification':'Pointwise |rate| > max(5*max(step differences),1e-10 s^-1).
    ↳ Diagnostic, not a formal error bound.'}
102 columns=['time_s','radius_cm','temperature_degC','C_kgkg_dry','D_m2_s','H_log','M_log'
    ↳ ','D_over_D0','T_rate_K_s','C_rate_kgkg_s','H_rate_s-1','M_rate_s-1','lnD_rate_s-
    ↳ 1','lnD_rate_step_error_s-1','net_sign_certified','rate_interpretable']
103 rows=[]
104 for j,p in enumerate(POSITIONS):
105     rows.append(np.column_stack([t,np.full_like(t,p/10),T[:,j],C[:,j],D[:,j],H[:,j],M
    ↳[:,j],D[:,j]/D0,
106         *(derived[1][k][:,j] for k in
    ↳ range(5)),err[:,j],sign[:,j],trustworthy.astype(int)]))
107 mat=np.concatenate(rows);np.savetxt(out/'data/mechanism_timeseries.csv',mat,delimiter
    ↳=',',fmt='%17g',header=''.join(columns),comments='')
108 np.savez_compressed(out/'data/mechanism_unrounded.npz',time_s=t,radius_cm=np.array([0
    ↳,1,2.]),T=T,C=C,D=D,H=H,M=M,D_over_D0=D/D0,
109     rate_h1=r1,rate_h2=derived[2][4],rate_h5=derived[5][4],H_rate=derived[1][2],M_rat
    ↳e=e=derived[1][3],rate_step_difference=err,rate_interpretable=trustworthy)
110 intervals=[];summary=[];table=[]
111 for j,p in enumerate(POSITIONS):
112     imax=int(np.argmax(D[:,j]));imin=int(np.argmin(D[:,j]));up=np.flatnonzero((lnratio
    ↳[:,j]<0)&(lnratio[1:,j]>=0))
113     back=float(t[up[0]]-lnratio[up[0],j]/(lnratio[up[0]+1,j]-lnratio[up[0],j])) if
    ↳ len(up) else None
114     d={'radius_cm':p/10,'H_3h':float(H[-1,j]),'M_3h':float(M[-1,j]),'net_log_3h':floo
    ↳t(t(lnratio[-1,j]),'D_3h':float(D[-1,j]),'D_ratio_3h':float(D[-1,j]/D0),
115     'global_sample_min_s':imin,'D_ratio_min':float(D[imin,j]/D0),'global_sample_ma
    ↳x_s':imax,'D_ratio_max':float(D[imax,j]/D0),'return_to_initial_D_s_linear_
    ↳ between_samples':back,
116     'last_half_hour_delta_H':float(H[-1,j]-H[9000,j]),'last_half_hour_delta_M':flo
    ↳at(M[-1,j]-M[9000,j]),'last_half_hour_D_relative_change':float(D[-1,j]/D[9
    ↳000,j]-1),
117     'rate_positive_time_fraction_2to3h':float(np.mean(sign[7200:10800,j]>0)), 'rate
    ↳_negative_time_fraction_2to3h':float(np.mean(sign[7200:10800,j]<0)),
118     'temperature_cooling_seconds_2to3h':int(np.sum(derived[1][0][7200:10800,j]<-1e
    ↳-9))}
119     summary.append(d)
120     for ti in range(0,10801,1800):table.append([ti/3600,p/10,T[ti,j],C[ti,j],H[ti,j],
    ↳ M[ti,j],D[ti,j],D[ti,j]/D0])
121     for lo in range(0,10800,60):
122         dh=H[lo+60,j]-H[lo,j];dm=M[lo+60,j]-M[lo,j]
123         intervals.append([lo,lo+60,p/10,dh,dm,dh+dm,dh/60,dm/60,(dh+dm)/60])
124 np.savetxt(out/'data/mechanism_table.csv',table,delimiter=',',fmt='%17g',header='tim
    ↳e_s,radius_cm,T_degC,C_kgkg,H,M,D_m2_s,D_over_D0',comments='')
125 np.savetxt(out/'data/interval_60s_contributions.csv',intervals,delimiter=',',fmt='%1
    ↳7g',header='start_s,end_s,radius_cm,delta_H,delta_M,delta_logD,mean_H_rate_s-1,me
    ↳an_M_rate_s-1,mean_logD_rate_s-1',comments='')
126 # Equilibrium mapping diagnostics keep the sampled state fixed; no extra PDE scenario.
127 b=a['environment'][:,1];cs=C[:,2];q=8e-7*(cs-b);eb=-b/(cs-b)
128 flux=np.column_stack([t,cs,b,cs-b,q,eb])
129 np.savetxt(out/'data/boundary_identifiability_diagnostic.csv',flux,delimiter=',',fmt=
    ↳ '%17g',header='time_s,C_surface,b_equivalent,driving_gap,effective_flux_m_s,elas
    ↳ ticity_to_b_at_fixed_state',comments='')
130 # Reuse same-grid ±20% trajectories; these are finite secants, not fitted
    ↳ uncertainties.
131 with np.load(sens_path,allow_pickle=False) as zz:s={k:zz[k] for k in zz.files}
132 assert np.array_equal(s['time_s'],t)

```

```

133 sr=[];elastic={};at_end=[]
134 for par in ['hm','hT']:
135     low,hi=par+'_0.8',par+'_1.2'
136     Cb=np.column_stack([s['baseline__moisture_dry_basis'],s['baseline__average_moisture_dry_basis']])
137     Tb=np.column_stack([s['baseline__temperature_degC'],s['baseline__average_temperature_degC']])
138     Cl=np.column_stack([s[low+'__moisture_dry_basis'],s[low+'__average_moisture_dry_basis']])
139     Ch=np.column_stack([s[hi+'__moisture_dry_basis'],s[hi+'__average_moisture_dry_basis']])
140     Tl=np.column_stack([s[low+'__temperature_degC'],s[low+'__average_temperature_degC']])
141     Th=np.column_stack([s[hi+'__temperature_degC'],s[hi+'__average_temperature_degC']])
142     ec=(Ch-Cl)/(.4*Cb);et=np.divide(Th-Tl,.4*(Tb-28),out=np.full_like(Tb,np.nan),where=(Tb-28)>=.05)
143     elastic[par]=ec
144     for j,loc in enumerate(['axis','r1cm','surface','rdr_mean']):
145         for ti in range(0,len(t)):
146             sr.append([par,loc,int(ti),Cb[ti,j],Cl[ti,j],Ch[ti,j],ec[ti,j],Tb[ti,j],Tl[ti,j],Th[ti,j],et[ti,j] if np.isfinite(et[ti,j]) else ''])
147             at_end.append({'parameter':par,'location':loc,'C_minus20':float(Cl[-1,j]),'C_plus20':float(Ch[-1,j]),'C_secant_elasticity':float(ec[-1,j]),
148                             'T_minus20':float(Tl[-1,j]),'T_base':float(Tb[-1,j]),'T_plus20':float(Th[-1,j]),'T_rise_secant_elasticity':float(et[-1,j]),
149                             'maximum_abs_C_change':float(max(np.max(abs(Cl[:,j]-Cb[:,j])),np.max(abs(Ch[:,j]-Cb[:,j])))),
150                             'maximum_abs_T_change':float(max(np.max(abs(Tl[:,j]-Tb[:,j])),np.max(abs(Th[:,j]-Tb[:,j]))))})
151 with (out/'data/parameter_sensitivity_timeseries.csv').open('w',newline='',encoding='utf-8') as f:
152     w=csv.writer(f);w.writerow(['parameter','location','time_s','C_base','C_minus20','C_plus20','C_secant_elasticity','T_base','T_minus20','T_plus20','T_rise_secant_elasticity_blank_if_rise_lt_0.05K']);w.writerows(sr)
153 with (out/'data/parameter_sensitivity_3h.csv').open('w',newline='',encoding='utf-8') as f:
154     w=csv.DictWriter(f,fieldnames=list(at_end[0]));w.writeheader();w.writerows(at_end)
155 # Empirical-density interpretation contradiction, not a new physical density estimate.
156 rho=650+128*C;rho_d_implied=rho/(1+C);rho_d0=(650+128*2.55)/(1+2.55)
157 volume=np.pi*.02**2*.25
158 val=json.loads(Path(validation_path).read_text('utf-8-sig'))
159 # Display only symbolic scaling for latent heat: energy = lambda * conditional lost mass.
160 mloss_conditional=rho_d0*volume*(2.55-a['average_moisture_dry_basis'][-1])
161 tail=env[env[:,0]>=10800]
162 time_mean=np.trapezoid(tail[:,1:],tail[:,0],axis=0)/3600
163 report={'scope':'New postprocessing only; no 3-hour PDE rerun and no corrected physical prediction.',
164         'source_hashes':{'accepted_npz':sha(raw),'sensitivity_projection':sha(sens_path),'raw_environment':sha(env_path),'reused_validation':sha(validation_path)},
165         'D_initial_m2_s':D0,'identity_max_abs_log_error':resid,'locations':summary,'derivative_check':ratecheck,'sensitivity_3h':at_end,
166         'boundary_fixed_state':{'q0_effective_m_s':float(q[0]),'q3h_effective_m_s':float(q[-1]),'b_elasticity_initial':float(eb[0]),'b_elasticity_3h':float(eb[-1]),'no_boundary_mapping_PDE_scenarios':True},
167         'density_diagnostic':{'interpretation':'Only if empirical rho is actual wet density, which is not the accepted static fixed-volume model.',
168                             'implied_rho_d_initial':rho_d0,'implied_rho_d_3h_0_1_2cm':rho_d_implied[-1].tolist(),
169                             'implied_ratio_to_initial_3h':(rho_d_implied[-1]/rho_d0).tolist(),
170                             'conditional_lost_water_kg_reference_dry_density':float(mloss_conditional),
171                             'reused_effective_heat_input_J':float(val['balance']['net_heat_input_J_m3']*volume),

```

```

172     'latent_equal_effective_heat_input_at_lambda_J_kg':float(val['balance']['net_hea
    ↪ t_input_J_m3']*volume/mloss_conditional),
173     'no_lambda_assumed_and_no_latent_PDE_run':True},
174     'time_scope':{'raw_data_s':[0,14400],'paper_s':[1800,3600,5400,7200,9000,10800],'e
    ↪ xcel_s':[1,10800],
175     'tail_sample_mean_3to4h':tail[:,1:].mean(axis=0).tolist(),'tail_time_mean_linear_
    ↪ 3to4h':time_mean.tolist(),'last_environment_value':env[-1,1:].tolist(),
176     'no_days_long_solution_or_threshold_in_this_review':True},
177     'response_3h':{'environment_T':float(a['environment'][-1,0]),'radial_T_span_K':flo
    ↪ at(np.ptp(T[-1])),
178     'max_T_lag_to_current_air_K':float(np.max(abs(T[-1]-a['environment'][-1,0])),
179     'radial_C_span_kgkg':float(np.ptp(C[-1])), 'remaining_effective_water_fraction':fl
    ↪ oat(a['average_moisture_dry_basis'][-1]/2.55)},
180     'runtime':{'python':sys.version,'numpy':np.__version__,'platform':platform.platfor
    ↪ m()})
181 json_write(out/'data/mechanism_validation.json',report)
182 if figures:draw(out,t,T,C,D,D0,H,M,derived,elastic,eb)
183 print(json.dumps({'identity_error':resid,'locations':summary,'derivatives':ratecheck,
    ↪ 'response_3h':report['response_3h']},ensure_ascii=False,indent=2))
184 return report
185
186 def draw(out,t,T,C,D,D0,H,M,derived,elastic,eb):
187     import matplotlib
188     matplotlib.use('Agg')
189     import matplotlib.pyplot as plt
190     plt.rcParams.update({'font.size':11,'axes.titlesize':13,'legend.fontsize':9,'svg.font
    ↪ size':11})
191     def finish(fig,ax,name,title,ylabel):
192         ax.set(xlabel='Time / h',ylabel=ylabel,title=title,xlim=(0,3));ax.grid(True,alpha
    ↪ =.25);ax.legend(loc='best');fig.tight_layout()
193         fig.savefig(out/'figures'/f'{name}.png',dpi=300);fig.savefig(out/'figures'/f'{nam
    ↪ e}.svg');plt.close(fig)
194     tx=t/3600
195     fig,ax=plt.subplots(figsize=(8.2,4.7))
196     for j in range(3):ax.plot(tx[:,5],(D[:,j]/D0)[:,5],label=LABELS[j])
197     ax.axhline(1,ls=':',label='Initial diffusivity')
198     finish(fig,ax,'01_D_competition','Local diffusivity: heating versus moisture
    ↪ loss','$D(t,r)/D_0$')
199     for j in range(3):
200         fig,ax=plt.subplots(figsize=(8.2,4.7));ax.plot(tx[:,5],H[:,5,j],label='Heating log
    ↪ contribution H')
201         ax.plot(tx[:,5],M[:,5,j],label='Moisture log contribution
    ↪ M',ls='--');ax.plot(tx[:,5],(H+M)[:,5,j],label='Net ln(D / D0)',ls='-.')
202         finish(fig,ax,f'0{j+2}_log_contributions','Cumulative contributions |
    ↪ '+LABELS[j],'Dimensionless log contribution')
203     for j in range(3):
204         fig,ax=plt.subplots(figsize=(8.2,4.7));mask=(t>=10)&(t<=10790)
205         tt=tx[mask][:,5]
206         ax.plot(tt,derived[1][2][mask,j][:,5]*3600,label='Thermal term (may become
    ↪ negative)')
207         ax.plot(tt,derived[1][3][mask,j][:,5]*3600,label='Moisture-loss term',ls='--')
208         ax.plot(tt,derived[1][4][mask,j][:,5]*3600,label='Net local rate',ls='-.')
209         finish(fig,ax,f'0{j+5}_instantaneous_rates','Instantaneous-rate diagnostic |
    ↪ '+LABELS[j],'Rate of ln(D) / h$^{-1}$')
210     for q,par in enumerate(['hm','hT']):
211         fig,ax=plt.subplots(figsize=(8.2,4.7))
212         for j,loc in enumerate(['Axis','r = 1 cm','Surface','r dr-weighted
    ↪ mean']):ax.plot(tx[:,5],elastic[par][:,5,j],label=loc)
213         finish(fig,ax,f'0{q+8}_sensitivity_{par}',f'Moisture response to {par}: same-grid
    ↪ +/-20% scenarios','Finite-secant normalized sensitivity')
214     fig,ax=plt.subplots(figsize=(8.2,4.7));ax.plot(tx[:,5],eb[:,5],label='b sensitivity at
    ↪ fixed simulated surface C');ax.axhline(0,ls=':')
215     finish(fig,ax,'10_mapping_flux_diagnostic','Boundary-input sensitivity: algebraic
    ↪ diagnostic, not a new PDE','d ln(q) / d ln(b), holding surface C fixed')

```

```

216     json_write(out/'figures/manifest.json',{'pairs':10,'format':['PNG 300
    ↪ dpi','SVG'],'source':'q2_mechanism.py; accepted unrounded data
    ↪ only','files':[p.name for p in sorted((out/'figures').glob('*')) if p.suffix in
    ↪ ('.png','.svg')])})
217
218 if __name__=='__main__':
219     root=Path(__file__).resolve().parents[1];p=argparse.ArgumentParser(description=__doc__)
220     p.add_argument('--out',type=Path,default=root);p.add_argument('--raw',type=Path,default=
    ↪ root/'reused/q2_final_delivery/output/q2_unrounded.npz')
221     p.add_argument('--sensitivity',type=Path,default=root/'reused/sensitivity_tracks.npz')
    ↪ );p.add_argument('--environment',type=Path,default=root/'reused/q2_final_delivery
    ↪ /inputs/附件1.xlsx')
222     p.add_argument('--validation',type=Path,default=root/'reused/q2_final_delivery/output
    ↪ /validation/validation.json');p.add_argument('--no-figures',action='store_true')
223     a=p.parse_args();run(a.out,a.raw,a.sensitivity,a.environment,a.validation,not
    ↪ a.no_figures)

```

A.19 第二三问全程统一轨迹

A.20 q3_solver

support/project/q4_complete_delivery/v1/q123_closeout/source/legacy_q3/q3_solver.py

```

1  """Q3: independently assembled conservative graph FV on graded cylindrical meshes.
2  SI units; T in Celsius except Arrhenius; C is dry-basis. No clipping or D floor.
3  Both actual early boundary knots and the future scenario are explicit.
4  """
5  from __future__ import annotations
6  import argparse, json, sys, sys, hashlib, platform, zipfile
7  from pathlib import Path
8  from dataclasses import dataclass, asdict
9  import xml.etree.ElementTree as ET
10 import numpy as np
11 import scipy
12 from scipy.integrate import solve_ivp
13 from scipy.interpolate import PchipInterpolator
14 from scipy.special import exp1
15 from scipy.optimize import brentq
16 from scipy.sparse import coo_matrix
17
18 NS={'s':'http://schemas.openxmlformats.org/spreadsheetml/2006/main'}
19
20 def read_xlsx(path):
21     """Read physical OOXML (not preview text), retaining numeric/string identity."""
22     with zipfile.ZipFile(path) as z:
23         if z.testzip(): raise ValueError('Corrupt xlsx')
24         strings=[]
25         if 'xl/sharedStrings.xml' in z.namelist():
26             for v in ET.fromstring(z.read('xl/sharedStrings.xml')).findall('s:si',NS):
27                 strings.append(''.join(t.text or '' for t in v.findall('..s:t',NS)))
28             rel={v.attrib['Id']:v.attrib['Target'] for v in
    ↪ ET.fromstring(z.read('xl/_rels/workbook.xml.rels'))}
29         out={}
30         for sh in ET.fromstring(z.read('xl/workbook.xml')).findall('s:sheets/s:sheet',NS):
31             rid=sh.attrib['{http://schemas.openxmlformats.org/officeDocument/2006/relatio
    ↪ nships}id']
32             target=rel[rid]; target=target[1:] if target.startswith('/') else 'xl/'+target
33             cells={}
34             for c in ET.fromstring(z.read(target)).findall('..s:sheetData/s:row/s:c',NS):
35                 kind=c.attrib.get('t','n');v=c.find('s:v',NS)
36                 if c.find('s:f',NS) is not None: raise ValueError('Unexpected formula')

```

```

37         if kind=='s': value=strings[int(v.text)]
38         elif kind=='inlineStr': value=''.join(t.text or '' for t in
    ↪ c.findall('.//s:t',NS))
39         elif v is None: value=None
40         elif kind in ('str','e'): value=v.text
41         else: value=float(v.text)
42         cells[c.attrib['r']]={'value':value,'type':kind,'style':int(c.attrib.get('
    ↪ 's','0'))}
43     out[sh.attrib['name']]=cells
44     return out
45
46 def load_input(path):
47     cells=read_xlsx(path)['Sheet1']
48     assert [cells[f'{c}1']['value'] for c in 'ABC']==['时间','温度','水分浓度']
49     a=np.array([[cells[f'{c}{i}']['value'] for c in 'ABC'] for i in range(2,243)],float)
50     assert a.shape==(241,3) and np.isfinite(a).all()
51     assert np.array_equal(a[:,0],np.arange(241)*60.)
52     return a
53
54 class Environment:
55     def __init__(self,data,scenario='mean',kind='linear'):
56         self.data=np.asarray(data,float);self.scenario=scenario;self.kind=kind
57         if scenario=='mean':self.future=self.data[self.data[:,0]>=10800,1:].mean(axis=0)
58         elif scenario=='last':self.future=self.data[-1,1:].copy()
59         elif scenario=='nominal':self.future=np.array([50.,.05])
60         else:raise ValueError(scenario)
61         self.interp=PchipInterpolator(self.data[:,0],self.data[:,1:],axis=0) if
    ↪ kind=='pchip' else None
62     def __call__(self,t):
63         t=np.asarray(t);v=(self.interp(t) if self.interp is not None else
    ↪ np.stack([np.interp(t,self.data[:,0],self.data[:,k]) for k in (1,2)],axis=-1))
64         return np.where((t>14400)[...],None,self.future,v)
65     def segment(self,a,b):
66         # One-sided boundary at t=14400: no spurious jump in the pre-4h segment.
67         if a>=14400:return lambda t: np.broadcast_to(self.future,np.shape(t)+(2,))
68         return self
69
70 def props(T,C):
71     if np.any(C<=0) or np.any(T<=-273.15):raise FloatingPointError('Nonpositive
    ↪ accepted/trial C or Kelvin temperature')
72     rho=650.+128*C;cp=1450.+2736*C/(1+C)
73     S=rho*cp;k=.21+.38*C/(1+C)
74     Sc=128*cp+rho*2736/(1+C)**2;kc=.38/(1+C)**2
75     A=.0024*np.exp(-3850/(T+273.15));D=A*np.exp(-.45/C)
76     return S,k,D,Sc,kc
77
78 def F(C):
79     C=np.asarray(C)
80     if np.any(C<=0):raise FloatingPointError('F requires C>0')
81     return C*np.exp(-.45/C)-.45*exp1(.45/C)
82
83 def Fdiff(a,b):
84     """Stable primitive difference. 4-point Gaussian integration when close.
85     The switch is numerical evaluation only; no modification of diffusivity.
86     """
87     a,b=np.broadcast_arrays(a,b);mid=(a+b)/2;delta=a-b
88     close=np.abs(delta)<.002*mid
89     ans=np.empty_like(a)
90     if np.any(~close):ans[~close]=F(a[~close])-F(b[~close])
91     if np.any(close):
92         m=mid[close];d=delta[close]
93         v=np.zeros_like(m)

```

```

94         for x,w in
95             ↪ ((-.8611363115940526,.3478548451374539),(-.3399810435848563,.6521451548625461))
96             ↪ (,.3399810435848563,.6521451548625461),(.8611363115940526,.3478548451374539)):
97                 v+=w*np.exp(-.45/(m+d*x/2))
98         ans[close]=d*v/2
99     return ans
100
101 @dataclass(frozen=True)
102 class Config:
103     n:int=256
104     nz:int=0
105     grading:float=2.
106     zgrading:float=2.
107     flux:str='integral'
108     scenario:str='mean'
109     interpolation:str='linear'
110     hT:float=25.
111     hm:float=8e-7
112     end_factor:float=1.
113     rtol:float=2e-9
114     atol_T:float=2e-10
115     atol_C:float=2e-12
116     max_step:float=240.
117     early_step:float=30.
118     method:str='BDF'
119     end_s:float=360000.
120     isothermal_after:float=0.
121     quadrature:bool=False
122     full_samples:bool=False
123     execution_time_s:float=0.
124
125 class Operator:
126     def __init__(self,cfg:Config):
127         self.cfg=cfg;self.R=.02;self.Lhalf=.125
128         self.r=self.R*(1-(1-np.linspace(0,1,cfg.n+1))**cfg.grading)
129         rf=np.r_[0,(self.r[:-1]+self.r[1:])/2,self.R];wr=np.diff(rf**2)/2
130         if cfg.nz:
131             self.z=self.Lhalf*(1-(1-np.linspace(0,1,cfg.nz+1))**cfg.zgrading)
132             zf=np.r_[0,(self.z[:-1]+self.z[1:])/2,self.Lhalf];wz=np.diff(zf)
133         else:self.z=np.array([0.]);wz=np.ones(1)
134         self.shape=(len(self.r),len(self.z)); self.size=np.prod(self.shape).item()
135         ids=np.arange(self.size).reshape(self.shape)
136         self.w=(wr[:,None]*wz).ravel();self.vol=self.w.sum()
137         l=[ids[:-1].ravel()];r=[ids[1:].ravel()]
138         ge=[((rf[1:-1]/np.diff(self.r))[:,None]*wz).ravel()]
139         if cfg.nz:
140             l.append(ids[:, :-1].ravel());r.append(ids[:, 1:].ravel())
141             ge.append((wr[:,None]/np.diff(self.z)).ravel())
142         self.left=np.concatenate(l);self.right=np.concatenate(r);self.geo=np.concatenate(
143             ↪ ge)
144         self.bound=np.zeros(self.shape);self.bound[-1]+=self.R*wz
145         if cfg.nz:self.bound[:, -1]+=cfg.end_factor*wr
146         self.bound=self.bound.ravel()
147         # Fixed sparse block pattern, row divided by its own storage.
148         rows=[];cols=[]; edgeid=[]; signs=[]
149         for dest,sgn in ((self.left,-1.),(self.right,1.)):
150             for src in (self.left,self.right):
151                 for a in range(2):
152                     for b in range(2):
153                         rows.append(2*dest+a);cols.append(2*src+b)
154         self.jrows=np.r_[np.concatenate(rows),2*np.arange(self.size),2*np.arange(self.siz
155             ↪ e),2*np.arange(self.size)+1]
156         self.jcols=np.r_[np.concatenate(cols),2*np.arange(self.size)+1,2*np.arange(self.s
157             ↪ ize),2*np.arange(self.size)+1]

```

```

153 def rhs(self,t,y,env,jac=False):
154     q=y.reshape(self.size,2);T=q[:,0];C=q[:,1]
155     S,k,D,Sc,kc=props(T,C);l=self.left;r=self.right;g=self.geo
156     kl=k[l];kr=k[r];ks=kl+kr;kf=2*kl*kr/ks
157     dT=T[l]-T[r];dC=C[l]-C[r]
158     tq=g*kf*dT
159     if self.cfg.flux=='integral':
160         Tf=(T[l]+T[r])/2;Af=.0024*np.exp(-3850/(Tf+273.15));at=3850/(Tf+273.15)**2/2
161         mf=g*Af*Fdiff(C[l],C[r])
162         mcl=g*Af*np.exp(-.45/C[l]);mcr=-g*Af*np.exp(-.45/C[r])
163         mtl=mf*at;mtr=mtl
164     else:
165         Dl=D[l];Dr=D[r]
166         if self.cfg.flux=='harmonic':
167             ds=Dl+Dr;df=2*Dl*Dr/ds;fl=2*(Dr/ds)**2;fr=2*(Dl/ds)**2
168         elif self.cfg.flux=='arithmetic':df=(Dl+Dr)/2;fl=fr=.5
169         else:raise ValueError(self.cfg.flux)
170         mf=g*df*dC
171         mcl=g*(df+fl*Dl*.45/C[l]**2*dC);mcr=g*(-df+fr*Dr*.45/C[r]**2*dC)
172         mtl=g*fl*Dl*3850/(T[l]+273.15)**2*dC;mtr=g*fr*Dr*3850/(T[r]+273.15)**2*dC
173     rhs=np.empty_like(q)
174     rhs[:,0]=np.bincount(r,weights=tq,minlength=self.size)-np.bincount(l,weights=tq,m
    ↪ inlength=self.size)
175     rhs[:,1]=np.bincount(r,weights=mf,minlength=self.size)-np.bincount(l,weights=mf,m
    ↪ inlength=self.size)
176     et,ec=env(t);rhs[:,0]=-self.bound*self.cfg.hT*(T-et);rhs[:,1]=-self.bound*self.cf
    ↪ g.hm*(C-ec)
177     storage=self.w[:,None]*np.column_stack([S,np.ones(self.size)])
178     rhs/=storage
179     iso=self.cfg.isothermal_after>0 and t>=self.cfg.isothermal_after
180     if iso:rhs[:,0]=0
181     if not jac:return rhs.ravel()
182     bl=np.empty((len(l),2,2));br=np.empty_like(bl)
183     bl[:,0,0]=g*kf;br[:,0,0]=-g*kf
184     bl[:,0,1]=g*2*(kr/ks)**2*kc[l]*dT;br[:,0,1]=g*2*(kl/ks)**2*kc[r]*dT
185     bl[:,1,0]=mtl;br[:,1,0]=mtr;bl[:,1,1]=mcl;br[:,1,1]=mcr
186     if iso:bl[:,0,:]=0;br[:,0,:]=0
187     vals=[]
188     for dest,sgn in ((l,-1.),(r,1.)):
189         for deriv in (bl,br):
190             for a in range(2):
191                 for b in range(2):vals.append(sgn*deriv[:,a,b]/storage[dest,a])
192     vals.extend([-rhs[:,0]*Sc/S,-self.bound*self.cfg.hT/storage[:,0]*(0 if iso else
    ↪ 1),-self.bound*self.cfg.hm/self.w])
193     return coo_matrix((np.concatenate(vals),(self.jrows,self.jcols)),shape=(2*self.si
    ↪ ze,2*self.size)).tocsc()
194 def profile(self,y):
195     field=np.asarray(y).reshape(self.shape+(2,))[:,0,:]
196     return PchipInterpolator(self.r**2,field,axis=0)(np.linspace(0,self.R,21)**2)
197
198
199 def derivative_dense(poly,t):
200     if hasattr(poly,'D'):
201         x=(t-poly.t_shift)/poly.denom;prod=1.;der=0.;ans=np.zeros(poly.D.shape[1])
202         for j in range(poly.order):
203             der=der*x[j]+prod/poly.denom[j];prod*=x[j];ans+=poly.D[j+1]*der
204         return ans
205     # Radau dense polynomial y0 + Q*[x,x^2,...]
206     x=(t-poly.t_old)/poly.h
207     return poly.Q@((np.arange(poly.order+1)+1)*x**np.arange(poly.order+1))/poly.h
208
209
210 def solve(cfg:Config,path:Path,input_path:Path):

```

```

211 if path.with_suffix('.json').exists(): raise FileExistsError(f'Refusing overwrite:
    ↳ {path}')
212 data=load_input(input_path);env=Environment(data,cfg.scenario,cfg.interpolation);op=0
    ↳ perator(cfg)
213 y=np.tile([28.,2.55],op.size);times=[0.];samples=[op.profile(y)];snapt=[0.];snaps=[y.
    ↳ copy()]
214 fulls=[y.copy()] if cfg.full_samples else []
215 event_times=[];event_fields=[]
216 diagnostics=[];balances=[];stats={'config':asdict(cfg),'python':sys.version,'numpy':n
    ↳ p.__version__, 'scipy':scipy.__version__, 'future':env.future.tolist(),'nodes':op.s
    ↳ ize,'min_dr_m':float(np.diff(op.r).min()),'max_dr_m':float(np.diff(op.r).max()),'
    ↳ nfev':0,'njev':0,'nlu':0,'steps':0}
217 cuts=np.unique(np.r_[data[:,0],np.arange(21600.,cfg.end_s,21600.),cfg.end_s,([cfg.iso
    ↳ thermal_after] if cfg.isothermal_after else [])]);cuts=cuts[cuts<=cfg.end_s]
218 tic=time.perf_counter();quad=np.polynomial.legendre.leggauss(4);qint=jint=stored=0.;e
    ↳ vent=None;projections=[]
219 minC=2.55;maxC=2.55;minT=28.;maxT=28.;max_up=0.;max_radial=0.;max_axial=0.;max_argmax=0
220 root_slope=0.;surface_event=None;mean_event=None
221 nextout=60.
222 for a,b in zip(cuts[:-1],cuts[1:]):
223     if cfg.isothermal_after and a==cfg.isothermal_after:
224         dy=np.max(abs(y[:2]-env.future[0]));projections.append({'t_s':float(a),'max_
            ↳ temperature_projection_K':float(dy)})
225         y[:2]=env.future[0]
226         e=env.segment(a,b)
227         fun=lambda t,v:op.rhs(t,v,e)
228         def evt(t,v):return np.max(v[1:2])-.15
229         evt.direction=-1;evt.terminal=False
230         sol=solve_ivp(fun,(a,b),y,method=cfg.method,jac=lambda
            ↳ t,v:op.rhs(t,v,e,True),rtol=cfg.rtol,atol=np.tile([cfg.atol_T,cfg.atol_C],op.
            ↳ size),max_step=cfg.early_step if a<14400 else
            ↳ cfg.max_step,dense_output=True,events=evt)
231         if not sol.success:raise RuntimeError(f'Failed on {(a,b)}: {sol.message}')
232         if not np.isfinite(sol.y).all() or np.any(sol.y[1:2]<=0):raise
            ↳ FloatingPointError('Invalid accepted field')
233         stats['steps']+=len(sol.t)-1
234         for k in ('nfev','njev','nlu'):stats[k]+=int(getattr(sol,k))
235         if len(sol.t_events[0]):
236             root=float(sol.t_events[0][0]);delta=.01
237             # Bracket width is reporting evidence, NOT a bound for discretization error.
238             tm=root-delta;tp=root+delta
239             yr=sol.sol(root);ym=sol.sol(tm);yp=sol.sol(tp)
240             slope=float(fun(root,yr)[1:2][np.argmax(yr[1:2])])
241             event={'critical_s':root,'critical_h':root/3600,'critical_max':float(np.max(y
                ↳ r[1:2])), 't_minus_s':tm,'max_minus':float(np.max(ym[1:2])), 't_plus_s':t
                ↳ p,'max_plus':float(np.max(yp[1:2])), 'critical_slope_kgkg_s':slope,'brack
                ↳ et_s':2*delta,'argmax_flat_index':int(np.argmax(yr[1:2]))}
242             actual_end=cfg.execution_time_s if cfg.execution_time_s else tp
243             if not (actual_end>root and actual_end<=b):raise ValueError('Execution time
                ↳ must be after the root and in its resolved segment')
244             yexec=sol.sol(actual_end)
245             event['execution_s']=float(actual_end);event['execution_h']=float(actual_end/
                ↳ 3600)
246             event['execution_max']=float(np.max(yexec[1:2]))
247             event['strict_execution_pass']=bool(np.max(yexec[1:2])<.15)
248             if not event['strict_execution_pass']:raise AssertionError('Execution state
                ↳ not strictly qualified')
249             event_times=[tm,root,tp,actual_end];event_fields=[ym.copy(),yr.copy(),yp.copy
                ↳ (),yexec.copy()]
250         else:actual_end=b
251         qtimes=np.arange(nextout,actual_end+1e-8,60.)
252         if len(qtimes):
253             vals=sol.sol(qtimes)
254             for i,t in enumerate(qtimes):

```

```

255         samples.append(op.profile(vals[:,i]));times.append(float(t))
256         if cfg.full_samples:fulls.append(vals[:,i].copy())
257         nextout=float(qtimes[-1]+60)
258     # Full mesh checks at each accepted time and each mid-step, not just 21 exported
    ↪ radii.
259     ct=np.unique(np.r_[sol.t[sol.t<=actual_end],(sol.t[-1]+sol.t[1:])/2,actual_end])
    ↪ ;ct=ct[ct<=actual_end]
260     for t in ct:
261         u=sol.sol(t).reshape(op.shape+(2,));cc=u[:,1];tt=u[:,0]
262         minC=min(minC,float(cc.min()));maxC=max(maxC,float(cc.max()));minT=min(minT,f
    ↪ loat(tt.min()));maxT=max(maxT,float(tt.max()))
263         max_radial=max(max_radial,float(np.diff(cc,axis=0).max()));max_argmax=max(max
    ↪ _argmax,int(np.argmax(cc)))
264         if cfg.nz:max_axial=max(max_axial,float(np.diff(cc,axis=1).max()))
265     for i,t in enumerate(sol.t):
266         if t>actual_end:break
267         u=sol.y[:,i].reshape(op.size,2);mean=float(op.w@u[:,1]/op.vol)
268         if diagnostics:max_up=max(max_up,float(np.max(u[:,1])-diagnostics[-1][1]))
269         diagnostics.append([float(t),float(np.max(u[:,1])),float(np.min(u[:,1])),mean
    ↪ ,float(u[0,0]),float(u[-1,0]),float(np.sum(op.bound*cfg.hm*(u[:,1]-e(t)[1
    ↪ ]))/op.vol)])
270     # Surface and weighted-average threshold events are diagnostic only.
271     for typ,func in [('surface',lambda
    ↪ v:v.reshape(op.shape+(2,))[-1,0,1]-.15),('mean',lambda
    ↪ v:op.w@v[1::2]/op.vol-.15)]:
272         if (surface_event if typ=='surface' else mean_event) is None:
273             if func(sol.y[:,0])>=0 and func(sol.y[:,1])<0:
274                 tr=brentq(lambda t:func(sol.sol(t)),a,b,xtol=1e-8)
275                 if typ=='surface':surface_event=tr
276                 else:mean_event=tr
277     if cfg.quadrature:
278         for lo,hi,poly in zip(sol.t[:-1],sol.t[1:],sol.sol.interpolants):
279             if lo>=actual_end:break
280             hi=min(hi,actual_end)
281             for g,wg in zip(*quad):
282                 tq=(lo+hi)/2+(hi-lo)/2*g;wt=wg*(hi-lo)/2
283                 uu=poly(tq).reshape(op.size,2);du=derivative_dense(poly,tq).reshape(o
    ↪ p.size,2)
284                 S=props(uu[:,0],uu[:,1])[0];et,ec=e(tq)
285                 jrate=np.sum(op.bound*cfg.hm*(uu[:,1]-ec))/op.vol;qrate=np.sum(op.bou
    ↪ nd*cfg.ht*(uu[:,0]-et))/op.vol
286                 jint+=wt*jrate;qint+=wt*qrate;stored+=wt*(op.w@(S*du[:,0]))/op.vol
287                 endy=sol.sol(actual_end).reshape(op.size,2)
288                 balances.append([actual_end,float(op.w@endy[:,1]/op.vol-2.55+jint),stored+qin
    ↪ t,jint,qint,stored])
289     if b in (1800,3600,5400,7200,9000,10800,14400,21600) or b%21600==0:
290         if b<=actual_end:snap.append(float(b));snaps.append(sol.y[:,1].copy())
291     y=sol.sol(actual_end)
292     if event:
293         times.append(actual_end);samples.append(op.profile(y));snapt.append(actual_en
    ↪ d);snaps.append(y.copy())
294         if cfg.full_samples:fulls.append(y.copy())
295         break
296     if b%21600==0 or b==14400:print(f'n={cfg.n} nz={cfg.nz} {cfg.flux} t={b/3600:g}h
    ↪ M={y[1::2].max():.8f} elapsed={time.perf_counter()-tic:.1f}s',flush=True)
297     # Free the previous segment's dense history before allocating the next one.
298     sol=None
299     stats.update({'elapsed_s':time.perf_counter()-tic,'event':event,'global_C_min':minC,'
    ↪ global_C_max':maxC,'global_T_min':minT,'global_T_max':maxT,'max_radial_increase':
    ↪ max_radial,'max_axial_increase':max_axial,'max_argmax_flat_index':max_argmax,'max
    ↪ _M_increase_accepted':max_up,'surface_crossing_s':surface_event,'mean_crossing_s':
    ↪ mean_event,'projections':projections})
300     if balances:

```

```

301         bb=np.array(balances);stats['max_water_balance_kgkg']=float(abs(bb[:,1]).max());s
        ↳ tats['max_effective_heat_balance_J_m3']=float(abs(bb[:,2]).max());stats['effe
        ↳ ctive_heat_relative']=float(abs(bb[:,2]).max()/max(1.,abs(qint)))
302     result={'time_s':np.array(times),'radius_cm':np.linspace(0,2,21),'sample_TC':np.array
        ↳ (samples),'r_m':op.r,'z_m':op.z,'weights':op.w.reshape(op.shape),'snapshot_time_s'
        ↳ :np.array(snapt),'snapshots':np.array(snaps).reshape((-1,)+op.shape+(2,)), 'diagno
        ↳ stics':np.array(diagnostics),'balance':np.array(balances)}
303     result['event_time_s']=np.array(event_times)
304     result['event_fields_TC']=np.array(event_fields).reshape((-1,)+op.shape+(2,))
305     stats['source_sha256']=hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
306     stats['input_sha256']=hashlib.sha256(input_path.read_bytes()).hexdigest()
307     if cfg.full_samples:result['full_TC']=np.array(fulls).reshape((-1,)+op.shape+(2,))
308     path.parent.mkdir(parents=True,exist_ok=True);np.savez_compressed(path.with_suffix('.
        ↳ npz'),**result)
309     path.with_suffix('.json').write_text(json.dumps(stats,ensure_ascii=False,indent=2),en
        ↳ coding='utf-8')
310     print(json.dumps(stats,ensure_ascii=False),flush=True)
311     return stats
312
313 def main():
314     p=argparse.ArgumentParser();p.add_argument('--input',type=Path,default=Path(__file__)
        ↳ .resolve().parents[1]/'inputs/attachment1.xlsx');p.add_argument('--out',type=Path
        ↳ ,required=True);p.add_argument('--config',type=Path)
315     p.add_argument('--n',type=int,default=256);p.add_argument('--nz',type=int,default=0);
        ↳ p.add_argument('--flux',default='integral');p.add_argument('--scenario',default='
        ↳ mean');p.add_argument('--rtol',type=float,default=2e-9);p.add_argument('--max-ste
        ↳ p',type=float,default=240.);p.add_argument('--quadrature',action='store_true');p.
        ↳ add_argument('--full-samples',action='store_true')
316     args=p.parse_args();cfg=Config(**json.loads(args.config.read_text())) if args.config
        ↳ else
        ↳ Config(n=args.n,nz=args.nz,flux=args.flux,scenario=args.scenario,rtol=args.rtol,m
        ↳ ax_step=args.max_step,quadrature=args.quadrature,full_samples=args.full_samples)
317     solve(cfg,args.out,args.input)
318 if __name__=='__main__':main()

```

A.21 q3_reference

support/project/q4_complete_delivery/v1/q123_closeout/source/legacy_q3/q3_reference.py

```

1  """Independent global Chebyshev collocation with primitive-gradient moisture flux.
2  No FV control volumes, face weights, or FV spatial operator are reused.
3  Full coupled heat/moisture evolution from t=0; Robin boundary algebraically solved.
4  Grid/differentiation construction follows the audited q2_spectral.py approach;
5  moisture flux and its boundary Jacobian are newly derived for long-time Q3.
6  """
7  from __future__ import annotations
8  import argparse,json,time
9  from pathlib import Path
10 import numpy as np
11 from numpy.polynomial import Chebyshev
12 from scipy.special import exp1
13 from scipy.optimize import brentq
14 from scipy.integrate import solve_ivp
15 from scipy.fft import dct
16 from q3_solver import load_input,Environment
17
18 def primitive(c):
19     if np.any(np.real(c)<=0):raise FloatingPointError('C<=0 in reference')
20     return c*np.exp(-.45/c)-.45*exp1(.45/c)
21

```

```

22 def grid(n):
23     x=(1-np.cos(np.pi*np.arange(n+1)/n))/2
24     w=(-1.)*np.arange(n+1);w[[0,-1]]*=-.5
25     dx=x[:,None]-x[None,:]
26     G=np.divide(w[None,:]/w[:,None],dx,out=np.zeros_like(dx),where=dx!=0)
27     G[np.diag_indices(n+1)]=-G.sum(axis=1);G[:, -1]=-G[:, : -1].sum(axis=1)
28     return x,G,w
29
30 def interp(x,w,target):
31     d=target[:,None]-x
32     out=np.empty_like(d)
33     for j,dd in enumerate(d):
34         k=np.argmin(abs(dd))
35         if abs(dd[k])<1e-14:out[j]=0;out[j,k]=1
36         else:v=w/dd;out[j]=v/v.sum()
37     return out
38
39 def inverse_primitive(v,guess):
40     c=np.asarray(guess).copy()
41     for _ in range(40):
42         delta=(primitive(c)-v)/np.exp(-.45/c)
43         cnew=c-delta
44         # Globalize Newton without altering the equation or accepted value.
45         bad=np.real(cnew)<=0
46         while np.any(bad):delta[bad]*=-.5;cnew=c-delta;bad=np.real(cnew)<=0
47         c=cnew
48         if np.max(abs(delta))<3e-14:break
49     else:raise RuntimeError('Inverse primitive failed')
50     return c
51
52 class Reference:
53     def __init__(self,n):
54         self.n=n;self.x,self.G,self.w=grid(n);self.row=self.G[-1,:-1];self.diag=self.G[-1,-1]
55         self.fac=4/.02**2;self.bd=2/.02
56         self.BT0=np.zeros((n+1,2*n));self.BC0=np.zeros_like(self.BT0)
57         self.BT0[np.arange(n),2*np.arange(n)]=1;self.BC0[np.arange(n),2*np.arange(n)+1]=1
58         self.I=interp(self.x,self.w,np.linspace(0,1,21)**2)
59         self.denseI=interp(self.x,self.w,np.linspace(0,1,8*n+1))
60         self.min_boundary_derivative=float('inf');self.max_bc_residual=0.;self.max_newton=0;self
        ↪ f_boundary_fallbacks=0
61     def surface(self,t,u,env,jac=False):
62         Ti,Ci=u.T;et,ec=env(t);Ui=primitive(Ci);anchor=Ci[-1];ua=Ui[-1]
63         gt=self.row@(Ti-et);gu=self.row@(Ui-ua)
64         def calc(c):
65             k=.21+.38*c/(1+c);kc=.38/(1+c)**2
66             den=self.bd*k*self.diag+25
67             Ts=et-self.bd*k*gt/den
68             Tsc=-self.bd*kc*gt*25/den**2
69             A=.0024*np.exp(-3850/(Ts+273.15));At=A*3850/(Ts+273.15)**2
70             ux=gu+self.diag*(primitive(c)-ua)
71             f=np.exp(-.45/c)
72             val=self.bd*A*ux+8e-7*(c-ec)
73             deriv=self.bd*(At*Tsc*ux+A*self.diag*f)+8e-7
74             return val,deriv,Ts,k,kc,A,At,ux
75         c=anchor
76         for it in range(25):
77             val,deriv,*_=calc(c);step=val/deriv
78             while c-step<=0:step*=-.5
79             c=c-step
80             if abs(step)<2e-14*(1+abs(c)):break
81         else:
82             self.boundary_fallbacks+=1
83             lo=min(ec,float(Ci.min()))*.1;hi=max(ec,float(Ci.max()))*1.1
84             c=brentq(lambda z:calc(z)[0],lo,hi,xtol=5e-15)

```

```

85     val,deriv,Ts,k,kc,A,At,ux=calc(c)
86     self.max_newton=max(self.max_newton,it+1);self.min_boundary_derivative=min(self.min_bou
↳ ndary_derivative,float(deriv));self.max_bc_residual=max(self.max_bc_residual,float(
↳ abs(val)))
87     full=np.vstack([u,[Ts,c]])
88     if not jac:return full
89     tx=self.row@(Ti-Ts)
90     B=np.array([[self.bd*k*self.diag+25,self.bd*k*tx],[self.bd*At*ux,self.bd*A*self.diag*n
↳ p.exp(-.45/c)+8e-7]])
91     Q=np.zeros((2,2*self.n));Q[0,:2]=self.bd*k*self.row;Q[1,1:2]=self.bd*A*self.row*np.ex
↳ p(-.45/Ci)
92     sens=-np.linalg.solve(B,Q);BT=self.BT0.copy();BC=self.BC0.copy();BT[-1]=sens[0];BC[-1]=
↳ sens[1]
93     return full,BT,BC
94     def evaluate(self,t,y,env,jac=False):
95         if jac:full,BT,BC=self.surface(t,y.reshape(self.n,2),env,True)
96         else:full=self.surface(t,y.reshape(self.n,2),env)
97         T,C=full.T
98         if np.any(T<=-273.15):raise FloatingPointError('Kelvin<=0')
99         rho=650+128*C;cp=1450+2736*C/(1+C);S=rho*cp;Sc=128*cp+rho*2736/(1+C)**2
100        k=.21+.38*C/(1+C);kc=.38/(1+C)**2;A=.0024*np.exp(-3850/(T+273.15));At=A*3850/(T+273.15)
↳ **2
101        U=primitive(C);gt=self.G@(T-T[-1]);gu=self.G@(U-U[-1]);qt=k*gt;qm=A*gu
102        ht=qt+self.x*(self.G@(qt-qt[-1]));hm=qm+self.x*(self.G@(qm-qm[-1]))
103        fT=self.fac*ht/S;fC=self.fac*hm
104        if not jac:return np.column_stack([fT[:-1],fC[:-1]]).ravel()
105        BU=np.exp(-.45/C)[: ,None]*BC
106        gtj=self.G@(BT-BT[-1]);guj=self.G@(BU-BU[-1])
107        qtj=k[: ,None]*gtj+(kc*gt)[: ,None]*BC
108        qmj=A[: ,None]*guj+(At*gu)[: ,None]*BT
109        htj=qtj+self.x[: ,None]*(self.G@(qtj-qtj[-1]));hmj=qmj+self.x[: ,None]*(self.G@(qmj-qmj[-
↳ 1]))
110        J=np.empty((2*self.n,2*self.n));J[:2]=
↳ (self.fac*htj/S[: ,None]-(fT*Sc/S)[: ,None]*BC)[: -1];J[1:2]=(self.fac*hmj)[: -1]
111        return J
112    def profiles(self,u):
113        T,C=u.T;U=primitive(C);v=self.I@U;guess=self.I@C
114        return np.column_stack([self.I@T,inverse_primitive(v,guess)])
115    def extrema(self,u):
116        # Interpolate U, not steep C. All real stationary points of the polynomial.
117        U=primitive(u[: ,1]);co=dcT(U[:-1],type=1)/self.n;co[[0,-1]]*=.5
118        p=Chebyshev(co);roots=p.deriv().roots();roots=roots[np.isreal(roots)].real;roots=roots[
↳ (roots>-1)&(roots<1)]
119        pts=np.r_[-1,roots,1];vals=p(pts);idx=np.argmax(vals)
120        maxi=inverse_primitive(np.array([vals[idx]]),np.array([u[: ,1].max()]))[0]
121        return {'max_C':float(maxi),'x_max':float((pts[idx]+1)/2),'stationary_points':len(roots)
↳ },'minimum_U':float(vals.min())}
122
123    def run(n,out,input_file,method='Radau',rtol=2e-11,atol_C=2e-13,max_step=120.):
124        if out.with_suffix('.json').exists():raise FileExistsError(out)
125        op=Reference(n);data=load_input(input_file);env=Environment(data,'mean');y=np.tile([28.,
↳ 2.55],n)
126        cuts=np.r_[data[: ,0],np.arange(21600,360001,21600.)]
127        times=[0.];vals=[np.tile([28.,2.55],(21,1))];snapt=[0.];snaps=[np.tile([28.,2.55],(n+1,1
↳ ))];ext=[]
128        nextt=60.;tic=time.perf_counter();nsteps=nfev=nlu=0;event=None;max_dense_overshoot=0.;gl
↳ obalmin=2.55
129        for a,b in zip(cuts[:-1],cuts[1:]):
130            e=env.segment(a,b)
131            fun=lambda t,y:op.evaluate(t,y,e)
132            def ev(t,y):return np.max(y[1:2])-.15
133            ev.terminal=False;ev.direction=-1

```

```

134 sol=solve_ivp(fun,(a,b),y,method=method,jac=lambda t,y:op.evaluate(t,y,e,True),rtol=rto
↪ 1,atol=np.tile([2e-11,atol_C],n),max_step=min(max_step,30.) if a<14400 else
↪ max_step,dense_output=True,events=ev)
135 if not sol.success:raise RuntimeError(sol.message)
136 nsteps+=len(sol.t)-1;nfev+=sol.nfev;nlu+=sol.nlu;end=b
137 if len(sol.t_events[0]):
138     root=float(sol.t_events[0][0]);end=root+.01;yr=sol.sol(root);ur=op.surface(root,yr.res
↪ hape(n,2),e)
139     event={'critical_s':root,'critical_h':root/3600,'t_minus_s':root-.01,'max_minus':float
↪ (np.max(sol.sol(root-.01)[1::2])), 't_plus_s':end,'max_plus':float(np.max(sol.sol(en
↪ d)[1::2])), 'slope':float(fun(root,yr)[1::2][np.argmax(yr[1::2]))], 'polynomial_extr
↪ ema':op.extrema(ur)}
140 ts=np.arange(nextt,end+1e-8,60)
141 if event:ts=np.r_[ts,end]
142 for t in ts:
143     uu=op.surface(t,sol.sol(t).reshape(n,2),e);times.append(float(t));vals.append(op.profi
↪ les(uu))
144     denseU=op.denseI@primitive(uu[:,1]);max_dense_overshoot=max(max_dense_overshoot,float(
↪ denseU.max()-primitive(uu[:,1]).max());globalmin=min(globalmin,float(uu[:,1].min(
↪ )))
145     if len(ts):nextt=60*(np.floor(ts[-1]/60)+1)
146     if b in (1800,3600,5400,7200,9000,10800,14400,21600) or b%21600==0 or event:
147         u=op.surface(end,sol.sol(end).reshape(n,2),e);snapt.append(float(end));snaps.append(u)
↪ ;ext.append({'t_s':float(end),**op.extrema(u)})
148         y=sol.sol(end)
149         if b%21600==0 or
↪ event:print(n,b/3600,float(y[1::2].max()),time.perf_counter()-tic,flush=True)
150         if event:break
151 stats={'n':n,'method':method,'rtol':rtol,'atol_C':atol_C,'max_step':max_step,'event':eve
↪ nt,'elapsed_s':time.perf_counter()-tic,'nsteps':nsteps,'nfev':nfev,'nlu':nlu,'min_sc
↪ alar_boundary_derivative':op.min_boundary_derivative,'max_boundary_residual':op.max_
↪ bc_residual,'max_boundary_newton':op.max_newton,'boundary_fallbacks':op.boundary_fal
↪ lbacks,'dense_U_overshoot':max_dense_overshoot,'global_C_min':globalmin,'extrema':ex
↪ t}
152 out.parent.mkdir(parents=True,exist_ok=True);out.with_suffix('.json').write_text(json.du
↪ mps(stats,indent=2));np.savez_compressed(out.with_suffix('.npz'),time_s=np.array(tim
↪ es),sample_TC=np.array(vals),x=op.x,snapshot_time_s=np.array(snapt),snapshots=np.arr
↪ ay(snaps))
153 print(json.dumps(stats),flush=True)
154 if __name__=='__main__':
155     p=argparse.ArgumentParser();p.add_argument('--n',type=int,default=80);p.add_argument('--
↪ out',type=Path,required=True);p.add_argument('--method',default='Radau');p.add_argum
↪ ent('--rtol',type=float,default=2e-11);p.add_argument('--input',type=Path,default=Pa
↪ th(__file__).resolve().parents[1]/'inputs/attachment1.xlsx');a=p.parse_args();run(a.
↪ n,a.out,a.input,a.method,a.rtol)

```

A.22 solve_unified_q23

support/project/q4_complete_delivery/v1/q123_closeout/source/solve_unified_q23.py

```

1 """Cold start a unified, simultaneous Appendix 3 T,C trajectory at every second.
2
3 The copied historical Chebyshev primitive-flux spatial implementation is used;
4 its floating-point output is retained, never cell-patched or spliced. Each data
5 knot is an integration boundary. After 4 h the same Q3 arithmetic mean applies.
6 Vectorization below changes only dense-output profile evaluation, not the ODE.
7 """
8 from pathlib import Path
9 import sys, json, time, argparse, hashlib, platform
10 sys.path.insert(0,str(Path(__file__).resolve().parent/'legacy_q3'))
11 import numpy as np

```

```

12 import scipy
13 from scipy.integrate import solve_ivp
14 from q3_reference import Reference, primitive, inverse_primitive
15 from q3_solver import Environment, load_input
16
17 def batched_profiles(op, ts, ys, env):
18     # ys = (samples,n,2); solve one Robin surface per sample.
19     Ti,Ci=ys[:,0],ys[:,1]
20     et,ec=np.asarray(env(ts)).T
21     Ui=primitive(Ci);anchor=Ci[:, -1];ua=Ui[:, -1]
22     gt=(Ti-et[:,None])@op.row;gu=(Ui-ua[:,None])@op.row
23     c=anchor.copy()
24     for it in range(40):
25         k=.21+.38*c/(1+c);kc=.38/(1+c)**2
26         den=op.bd*k*op.diag+25
27         Ts=et-op.bd*k*gt/den
28         Tsc=-op.bd*kc*gt*25/den**2
29         A=.0024*np.exp(-3850/(Ts+273.15));At=A*3850/(Ts+273.15)**2
30         ux=gu+op.diag*(primitive(c)-ua)
31         val=op.bd*A*ux+8e-7*(c-ec)
32         deriv=op.bd*(At*Tsc*ux+A*op.diag*np.exp(-.45/c))+8e-7
33         step=val/deriv
34         bad=c-step<=0
35         while bad.any(): step[bad]*=.5;bad=c-step<=0
36         c-=step
37         if np.max(abs(step)/(1+abs(c)))<2e-14:break
38     else: raise RuntimeError('Vectorized Robin Newton failed')
39     k=.21+.38*c/(1+c);Ts=et-op.bd*k*gt/(op.bd*k*op.diag+25)
40     full=np.concatenate([ys,np.stack([Ts,c],axis=1)[:,None,:]],axis=1)
41     T,C=full[:,0],full[:,1]
42     U=primitive(C)
43     profile=np.stack([T@op.I.T,inverse_primitive(U@op.I.T,C@op.I.T)],axis=-1)
44     return profile,full
45
46 def solve(n=160,end=206906.76,method='Radau',rtol=2e-12,max_step=60.,out=None,input_path=
↳ None,step=1.):
47     out=Path(out)
48     if out.with_suffix('.npz').exists() or out.with_suffix('.json').exists():raise
↳ FileExistsError(out)
49     out.parent.mkdir(parents=True,exist_ok=True)
50     op=Reference(n);data=load_input(input_path);env=Environment(data,'mean','linear')
51     ts=np.unique(np.r_[np.arange(0,end+1e-8,step),end]);v=np.empty((len(ts),21,2));v[0]=[
↳ 28.,2.55]
52     y=np.tile([28.,2.55],n)
53     cuts=np.unique(np.r_[data[:,0][data[:,0]<end],np.arange(21600.,end,21600.),end]);snap
↳ t=[0.];snaps=[np.tile([28.,2.55],(n+1,1))]
54     stats=dict(n=n,method=method,rtol=rtol,atol_T=2e-12,atol_C=2e-14,max_step_s=max_step,
↳ end_s=end,step_s=step,source_sha='anonymous-source',model='Appendix 3 effective
↳ radial, no explicit latent heat; same equation and boundary as
↳ Q2/Q3',environment='piecewise linear through 4 h; then arithmetic mean of 61
↳ samples from 3 to 4 h',future=env.future.tolist(),nsteps=0,nfev=0,nlu=0,events=[])
↳ ,runtime_versions={'python':platform.python_version(),'numpy':np.__version__,'sci
↳ py':scipy.__version__})
55     start=time.perf_counter();max_batch_diff=np.zeros(2)
56     for a,b in zip(cuts[:-1],cuts[1:]):
57         ee=env.segment(a,b)
58         def event(t,y):return y[1:2].max()-.15
59         event.terminal=False;event.direction=-1
60         sol=solve_ivp(lambda t,y:op.evaluate(t,y,ee),(a,b),y,method=method,jac=lambda
↳ t,y:op.evaluate(t,y,ee,True),rtol=rtol,atol=np.tile([2e-12,2e-14],n),max_step
↳ =min(max_step,20.) if a<14400 else max_step,dense_output=True,events=event)
61         if not sol.success:raise RuntimeError(sol.message)
62         stats['nsteps']+=len(sol.t)-1;stats['nfev']+=sol.nfev;stats['nlu']+=sol.nlu
63         ids=np.flatnonzero((ts>a)&(ts<=b+1e-8))

```

```

64     for chunk in np.array_split(ids,max(1,(len(ids)+2047)//2048)):
65         if len(chunk)==0:continue
66         yy=sol.sol(ts[chunk]).reshape(n,2,-1).transpose(2,0,1)
67         vv,full=batched_profiles(op,ts[chunk],yy,ee);v[chunk]=vv
68         # Scalar evaluation samples ensure vectorization has not changed equations.
69         for j in [0,len(chunk)-1]:
70             scalar=op.profiles(op.surface(ts[chunk[j]],yy[j],ee))
71             max_batch_diff=np.maximum(max_batch_diff,np.max(abs(scalar-vv[j]),axis=0))
72     for t in sol.t_events[0]:
73         state=op.surface(t,sol.sol(t).reshape(n,2),ee)
74         stats['events'].append({'root_s':float(t),'max_C':float(state[:,1].max()),'ex_
        ↪ trema':op.extrema(state)})
75     y=sol.y[:, -1]
76     if b%1800==0 or b==end:
77         sf=op.surface(b,y.reshape(n,2),ee);snapt.append(b);snaps.append(sf)
78         print(json.dumps({'n':n,'t_s':float(b),'Cmax':float(sf[:,1].max()),'elapsed_s_
        ↪ ':time.perf_counter()-start}),flush=True)
79     stats['elapsed_s']=time.perf_counter()-start;stats['vectorized_vs_scalar_max_abs_TC']
        ↪ =max_batch_diff.tolist();stats['end_max_C']=float(snaps[-1][:,1].max());stats['en_
        ↪ d_global_extrema']=op.extrema(snaps[-1]);stats['strict_1d_pass']=stats['end_globa_
        ↪ l_extrema'][:,'max_C']<.15
80     stats['input_sha256']=hashlib.sha256(Path(input_path).read_bytes()).hexdigest();stats
        ↪ ['source_sha256']=hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
81     np.savez_compressed(out.with_suffix('.npz'),time_s=ts,radius_cm=np.linspace(0,2,21),p_
        ↪ rofile_TC=v,environment=env(ts),x=op.x,snapshot_time_s=np.array(snapt),snapshot_f_
        ↪ ull_TC=np.array(snaps))
82     out.with_suffix('.json').write_text(json.dumps(stats,ensure_ascii=False,indent=2)+'\n')
83     print(json.dumps(stats,ensure_ascii=False),flush=True)
84     return stats
85
86 if __name__=='__main__':
87     p=argparse.ArgumentParser();p.add_argument('--n',type=int,default=160);p.add_argument
        ↪ ('--end',type=float,default=206906.76);p.add_argument('--method',default='Radau');
        ↪ p.add_argument('--rtol',type=float,default=2e-12);p.add_argument('--max-step',typ_
        ↪ e=float,default=60.);p.add_argument('--step',type=float,default=1.);p.add_argumen_
        ↪ t('--out',type=Path,required=True);p.add_argument('--input',type=Path,default=Pat_
        ↪ h(__file__).resolve().parents[1]/'inputs/attachment1.xlsx');a=p.parse_args();solv_
        ↪ e(a.n,a.end,a.method,a.rtol,a.max_step,a.out,a.input,a.step)

```

A.23 audit_unified_q23

support/project/q4_complete_delivery/v1/q123_closeout/source/audit_unified_q23.py

```

1  """Read-only numeric/source audit for the newly solved Q2/Q3 trajectory.
2  Frozen historical comparison inputs are included in inputs/historical_baseline.
3  All new evidence goes to --out, which must not already exist.
4  """
5  from pathlib import Path
6  import argparse,json,csv,hashlib,zipfile,xml.etree.ElementTree as ET
7  import numpy as np
8  NS='{http://schemas.openxmlformats.org/spreadsheetml/2006/main}'
9
10 def fp(p):return {'path':str(p),'bytes':p.stat().st_size,'sha256':hashlib.sha256(p.read_b_
    ↪ ytes()).hexdigest()}
11 def rounded(a):
12     r=np.round(a,4)
13     # Resolve numbers close enough to a tie to expose decimal formatting choices.
14     z=a*10000;ii=np.argwhere(abs(z-np.floor(z)-.5)<1e-9)
15     for ix in ii:
16         ix=tuple(ix);r[ix]=float(format(float(a[ix]),'.4f'))
17     return r

```

```

18
19 def cmp(a,b,time_s):
20     d=abs(a-b);rr=rounded(a)!=rounded(b)
21     report={'max_abs_TC':d.max(axis=(0,1)).tolist(),'rms_TC':np.sqrt(np.mean(d*d,axis=(0,1))).tolist(),'four_decimal_difference_count_TC':rr.sum(axis=(0,1)).tolist(),'values_per_field':int(np.prod(d.shape[:2]))}
22     report['argmax_TC']=[]
23     for k in range(2):
24         i,j=np.unravel_index(d[:, :, k].argmax(),d.shape[:2]);report['argmax_TC'].append({'time_s':float(time_s[i]),'radius_cm':float(j/10),'new':float(a[i,j,k]),'comparison':float(b[i,j,k])})
25     return report,rr
26
27 def old_workbook(p):
28     out=np.empty((10800,21,2));times=np.empty((10800,2))
29     with zipfile.ZipFile(p) as z:
30         rel={r.attrib['Id']:r.attrib['Target'] for r in
31             ET.fromstring(z.read('xl/_rels/workbook.xml.rels'))}
32         sheets=ET.fromstring(z.read('xl/workbook.xml')).find(NS+'sheets')
33         names=[]
34         for k,s in enumerate(sheets):
35             names.append(s.attrib['name']);target=rel[s.attrib['{http://schemas.openxmlformats.org/officeDocument/2006/relationships}id']];target=target.lstrip('/')
36             if target.startswith('/') else 'xl/'+target
37             for event,el in ET.iterparse(z.open(target),events=('end',)):
38                 if el.tag!=NS+'row':continue
39                 i=int(el.attrib['r'])-2
40                 if i<0:el.clear();continue
41                 if i>=10800:raise ValueError('Unexpected old Q2 row')
42                 for cell in el:
43                     addr=cell.attrib['r'];col=''.join(filter(str.isalpha,addr));v=cell.find(NS+'v')
44                     if v is None:continue
45                     if col=='A':times[i,k]=float(v.text)
46                     elif len(col)==1 and
47                         'B'<=col<='V':out[i,ord(col)-ord('B')],k]=float(v.text)
48                     el.clear()
49         assert names==['温度','水分浓度'] and np.array_equal(times[:,0],np.arange(1,10801)) and
50             np.array_equal(times[:,1],times[:,0])
51     return out
52
53 def run(repo,new,reference,out):
54     out.mkdir(parents=True,exist_ok=False)
55     z=np.load(new);t=z['time_s'];v=z['profile_TC'];ref=np.load(reference);rv=ref['profile_TC'];rt=ref['time_s'];ri=np.searchsorted(t,rt);assert np.array_equal(t[ri],rt)
56     old_path=repo/'q2_final_delivery/output/q2_unrounded.npz';o=np.load(old_path);ov=np.savez(tack([o['temperature_degC'],o['moisture_dry_basis']],axis=-1)
57     r={'source_sha':'anonymous-source','data_contract':{'shape':list(v.shape),'time_s_first':float(t[0]),float(t[-1]),'integer_seconds_0_through':206906,'extra_fields':206906.76,'all_integer_seconds_present':bool(np.array_equal(t[:-1],np.arange(206907))),'radius_cm':z['radius_cm'].tolist(),'joint_fields':['temperature_degC','C_kg_water_per_kg_dry_solid'],'finite':bool(np.isfinite(v).all())}}
58     r['historical_n160_BDF_reference_at_common_outputs'],rr=cmp(v[ri],rv,rt)
59     r['historical_n160_BDF_reference_at_common_outputs']['scope']='Previously completed and accepted independent 160-order BDF output, every 60s plus the same official execution endpoint; not every second and not claimed as newly run.'
60     r['old_Q2_first3h_unrounded'],ro=cmp(v[:10801],ov,t[:10801])
61     ow=old_workbook(repo/'q2_final_delivery/output/result2.xlsx');wcmp=rounded(v[1:10801])!=ow
62     r['old_Q2_actual_workbook']={'rows_per_sheet':10800,'values_per_sheet':226800,'four_decimal_difference_count_TC':wcmp.sum(axis=(0,1)).tolist(),'max_abs_new_rounded_vs_old_cells_TC':abs(rounded(v[1:10801])-ow).max(axis=(0,1)).tolist()}
63     q3_path=repo/'q3_refinement_delivery/output/solution.npz';q=np.load(q3_path);qt=q['time_s'];ix=np.searchsorted(t,qt);assert np.max(abs(t[ix]-qt))<1e-8

```

```

60     r['old_Q3_timestamp_binary_max_difference_s']=float(np.max(abs(t[ix]-qt)))
61     r['old_Q3_common60s_and_endpoint'],rq3=cmp(v[ix],q['profile_TC'],qt)
62     r['paper_table_difference_count_TC']=[]
63     for k,name in enumerate(['table_temperature.csv','table_moisture.csv']):
64         tab=np.loadtxt(repo/'q2_final_delivery/output'/name,delimiter=',',skiprows=1)
65         vals=rounded(v[(tab[:,0]*3600).astype(int),:,k][:,[0,5,10,15,20]])
66         r['paper_table_difference_count_TC'].append(int(np.count_nonzero(vals!=tab[:,1:])))
67     r['source_files']=[fp(p) for p in
68         ↪ [new,reference,old_path,repo/'q2_final_delivery/output/result2.xlsx',q3_path]]
69     r['interpretation']='All new T,C originate from one Appendix 3 cold-start solution.
70     ↪ Differences from old Q2 reflect finite-volume Richardson versus primitive-flux
71     ↪ collocation output, not parameter or physical-model changes. These are 1D
72     ↪ numerical checks, not a 2D geometry error bound.'
73     for name,a,b,tt,mask in [(('reference_four_decimal_differences',v[ri],rv,rt,rr),('old_
74     ↪ Q2_four_decimal_differences',v[:10801],ov,t[:10801],ro),('old_Q3_four_decimal_dif
75     ↪ ferences',v[ix],q['profile_TC'],qt,rq3))]:
76         with (out/(name+'.csv')).open('w',newline='') as f:
77             w=csv.writer(f);w.writerow(['t_s','radius_cm','field','new_unrounded','compar
78             ↪ ison_unrounded','new_4dp','comparison_4dp'])
79             for i,j,k in np.argwhere(mask):w.writerow([tt[i],j/10,['T_degC','C_dry_basis'
80             ↪ ][k],format(a[i,j,k],'.17g'),format(b[i,j,k],'.17g'),format(a[i,j,k],'.4f
81             ↪ '),format(b[i,j,k],'.4f')])
82     (out/'audit.json').write_text(json.dumps(r,ensure_ascii=False,indent=2)+'\n');print(j
83     ↪ son.dumps(r,ensure_ascii=False,indent=2));return r
84
85 if __name__=='__main__':
86     p=argparse.ArgumentParser();p.add_argument('--repo',type=Path,default=Path(__file__).
87     ↪ resolve().parents[1]/'inputs/historical_baseline');p.add_argument('--new',type=Pa
88     ↪ th,required=True);p.add_argument('--reference',type=Path,required=True);p.add_arg
89     ↪ ument('--out',type=Path,required=True);a=p.parse_args();run(a.repo,a.new,a.refere
90     ↪ nce,a.out)

```

A.24 第三问现行执行点与核验

A.25 q3_solver

support/project/q3_refinement_delivery/source/q3_solver.py

```

1  """Q3: independently assembled conservative graph FV on graded cylindrical meshes.
2  SI units; T in Celsius except Arrhenius; C is dry-basis. No clipping or D floor.
3  Both actual early boundary knots and the future scenario are explicit.
4  """
5  from __future__ import annotations
6  import argparse, json, time, sys, hashlib, platform, zipfile
7  from pathlib import Path
8  from dataclasses import dataclass, asdict
9  import xml.etree.ElementTree as ET
10 import numpy as np
11 import scipy
12 from scipy.integrate import solve_ivp
13 from scipy.interpolate import PchipInterpolator
14 from scipy.special import exp1
15 from scipy.optimize import brentq
16 from scipy.sparse import coo_matrix
17
18 NS={'s':'http://schemas.openxmlformats.org/spreadsheetml/2006/main'}
19
20 def read_xlsx(path):
21     """Read physical OOXML (not preview text), retaining numeric/string identity."""
22     with zipfile.ZipFile(path) as z:
23         if z.testzip(): raise ValueError('Corrupt xlsx')

```

```

24     strings=[]
25     if 'xl/sharedStrings.xml' in z.namelist():
26         for v in ET.fromstring(z.read('xl/sharedStrings.xml')).findall('s:si',NS):
27             strings.append(''.join(t.text or '' for t in v.findall('..s:t',NS)))
28     rel={v.attrib['Id']:v.attrib['Target'] for v in
29     ↪ ET.fromstring(z.read('xl/_rels/workbook.xml.rels'))}
29     out={}
30     for sh in ET.fromstring(z.read('xl/workbook.xml')).findall('s:sheets/s:sheet',NS):
31         rid=sh.attrib['{http://schemas.openxmlformats.org/officeDocument/2006/relatio
32 ↪ nships}id']
32         target=rel[rid]; target=target[1:] if target.startswith('/') else 'xl/'+target
33         cells={}
34         for c in ET.fromstring(z.read(target)).findall('..s:sheetData/s:row/s:c',NS):
35             kind=c.attrib.get('t','n');v=c.find('s:v',NS)
36             if c.find('s:f',NS) is not None: raise ValueError('Unexpected formula')
37             if kind=='s': value=strings[int(v.text)]
38             elif kind=='inlineStr': value=''.join(t.text or '' for t in
39 ↪ c.findall('..s:t',NS))
39             elif v is None: value=None
40             elif kind in ('str','e'): value=v.text
41             else: value=float(v.text)
42             cells[c.attrib['r']]={'value':value,'type':kind,'style':int(c.attrib.get(
43 ↪ 's','0'))}
43         out[sh.attrib['name']]=cells
44     return out
45
46 def load_input(path):
47     cells=read_xlsx(path)['Sheet1']
48     assert [cells[f'{c}1'][f'value'] for c in 'ABC']==['时间','温度','水分浓度']
49     a=np.array([[cells[f'{c}{i}'][f'value'] for c in 'ABC'] for i in range(2,243)],float)
50     assert a.shape==(241,3) and np.isfinite(a).all()
51     assert np.array_equal(a[:,0],np.arange(241)*60.)
52     return a
53
54 class Environment:
55     def __init__(self,data,scenario='mean',kind='linear'):
56         self.data=np.asarray(data,float);self.scenario=scenario;self.kind=kind
57         if scenario=='mean':self.future=self.data[self.data[:,0]>=10800,1:].mean(axis=0)
58         elif scenario=='last':self.future=self.data[-1,1:].copy()
59         elif scenario=='nominal':self.future=np.array([50.,.05])
60         else:raise ValueError(scenario)
61         self.interp=PchipInterpolator(self.data[:,0],self.data[:,1:],axis=0) if
62 ↪ kind=='pchip' else None
62     def __call__(self,t):
63         t=np.asarray(t);v=(self.interp(t) if self.interp is not None else
64 ↪ np.stack([np.interp(t,self.data[:,0],self.data[:,k]) for k in (1,2)],axis=-1))
64         return np.where((t>14400)[...],None,self.future,v)
65     def segment(self,a,b):
66         # One-sided boundary at t=14400: no spurious jump in the pre-4h segment.
67         if a>=14400:return lambda t: np.broadcast_to(self.future,np.shape(t)+(2,))
68         return self
69
70 def props(T,C):
71     if np.any(C<=0) or np.any(T<=-273.15):raise FloatingPointError('Nonpositive
72 ↪ accepted/trial C or Kelvin temperature')
72     rho=650.+128*C;cp=1450.+2736*C/(1+C)
73     S=rho*cp;k=.21+.38*C/(1+C)
74     Sc=128*cp+rho*2736/(1+C)**2;k=.38/(1+C)**2
75     A=.0024*np.exp(-3850/(T+273.15));D=A*np.exp(-.45/C)
76     return S,k,D,Sc,kc
77
78 def F(C):
79     C=np.asarray(C)
80     if np.any(C<=0):raise FloatingPointError('F requires C>0')

```

```

81     return C*np.exp(-.45/C)-.45*exp1(.45/C)
82
83 def Fdiff(a,b):
84     """Stable primitive difference. 4-point Gaussian integration when close.
85     The switch is numerical evaluation only; no modification of diffusivity.
86     """
87     a,b=np.broadcast_arrays(a,b);mid=(a+b)/2;delta=a-b
88     close=np.abs(delta)<.002*mid
89     ans=np.empty_like(a)
90     if np.any(~close):ans[~close]=F(a[~close])-F(b[~close])
91     if np.any(close):
92         m=mid[close];d=delta[close]
93         v=np.zeros_like(m)
94         for x,w in
95             ↪ ((-.8611363115940526,.3478548451374539),(-.3399810435848563,.6521451548625461))
96             ↪ ,(.3399810435848563,.6521451548625461),(.8611363115940526,.3478548451374539)):
97             v+=w*np.exp(-.45/(m+d*x/2))
98         ans[close]=d*v/2
99     return ans
100
101 @dataclass(frozen=True)
102 class Config:
103     n:int=256
104     nz:int=0
105     grading:float=2.
106     zgrading:float=2.
107     flux:str='integral'
108     scenario:str='mean'
109     interpolation:str='linear'
110     hT:float=25.
111     hm:float=8e-7
112     end_factor:float=1.
113     rtol:float=2e-9
114     atol_T:float=2e-10
115     atol_C:float=2e-12
116     max_step:float=240.
117     early_step:float=30.
118     method:str='BDF'
119     end_s:float=360000.
120     isothermal_after:float=0.
121     quadrature:bool=False
122     full_samples:bool=False
123     execution_time_s:float=0.
124
125 class Operator:
126     def __init__(self,cfg:Config):
127         self.cfg=cfg;self.R=.02;self.Lhalf=.125
128         self.r=self.R*(1-(1-np.linspace(0,1,cfg.n+1))*cfg.grading)
129         rf=np.r_[0,(self.r[:-1]+self.r[1:])/2,self.R];wr=np.diff(rf**2)/2
130         if cfg.nz:
131             self.z=self.Lhalf*(1-(1-np.linspace(0,1,cfg.nz+1))*cfg.zgrading)
132             zf=np.r_[0,(self.z[:-1]+self.z[1:])/2,self.Lhalf];wz=np.diff(zf)
133         else:self.z=np.array([0.]);wz=np.ones(1)
134         self.shape=(len(self.r),len(self.z)); self.size=np.prod(self.shape).item()
135         ids=np.arange(self.size).reshape(self.shape)
136         self.w=(wr[:,None]*wz).ravel();self.vol=self.w.sum()
137         l=[ids[:-1].ravel()];r=[ids[1:].ravel()]
138         ge=[((rf[1:-1]/np.diff(self.r))[:,None]*wz).ravel()]
139         if cfg.nz:
140             l.append(ids[:, :-1].ravel());r.append(ids[:, 1:].ravel())
141             ge.append((wr[:, None]/np.diff(self.z)).ravel())
142         self.left=np.concatenate(l);self.right=np.concatenate(r);self.geo=np.concatenate(
143             ↪ ge)
144         self.bound=np.zeros(self.shape);self.bound[-1]+=self.R*wz

```

```

142     if cfg.nz:self.bound[:, -1] += cfg.end_factor*wr
143     self.bound=self.bound.ravel()
144     # Fixed sparse block pattern, row divided by its own storage.
145     rows=[];cols=[]; edgeid=[]; signs=[]
146     for dest,sgn in ((self.left,-1.), (self.right,1.)):
147         for src in (self.left,self.right):
148             for a in range(2):
149                 for b in range(2):
150                     rows.append(2*dest+a);cols.append(2*src+b)
151     self.jrows=np.r_[np.concatenate(rows),2*np.arange(self.size),2*np.arange(self.size)
152     ↪ e),2*np.arange(self.size)+1]
153     self.jcols=np.r_[np.concatenate(cols),2*np.arange(self.size)+1,2*np.arange(self.s
154     ↪ ize),2*np.arange(self.size)+1]
155 def rhs(self,t,y,env,jac=False):
156     q=y.reshape(self.size,2);T=q[:,0];C=q[:,1]
157     S,k,D,Sc,kc=props(T,C);l=self.left;r=self.right;g=self.geo
158     kl=k[1];kr=k[r];ks=kl+kr;kf=2*kl*kr/ks
159     dT=T[1]-T[r];dC=C[1]-C[r]
160     tq=g*kf*dT
161     if self.cfg.flux=='integral':
162         Tf=(T[1]+T[r])/2;Af=.0024*np.exp(-3850/(Tf+273.15));at=3850/(Tf+273.15)**2/2
163         mf=g*Af*Fdiff(C[1],C[r])
164         mcl=g*Af*np.exp(-.45/C[1]);mcr=-g*Af*np.exp(-.45/C[r])
165         mtl=mf*at;mtr=mtl
166     else:
167         Dl=D[1];Dr=D[r]
168         if self.cfg.flux=='harmonic':
169             ds=Dl+Dr;df=2*Dl*Dr/ds;fl=2*(Dr/ds)**2;fr=2*(Dl/ds)**2
170             elif self.cfg.flux=='arithmetic':df=(Dl+Dr)/2;fl=fr=.5
171             else:raise ValueError(self.cfg.flux)
172         mf=g*df*dC
173         mcl=g*(df+fl*Dl*.45/C[1]**2*dC);mcr=g*(-df+fr*Dr*.45/C[r]**2*dC)
174         mtl=g*fl*Dl*3850/(T[1]+273.15)**2*dC;mtr=g*fr*Dr*3850/(T[r]+273.15)**2*dC
175     rhs=np.empty_like(q)
176     rhs[:,0]=np.bincount(r,weights=tq,minlength=self.size)-np.bincount(l,weights=tq,m
177     ↪ inlength=self.size)
178     rhs[:,1]=np.bincount(r,weights=mf,minlength=self.size)-np.bincount(l,weights=mf,m
179     ↪ inlength=self.size)
180     et,ec=env(t);rhs[:,0]-=self.bound*self.cfg.hT*(T-et);rhs[:,1]-=self.bound*self.cf
181     ↪ g.hm*(C-ec)
182     storage=self.w[:,None]*np.column_stack([S,np.ones(self.size)])
183     rhs/=storage
184     iso=self.cfg.isothermal_after>0 and t>=self.cfg.isothermal_after
185     if iso:rhs[:,0]=0
186     if not jac:return rhs.ravel()
187     bl=np.empty((len(l),2,2));br=np.empty_like(bl)
188     bl[:,0,0]=g*kf;br[:,0,0]=-g*kf
189     bl[:,0,1]=g*2*(kr/ks)**2*kc[1]*dT;br[:,0,1]=g*2*(kl/ks)**2*kc[r]*dT
190     bl[:,1,0]=mtl;br[:,1,0]=mtr;bl[:,1,1]=mcl;br[:,1,1]=mcr
191     if iso:bl[:,0,:]=0;br[:,0,:]=0
192     vals=[]
193     for dest,sgn in ((1,-1.), (r,1.)):
194         for deriv in (bl,br):
195             for a in range(2):
196                 for b in range(2):vals.append(sgn*deriv[:,a,b]/storage[dest,a])
197     vals.extend([-rhs[:,0]*Sc/S,-self.bound*self.cfg.hT/storage[:,0]*(0 if iso else
198     ↪ 1),-self.bound*self.cfg.hm/self.w])
199     return coo_matrix((np.concatenate(vals),(self.jrows,self.jcols)),shape=(2*self.s
200     ↪ ize,2*self.size)).tocsc()
201 def profile(self,y):
202     field=np.asarray(y).reshape(self.shape+(2,))[0,:0,:1]
203     return PchipInterpolator(self.r**2,field,axis=0)(np.linspace(0,self.R,21)**2)

```

```

199 def derivative_dense(poly,t):
200     if hasattr(poly,'D'):
201         x=(t-poly.t_shift)/poly.denom;prod=1.;der=0.;ans=np.zeros(poly.D.shape[1])
202         for j in range(poly.order):
203             der=der*x[j]+prod/poly.denom[j];prod*=x[j];ans+=poly.D[j+1]*der
204         return ans
205     # Radau dense polynomial y0 + Q*[x,x^2,...]
206     x=(t-poly.t_old)/poly.h
207     return poly.Q@((np.arange(poly.order+1)+1)*x**np.arange(poly.order+1))/poly.h
208
209
210 def solve(cfg:Config,path:Path,input_path:Path):
211     if path.with_suffix('.json').exists(): raise FileExistsError(f'Refusing overwrite:
↳ {path}')
212     data=load_input(input_path);env=Environment(data,cfg.scenario,cfg.interpolation);op=0
↳ perator(cfg)
213     y=np.tile([28.,2.55],op.size);times=[0.];samples=[op.profile(y)];snapt=[0.];snaps=[y.
↳ copy()]
214     fulls=[y.copy()] if cfg.full_samples else []
215     event_times=[];event_fields=[]
216     diagnostics=[];balances=[];stats={'config':asdict(cfg),'python':sys.version,'numpy':n
↳ p.__version__, 'scipy':scipy.__version__, 'future':env.future.tolist(),'nodes':op.s
↳ ize,'min_dr_m':float(np.diff(op.r).min()),'max_dr_m':float(np.diff(op.r).max()),'
↳ nfev':0,'njev':0,'nlu':0,'steps':0}
217     cuts=np.unique(np.r_[data[:,0],np.arange(21600.,cfg.end_s,21600.),cfg.end_s,([cfg.iso
↳ thermal_after] if cfg.isothermal_after else [])]);cuts=cuts[cuts<=cfg.end_s]
218     tic=time.perf_counter();quad=np.polynomial.legendre.leggauss(4);qint=jint=stored=0.;e
↳ vent=None;projections=[]
219     minC=2.55;maxC=2.55;minT=28.;maxT=28.;max_up=0.;max_radial=0.;max_axial=0.;max_argmax=0
220     root_slope=0.;surface_event=None;mean_event=None
221     nextout=60.
222     for a,b in zip(cuts[:-1],cuts[1:]):
223         if cfg.isothermal_after and a==cfg.isothermal_after:
224             dy=np.max(abs(y[:,2]-env.future[0]));projections.append({'t_s':float(a),'max_
↳ temperature_projection_K':float(dy)})
225             y[:,2]=env.future[0]
226             e=env.segment(a,b)
227             fun=lambda t,v:op.rhs(t,v,e)
228             def evt(t,v):return np.max(v[1:2])-.15
229             evt.direction=-1;evt.terminal=False
230             sol=solve_ivp(fun,(a,b),y,method=cfg.method,jac=lambda
↳ t,v:op.rhs(t,v,e,True),rtol=cfg.rtol,atol=np.tile([cfg.atol_T,cfg.atol_C],op.
↳ size),max_step=cfg.early_step if a<14400 else
↳ cfg.max_step,dense_output=True,events=evt)
231             if not sol.success:raise RuntimeError(f'Failed on {(a,b)}: {sol.message}')
232             if not np.isfinite(sol.y).all() or np.any(sol.y[1:2]<=0):raise
↳ FloatingPointError('Invalid accepted field')
233             stats['steps']+=len(sol.t)-1
234             for k in ('nfev','njev','nlu'):stats[k]+=int(getattr(sol,k))
235             if len(sol.t_events[0]):
236                 root=float(sol.t_events[0][0]);delta=.01
237                 # Bracket width is reporting evidence, NOT a bound for discretization error.
238                 tm=root-delta;tp=root+delta
239                 yr=sol.sol(root);ym=sol.sol(tm);yp=sol.sol(tp)
240                 slope=float(fun(root,yr)[1:2][np.argmax(yr[1:2])])
241                 event={'critical_s':root,'critical_h':root/3600,'critical_max':float(np.max(y
↳ r[1:2])), 't_minus_s':tm,'max_minus':float(np.max(ym[1:2])), 't_plus_s':t
↳ p,'max_plus':float(np.max(yp[1:2])), 'critical_slope_kgkg_s':slope,'brack
↳ et_s':2*delta,'argmax_flat_index':int(np.argmax(yr[1:2]))}
242                 actual_end=cfg.execution_time_s if cfg.execution_time_s else tp
243                 if not (actual_end>root and actual_end<=b):raise ValueError('Execution time
↳ must be after the root and in its resolved segment')
244                 yexec=sol.sol(actual_end)

```

```

245     event['execution_s']=float(actual_end);event['execution_h']=float(actual_end/
    ↪ 3600)
246     event['execution_max']=float(np.max(yexec[1::2]))
247     event['strict_execution_pass']=bool(np.max(yexec[1::2])<.15)
248     if not event['strict_execution_pass']:raise AssertionError('Execution state
    ↪ not strictly qualified')
249     event_times=[tm,root,tp,actual_end];event_fields=[ym.copy(),yr.copy(),yp.copy()
    ↪ (),yexec.copy())]
250 else:actual_end=b
251 qtimes=np.arange(nextout,actual_end+1e-8,60.)
252 if len(qtimes):
253     vals=sol.sol(qtimes)
254     for i,t in enumerate(qtimes):
255         samples.append(op.profile(vals[:,i]));times.append(float(t))
256         if cfg.full_samples:fulls.append(vals[:,i].copy())
257     nextout=float(qtimes[-1]+60)
258 # Full mesh checks at each accepted time and each mid-step, not just 21 exported
    ↪ radii.
259 ct=np.unique(np.r_[sol.t[sol.t<=actual_end],(sol.t[:-1]+sol.t[1:])/2,actual_end])
    ↪ ;ct=ct[ct<=actual_end]
260 for t in ct:
261     u=sol.sol(t).reshape(op.shape+(2,));cc=u[:, :, 1];tt=u[:, :, 0]
262     minC=min(minC,float(cc.min()));maxC=max(maxC,float(cc.max()));minT=min(minT,f
    ↪ loat(tt.min()));maxT=max(maxT,float(tt.max()))
263     max_radial=max(max_radial,float(np.diff(cc,axis=0).max()));max_argmax=max(max
    ↪ _argmax,int(np.argmax(cc)))
264     if cfg.nz:max_axial=max(max_axial,float(np.diff(cc,axis=1).max()))
265 for i,t in enumerate(sol.t):
266     if t>actual_end:break
267     u=sol.y[:,i].reshape(op.size,2);mean=float(op.w@u[:,1]/op.vol)
268     if diagnostics:max_up=max(max_up,float(np.max(u[:,1])-diagnostics[-1][1]))
269     diagnostics.append([float(t),float(np.max(u[:,1])),float(np.min(u[:,1])),mean
    ↪ ,float(u[0,0]),float(u[-1,0]),float(np.sum(op.bound*cfg.hm*(u[:,1]-e(t)[1]
    ↪ ))/op.vol)])
270 # Surface and weighted-average threshold events are diagnostic only.
271 for typ,func in [('surface',lambda
    ↪ v:v.reshape(op.shape+(2,))[-1,0,1]-.15),('mean',lambda
    ↪ v:op.w@v[1::2]/op.vol-.15)]:
272     if (surface_event if typ=='surface' else mean_event) is None:
273         if func(sol.y[:,0])>=0 and func(sol.y[:, -1])<0:
274             tr=brentq(lambda t:func(sol.sol(t)),a,b,xtol=1e-8)
275             if typ=='surface':surface_event=tr
276             else:mean_event=tr
277 if cfg.quadrature:
278     for lo,hi,poly in zip(sol.t[:-1],sol.t[1:],sol.sol.interpolants):
279         if lo>=actual_end:break
280         hi=min(hi,actual_end)
281         for g,wg in zip(*quad):
282             tq=(lo+hi)/2+(hi-lo)/2*g;wt=wg*(hi-lo)/2
283             uu=poly(tq).reshape(op.size,2);du=derivative_dense(poly,tq).reshape(o
    ↪ p.size,2)
284             S=props(uu[:,0],uu[:,1])[0];et,ec=e(tq)
285             jrate=np.sum(op.bound*cfg.hm*(uu[:,1]-ec))/op.vol;qrate=np.sum(op.bou
    ↪ nd*cfg.hT*(uu[:,0]-et))/op.vol
286             jint+=wt*jrate;qint+=wt*qrate;stored+=wt*(op.w@(S*du[:,0]))/op.vol
287             endy=sol.sol(actual_end).reshape(op.size,2)
288             balances.append([actual_end,float(op.w@endy[:,1]/op.vol-2.55+jint),stored+qin
    ↪ t,t,jint,qint,stored])
289 if b in (1800,3600,5400,7200,9000,10800,14400,21600) or b%21600==0:
290     if b<=actual_end:snap.append(float(b));snaps.append(sol.y[:, -1].copy())
291 y=sol.sol(actual_end)
292 if event:
293     times.append(actual_end);samples.append(op.profile(y));snap.append(actual_en
    ↪ d);snaps.append(y.copy())

```

```

294         if cfg.full_samples:fulls.append(y.copy())
295         break
296         if b%21600==0 or b==14400:print(f'n={cfg.n} nz={cfg.nz} {cfg.flux} t={b/3600:g}h
↪ M={y[1::2].max():.8f} elapsed={time.perf_counter()-tic:.1f}s',flush=True)
297         # Free the previous segment's dense history before allocating the next one.
298         sol=None
299         stats.update({'elapsed_s':time.perf_counter()-tic,'event':event,'global_C_min':minC,'
↪ global_C_max':maxC,'global_T_min':minT,'global_T_max':maxT,'max_radial_increase':
↪ max_radial,'max_axial_increase':max_axial,'max_argmax_flat_index':max_argmax,'max
↪ _M_increase_accepted':max_up,'surface_crossing_s':surface_event,'mean_crossing_s':
↪ mean_event,'projections':projections})
300         if balances:
301             bb=np.array(balances);stats['max_water_balance_kgkg']=float(abs(bb[:,1]).max());s
↪ tats['max_effective_heat_balance_J_m3']=float(abs(bb[:,2]).max());stats['effe
↪ ctive_heat_relative']=float(abs(bb[:,2]).max()/max(1.,abs(qint)))
302         result={'time_s':np.array(times),'radius_cm':np.linspace(0,2,21),'sample_TC':np.array
↪ (samples),'r_m':op.r,'z_m':op.z,'weights':op.w.reshape(op.shape),'snapshot_time_s'
↪ :np.array(snapt),'snapshots':np.array(snaps).reshape((-1,)+op.shape+(2,)), 'diagno
↪ stics':np.array(diagnostics),'balance':np.array(balances)}
303         result['event_time_s']=np.array(event_times)
304         result['event_fields_TC']=np.array(event_fields).reshape((-1,)+op.shape+(2,))
305         stats['source_sha256']=hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
306         stats['input_sha256']=hashlib.sha256(input_path.read_bytes()).hexdigest()
307         if cfg.full_samples:result['full_TC']=np.array(fulls).reshape((-1,)+op.shape+(2,))
308         path.parent.mkdir(parents=True,exist_ok=True);np.savez_compressed(path.with_suffix('.
↪ npz'),**result)
309         path.with_suffix('.json').write_text(json.dumps(stats,ensure_ascii=False,indent=2),en
↪ coding='utf-8')
310         print(json.dumps(stats,ensure_ascii=False),flush=True)
311         return stats
312
313     def main():
314         p=argparse.ArgumentParser();p.add_argument('--input',type=Path,default=Path(__file__)
↪ .resolve().parents[1]/'inputs/attachment1.xlsx');p.add_argument('--out',type=Path
↪ ,required=True);p.add_argument('--config',type=Path)
315         p.add_argument('--n',type=int,default=256);p.add_argument('--nz',type=int,default=0);
↪ p.add_argument('--flux',default='integral');p.add_argument('--scenario',default='
↪ mean');p.add_argument('--rtol',type=float,default=2e-9);p.add_argument('--max-ste
↪ p',type=float,default=240.);p.add_argument('--quadrature',action='store_true');p.
↪ add_argument('--full-samples',action='store_true')
316         args=p.parse_args();cfg=Config(**json.loads(args.config.read_text())) if args.config
↪ else
↪ Config(n=args.n,nz=args.nz,flux=args.flux,scenario=args.scenario,rtol=args.rtol,m
↪ ax_step=args.max_step,quadrature=args.quadrature,full_samples=args.full_samples)
317         solve(cfg,args.out,args.input)
318     if __name__=='__main__':main()

```

A.26 q3_reference

support/project/q3_refinement_delivery/source/q3_reference.py

```

1  """Independent global Chebyshev collocation with primitive-gradient moisture flux.
2  No FV control volumes, face weights, or FV spatial operator are reused.
3  Full coupled heat/moisture evolution from t=0; Robin boundary algebraically solved.
4  Grid/differentiation construction follows the audited q2_spectral.py approach;
5  moisture flux and its boundary Jacobian are newly derived for long-time Q3.
6  """
7  from __future__ import annotations
8  import argparse,json,time
9  from pathlib import Path
10 import numpy as np

```

```

11 from numpy.polynomial import Chebyshev
12 from scipy.special import exp1
13 from scipy.optimize import brentq
14 from scipy.integrate import solve_ivp
15 from scipy.fft import dct
16 from q3_solver import load_input, Environment
17
18 def primitive(c):
19     if np.any(np.real(c)<=0):raise FloatingPointError('C<=0 in reference')
20     return c*np.exp(-.45/c)-.45*exp1(.45/c)
21
22 def grid(n):
23     x=(1-np.cos(np.pi*np.arange(n+1)/n))/2
24     w=(-1.)*np.arange(n+1);w[[0,-1]]*=-.5
25     dx=x[:,None]-x[None,:]
26     G=np.divide(w[None,:]/w[:,None],dx,out=np.zeros_like(dx),where=dx!=0)
27     G[np.diag_indices(n+1)]=-G.sum(axis=1);G[:, -1]=-G[:, : -1].sum(axis=1)
28     return x,G,w
29
30 def interp(x,w,target):
31     d=target[:,None]-x
32     out=np.empty_like(d)
33     for j,dd in enumerate(d):
34         k=np.argmin(abs(dd))
35         if abs(dd[k])<1e-14:out[j]=0;out[j,k]=1
36         else:v=w/dd;out[j]=v/v.sum()
37     return out
38
39 def inverse_primitive(v,guess):
40     c=np.asarray(guess).copy()
41     for _ in range(40):
42         delta=(primitive(c)-v)/np.exp(-.45/c)
43         cnew=c-delta
44         # Globalize Newton without altering the equation or accepted value.
45         bad=np.real(cnew)<=0
46         while np.any(bad):delta[bad]*=-.5;cnew=c-delta;bad=np.real(cnew)<=0
47         c=cnew
48         if np.max(abs(delta))<3e-14:break
49     else:raise RuntimeError('Inverse primitive failed')
50     return c
51
52 class Reference:
53     def __init__(self,n):
54         self.n=n;self.x,self.G,self.w=grid(n);self.row=self.G[-1,:-1];self.diag=self.G[-1,-1]
55         self.fac=4/.02**2;self.bd=2/.02
56         self.BT0=np.zeros((n+1,2*n));self.BC0=np.zeros_like(self.BT0)
57         self.BT0[np.arange(n),2*np.arange(n)]=1;self.BC0[np.arange(n),2*np.arange(n)+1]=1
58         self.I=interp(self.x,self.w,np.linspace(0,1,21)**2)
59         self.denseI=interp(self.x,self.w,np.linspace(0,1,8*n+1))
60         self.min_boundary_derivative=float('inf');self.max_bc_residual=0.;self.max_newton=0;self
        ↪ f.boundary_fallbacks=0
61     def surface(self,t,u,env,jac=False):
62         Ti,Ci=u.T;et,ec=env(t);Ui=primitive(Ci);anchor=Ci[-1];ua=Ui[-1]
63         gt=self.row@(Ti-et);gu=self.row@(Ui-ua)
64         def calc(c):
65             k=.21+.38*c/(1+c);kc=.38/(1+c)**2
66             den=self.bd*k*self.diag+25
67             Ts=et-self.bd*k*gt/den
68             Tsc=-self.bd*kc*gt*25/den**2
69             A=.0024*np.exp(-3850/(Ts+273.15));At=A*3850/(Ts+273.15)**2
70             ux=gu+self.diag*(primitive(c)-ua)
71             f=np.exp(-.45/c)
72             val=self.bd*A*ux+8e-7*(c-ec)
73             deriv=self.bd*(At*Tsc*ux+A*self.diag*f)+8e-7

```

```

74     return val,deriv,Ts,k,kc,A,At,ux
75     c=anchor
76     for it in range(25):
77         val,deriv,*_=calc(c);step=val/deriv
78         while c-step<=0:step*=-.5
79         c=c-step
80         if abs(step)<2e-14*(1+abs(c)):break
81     else:
82         self.boundary_fallbacks+=1
83         lo=min(ec,float(Ci.min()))*.1;hi=max(ec,float(Ci.max()))*1.1
84         c=brentq(lambda z:calc(z)[0],lo,hi,xtol=5e-15)
85     val,deriv,Ts,k,kc,A,At,ux=calc(c)
86     self.max_newton=max(self.max_newton,it+1);self.min_boundary_derivative=min(self.min_bou
↵ ndary_derivative,float(deriv));self.max_bc_residual=max(self.max_bc_residual,float(
↵ abs(val)))
87     full=np.vstack([u,[Ts,c]])
88     if not jac:return full
89     tx=self.row@(Ti-Ts)
90     B=np.array([[self.bd*k*self.diag+25,self.bd*k*tx],[self.bd*At*ux,self.bd*A*self.diag*n
↵ p.exp(-.45/c)+8e-7]])
91     Q=np.zeros((2,2*self.n));Q[0,:2]=self.bd*k*self.row;Q[1,1:2]=self.bd*A*self.row*np.ex
↵ p(-.45/Ci)
92     sens=-np.linalg.solve(B,Q);BT=self.BT0.copy();BC=self.BC0.copy();BT[-1]=sens[0];BC[-1]=
↵ sens[1]
93     return full,BT,BC
94 def evaluate(self,t,y,env,jac=False):
95     if jac:full,BT,BC=self.surface(t,y.reshape(self.n,2),env,True)
96     else:full=self.surface(t,y.reshape(self.n,2),env)
97     T,C=full.T
98     if np.any(T<=-273.15):raise FloatingPointError('Kelvin<=0')
99     rho=650+128*C;cp=1450+2736*C/(1+C);S=rho*cp;Sc=128*cp+rho*2736/(1+C)**2
100    k=.21+.38*C/(1+C);kc=.38/(1+C)**2;A=.0024*np.exp(-3850/(T+273.15));At=A*3850/(T+273.15)
↵ **2
101    U=primitive(C);gt=self.G@(T-T[-1]);gu=self.G@(U-U[-1]);qt=k*gt;qm=A*gu
102    ht=qt+self.x*(self.G@(qt-qt[-1]));hm=qm+self.x*(self.G@(qm-qm[-1]))
103    fT=self.fac*ht/S;fC=self.fac*hm
104    if not jac:return np.column_stack([fT[: -1],fC[: -1]]).ravel()
105    BU=np.exp(-.45/C)[: ,None]*BC
106    gtj=self.G@(BT-BT[-1]);guj=self.G@(BU-BU[-1])
107    qtj=k[: ,None]*gtj+(kc*gt)[: ,None]*BC
108    qmj=A[: ,None]*guj+(At*gu)[: ,None]*BT
109    htj=qtj+self.x[: ,None]*(self.G@(qtj-qtj[-1]));hmj=qmj+self.x[: ,None]*(self.G@(qmj-qmj[-
↵ 1]))
110    J=np.empty((2*self.n,2*self.n));J[:2,:2]=
↵ (self.fac*htj/S[: ,None]-(fT*Sc/S)[: ,None]*BC)[: -1];J[1:2,:2]=(self.fac*hmj)[: -1]
111    return J
112 def profiles(self,u):
113     T,C=u.T;U=primitive(C);v=self.I@U;guess=self.I@C
114     return np.column_stack([self.I@T,inverse_primitive(v,guess)])
115 def extrema(self,u):
116     # Interpolate U, not steep C. All real stationary points of the polynomial.
117     U=primitive(u[: ,1]);co=dct(U[: -1],type=1)/self.n;co[[0,-1]]*=-.5
118     p=Chebyshev(co);roots=p.deriv().roots();roots=roots[np.isreal(roots)].real;roots=roots[
↵ (roots>-1)&(roots<1)]
119     pts=np.r_[-1,roots,1];vals=p(pts);idx=np.argmax(vals)
120     maxi=inverse_primitive(np.array([vals[idx]]),np.array([u[: ,1].max()]))[0]
121     return {'max_C':float(maxi),'x_max':float((pts[idx]+1)/2),'stationary_points':len(roots)
↵ },'minimum_U':float(vals.min())}
122
123 def run(n,out,input_file,method='Radau',rtol=2e-11,atol_C=2e-13,max_step=120.):
124     if out.with_suffix('.json').exists():raise FileExistsError(out)
125     op=Reference(n);data=load_input(input_file);env=Environment(data,'mean');y=np.tile([28.,
↵ 2.55],n)
126     cuts=np.r_[data[: ,0],np.arange(21600,360001,21600.)]

```

```

127 times=[0.];vals=[np.tile([28.,2.55],[21,1))];snapt=[0.];snaps=[np.tile([28.,2.55],[n+1,1
↪ )]);ext=[]
128 nextt=60.;tic=time.perf_counter();nsteps=nfev=nlu=0;event=None;max_dense_overshoot=0.;gl
↪ obalmin=2.55
129 for a,b in zip(cuts[:-1],cuts[1:]):
130     e=env.segment(a,b)
131     fun=lambda t,y:op.evaluate(t,y,e)
132     def ev(t,y):return np.max(y[1::2])-.15
133     ev.terminal=False;ev.direction=-1
134     sol=solve_ivp(fun,(a,b),y,method=method,jac=lambda t,y:op.evaluate(t,y,e,True),rtol=rto
↪ l,atol=np.tile([2e-11,atol_C],n),max_step=min(max_step,30.) if a<14400 else
↪ max_step,dense_output=True,events=ev)
135     if not sol.success:raise RuntimeError(sol.message)
136     nsteps+=len(sol.t)-1;nfev+=sol.nfev;nlu+=sol.nlu;end=b
137     if len(sol.t_events[0]):
138         root=float(sol.t_events[0][0]);end=root+.01;yr=sol.sol(root);ur=op.surface(root,yr.res
↪ hape(n,2),e)
139         event={'critical_s':root,'critical_h':root/3600,'t_minus_s':root-.01,'max_minus':float
↪ (np.max(sol.sol(root-.01)[1::2])), 't_plus_s':end,'max_plus':float(np.max(sol.sol(en
↪ d)[1::2])), 'slope':float(fun(root,yr)[1::2][np.argmax(yr[1::2]))], 'polynomial_extr
↪ ema':op.extrema(ur)}
140     ts=np.arange(nextt,end+1e-8,60)
141     if event:ts=np.r_[ts,end]
142     for t in ts:
143         uu=op.surface(t,sol.sol(t).reshape(n,2),e);times.append(float(t));vals.append(op.profi
↪ les(uu))
144         denseU=op.denseI@primitive(uu[:,1]);max_dense_overshoot=max(max_dense_overshoot,float(
↪ denseU.max()-primitive(uu[:,1]).max()));globalmin=min(globalmin,float(uu[:,1].min(
↪ )))
145         if len(ts):nextt=60*(np.floor(ts[-1]/60)+1)
146         if b in (1800,3600,5400,7200,9000,10800,14400,21600) or b%21600==0 or event:
147             u=op.surface(end,sol.sol(end).reshape(n,2),e);snapt.append(float(end));snaps.append(u)
↪ ;ext.append({'t_s':float(end),**op.extrema(u)})
148             y=sol.sol(end)
149             if b%21600==0 or
↪ event:print(n,b/3600,float(y[1::2].max()),time.perf_counter()-tic,flush=True)
150             if event:break
151     stats={'n':n,'method':method,'rtol':rtol,'atol_C':atol_C,'max_step':max_step,'event':eve
↪ nt,'elapsed_s':time.perf_counter()-tic,'nsteps':nsteps,'nfev':nfev,'nlu':nlu,'min_sc
↪ alar_boundary_derivative':op.min_boundary_derivative,'max_boundary_residual':op.max_
↪ bc_residual,'max_boundary_newton':op.max_newton,'boundary_fallbacks':op.boundary_fal
↪ lbacks,'dense_U_overshoot':max_dense_overshoot,'global_C_min':globalmin,'extrema':ex
↪ t}
152     out.parent.mkdir(parents=True,exist_ok=True);out.with_suffix('.json').write_text(json.du
↪ mps(stats,indent=2));np.savez_compressed(out.with_suffix('.npz'),time_s=np.array(tim
↪ es),sample_TC=np.array(vals),x=op.x,snapshot_time_s=np.array(snapt),snapshots=np.arr
↪ ay(snaps))
153     print(json.dumps(stats),flush=True)
154 if __name__=='__main__':
155     p=argparse.ArgumentParser();p.add_argument('--n',type=int,default=80);p.add_argument('--
↪ out',type=Path,required=True);p.add_argument('--method',default='Radau');p.add_argum
↪ ent('--rtol',type=float,default=2e-11);p.add_argument('--input',type=Path,default=Pa
↪ th(__file__).resolve().parents[1]/'inputs/attachment1.xlsx');a=p.parse_args();run(a.
↪ n,a.out,a.input,a.method,a.rtol)

```

A.27 q3_refined_reference

support/project/q3_refinement_delivery/source/q3_refined_reference.py

```
1 """Refined Q3 reference solver used for the frozen refinement delivery.
2
3 Main features relative to the historical reference solver:
4 - future-environment scenario is explicit (mean / nominal / time_mean / last);
5 - strict execution endpoint is derived automatically from the event root,
6   numerical budget and output quantum; no historical endpoint is hard-coded;
7 - the execution endpoint is actually evaluated on the same continuous solution;
8 - all official 60 s / 0.1 cm output values come from one unrounded trajectory;
9 - output refuses overwrite if either JSON or NPZ already exists.
10
11 The spatial method remains the audited global Chebyshev collocation in  $x=(r/R)^2$ 
12 with primitive-gradient moisture flux and algebraic Robin surface conditions.
13 """
14 from __future__ import annotations
15
16 import argparse
17 import hashlib
18 import json
19 import math
20 import os
21 import tempfile
22 import time
23 from pathlib import Path
24
25 import numpy as np
26 from scipy.integrate import solve_ivp
27
28 from q3_reference import Reference
29 from q3_solver import Environment, load_input
30
31
32 class RefinedEnvironment(Environment):
33     def __init__(self, data, scenario="mean", kind="linear"):
34         if scenario == "time_mean":
35             # Initialize interpolation machinery without accepting an unknown scenario.
36             super().__init__(data, scenario="mean", kind=kind)
37             mask = self.data[:, 0] >= 10800.0
38             tail = self.data[mask]
39             duration = tail[-1, 0] - tail[0, 0]
40             self.future = np.trapezoid(tail[:, 1:], tail[:, 0], axis=0) / duration
41             self.scenario = scenario
42         else:
43             super().__init__(data, scenario=scenario, kind=kind)
44
45
46 def _atomic_npz(path: Path, **arrays) -> None:
47     path.parent.mkdir(parents=True, exist_ok=True)
48     fd, tmp_name = tempfile.mkstemp(prefix=path.name + ".", suffix=".tmp.npz",
49     ↪ dir=path.parent)
50     os.close(fd)
51     tmp = Path(tmp_name)
52     try:
53         np.savez_compressed(tmp, **arrays)
54         os.replace(tmp, path)
55     finally:
56         if tmp.exists():
57             tmp.unlink()
58
59 def _atomic_text(path: Path, text: str) -> None:
```

```

60     path.parent.mkdir(parents=True, exist_ok=True)
61     fd, tmp_name = tempfile.mkstemp(prefix=path.name + ".", suffix=".tmp", dir=path.parent)
62     os.close(fd)
63     tmp = Path(tmp_name)
64     try:
65         tmp.write_text(text, encoding="utf-8")
66         os.replace(tmp, path)
67     finally:
68         if tmp.exists():
69             tmp.unlink()
70
71
72 def derive_execution_time(root_s: float, budget_s: float, quantum_s: float) -> float:
73     if budget_s < 0 or quantum_s <= 0:
74         raise ValueError("budget_s must be >= 0 and quantum_s must be > 0")
75     # Tiny subtraction avoids an accidental extra quantum from binary representation
76     # when the mathematical quotient is already an integer.
77     return quantum_s * math.ceil((root_s + budget_s) / quantum_s - 1e-13)
78
79
80 def run(
81     *,
82     n: int,
83     out: Path,
84     input_file: Path,
85     scenario: str = "mean",
86     method: str = "Radau",
87     rtol: float = 2e-11,
88     atol_c: float = 2e-13,
89     max_step: float = 120.0,
90     budget_s: float = 0.03,
91     quantum_s: float = 0.36,
92     interpolation: str = "linear",
93 ):
94     json_path = out.with_suffix(".json")
95     npz_path = out.with_suffix(".npz")
96     if json_path.exists() or npz_path.exists():
97         raise FileExistsError(f"Refusing overwrite: {json_path} / {npz_path}")
98
99     data = load_input(input_file)
100     env = RefinedEnvironment(data, scenario=scenario, kind=interpolation)
101     op = Reference(n)
102     y = np.tile([28.0, 2.55], n)
103
104     # Segment at all observed boundary knots; after 4 h, use wider blocks only.
105     cuts = np.unique(np.r_[data[:, 0], np.arange(21600.0, 360001.0, 21600.0)])
106     times = [0.0]
107     profiles = [np.tile([28.0, 2.55], (21, 1))]
108     snapshot_times = [0.0]
109     snapshot_full = [np.tile([28.0, 2.55], (n + 1, 1))]
110     event_full = []
111     event_times = []
112     extrema_log = []
113     next_output = 60.0
114
115     tic = time.perf_counter()
116     nsteps = nfev = nl = 0
117     critical = None
118     execution = None
119     execution_full = None
120
121     for a, b in zip(cuts[:-1], cuts[1:]):
122         seg_env = env.segment(a, b)
123         fun = lambda t, yy: op.evaluate(t, yy, seg_env)

```

```

124
125     def event(t, yy):
126         return float(np.max(yy[1::2]) - 0.15)
127
128     event.terminal = False
129     event.direction = -1
130
131     sol = solve_ivp(
132         fun,
133         (a, b),
134         y,
135         method=method,
136         jac=lambda t, yy: op.evaluate(t, yy, seg_env, True),
137         rtol=rtol,
138         atol=np.tile([2e-11, atol_c], n),
139         max_step=min(max_step, 30.0) if a < 14400.0 else max_step,
140         dense_output=True,
141         events=event,
142     )
143     if not sol.success:
144         raise RuntimeError(sol.message)
145     nsteps += len(sol.t) - 1
146     nfev += sol.nfev
147     nlu += sol.nlu
148
149     segment_end = b
150     if len(sol.t_events[0]):
151         root = float(sol.t_events[0][0])
152         execution = derive_execution_time(root, budget_s, quantum_s)
153         if execution > b:
154             raise RuntimeError("Derived execution endpoint crosses unresolved
155                               ↪ integration segment")
156         segment_end = execution
157
158         y_root = sol.sol(root)
159         root_full = op.surface(root, y_root.reshape(n, 2), seg_env)
160         y_minus = sol.sol(root - 0.01)
161         y_plus = sol.sol(root + 0.01)
162         minus_full = op.surface(root - 0.01, y_minus.reshape(n, 2), seg_env)
163         plus_full = op.surface(root + 0.01, y_plus.reshape(n, 2), seg_env)
164         y_exec = sol.sol(execution)
165         execution_full = op.surface(execution, y_exec.reshape(n, 2), seg_env)
166
167         critical = {
168             "critical_s": root,
169             "critical_h": root / 3600.0,
170             "t_minus_s": root - 0.01,
171             "max_minus": float(np.max(minus_full[:, 1])),
172             "critical_max": float(np.max(root_full[:, 1])),
173             "t_plus_s": root + 0.01,
174             "max_plus": float(np.max(plus_full[:, 1])),
175             "critical_slope_kgkg_s": float(fun(root,
176                               ↪ y_root)[1::2][np.argmax(y_root[1::2])]),
177             "execution_s": float(execution),
178             "execution_h": float(execution / 3600.0),
179             "execution_max": float(np.max(execution_full[:, 1])),
180             "execution_argmax_x": float(op.extrema(execution_full)[x_max]),
181             "strict_execution_pass": bool(np.max(execution_full[:, 1]) < 0.15),
182             "budget_s": budget_s,
183             "quantum_s": quantum_s,
184         }
185     if not critical["strict_execution_pass"]:
186         raise AssertionError("Derived execution endpoint is not strictly
187                               ↪ qualified")

```

```

185         event_times = [root - 0.01, root, root + 0.01, execution]
186         event_full = [minus_full, root_full, plus_full, execution_full]
187
188     out_times = np.arange(next_output, segment_end + 1e-8, 60.0)
189     # For the event segment the execution endpoint is not an integer minute.
190     if critical is not None:
191         out_times = out_times[out_times < execution - 1e-9]
192     for t in out_times:
193         full = op.surface(t, sol.sol(t).reshape(n, 2), seg_env)
194         times.append(float(t))
195         profiles.append(op.profiles(full))
196     if len(out_times):
197         next_output = 60.0 * (np.floor(out_times[-1] / 60.0) + 1.0)
198
199     if b in (1800.0, 3600.0, 5400.0, 7200.0, 9000.0, 10800.0, 14400.0, 21600.0) or b %
200     ↪ 21600.0 == 0 or critical is not None:
201         snap_t = segment_end if critical is not None else b
202         full = op.surface(snap_t, sol.sol(snap_t).reshape(n, 2), seg_env)
203         snapshot_times.append(float(snap_t))
204         snapshot_full.append(full)
205         extrema_log.append({"t_s": float(snap_t), **op.extrema(full)})
206
207     y = sol.sol(segment_end)
208     if critical is not None:
209         times.append(float(execution))
210         profiles.append(op.profiles(execution_full))
211         break
212
213     if critical is None:
214         raise RuntimeError("Threshold event not reached")
215
216     stats = {
217         "model": "global Chebyshev collocation in  $x=(r/R)^2$  with primitive-gradient
218         ↪ moisture flux",
219         "n": n,
220         "method": method,
221         "rtol": rtol,
222         "atol_C": atol_c,
223         "max_step_s": max_step,
224         "scenario": scenario,
225         "interpolation": interpolation,
226         "future": env.future.tolist(),
227         "event": critical,
228         "elapsed_s": time.perf_counter() - tic,
229         "nsteps": nsteps,
230         "nfev": nfev,
231         "nlu": nlu,
232         "min_scalar_boundary_derivative": op.min_boundary_derivative,
233         "max_boundary_residual": op.max_bc_residual,
234         "max_boundary_newton": op.max_newton,
235         "boundary_fallbacks": op.boundary_fallbacks,
236         "extrema": extrema_log,
237         "input_sha256": hashlib.sha256(input_file.read_bytes()).hexdigest(),
238         "source_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
239     }
240
241     arrays = {
242         "time_s": np.asarray(times),
243         "radius_cm": np.linspace(0.0, 2.0, 21),
244         "profile_IC": np.asarray(profiles),
245         "x": op.x,
246         "snapshot_time_s": np.asarray(snapshot_times),
247         "snapshot_full_IC": np.asarray(snapshot_full),
248         "event_time_s": np.asarray(event_times),

```

```

247         "event_full_TC": np.asarray(event_full),
248     }
249     _atomic_npz(npz_path, **arrays)
250     _atomic_text(json_path, json.dumps(stats, ensure_ascii=False, indent=2) + "\n")
251     return stats
252
253
254 def main():
255     p = argparse.ArgumentParser()
256     p.add_argument("--input", type=Path, required=True)
257     p.add_argument("--out", type=Path, required=True)
258     p.add_argument("--n", type=int, default=80)
259     p.add_argument("--scenario", choices=["mean", "nominal", "time_mean", "last"],
260                    ↪ default="mean")
261     p.add_argument("--method", default="Radau")
262     p.add_argument("--rtol", type=float, default=2e-11)
263     p.add_argument("--atol-c", type=float, default=2e-13)
264     p.add_argument("--max-step", type=float, default=120.0)
265     p.add_argument("--budget-s", type=float, default=0.03)
266     p.add_argument("--quantum-s", type=float, default=0.36)
267     p.add_argument("--interpolation", choices=["linear", "pchip"], default="linear")
268     a = p.parse_args()
269     result = run(
270         n=a.n,
271         out=a.out,
272         input_file=a.input,
273         scenario=a.scenario,
274         method=a.method,
275         rtol=a.rtol,
276         atol_c=a.atol_c,
277         max_step=a.max_step,
278         budget_s=a.budget_s,
279         quantum_s=a.quantum_s,
280         interpolation=a.interpolation,
281     )
282     print(json.dumps(result, ensure_ascii=False))
283
284 if __name__ == "__main__":
285     main()

```

A.28 future_scenarios_from_checkpoint

support/project/q3_refinement_delivery/source/future_scenarios_from_checkpoint.py

```

1  """High-resolution N=2048 future-boundary comparisons from the verified 4 h full state.
2
3  This script reuses only a full internal checkpoint whose provenance is fixed by SHA256.
4  It does not reuse the 21-point display profile. From 4 h onward each requested
5  future environment is integrated with the same FV operator, tolerances, and all-domain
6  event definition as the historical main model.
7  """
8  from __future__ import annotations
9  import argparse, hashlib, json, time, math
10 from pathlib import Path
11 import numpy as np
12 from scipy.integrate import solve_ivp
13 from q3_solver import Config, Operator, load_input
14
15
16 def future_value(data, scenario):
17     if scenario == 'mean':

```

```

18         return data[data[:,0] >= 10800,1:].mean(axis=0)
19     if scenario == 'time_mean':
20         tail=data[data[:,0] >= 10800]
21         return np.trapezoid(tail[:,1:], tail[:,0], axis=0)/(tail[-1,0]-tail[0,0])
22     if scenario == 'nominal':
23         return np.array([50.0,0.05])
24     if scenario == 'last':
25         return data[-1,1:].copy()
26     raise ValueError(scenario)
27
28
29 def run(checkpoint, input_file, scenario, out):
30     if out.exists():
31         raise FileExistsError(out)
32     data=load_input(input_file)
33     z=np.load(checkpoint)
34     if 'state_TC' in z.files:
35         if abs(float(z['time_s'])-14400.0)>1e-8: raise ValueError('Checkpoint is not at 4
↪ h')
36         y=z['state_TC'].reshape(-1)
37     else:
38         st=z['snapshot_time_s']
39         idx=np.where(np.isclose(st,14400.0))[0]
40         if len(idx)!=1: raise ValueError('No unique 4 h checkpoint')
41         y=z['snapshots'][idx[0]].reshape(-1)
42     cfg=Config(n=2048,rtol=2e-12,atol_T=2e-12,atol_C=2e-14,max_step=60,early_step=15,quad
↪ rature=False,full_samples=False,execution_time_s=0)
43     op=Operator(cfg)
44     if y.size != 2*op.size: raise ValueError('Checkpoint grid mismatch')
45     fut=future_value(data,scenario)
46     env=lambda t: np.broadcast_to(fut,np.shape(t)+(2,))
47     fun=lambda t,v: op.rhs(t,v,env)
48     def ev(t,v): return float(np.max(v[1:2]))-0.15)
49     ev.direction=-1;ev.terminal=False
50     tic=time.perf_counter()
51     sol=solve_ivp(fun,(14400.0,206901.0),y,method='BDF',jac=lambda
↪ t,v:op.rhs(t,v,env,True),rtol=cfg.rtol,atol=np.tile([cfg.atol_T, cfg.atol_C],op.si
↪ ze),max_step=cfg.max_step,dense_output=True,events=ev)
52     if not sol.success or not len(sol.t_events[0]): raise RuntimeError(sol.message or
↪ 'event not reached')
53     root=float(sol.t_events[0][0]); yr=sol.sol(root); rootmax=float(np.max(yr[1:2]));
↪ argmax=int(np.argmax(yr[1:2])); execution=0.36*math.ceil((root+0.03)/0.36-1e-13);
↪ yexec=sol.sol(execution)
54     state_206901=sol.sol(206901.0)
55     obj={
56         'scenario':scenario,'future_T_C':float(fut[0]),'future_C_kgkg':float(fut[1]),
57         'critical_s':root,'critical_h':root/3600,'critical_max':rootmax,'critical_argmax_fl
↪ at_index':argmax,'execution_s':execution,'execution_h':execution/3600,'executio
↪ n_max':float(np.max(yexec[1:2])), 'strict_execution_pass':bool(np.max(yexec[1:
↪ 2])<0.15),
58         'Cmax_206901':None if state_206901 is None else float(np.max(state_206901[1:2])),
59         'strict_at_206901':None if state_206901 is None else
↪ bool(np.max(state_206901[1:2])<0.15),
60         'elapsed_s':time.perf_counter()-tic,'nfev':sol.nfev,'njev':sol.njev,'nlu':sol.nlu,
61         'checkpoint_sha256':hashlib.sha256(checkpoint.read_bytes()).hexdigest(),
62         'input_sha256':hashlib.sha256(input_file.read_bytes()).hexdigest(),
63         'source_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
64     }
65     out.parent.mkdir(parents=True,exist_ok=True);out.write_text(json.dumps(obj,ensure_asc
↪ ii=False,indent=2)+'\n',encoding='utf-8')
66     print(json.dumps(obj,ensure_ascii=False))
67
68 if __name__=='__main__':

```

```

69 p=argparse.ArgumentParser();p.add_argument('--checkpoint',type=Path,required=True);p.
↪ add_argument('--input',type=Path,required=True);p.add_argument('--scenario',choic
↪ es=['mean','time_mean','nominal','last'],required=True);p.add_argument('--out',ty
↪ pe=Path,required=True);a=p.parse_args();run(a.checkpoint,a.input,a.scenario,a.out)

```

A.29 export_workbook

support/project/q3_refinement_delivery/source/export_workbook.py

```

1  """Export the frozen Q3 refinement workbook from output/solution.npz.
2
3  The official workbook in this package was produced with artifact_tool. This
4  module provides the same artifact_tool path plus a standard-library OOXML
5  fallback for ordinary local reproduction. No formulas are used.
6  """
7  from __future__ import annotations
8  import argparse, io, json, os, zipfile
9  from pathlib import Path
10 import numpy as np
11
12
13 def payload(root: Path):
14     a=np.load(root/'output/solution.npz')
15     t=a['time_s'][1:]
16     c=a['profile_TC'][1:,:1]
17     if c.shape!=(len(t),21) or not np.all(np.diff(t)>0) or not np.isfinite(c).all():
18         raise ValueError('Invalid final data')
19     if abs(t[0]-60)>1e-10:
20         raise ValueError('Workbook must start at 60 s')
21     if not np.array_equal(t[:-1],np.arange(1,len(t))*60.0):
22         raise ValueError('Missing 60 s rows')
23     if c[-1].max()>=0.15:
24         raise ValueError('Final row is not strictly qualified')
25     return t,c
26
27
28 def export_artifact(root: Path, out: Path):
29     if out.exists(): raise FileExistsError(out)
30     os.environ.setdefault('ARTIFACT_TOOL_RPC_DAEMON_STARTUP_TIMEOUT_S','35')
31     from artifact_tool import Blob, SpreadsheetFile
32     root=Path(root);t,c=payload(root);n=len(t)+1
33     wb=SpreadsheetFile.import_xlsx(Blob.load(str(root/'inputs/result3_blank.xlsx')))
34     sh=wb.worksheets.get_item('Sheet1')
35     sh.get_range(f'A1:V{n}').values=[[ '时间/s\\到药材中心的距离/cm']+np.linspace(0,2,21).to
↪ list()+np.column_stack([t,c]).tolist()
36     sh.get_range(f'A2:A{n}').format.number_format='0.00'
37     sh.get_range(f'B2:V{n}').format.number_format='0.0000'
38     SpreadsheetFile.export_xlsx(wb).save(str(out))
39     return {'engine':'artifact_tool','rows':len(t),'columns':22,'bytes':out.stat().st_size}
40
41
42 def portable_bytes(root: Path):
43     t,c=payload(root);n=len(t)+1;ns='http://schemas.openxmlformats.org/spreadsheetml/2006'
↪ /main'
44     parts={

```

```

45 '[Content_Types].xml': ''<Types
    ↪ xmlns="http://schemas.openxmlformats.org/package/2006/content-types"><Default
    ↪ Extension="rels"
    ↪ ContentType="application/vnd.openxmlformats-package.relationships+xml"/><Default
    ↪ Extension="xml" ContentType="application/xml"/><Override
    ↪ PartName="/xl/workbook.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet.main+xml"/><Override PartName="/xl/styles.xml"
    ↪ ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.styles+xml"/><Override PartName="/xl/worksheets/sheet1.xml"
    ↪ ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.worksheet+xml"/></Types>''',
46 '_rels/.rels': ''<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships"><Relationship Id="rId1"
    ↪ Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/officeDocument" Target="xl/workbook.xml"/></Relationships>'',
47 'xl/workbook.xml': f''<workbook xmlns="{ns}" xmlns:r="http://schemas.openxmlformats.org/officeDocument/2006/relationships"><sheets><sheet name="Sheet1" sheetId="1"
    ↪ r:id="rId1"/></sheets></workbook>'',
48 'xl/_rels/workbook.xml.rels': ''<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships"><Relationship Id="rId1"
    ↪ Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/worksheets" Target="worksheets/sheet1.xml"/><Relationship Id="rId2"
    ↪ Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/styleSheet" Target="styles.xml"/></Relationships>'',
49 'xl/styles.xml': f''<styleSheet xmlns="{ns}"><numFmts count="2"><numFmt
    ↪ numFmtId="164" formatCode="0.0000"/><numFmt numFmtId="165"
    ↪ formatCode="0.00"/></numFmts><fonts count="1"><font><sz val="10"/><name
    ↪ val="Arial"/></font></font><fills count="2"><fill><patternFill
    ↪ patternType="none"/></fill><fill><patternFill
    ↪ patternType="gray125"/></fill></fills><borders count="1"><border><left/><right/><top/><bottom/><diagonal/></border></borders><cellStyleXfs count="1"><xf
    ↪ numFmtId="0" fontId="0" fillId="0" borderId="0"/></cellStyleXfs><cellXfs
    ↪ count="3"><xf numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0"/><xf
    ↪ numFmtId="164" fontId="0" fillId="0" borderId="0" xfId="0"
    ↪ applyNumberFormat="1"/><xf numFmtId="165" fontId="0" fillId="0" borderId="0"
    ↪ xfId="0" applyNumberFormat="1"/></cellXfs><cellStyles count="1"><cellStyle
    ↪ name="Normal" xfId="0" builtinId="0"/></cellStyles></styleSheet>''
50 }
51 b=io.BytesIO()
52 with zipfile.ZipFile(b,'w',zipfile.ZIP_DEFLATED,compresslevel=6) as z:
53     for name,text in parts.items():
54         z.writestr(name,(f'<?xml version="1.0" encoding="UTF-8"?>'+text).encode())
55     with z.open('xl/worksheets/sheet1.xml','w') as f:
56         hdr='<worksheet xmlns="%s"><dimension ref="A1:V%d"/><sheetData><row r="1"><c
    ↪ r="A1" t="inlineStr"><is><t>时间/s\\到药材中心的距离/cm</t></is></c>'%(ns,n)
57         hdr+=''.join(f'<c r="{chr(66+j)}" t="n"><v>{j/10:.1f}</v></c>' for j in
    ↪ range(21))+</row>'
58         f.write(hdr.encode())
59         for i,(tt,cc) in enumerate(zip(t,c),2):
60             row=f'<row r="{i}"><c r="A{i}" t="n" s="2"><v>{float(tt):.17g}</v></c>'
61             row+=''.join(f'<c r="{chr(66+j)}" t="n"
    ↪ s="1"><v>{float(v):.17g}</v></c>' for j,v in enumerate(cc))+</row>'
62             f.write(row.encode())
63         f.write(b'</sheetData></worksheet>')
64     return b.getvalue()
65
66
67 def export_portable(root: Path, out: Path):
68     if out.exists(): raise FileExistsError(out)
69     out.write_bytes(portable_bytes(root))
70     return {'engine':'standard-library OOXML','bytes':out.stat().st_size}
71
72

```

```

73 if __name__ == '__main__':
74     p=argparse.ArgumentParser();p.add_argument('--root',type=Path,default=Path(__file__).
    ↪ resolve().parents[1]);p.add_argument('--out',type=Path,required=True);p.add_argum
    ↪ ent('--engine',choices=['artifact','portable'],default='portable');a=p.parse_args
    ↪ ()
75     print(json.dumps(export_artifact(a.root,a.out) if a.engine=='artifact' else
    ↪ export_portable(a.root,a.out)))

```

A.30 validate_xlsx

support/project/q3_refinement_delivery/source/validate_xlsx.py

```

1  """Strict OOXML readback validator for the frozen Q3 workbook.
2
3  This validator is read-only. It checks the exact allowed Sheet1 rectangle,
4  header semantics, numeric types, four-decimal moisture format, row times, radii,
5  all moisture values against output/solution.npz, and the final strict endpoint.
6  """
7  from __future__ import annotations
8  import argparse, json, re, zipfile
9  from pathlib import Path
10 import xml.etree.ElementTree as ET
11 import numpy as np
12
13 NS={'s':'http://schemas.openxmlformats.org/spreadsheetml/2006/main'}
14 RID='{http://schemas.openxmlformats.org/officeDocument/2006/relationships}id'
15 CELL_RE=re.compile(r'^([A-Z]+)([1-9][0-9]*)$')
16
17 def colnum(s):
18     n=0
19     for ch in s:n=n*26+ord(ch)-64
20     return n
21
22 def read_book(path:Path):
23     with zipfile.ZipFile(path) as z:
24         if z.testzip(): raise ValueError('Corrupt XLSX')
25         wb=ET.fromstring(z.read('xl/workbook.xml'))
26         relroot=ET.fromstring(z.read('xl/_rels/workbook.xml.rels'))
27         rel={e.attrib['Id']:e.attrib['Target'] for e in relroot}
28         sheets=wb.findall('s:sheets/s:sheet',NS)
29         names=[s.attrib['name'] for s in sheets]
30         if names!=['Sheet1']: raise ValueError(f'Unexpected sheets: {names}')
31         target=rel[sheets[0].attrib[RID]]
32         target=target[1:] if target.startswith('/') else 'xl/'+target
33         root=ET.fromstring(z.read(target))
34         strings=[]
35         if 'xl/sharedStrings.xml' in z.namelist():
36             sr=ET.fromstring(z.read('xl/sharedStrings.xml'))
37             for si in sr.findall('s:si',NS):
38                 strings.append(''.join(t.text or '' for t in si.findall('.//s:t',NS)))
39         # numFmt map by style id.
40         styles=ET.fromstring(z.read('xl/styles.xml'))
41         custom={int(e.attrib['numFmtId']):e.attrib['formatCode'] for e in
    ↪ styles.findall('s:numFmts/s:numFmt',NS)}
42         builtin={0:'General',1:'0',2:'0.00',3:'#,##0',4:'#,##0.00'}
43         xfs=styles.findall('s:cellXfs/s:xf',NS)
44         style_fmt=[custom.get(int(x.attrib.get('numFmtId','0')),builtin.get(int(x.attrib.
    ↪ get('numFmtId','0')),str(x.attrib.get('numFmtId','0')))) for x in xfs]
45         cells={}
46         max_row=max_col=0
47         for c in root.findall('.//s:sheetData/s:row/s:c',NS):
48             ref=c.attrib['r'];m=CELL_RE.match(ref)

```

```

49         if not m:raise ValueError(ref)
50         col,row=m.group(1),int(m.group(2));max_row=max(max_row,row);max_col=max(max_c
    ↪ ol,colnum(col))
51         if c.find('s:f',NS) is not None:raise ValueError(f'Formula not allowed: {ref}')
52         typ=c.attrib.get('t','n');v=c.find('s:v',NS)
53         if typ=='s': value=strings[int(v.text)]
54         elif typ=='inlineStr': value=''.join(t.text or '' for t in
    ↪ c.findall('.//s:t',NS))
55         elif v is None:value=None
56         elif typ in ('str','e'):value=v.text
57         else:value=float(v.text)
58         sid=int(c.attrib.get('s','0'));fmt=style_fmt[sid] if sid<len(style_fmt) else
    ↪ None
59         cells[ref]=(value,typ,fmt)
60     return cells,max_row,max_col
61
62 def validate(xlsx:Path, solution:Path):
63     cells,max_row,max_col=read_book(xlsx)
64     z=np.load(solution);t=z['time_s'][1:];C=z['profile_TC'][1:,:1];r=z['radius_cm']
65     n=len(t)+1
66     if (max_row,max_col)!=(n,22):raise ValueError(f'Exact extent must be A1:V{n}, got
    ↪ row={max_row}, col={max_col}')
67     expected_refs=set()
68     for row in range(1,n+1):
69         for col in range(1,23):
70             x=col;letters=''
71             while x:
72                 x,rem=divmod(x-1,26);letters=chr(65+rem)+letters
73             expected_refs.add(f'{letters}{row}')
74     if set(cells)!=expected_refs:
75         extra=sorted(set(cells)-expected_refs)[:10];missing=sorted(expected_refs-set(cell
    ↪ s)[:10])
76         raise ValueError(f'Unexpected cell set; extra={extra}, missing={missing}')
77     if cells['A1'][0] != '时间/s\\到药材中心的距离/cm':raise ValueError('Wrong A1
    ↪ title/unit')
78     for j in range(21):
79         ref=f'{chr(66+j)}1'
80         if abs(float(cells[ref][0])-r[j])>1e-12:raise ValueError(f'Wrong radius {ref}')
81     max_t=max_c=0.0;bad_format=[]
82     for i in range(len(t)):
83         row=i+2
84         v,typ,_=cells[f'A{row}']
85         if typ not in ('n',None):raise ValueError(f'Non-numeric time A{row}')
86         max_t=max(max_t,abs(float(v)-float(t[i])))
87         for j in range(21):
88             ref=f'{chr(66+j)}{row}';v,typ,fmt=cells[ref]
89             if typ not in ('n',None):raise ValueError(f'Non-numeric moisture {ref}')
90             max_c=max(max_c,abs(float(v)-float(C[i,j])))
91             if fmt!='0.0000':bad_format.append((ref,fmt))
92     if max_t!=0 or max_c!=0:raise ValueError(f'Numeric mismatch max_t={max_t},
    ↪ max_C={max_c}')
93     if bad_format:raise ValueError(f'Wrong moisture format: {bad_format[:5]}')
94     if C[-1].max()>=0.15:raise ValueError('Final unrounded field not strictly qualified')
95     return
    ↪ {'passed':True,'sheet':'Sheet1','rows_including_header':n,'columns':22,'data_rows'
    ↪ :len(t),'moisture_cells':int(C.size),'max_time_difference_s':max_t,'max_moisture_
    ↪ difference_kgkg':max_c,'last_time_s':float(t[-1]),'last_max_C':float(C[-1].max())}
96
97 if __name__=='__main__':

```

```

98 p=argparse.ArgumentParser();p.add_argument('--xlsx',type=Path,required=True);p.add_ar
↪ gument('--solution',type=Path,required=True);p.add_argument('--json-out',type=Pat
↪ h);a=p.parse_args();obj=validate(a.xlsx,a.solution);txt=json.dumps(obj,ensure_asc
↪ ii=False,indent=2);print(txt);a.json_out and
↪ a.json_out.write_text(txt+'\n',encoding='utf-8')

```

A.31 verify_frozen

support/project/q3_refinement_delivery/source/verify_frozen.py

```

1  """Read-only verifier for a frozen q3_refinement_delivery directory."""
2  from __future__ import annotations
3  import argparse, hashlib, json, os, sys
4  from pathlib import Path
5  import numpy as np
6
7  sys.dont_write_bytecode = True
8  from validate_xlsx import validate as validate_xlsx
9
10
11 def sha256(path: Path) -> str:
12     h=hashlib.sha256()
13     with path.open('rb') as f:
14         for b in iter(lambda:f.read(1024*1024),b''):h.update(b)
15     return h.hexdigest()
16
17
18 def snapshot(root: Path):
19     return {str(p.relative_to(root)):sha256(p) for p in sorted(root.rglob('*')) if
↪ p.is_file() and '__pycache__' not in p.parts}
20
21
22 def read_manifest(path:Path):
23     out={}
24     for line in path.read_text(encoding='utf-8').splitlines():
25         if not line.strip():continue
26         digest,rel=line.split(' ',1);out[rel]=digest
27     return out
28
29
30 def verify(root:Path,audit_out:Path):
31     root=root.resolve();audit_out=audit_out.resolve()
32     try:audit_out.relative_to(root)
33     except ValueError:pass
34     else:raise ValueError('audit-out must be outside the frozen root')
35     if audit_out.exists():raise FileExistsError(audit_out)
36     before=snapshot(root)
37     manifest=read_manifest(root/'MANIFEST.sha256')
38     actual={k:v for k,v in before.items() if k!='MANIFEST.sha256'}
39     if set(manifest)!=set(actual):
40         raise ValueError(f'manifest coverage mismatch:
↪ missing={sorted(set(actual)-set(manifest))[:5]},
↪ stale={sorted(set(manifest)-set(actual))[:5]}')
41     bad=[k for k in manifest if manifest[k]!=actual[k]]
42     if bad:raise ValueError(f'manifest mismatch: {bad[:8]}')
43     xlsx=validate_xlsx(root/'output/result3.xlsx',root/'output/solution.npz')
44     event=json.loads((root/'output/end_event.json').read_text(encoding='utf-8'))
45     z=np.load(root/'output/solution.npz')
46     maxC=float(z['profile_TC'][-1,:1].max())
47     checks={
48         'event_execution_matches_solution':abs(float(z['time_s'][-1])-event['execution_s'])
↪ <1e-8,

```

```

49     'strict_endpoint':maxC<0.15,
50     'event_execution_max_matches_profile':abs(maxC-event['execution_max'])<1e-12,
51     'table5_exists':(root/'output/table5.csv').is_file(),
52     'xlsx':xlsx,
53 }
54 if not all(v if isinstance(v,bool) else True for v in checks.values()):raise
55 ↪ AssertionError(checks)
56 after=snapshot(root)
57 if before!=after:raise AssertionError('Frozen root changed during verification')
58 audit_out.mkdir(parents=True)
59 obj={'passed':True,'root':str(root),'root_unchanged':True,'manifest_files':len(manife
60 ↪ st),'endpoint_max_C':maxC,'checks':checks}
61 (audit_out/'verify.json').write_text(json.dumps(obj,ensure_ascii=False,indent=2)+'\n'
62 ↪ ,encoding='utf-8')
63 return obj
64
65 if __name__=='__main__':
66     p=argparse.ArgumentParser();p.add_argument('--root',type=Path,required=True);p.add_ar
67     ↪ gument('--audit-out',type=Path,required=True);a=p.parse_args();print(json.dumps(v
68     ↪ erify(a.root,a.audit_out),ensure_ascii=False,indent=2))

```

A.32 第四问及有限圆柱验证

A.33 q4_common

support/project/q4_complete_delivery/v1/source/q4_common.py

```

1  """Inputs and geometry, all in SI. Only reads source workbooks."""
2  from pathlib import Path
3  import hashlib, json
4  import numpy as np
5  from openpyxl import load_workbook
6  from scipy.interpolate import PchipInterpolator
7
8  ROOT=Path(__file__).resolve().parents[1]
9
10 def read_numeric(path, columns):
11     wb=load_workbook(path,read_only=True,data_only=True)
12     ws=wb.worksheets[0];rows=list(ws.values);wb.close()
13     data=np.asarray([r[:columns] for r in rows[1:] if r[0] is not None],float)
14     if not np.isfinite(data).all() or np.any(np.diff(data[:,0])<=0):
15         raise ValueError('Nonfinite input, duplicate or unordered time')
16     return rows[0],data
17
18 def load_input(path):return read_numeric(path,3)[1]
19
20 class Environment:
21     def __init__(self,data):
22         self.data=np.asarray(data);self.future=self.data[self.data[:,0]>=10800,1:].mean(a
23         ↪ xis=0)
24     def __call__(self,t):
25         if t>14400:return self.future
26         return np.array([np.interp(t,self.data[:,0],self.data[:,k]) for k in (1,2)])
27     def segment(self,a,b):return (lambda t:self.future) if a>=14400 else self
28
29 class Radius:
30     def __init__(self,data,kind='linear'):
31         self.data=np.asarray(data).copy();self.data[:,1]*=.01;self.kind=kind
32         if np.any(self.data[:,1]<=0) or np.any(np.diff(self.data[:,1])>0):raise
33         ↪ ValueError('Invalid measured shrinkage')
34         self.pchip=PchipInterpolator(self.data[:,0],self.data[:,1],extrapolate=False)

```

```

33     def __call__(self,t):
34         if self.kind=='fixed':return .02
35         if self.kind=='pchip' and 0<=t<=self.data[-1,0]:return float(self.pchip(t))
36         return float(np.interp(t,self.data[:,0],self.data[:,1]))
37     def derivative(self,t):
38         if self.kind=='fixed' or t>=self.data[-1,0]:return 0.
39         if self.kind=='pchip':return float(self.pchip.derivative()(t))
40         i=np.clip(np.searchsorted(self.data[:,0],t,side='right')-1,0,len(self.data)-2)
41         return float(np.diff(self.data[i:i+2,1])[0]/np.diff(self.data[i:i+2,0])[0])
42
43 def input_audit(out):
44     result={}
45     for name,n in [('attachment1.xlsx',3),('attachment2.xlsx',2)]:
46         p=ROOT/'inputs'/name;head,a=read_numeric(p,n)
47         result[name]={ 'sha256':hashlib.sha256(p.read_bytes()).hexdigest(), 'headers':list(
48             ↪ head),
49             'rows':len(a), 'first':a[0].tolist(), 'last':a[-1].tolist(), 'missing':int(np.isna
50             ↪ n(a).sum()),
51             'time_step_unique_s':np.unique(np.diff(a[:,0])).tolist(), 'min':a.min(axis=0).to
52             ↪ list(), 'max':a.max(axis=0).tolist()}
53     _,a=read_numeric(ROOT/'inputs/attachment2.xlsx',2)
54     result['radius']={ 'units_input':'cm', 'units_internal':'m', 'monotone_nonincreasing':bo
55     ↪ ol(np.all(np.diff(a[:,1])<=0)),
56     'plateau_intervals':int(np.sum(np.diff(a[:,1])==0)), 'measurement_increment_cm':.001,
57     'primary_interpolation':'piecewise linear, no overshoot; restart at knots',
58     'beyond_72h':'hold last observed radius; conditional extension only if needed',
59     'length_m':.25, 'length_assumption':'constant; homogeneous radial material
60     ↪ contraction'}
61     result['future_environment']=Environment(load_input(ROOT/'inputs/attachment1.xlsx')).
62     ↪ future.tolist()
63     Path(out).write_text(json.dumps(result,ensure_ascii=False,indent=2))
64     return result

```

A.34 q4_spectral

support/project/q4_complete_delivery/v1/source/q4_spectral.py

```

1  """Q4 material-coordinate Chebyshev collocation, adapted from audited Q3 reference.
2  Changes: Appendix 4 local coefficients; radius(t); material-coordinate derivation.
3  Source commit anonymous-source.
4  Independent global Chebyshev collocation with primitive-gradient moisture flux.
5  No FV control volumes, face weights, or FV spatial operator are reused.
6  Full coupled heat/moisture evolution from t=0; Robin boundary algebraically solved.
7  Grid/differentiation construction follows the audited q2_spectral.py approach;
8  moisture flux and its boundary Jacobian are newly derived for long-time Q3.
9  """
10 from __future__ import annotations
11 import argparse,json,time
12 from pathlib import Path
13 import numpy as np
14 from numpy.polynomial import Chebyshev
15 from scipy.special import exp1
16 from scipy.optimize import brentq
17 from scipy.integrate import solve_ivp
18 from scipy.fft import dct
19 from q4_common import load_input,Environment
20
21 def primitive(c):
22     if np.any(np.real(c)<=0):raise FloatingPointError('C<=0 in reference')
23     return c*np.exp(-.30/c)-.30*exp1(.30/c)
24
25 def grid(n):

```

```

26 x=(1-np.cos(np.pi*np.arange(n+1)/n))/2
27 w=(-1.)*np.arange(n+1);w[[0,-1]]*=.5
28 dx=x[:,None]-x[None,:]
29 G=np.divide(w[None,:]/w[:,None],dx,out=np.zeros_like(dx),where=dx!=0)
30 G[np.diag_indices(n+1)]=-G.sum(axis=1);G[:, -1]=-G[:, :-1].sum(axis=1)
31 return x,G,w
32
33 def interp(x,w,target):
34     d=target[:,None]-x
35     out=np.empty_like(d)
36     for j,dd in enumerate(d):
37         k=np.argmin(abs(dd))
38         if abs(dd[k])<1e-14:out[j]=0;out[j,k]=1
39         else:v=w/dd;out[j]=v/v.sum()
40     return out
41
42 def inverse_primitive(v,guess):
43     c=np.asarray(guess).copy()
44     for _ in range(40):
45         delta=(primitive(c)-v)/np.exp(-.30/c)
46         cnew=c-delta
47         # Globalize Newton without altering the equation or accepted value.
48         bad=np.real(cnew)<=0
49         while np.any(bad):delta[bad]*=.5;cnew=c-delta;bad=np.real(cnew)<=0
50         c=cnew
51         if np.max(abs(delta))<3e-14:break
52     else:raise RuntimeError('Inverse primitive failed')
53     return c
54
55 class Reference:
56     def __init__(self,n,radius):
57         self.radius=radius
58         self.n=n;self.x,self.G,self.w=grid(n);self.row=self.G[-1,:-1];self.diag=self.G[-1,-1]
59         self.fac=4/.02**2;self.bd=2/.02
60         self.BT0=np.zeros((n+1,2*n));self.BC0=np.zeros_like(self.BT0)
61         self.BT0[np.arange(n),2*np.arange(n)]=1;self.BC0[np.arange(n),2*np.arange(n)+1]=1
62         self.I=interp(self.x,self.w,np.linspace(0,1,21)**2)
63         self.denseI=interp(self.x,self.w,np.linspace(0,1,8*n+1))
64         self.min_boundary_derivative=float('inf');self.max_bc_residual=0.;self.max_newton=0;self
        ↪ f_boundary_fallbacks=0
65     def surface(self,t,u,env,jac=False):
66         R=self.radius(t);self.fac=4/R**2;self.bd=2/R
67         Ti,Ci=u.T;et,ec=env(t);Ui=primitive(Ci);anchor=Ci[-1];ua=Ui[-1]
68         gt=self.row@(Ti-et);gu=self.row@(Ui-ua)
69         def calc(c):
70             k=.12+.20*c/(1+c);kc=.20/(1+c)**2
71             den=self.bd*k*self.diag+25
72             Ts=et-self.bd*k*gt/den
73             Tsc=-self.bd*kc*gt*25/den**2
74             A=.00042*np.exp(-3850/(Ts+273.15));At=A*3850/(Ts+273.15)**2
75             ux=gu+self.diag*(primitive(c)-ua)
76             f=np.exp(-.30/c)
77             val=self.bd*A*ux+8e-7*(c-ec)
78             deriv=self.bd*(At*Tsc*ux+A*self.diag*f)+8e-7
79             return val,deriv,Ts,k,kc,A,At,ux
80         c=anchor
81         for it in range(25):
82             val,deriv,*_=calc(c);step=val/deriv
83             while c-step<=0:step*=.5
84             c=c-step
85             if abs(step)<2e-14*(1+abs(c)):break
86         else:
87             self.boundary_fallbacks+=1
88             lo=min(ec,float(Ci.min()))*.1;hi=max(ec,float(Ci.max()))*1.1

```

```

89     c=brentq(lambda z:calc(z)[0],lo,hi,xtol=5e-15)
90     val,deriv,Ts,k,kc,A,At,ux=calc(c)
91     self.max_newton=max(self.max_newton,it+1);self.min_boundary_derivative=min(self.min_bou
↪ ndary_derivative,float(deriv));self.max_bc_residual=max(self.max_bc_residual,float(
↪ abs(val)))
92     full=np.vstack([u,[Ts,c]])
93     if not jac:return full
94     tx=self.row@(Ti-Ts)
95     B=np.array([[self.bd*k*self.diag+25,self.bd*k*tx],[self.bd*At*ux,self.bd*A*self.diag*n
↪ p.exp(-.30/c)+8e-7]])
96     Q=np.zeros((2,2*self.n));Q[0,:2]=self.bd*k*self.row;Q[1,1:2]=self.bd*A*self.row*np.ex
↪ p(-.30/Ci)
97     sens=-np.linalg.solve(B,Q);BT=self.BT0.copy();BC=self.BC0.copy();BT[-1]=sens[0];BC[-1]=
↪ sens[1]
98     return full,BT,BC
99     def evaluate(self,t,y,env,jac=False):
100         if jac:full,BT,BC=self.surface(t,y.reshape(self.n,2),env,True)
101         else:full=self.surface(t,y.reshape(self.n,2),env)
102         T,C=full.T
103         if np.any(T<=-273.15):raise FloatingPointError('Kelvin<=0')
104         rho=760+90*C;cp=1850+2150*C/(1+C);S=rho*cp;Sc=90*cp+rho*2150/(1+C)**2
105         k=.12+.20*C/(1+C);kc=.20/(1+C)**2;A=.00042*np.exp(-3850/(T+273.15));At=A*3850/(T+273.15
↪ )**2
106         U=primitive(C);gt=self.G@(T-T[-1]);gu=self.G@(U-U[-1]);qt=k*gt;qm=A*gu
107         ht=qt+self.x*(self.G@(qt-qt[-1]));hm=qm+self.x*(self.G@(qm-qm[-1]))
108         fT=self.fac*ht/S;fC=self.fac*hm
109         if not jac:return np.column_stack([fT[:-1],fC[:-1]]).ravel()
110         BU=np.exp(-.30/C)[: ,None]*BC
111         gtj=self.G@(BT-BT[-1]);guj=self.G@(BU-BU[-1])
112         qtj=k[: ,None]*gtj+(kc*gt)[: ,None]*BC
113         qmj=A[: ,None]*guj+(At*gu)[: ,None]*BT
114         htj=qtj+self.x[: ,None]*(self.G@(qtj-qtj[-1]));hmj=qmj+self.x[: ,None]*(self.G@(qmj-qmj[-
↪ 1]))
115         J=np.empty((2*self.n,2*self.n));J[:2]=
↪ (self.fac*htj/S[: ,None]-(fT*Sc/S)[: ,None]*BC)[: -1];J[1:2]=(self.fac*hmj)[: -1]
116         return J
117     def profiles(self,u):
118         T,C=u.T;U=primitive(C);v=self.I@U;guess=self.I@C
119         return np.column_stack([self.I@T,inverse_primitive(v,guess)])
120     def extrema(self,u):
121         # Interpolate U, not steep C. All real stationary points of the polynomial.
122         U=primitive(u[:,1]);co=dcT(U[: -1],type=1)/self.n;co[[0,-1]]*=.5
123         p=Chebyshev(co);roots=p.deriv().roots();roots=roots[np.isreal(roots)].real;roots=roots[
↪ (roots>-1)&(roots<1)]
124         pts=np.r_[-1,roots,1];vals=p(pts);idx=np.argmax(vals)
125         maxi=inverse_primitive(np.array([vals[idx]]),np.array([u[:,1].max()]))[0]
126         return {'max_C':float(maxi),'x_max':float((pts[idx]+1)/2),'stationary_points':len(roots
↪ ),'minimum_U':float(vals.min())}

```

A.35 run_q4

support/project/q4_complete_delivery/v1/source/run_q4.py

```

1  """Cold-start fully coupled Q4 material-coordinate calculation.
2
3  The effective equations are  $S(C)$   $D_s T = \text{div}(k \text{ grad } T)$ ,  $D_s C = \text{div}(D \text{ grad } C)$ .
4  With  $v_s = w(R'/R)r$ ,  $x = (r/R)^2$ , radial  $\text{div} = 4/R^2 d_x(x \text{ flux}_x)$ .
5  No mass-density dilution term is added to dry-basis C.  $\text{Rho}(C)$  is only in S.
6  """
7  import argparse,json,time,hashlib,platform
8  from pathlib import Path
9  import numpy as np

```

```

10 import scipy
11 from scipy.integrate import solve_ivp
12 from numpy.polynomial.chebyshev import chebvander
13 from q4_common import ROOT, Environment, Radius, read_numeric, load_input, input_audit
14 from q4_spectral import Reference, interp, primitive, inverse_primitive
15
16 def run(n, out, kind='linear', method='Radau', rtol=2e-10, max_step=180., audit=False):
17     out=Path(out)
18     if out.with_suffix('.json').exists() or out.with_suffix('.npz').exists():raise
19     ↪ FileExistsError(out)
19     out.parent.mkdir(parents=True, exist_ok=True)
20     data=load_input(ROOT/'inputs/attachment1.xlsx');rd=read_numeric(ROOT/'inputs/attachme
21     ↪ nt2.xlsx',2)[1]
21     env=Environment(data);radius=Radius(rd,kind);op=Reference(n,radius)
22     weights=np.linalg.solve(chebvander(2*op.x-1,n).T,np.array([0. if j%2 else 1/(1-j*j)
23     ↪ for j in range(n+1)]))
23     target=np.linspace(0,.02,21);times=[0.];states=[np.tile([28.,2.55],(n+1,1))]
24     radii=[.02];fixed=[np.tile([28.,2.55],(21,1))];surf=[[28.,2.55]];meanC=[2.55]
25     maxima=[2.55];positions=[0.];rate_checks=[];cumulative_out=0.;balance=[]
26     y=np.tile([28.,2.55],n);tic=time.perf_counter();steps=nfev=nlu=0;event_info=None
27     cuts=np.unique(np.r_[data[:,0],rd[:,0],np.arange(259200,518401,1800.)]);next_out=60.
28     gauss_x,gauss_w=np.polynomial.legendre.leggauss(4)
29     def record(t,u,e):
30         R=radius(t);inside=target<=R+1e-14;v=np.full((21,2),np.nan)
31         I=interp(op.x,op.w,(target[inside]/R)**2)
32         v[inside,0]=I@u[:,0];v[inside,1]=inverse_primitive(I@primitive(u[:,1]),I@u[:,1])
33         ex=op.extrema(u)
34         times.append(float(t));states.append(u);radii.append(R);fixed.append(v);surf.appe
35         ↪ nd(u[-1]);meanC.append(float(weights@u[:,1]));maxima.append(ex['max_C']);posi
36         ↪ tions.append(float(R*np.sqrt(ex['x_max'])))
35     for a,b in zip(cuts[:-1],cuts[1:]):
36         e=env.segment(a,b);fun=lambda t,y:op.evaluate(t,y,e)
37         def ev(t,y):return float(np.max(y[1::2])-.15)
38         ev.direction=-1;ev.terminal=False
39         sol=solve_ivp(fun,(a,b),y,method=method,jac=lambda t,y:op.evaluate(t,y,e,True),
40         ↪ rtol=rtol,atol=np.tile([rtol*.1,rtol*.01],n),max_step=min(max_step,30.) if
41         ↪ a<14400 else max_step,dense_output=True,events=ev)
41     if not sol.success:raise RuntimeError(sol.message)
42     steps+=len(sol.t)-1;nfev+=sol.nfev;nlu+=sol.nlu;end=b
43     if len(sol.t_events[0]):
44         root=float(sol.t_events[0][0]);end=.36*np.ceil(root/.36)
45         # Exact evaluations on an explicitly stored grid allow the final
46         # execution margin to be selected AFTER comparing numerical runs.
47         near=np.unique(np.r_[root+np.array([-2.,-1.,0.,1.,2.]),
48         ↪ np.arange(np.floor(root/.36)-3,np.ceil(root/.36)+10)*.36])
49         if near.min()<a or near.max()>b:raise RuntimeError('Event neighborhood crosses
50         ↪ segment; refine cuts')
50         near_full=np.array([op.surface(t,sol.sol(t).reshape(n,2),e) for t in near])
51         event_info={'critical_s':root,'critical_h':root/3600,'preliminary_execution_s
52         ↪ ':end,
52         ↪ 'preliminary_execution_max_C':op.extrema(op.surface(end,sol.sol(end).res
53         ↪ hape(n,2),e))['max_C'],
53         ↪ 'critical_slope':float(fun(root,sol.sol(root))[1]),'root_extrema':op.ext
54         ↪ rema(op.surface(root,sol.sol(root).reshape(n,2),e)),
54         ↪ 'radius_m':radius(root),'length_m':.25,'root_is_1d':True}
55         ts=np.arange(next_out,end+1e-9,60.)
56         if event_info:ts=np.r_[ts[ts<end-1e-8],end]
57         for t in ts:record(t,op.surface(t,sol.sol(t).reshape(n,2),e),e)
58         if len(ts):next_out=60*(np.floor(ts[-1]/60)+1)
59         # Independent quadrature of the boundary loss; dry mass cancels.
60         if audit:
61             for sa,sb in zip(sol.t[:-1],sol.t[1:]):
62                 sb=min(sb,end)
63                 if sa>=end:break

```

```

64         tq=(sa+sb)/2+(sb-sa)/2*gauss_x
65         rates=[]
66         for t in tq:
67             u=op.surface(t,sol.sol(t).reshape(n,2),e)
68             rates.append(2/radius(t)*8e-7*(u[-1,1]-e(t)[1]))
69             cumulative_out+=(sb-sa)/2*np.dot(gauss_w,rates)
70             u=op.surface(end,sol.sol(end).reshape(n,2),e)
71             balance.append([float(end),float(weights@u[:,1]),float(cumulative_out),float(
↪ weights@u[:,1]+cumulative_out-2.55)])
72         if b%21600==0 or event_info:
73             u=op.surface(end,sol.sol(end).reshape(n,2),e)
74             f=fun(end,sol.sol(end)).reshape(n,2)
75             # Pointwise differentiated polynomial at the boundary supplies rate
76             # solely for this spatial quadrature identity, not a PDE unknown.
77             T,C=u.T;R=radius(end);S=(760+90*C)*(1850+2150*C/(1+C));k=.12+.20*C/(1+C)
78             U=primitive(C);gt=op.G@(T-T[-1]);gu=op.G@(U-U[-1]);qt=k*gt;qm=.00042*np.exp(-
↪ 3850/(T+273.15))*gu
79             ft=4/R**2*(qt+op.x*(op.G@(qt-qt[-1])))/S;fc=4/R**2*(qm+op.x*(op.G@(qm-qm[-1])))
80             rate_checks.append({'t_s':end,'water_rate_residual':float(weights@fc+2/R*8e-7
↪ *(C[-1]-e(end)[1])),
81                                 'effective_heat_rate_residual':float(weights@(S*ft)+2/R*25*(T[-1]-e(end)[
↪ 0]))})
82             print(json.dumps({'n':n,'kind':kind,'time_h':end/3600,'max_C':float(C.max()),
↪ 'elapsed_s':time.perf_counter()-tic}),flush=True)
83         y=sol.sol(end)
84         if event_info:break
85     if event_info is None:raise RuntimeError('No threshold by 144h; no fabricated
↪ endpoint')
86     stats={'n':n,'method':method,'rtol':rtol,'atol_TC':[rtol*.1,rtol*.01],'max_step_s':ma
↪ x_step,'radius_interpolation':kind,
87            'future_environment':env.future.tolist(),'event':event_info,'elapsed_s':time.perf_
↪ counter()-tic,'steps':steps,'nfev':nfev,'nlu':nlu,
88            'max_boundary_residual':op.max_bc_residual,'boundary_fallbacks':op.boundary_fallba
↪ cks,'rate_checks':rate_checks,
89            'mass_balance_max_abs_in_mean_C':float(np.max(np.abs(np.array(balance)[:,-1]))) if
↪ balance else None,
90            'numpy':np.__version__,'scipy':scipy.__version__,'python':platform.python_version(),
91            'source_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest()}
92     np.savez_compressed(out.with_suffix('.npz'),time_s=times,radius_m=radii,full_TC=state
↪ s,x=op.x,
93                        fixed_radius_m=target,sample_TC=fixed,surface_TC=surf,mean_C=meanC,max_C=maxima
↪ ,max_radius_m=positions,
94                        event_time_s=near,event_full_TC=near_full,quadrature_weights=weights,balance=ba
↪ lance)
95     out.with_suffix('.json').write_text(json.dumps(stats,ensure_ascii=False,indent=2))
96     print(json.dumps(stats,ensure_ascii=False),flush=True)
97     return stats
98
99 if __name__=='__main__':
100     p=argparse.ArgumentParser();p.add_argument('--n',type=int,default=80);p.add_argument(
↪ '--out',type=Path,required=True)
101     p.add_argument('--kind',choices=['linear','fixed','pchip'],default='linear');p.add_ar
↪ gument('--method',default='Radau')
102     p.add_argument('--rtol',type=float,default=2e-10);p.add_argument('--max-step',type=fl
↪ oat,default=180.);p.add_argument('--audit',action='store_true')
103     a=p.parse_args();run(a.n,a.out,a.kind,a.method,a.rtol,a.max_step,a.audit)

```

A.36 check_numerics

support/project/q4_complete_delivery/v1/source/check_numerics.py

```
1 """Read-only comparisons of real saved runs. No output-state edits."""
2 import argparse,json,hashlib
3 from pathlib import Path
4 from decimal import Decimal,ROUND_HALF_UP
5 import numpy as np
6 from q4_common import ROOT
7 from q4_spectral import interp,primitive,inverse_primitive
8
9 def check(base=ROOT):
10     base=Path(base);v=base/'validation';z=np.load(base/'output/main.npz');m=json.loads((b_
11     ↪ ase/'output/main.json').read_text())
12     result={'main':'output/main.npz','comparisons':{},'main_mass_balance_max_mean_C':m['m_
13     ↪ ass_balance_max_abs_in_mean_C'],
14     'main_max_boundary_residual':m['max_boundary_residual'],'main_total_loss_per_unit_
15     ↪ dry_mass':float(2.55-z['mean_C'][-1]),
16     'main_end_mean_C':float(z['mean_C'][-1]),'domain_masks':{}}
17     for name in ['spectral80','spectral160','time120_bdf']:
18         p=v/(name+'.npz')
19         if not p.exists():result['comparisons'][name]={'status':'not complete';
20         ↪ excluded'};continue
21         q=np.load(p);d=json.loads(p.with_suffix('.json').read_text());common,ia,ib=np.int_
22         ↪ ersect1d(z['time_s'],q['time_s'],return_indices=True)
23         err=z['sample_TC'][ia]-q['sample_TC'][ib];finite=np.isfinite(err)
24         rnd=lambda
25         ↪ x:Decimal(str(float(x))).quantize(Decimal('.0001'),rounding=ROUND_HALF_UP)
26         mismatch=sum(rnd(a)!=rnd(b) for a,b in
27         ↪ zip(z['sample_TC'][ia][finite],q['sample_TC'][ib][finite]))
28         result['comparisons'][name]={'status':'actual saved run
29         ↪ read','common_times':len(common),
30         'max_abs_T_C':float(np.nanmax(abs(err[:, :, 0]))),'max_abs_C':float(np.nanmax(a_
31         ↪ bs(err[:, :, 1]))),
32         'four_decimal_mismatch_count_TC':int(mismatch),'root_difference_s':d['event']_
33         ↪ ['critical_s']-m['event']['critical_s']}
34     g=v/'geometry';p=g/'q4_640x0_integral_rtol2e-11.npz'
35     if p.exists():
36         f=np.load(p);common,ia,ib=np.intersect1d(z['time_s'],f['time_s'],return_indices=T_
37         ↪ rue)
38         n=len(z['x'])-1;w=(-1.)*np.arange(n+1);w[[0,-1]]*=.5
39         I=interp(z['x'],w,np.linspace(0,1,21)**2);u=z['full_TC'][ia]
40         mt=np.einsum('ij,tj->ti',I,u[:, :, 0]);mc=inverse_primitive(np.einsum('ij,tj->ti',I_
41         ↪ ,primitive(u[:, :, 1])),np.einsum('ij,tj->ti',I,u[:, :, 1]))
42         result['independent_fv']={'path':str(p.relative_to(base)),'common_times':len(comm_
43         ↪ on),'coordinate':'matched material xi=0:.05:1, not fixed physical r',
44         'max_abs_T':float(np.max(abs(mt-f['mid'][ib, :, 0]))),'max_abs_C':float(np.max(_
45         ↪ abs(mc-f['mid'][ib, :, 1]))),
46         'last_pair_extrapolation_is_estimate':True}
47     mask=z['fixed_radius_m'][None, :]>z['radius_m'][:, None]+1e-14
48     result['domain_masks']={'outside_count':int(mask.sum()),'outside_are_nan':bool(np.isn_
49     ↪ an(z['sample_TC'][:, :, 1][mask]).all()),
50     'inside_are_finite':bool(np.isfinite(z['sample_TC'][:, :, 1][~mask]).all())}
51     eq=np.isclose(z['fixed_radius_m'][None, :],z['radius_m'][:, None],atol=1e-14,rtol=0);ii_
52     ↪ ,jj=np.where(eq)
53     result['surface_fixed_coincidence']={'count':len(ii),'max_abs_C_difference':float(np._
54     ↪ max(abs(z['sample_TC'][ii,jj,1]-z['surface_TC'][ii,1])))}
55     result['polynomial_maximum']={'max_C_minus_axis_C':float(np.max(abs(z['max_C']-z['ful_
56     ↪ l_TC'][:, 0, 1]))),
57     'root_extrema':m['event']['root_extrema'],'early_argmax_note':'nearly uniform
58     ↪ profiles have floating-point nonunique locations'}
59     (v/'numeric_comparison.json').write_text(json.dumps(result,ensure_ascii=False,indent=
60     ↪ 2));print(json.dumps(result,ensure_ascii=False,indent=2))
```

```

41     return result
42 if __name__=='__main__':
43     p=argparse.ArgumentParser();p.add_argument('--base',type=Path,default=ROOT);a=p.parse_
    ↪ _args();check(a.base)

```

A.37 finalize_q4

support/project/q4_complete_delivery/v1/source/finalize_q4.py

```

1  """Select a strict execution point using computed numerical discrepancies.
2  All reported cells, tables and trajectories originate from the same main run.
3  The budget is empirical numerical evidence, not a rigorous error bound.
4  """
5  import argparse,json,math,csv
6  from pathlib import Path
7  import numpy as np
8  from q4_common import ROOT
9
10 def finalize(base):
11     base=Path(base);out=base/'output';v=base/'validation';g=v/'geometry'
12     if (out/'result4_data.json').exists():raise FileExistsError('Refuse overwrite final
    ↪ derived results')
13     read=lambda p:json.loads(Path(p).read_text())
14     main=read(out/'main.json');root=main['event']['critical_s']
15     a=read(v/'spectral80.json')['event']['critical_s']
16     b=read(v/'time120_bdf.json')['event']['critical_s']
17     f320=read(g/'q4_320x0_integral_rtol2e-10.json')['event_root_s']
18     f640=read(g/'q4_640x0_integral_rtol2e-10.json')['event_root_s']
19     floose=read(g/'q4_320x0_integral.json')['event_root_s']
20     fex=(4*f640-f320)/3
21     discrepancies={'spectral80_to_main_s':abs(a-root),'same_n_time_method_change_s':abs(b
    ↪ -root),
22                   'independent_fv_extrapolation_to_main_s':abs(fex-root),
23                   'independent_fv_temporal_change_s':abs(f320-floose),
24                   'independent_fv_last_spatial_correction_s':abs(f640-fex)}
25     budget=math.ceil(2000*max(discrepancies.values()))/1000
26     execute=.36*math.ceil((root+budget)/.36)
27     z=dict(np.load(out/'main.npz'));candidates=z['event_time_s'];idx=int(np.argmin(abs(ca
    ↪ ndidates-execute)))
28     if abs(candidates[idx]-execute)>1e-7:raise ValueError('Execution point not actually
    ↪ solved')
29     full=z['event_full_TC'][idx]
30     # Current implementation records exact candidates around the event. A shifted
31     # final endpoint must first be regenerated from that complete state.
32     if abs(z['time_s'][-1]-execute)>1e-7:raise ValueError('Regenerate final row from event
    ↪ full state before export')
33     if float(full[:,1].max())>=.15:raise AssertionError('Strict C<.15 failed')
34     before=z['event_full_TC'][np.argmin(abs(candidates-(root-1)))]
35     after=z['event_full_TC'][np.argmin(abs(candidates-(root+1)))]
36     final={'critical_s':root,'critical_h':root/3600,'critical_display_h':round(root/3600,
    ↪ 4),
37           'execution_s':execute,'execution_h':execute/3600,'execution_max_C':float(full[:,1]
    ↪ .max()),
38           'actual_margin_s':execute-root,'empirical_budget_s':budget,'budget_rule':'ceil(2*m
    ↪ ax(recorded discrepancies), 0.001 s)',
39           'budget_is_rigorous_bound':False,'discrepancies':discrepancies,'fv_extrapolated_ro
    ↪ t_s':fex,
40           'one_second_before_max_C':float(before[:,1].max()),'one_second_after_max_C':float(
    ↪ after[:,1].max()),
41           'radius_m':float(z['radius_m'][-1]),'length_m':.25,'domain':'0<=r<=R(t);
    ↪ -0.125<=z<=0.125 m',

```

```

42     'source': 'output/main.npz; official 1D endpoint, matched 2D applicability and
    ↪ whole-domain evidence are reported separately'}
43 (out/'end_event.json').write_text(json.dumps(final,ensure_ascii=False,indent=2))
44 keep=z['time_s']>0;Cf=z['sample_TC'][keep,:1]
45 doc={'time_s':z['time_s'][keep].tolist(),'radius_m':z['radius_m'][keep].tolist(),
46     'C_fixed':[[None if not np.isfinite(x) else float(x) for x in row] for row in Cf],
47     'C_surface':z['surface_TC'][keep,1].tolist()}
48 (out/'result4_data.json').write_text(json.dumps(doc,ensure_ascii=False,allow_nan=False,
    ↪ indent=2))
49 with (out/'trajectory_60s_unrounded.csv').open('w',newline='') as f:
50     w=csv.writer(f);w.writerow(['time_s','radius_m','max_C','max_radius_m','mean_C','']
    ↪ 'T_surface_C','C_surface')+
51     [f'T_r{j/10:.1f}cm_C' for j in range(21)]+[f'C_r{j/10:.1f}cm' for j in
    ↪ range(21)])
52     for i,t in enumerate(z['time_s']):
53         vals=[t,z['radius_m'][i],z['max_C'][i],z['max_radius_m'][i],z['mean_C'][i],*z
    ↪ ['surface_TC'][i],*z['sample_TC'][i,:0],*z['sample_TC'][i,:1]]
54         w.writerow(['' if not np.isfinite(x) else format(float(x),'.17g') for x in
    ↪ vals])
55     rows=[]
56     for t in np.r_[np.arange(21600,execute,21600),execute]:
57         i=int(np.argmax(abs(z['time_s']-t)))
58         if abs(z['time_s'][i]-t)>1e-7:raise ValueError('Missing actual table time')
59         rows.append([t/3600,*z['sample_TC'][i,:5,1],z['surface_TC'][i,1],z['radius_m'][i]
    ↪ *100])
60     with (out/'table6_unrounded.csv').open('w',newline='') as f:
61         w=csv.writer(f);w.writerow(['time_h','r0cm','r0.5cm','r1cm','r1.5cm','r2cm','surf_
    ↪ ace','radius_cm'])
62         for row in rows:w.writerow(['' if not np.isfinite(x) else format(float(x),'.17g')
    ↪ for x in row])
63     md='|时间/h|0 cm|0.5 cm|1 cm|1.5 cm|2
    ↪ cm|药材表面|半径/cm|\n|---:|---:|---:|---:|---:|---:|---:|---:|
64     for row in rows:md+='|'+'|'.join('—' if not np.isfinite(x) else f'{x:.4f}' for x in
    ↪ row)+'|\n'
65     md+='\n “—” 表示固定位置已在当前材料域外。末行轴心显示0.1500是四位舍入;
    ↪ 未舍入最大值见end_event.json, 严格小于0.15。 \n'
66     (out/'表 6.md').write_text(md)
67     print(json.dumps(final,ensure_ascii=False,indent=2));print(md)
68     return final
69 if __name__=='__main__':
70     p=argparse.ArgumentParser();p.add_argument('--base',type=Path,default=R00T);a=p.parse
    ↪ _args();finalize(a.base)

```

A.38 package_workbooks

support/project/q4_complete_delivery/v1/source/package_workbooks.py

```

1  """Prepare arrays, execute artifact-tool chunks, package their unchanged OOXML.
2
3  This module does not author spreadsheet cells: all cells and their final row
4  addresses are generated by export_workbooks.mjs. Standard-library zipfile only
5  joins already-authored sheetData rows after checking common styles/strings.
6  Openpyxl is used exclusively for reading the final file.
7  """
8  from __future__ import annotations
9  import argparse
10 import hashlib
11 import json
12 import os
13 from pathlib import Path
14 import re
15 import shutil

```

```

16 import subprocess
17 import time
18 import threading
19 import zipfile
20 import xml.etree.ElementTree as ET
21 from xml.sax.saxutils import escape
22 from concurrent.futures import ThreadPoolExecutor
23 from decimal import Decimal, ROUND_HALF_UP
24 import numpy as np
25 import openpyxl
26
27
28 def sha(path):
29     h = hashlib.sha256()
30     with open(path, "rb") as f:
31         for b in iter(lambda: f.read(2**20), b""):
32             h.update(b)
33     return h.hexdigest()
34
35
36 def rounded4(array):
37     a = np.asarray(array, dtype=float)
38     scaled = np.abs(a) * 1e4
39     out = np.sign(a) * np.floor(scaled + 0.5) / 1e4
40     ties = np.abs(scaled - np.floor(scaled) - 0.5) <= 1e-8
41     for idx in zip(*np.where(ties)):
42         out[idx] = float(Decimal(str(float(a[idx]))).quantize(Decimal("0.0001"),
43             ↳ rounding=ROUND_HALF_UP))
44     return out
45
46
47 def verify_common_parts(base, part):
48     for common in ["xl/styles.xml", "xl/sharedStrings.xml"]:
49         if part.read(common) != base.read(common):
50             raise ValueError(f"Chunk metadata differs: {common}")
51     # Each export legitimately creates fresh relationship IDs. Keep the first
52     # package's workbook and .rels unchanged; compare their resolved meaning.
53     def signature(z):
54         rel=ET.fromstring(z.read("xl/_rels/workbook.xml.rels"))
55         targets={e.attrib["Id"]:e.attrib["Target"] for e in rel}
56         book=ET.fromstring(z.read("xl/workbook.xml"))
57         main="http://schemas.openxmlformats.org/spreadsheetml/2006/main"
58         relation="http://schemas.openxmlformats.org/officeDocument/2006/relationships"
59         sheets=[(e.attrib["name"],targets[e.attrib[f"{{{relation}}}id"]]) for e in
60             ↳ book.findall(f"{{{main}}}sheets/{{{main}}}sheet")]
61         return sheets,sorted((e.attrib["Type"],e.attrib["Target"]) for e in rel)
62     if signature(base)!=signature(part):
63         raise ValueError("Chunk workbook/sheet relationship meaning differs")
64
65
66 def combine_chunks(chunks, destination):
67     """Transport artifact-authored XML rows verbatim; never calculate a cell."""
68     destination = Path(destination)
69     temp = destination.with_suffix(".partial.xlsx")
70     row_re = re.compile(rb'<x:row\b[^\>]*\br="(\d+)"[^\>]*(?:/>|.??</x:row>)', re.S)
71     evidence = {"method": "artifact-tool authoring; verbatim sheetData row packaging",
72         ↳ "chunks": [], "sheets": {}}
73     with zipfile.ZipFile(chunks[0][2]) as base, zipfile.ZipFile(temp, "w",
74         ↳ compression=zipfile.ZIP_DEFLATED, compresslevel=6, allowZip64=True) as final:
75         names = base.namelist()
76         sheets = [n for n in names if re.fullmatch(r"xl/worksheets/sheet[12]\.xml", n)]
77         for name in names:
78             if name not in sheets:
79                 final.writestr(name, base.read(name))

```

```

76     for sheet in sheets:
77         previous = 0
78         count = 0
79         with final.open(sheet, "w", force_zip64=True) as output:
80             base_xml = base.read(sheet)
81             before, rest = base_xml.split(b"<x:sheetData>", 1)
82             _, after = rest.split(b"</x:sheetData>", 1)
83             # artifact-tool currently omits dimension. Reject a future
84             # per-chunk dimension rather than silently authoring metadata.
85             if b"<x:dimension" in before:
86                 raise ValueError("Unexpected per-chunk dimension metadata")
87             output.write(before + b"<x:sheetData>")
88             for lo, hi, filename in chunks:
89                 with zipfile.ZipFile(filename) as part:
90                     verify_common_parts(base, part)
91                     xml = part.read(sheet)
92                     expected = lo + 1
93                     for m in row_re.finditer(xml):
94                         row = int(m[1])
95                         if row == 1 and previous == 0:
96                             output.write(m[0]); previous = 1; count += 1
97                         elif lo + 1 <= row <= hi:
98                             if row != expected or row != previous + 1:
99                                 raise ValueError(f"Missing/duplicate row {row}; expected
100                                     ↪ {expected}")
101                             output.write(m[0]); previous = row; expected += 1; count += 1
102                         if expected != hi + 1:
103                             raise ValueError(f"Incomplete chunk: {filename}")
104                     output.write(b"</x:sheetData>" + after)
105             evidence["sheets"][sheet] = {"total_rows_including_header": count, "last_row":
106                 ↪ previous}
107         temp.replace(destination)
108     for lo, hi, p in chunks:
109         evidence["chunks"].append({"source_index_lo": lo, "source_index_hi_exclusive": hi,
110             ↪ "sha256": sha(p), "bytes": Path(p).stat().st_size})
111     evidence["final_sha256"] = sha(destination)
112     return evidence
113
114 def validate_q2(path, time_s, profiles, radius_cm):
115     report = {"file": str(path), "sha256": sha(path), "rounding": "decimal ROUND_HALF_UP, 4
116         ↪ places; time unrounded", "sheets": {}}
117     wb = openpyxl.load_workbook(path, read_only=True, data_only=False)
118     if wb.sheetnames != [" 温度", " 水分浓度"]:
119         raise ValueError("Template sheet names/order changed")
120     for name, component in [( " 温度", 0), ( " 水分浓度", 1)]:
121         s = wb[name]
122         rows = s.iter_rows(values_only=True)
123         header = next(rows)
124         if header != tuple([" 时间\\到药材中心的距离", *radius_cm]):
125             raise ValueError(f"Unexpected header {header}")
126         expected = rounded4(profiles[1:, :, component])
127         mismatch = 0; maxdiff = 0.0; n = 0; badtime = 0; badtypes = 0
128         for n, row in enumerate(rows, 1):
129             if n >= len(time_s):
130                 raise ValueError("Unexpected trailing row")
131             if len(row) != 22:
132                 raise ValueError(f"Wrong column count at row {n+1}")
133             badtime += row[0] != float(time_s[n])
134             badtypes += sum(not isinstance(v, (int, float)) for v in row)
135             d = np.abs(np.asarray(row[1:], dtype=float) - expected[n-1])
136             mismatch += int(np.count_nonzero(d)); maxdiff = max(maxdiff, float(d.max()))

```

```

134     report["sheets"][name] = {"data_rows": n, "expected_data_rows": len(time_s)-1,
    ↪ "numeric_cells_compared": n*21, "time_mismatches": badtime,
    ↪ "value_mismatches": mismatch, "max_abs_difference_after_rounding": maxdiff,
    ↪ "non_numeric_cells": badtypes, "first_time_s": float(time_s[1]),
    ↪ "last_time_s": float(time_s[-1])}
135     if n != len(time_s)-1 or mismatch or badtime or badtypes:
136         raise ValueError(json.dumps(report["sheets"][name]))
137     wb.close()
138     report["passed"] = True
139     return report
140
141
142 def validate_q4(path, source):
143     indexes=[i for i,t in enumerate(source["time_s"]) if t>0]
144     expected_times=np.array([source["time_s"][i] for i in indexes])
145     if not np.array_equal(expected_times[:-1],np.arange(60,expected_times[-1],60)):
146         raise ValueError("Q4 time grid is not every 60 s plus final time")
147     wb=openpyxl.load_workbook(path,read_only=True,data_only=False)
148     if wb.sheetnames != ["Sheet1"]:
149         raise ValueError("Q4 template sheet name changed")
150     it=wb["Sheet1"].iter_rows(max_col=25,values_only=True)
151     header=next(it)
152     if header[:23]!=tuple(["时间\\到药材中心的距离",*[j/10 for j in range(21)],"药材表面"]):
153         raise ValueError("Q4 template core header changed")
154     errors=[]; cells=0; blanks=0; coincident=0; radius_error=0.0; n=0
155     for n,row in enumerate(it,1):
156         if n>len(indexes):
157             raise ValueError("Q4 unexpected trailing data row")
158         i=indexes[n-1]; radius=source["radius_m"][i]
159         if row[0]!=source["time_s"][i]:
160             errors.append([n+1,"time"])
161         want=source["C_fixed"][i]+[source["C_surface"][i]]
162         expected=[None if x is None else
    ↪ float(Decimal(str(x)).quantize(Decimal('0.0001'),rounding=ROUND_HALF_UP)) for x
    ↪ in want]
163         if list(row[1:23])!=expected:
164             errors.append([n+1,"value"])
165         for j in range(21):
166             outside=j/1000>radius+1e-12
167             if (row[j+1] is None)!=outside:
168                 errors.append([n+1,j+2,"domain_mask"])
169             if outside: blanks+=1
170             else: cells+=1
171             if abs(j/1000-radius)<=1e-12:
172                 coincident+=1
173                 if row[j+1]!=row[22]: errors.append([n+1,j+2,"surface_coincidence"])
174             cells+=1
175             if row[23] is not None: errors.append([n+1,"spacer"])
176             radius_error=max(radius_error,abs(row[24]-radius*100))
177     wb.close()
178     report={"file":str(path),"sha256":sha(path),"rounding":"decimal ROUND_HALF_UP, 4
    ↪ places; time unrounded","data_rows":n,"expected_data_rows":len(indexes),"numeric_
    ↪ moisture_cells_compared":cells,"outside_domain_blank_cells":blanks,"surface_coinc
    ↪ idences_checked":coincident,"time_grid":"60 s, 120 s, ... plus actual final time",
    ↪ "first_time_s":float(expected_times[0]),"last_time_s":float(expected_times[-1]),"
    ↪ radius_metadata_max_abs_error_cm":radius_error,"errors":errors[:100],"passed":n==
    ↪ len(indexes) and not errors and radius_error<1e-12}
179     if not report["passed"]: raise ValueError(json.dumps(report))
180     return report
181
182
183 def main():
184     p=argparse.ArgumentParser()
185     p.add_argument("--root", type=Path, default=Path(__file__).resolve().parents[1])

```

```

186 p.add_argument("--q2-npz", type=Path)
187 p.add_argument("--q4-json", type=Path)
188 p.add_argument("--out-dir", type=Path, help="New delivery directory for a cold-start
↳ rebuild; refuses to overwrite an existing formal XLSX")
189 p.add_argument("--chunk-rows", type=int, default=10000)
190 p.add_argument("--font", default="Arial")
191 p.add_argument("--resume", action="store_true")
192 p.add_argument("--workers", type=int, choices=[1,2], default=2)
193 a=p.parse_args()
194 root=a.root.resolve(); source_delivery=root if
↳ (root/"source/export_workbooks.mjs").exists() else root/"q4_complete_delivery/v1"
195 delivery=a.out_dir.resolve() if a.out_dir else source_delivery
196 for relative in ["output", "validation", "q123_closeout"]:
197     (delivery/relative).mkdir(parents=True, exist_ok=True)
198 targets=[]
199 if a.q4_json: targets.append(delivery/"output/result4.xlsx")
200 if a.q2_npz: targets.append(delivery/"q123_closeout/result2.xlsx")
201 for target in targets:
202     if target.exists(): raise FileExistsError(f"Refusing to overwrite {target}; choose
↳ a new --out-dir")
203 temp=Path(os.environ.get("DRYING_WORKBOOK_TEMP", str(Path.cwd()/"workbook_temp")));
↳ temp.mkdir(parents=True, exist_ok=True)
204 node=os.environ["CODEX_PRIMARY_RUNTIME_NODE"]
205 modules=Path(os.environ["CODEX_PRIMARY_RUNTIME_NODE_MODULES"])
206 if not (temp/"node_modules").exists():
207     (temp/"node_modules").symlink_to(modules, target_is_directory=True)
208 shutil.copyfile(source_delivery/"source/export_workbooks.mjs",
↳ temp/"export_workbooks.mjs")
209 font_dir=source_delivery/"inputs/fonts"
210 node_env=os.environ.copy()
211 if font_dir.exists():
212     config=temp/"fonts.conf"
213     config.write_text(f'<?xml version="1.0"?><!DOCTYPE fontconfig SYSTEM
↳ "urn:fontconfig:fonts.dtd"><fontconfig><include
↳ ignore_missing="yes">/etc/fonts/fonts.conf</include><dir>{escape(str(font_dir)
↳ )}</dir><cachedir>{escape(str(temp/"font-cache"))}</cachedir></fontconfig>')
214     node_env["FONTCONFIG_FILE"]=str(config)
215 def template(name):
216     supplied=source_delivery/"inputs"/name.replace('.xlsx', '_template.xlsx')
217     return supplied if supplied.exists() else root/"A 题/附件/附件 3"/name
218 if a.q4_json:
219     previewdir=delivery/"validation/workbook_previews"; previewdir.mkdir(exist_ok=True)
220     final=delivery/"output/result4.xlsx"
221     with open(delivery/"validation/workbook_result4_build.log", "w") as f:
222         subprocess.run([node, "--max-old-space-size=2048", str(temp/"export_workbooks.mjs"),
↳ "--mode=q4", f"--root={root}", f"--input={a.q4_json.resolve()}", f"--output={final}", f"--font={a.font}", f"--previewDir={previewdir}"], cwd=temp, env=node_env, stdout=f, stderr=subprocess.STDOUT, check=True)
223     source=json.loads(a.q4_json.read_text())
224     report=validate_q4(final, source)
225     report["input_json_sha256"]=sha(a.q4_json)
226     report["template_sha256"]=sha(template('result4.xlsx'))
227     sidecar=Path(str(final)+".inspect.ndjson")
228     if sidecar.exists():
229         ↳ sidecar.replace(delivery/"validation/workbook_result4_inspect.ndjson")
229     (delivery/"validation/workbook_result4.json").write_text(json.dumps(report, ensure_ascii=False, indent=2)+"\n")
230     print(json.dumps({"stage": "result4_validated", "report": report}, ensure_ascii=False), flush=True)
231     if not a.q2_npz: return
232 if not a.q2_npz:
233     p.error("--q2-npz or --q4-json is required")
234 with np.load(a.q2_npz) as data:
235     times=np.asarray(data["time_s"], dtype="<f8")

```

```

236     profile=np.asarray(data["profile_TC"], dtype="<f8")
237     radius=np.asarray(data["radius_cm"], dtype=float)
238     assert profile.shape == (len(times),21,2)
239     assert times[0]==0 and times[-1]==206906.76 and np.array_equal(times[1:-1],
    ↪ np.arange(1,206907))
240     source_hash=sha(a.q2_npz);
    ↪ builder_hash=sha(source_delivery/"source/export_workbooks.mjs")
241     metadata=temp/"q2_meta.json"
242     if a.resume and metadata.exists():
243         previous=json.loads(metadata.read_text())
244         if previous.get("source_sha256")!=source_hash or
    ↪ previous.get("builder_sha256")!=builder_hash:
245             raise ValueError("Resume cache belongs to different inputs or builder; omit
    ↪ --resume for a fresh rebuild")
246     times.tofile(temp/"q2_time.f64"); profile.tofile(temp/"q2_profile.f64")
247     metadata.write_text(json.dumps({"shape":profile.shape,"radius_cm":radius.tolist(),"so
    ↪ urce_sha256":source_hash,"builder_sha256":builder_hash}))
248     chunks=[]; start=time.monotonic()
249     previewdir=delivery/"validation/workbook_previews"; previewdir.mkdir(exist_ok=True)
250     logdir=delivery/"validation/workbook_chunks"; logdir.mkdir(exist_ok=True)
251     slots=threading.Condition(); active=0
252     def make_chunk(lo):
253         nonlocal active
254         hi=min(lo+a.chunk_rows,len(times)); outfile=temp/f"q2_chunk_{lo}_{hi}.xlsx"
255         logfile=logdir/f"q2_chunk_{lo}_{hi}.log"
256         reusable=False
257         if a.resume and outfile.exists():
258             try:
259                 with zipfile.ZipFile(outfile) as z:
260                     reusable=z.testzip() is None
261             except zipfile.BadZipFile:
262                 reusable=False
263         if not reusable:
264             cmd=[node,"--max-old-space-size=4096",str(temp/"export_workbooks.mjs"),"--mod
    ↪ e=q2chunk",f"--root={root}",f"--inputDir={temp}",f"--lo={lo}",f"--hi={hi}
    ↪ ",f"--out={outfile}",f"--font={a.font}",f"--previewDir={previewdir}",f"--
    ↪ preview={'true' if lo==1 else 'false'}"]
265             with slots:
266                 while True:
267                     control=temp/"q2_workers.txt"
268                     cap=min(a.workers,int(control.read_text().strip())) if
    ↪ control.exists() else a.workers
269                     available=int(re.search(r"MemAvailable:\s+(\d+)",Path('/proc/meminfo'
    ↪ ).read_text())[1])/1024**2
270                     if available<8: cap=1
271                     if active<cap and available>4:
272                         active+=1; break
273                     slots.wait(timeout=5)
274             try:
275                 with open(logfile,"w") as f:
276                     subprocess.run(cmd,cwd=temp,env=node_env,stdout=f,stderr=subprocess.S
    ↪ TDOUT,check=True)
277             finally:
278                 with slots:
279                     active-=1; slots.notify_all()
280             print(json.dumps({"stage":"chunk_complete","lo":lo,"hi":hi,"elapsed_s":time.monot
    ↪ onic()-start}),flush=True)
281             return lo,hi,outfile
282     with ThreadPoolExecutor(max_workers=a.workers) as pool:
283         for chunk in pool.map(make_chunk,range(1,len(times),a.chunk_rows)):
284             chunks.append(chunk)
285             if len(chunks)>1:
286                 with zipfile.ZipFile(chunks[0][2]) as first, zipfile.ZipFile(chunk[2]) as
    ↪ current:

```

```

287         verify_common_parts(first,current)
288     final=delivery/"q123_closeout/result2.xlsx"
289     package=combine_chunks(chunks,final)
290     package.update({"input_npz_sha256":sha(a.q2_npz),"artifact_builder_sha256":sha(source_
↵ _delivery/"source/export_workbooks.mjs"),"chunk_rows":a.chunk_rows,"elapsed_s":tim
↵ e.monotonic()-start}))
291     package["template_sha256"]=sha(template('result2.xlsx'))
292     (delivery/"validation/workbook_packaging.json").write_text(json.dumps(package,ensure_
↵ ascii=False,indent=2)+"\n")
293     report=validate_q2(final,times,profile,radius.tolist())
294     (delivery/"validation/workbook_result2.json").write_text(json.dumps(report,ensure_asc
↵ ii=False,indent=2)+"\n")
295     print(json.dumps({"stage":"result2_validated","report":report},ensure_ascii=False),fl
↵ ush=True)
296
297
298 if __name__ == "__main__":
299     main()

```

A.39 export_workbooks

support/project/q4_complete_delivery/v1/source/export_workbooks.mjs

```

1 // Run a copy of this file in an OS temporary directory whose node_modules
2 // symlink points to CODEX_PRIMARY_RUNTIME_NODE_MODULES, using CODEX_PRIMARY_RUNTIME_NODE.
3 // All XLSX authoring uses the documented @oai/artifact-tool API.
4 import fs from 'node:fs/promises';
5 import path from 'node:path';
6 import { FileBlob, SpreadsheetFile, Workbook } from '@oai/artifact-tool';
7
8 const args=Object.fromEntries(process.argv.slice(2).map(s=>{const p=s.indexOf('=');return
↵ [s.slice(0,p).replace(/^-\/,','),s.slice(p+1)]}));
9 const root=args.root||process.cwd();
10 const mode=args.mode||'probe';
11 function report(stage, extra={}) { console.log(JSON.stringify({stage,elapsed_s:(Date.now(
↵ )-start)/1000,rss_MB:process.memoryUsage().rss/2**20,...extra})); }
12 const start=Date.now();
13 const font=args.font||'Arial';
14 async function templateFile(name) {
15     const bundled=name.replace('.xlsx','_template.xlsx');
16     for(const candidate of [path.join(root,'inputs',bundled),path.join(root,'q4_complete_de
↵ livery/v1/inputs',bundled),path.join(root,'A题/附件/附件3',name)]) {
17         try { await fs.access(candidate); return candidate; } catch {}
18     }
19     throw new Error(`Missing supplied template ${bundled}`);
20 }
21 // Decimal ROUND_HALF_UP of the canonical decimal representation of each
22 // double. Values far from a half-way case use an equivalent fast path.
23 function halfUp4(value) {
24     if(value===null) return null;
25     if(!Number.isFinite(value)) throw new Error('Non-finite in-domain value');
26     const sign=value<0?-1:1,v=Math.abs(value),scaled=v*1e4;
27     if(Math.abs(scaled-Math.floor(scaled)-0.5)>1e-8) return sign*Math.floor(scaled+0.5)/1e4;
28     const [mant,expText='0']=v.toString().split('e');
29     const [whole,fraction='']=mant.split('.');
30     const integer=BigInt(whole+fraction);
31     const places=fraction.length-Number(expText)-4;
32     const rounded=places<=0?integer*10n**BigInt(-places):(integer+5n*10n**BigInt(places-1))
↵ /(10n**BigInt(places));
33     return sign*Number(rounded)/1e4;
34 }
35

```

```

36 async function saveCheck(wb, outfile, label, sampleSheets=[]) {
37   wb.recalculate();
38   const checks=[];
39   for(const [sn,rg] of sampleSheets) {
40     const chk=await wb.inspect({kind:'table',range:`${sn}!${rg}`,include:'values,formulas',
41     ↪ as',tableMaxRows:3,tableMaxCols:24,maxChars:5000});
42     checks.push(JSON.parse(chk.ndjson.split('\n').filter(Boolean)[0]));
43   }
44   const errors=await
45   ↪ wb.inspect({kind:'match',searchTerm:'#REF!|#DIV/0!|#VALUE!|#NAME\\?|#N/A|#NUM!|#NUL
46   ↪ L!|#SPILL!|#CALC!',options:{useRegex:true,maxResults:10},maxChars:2000});
47   report('artifact_checked',{label,checks,errors:errors.ndjson});
48   await fs.mkdir(path.dirname(outfile),{recursive:true});
49   const output=await SpreadsheetFile.exportXlsx(wb);
50   await output.save(outfile);
51   report('artifact_saved',{label,outfile});
52 }
53
54 function formatSheet(sh,firstRow,lastRow,lastCol,header) {
55   sh.getRange('A1:F5').clear({applyTo:'all'});
56   sh.getRangeByIndexes(0,0,1,header.length).values=[header];
57   sh.getRangeByIndexes(0,0,1,header.length).format={font:{name:font,size:10,bold:true},verticalAlignment:'center',horizontalAlignment:'center',wrapText:true,rowHeight:31};
58   sh.getRange('A1').format.columnWidth=28;
59   sh.getRangeByIndexes(0,1,1,header.length-1).format.columnWidth=10;
60   sh.getRangeByIndexes(firstRow-1,0,lastRow-firstRow+1,lastCol).format={font:{name:font,size:10},verticalAlignment:'center',horizontalAlignment:'right'};
61   sh.getRangeByIndexes(firstRow-1,0,lastRow-firstRow+1,1).setNumberFormat('0.#####');
62   sh.getRangeByIndexes(firstRow-1,1,lastRow-firstRow+1,lastCol-1).setNumberFormat('0.0000');
63   sh.freezePanes.freezeRows(1);
64   sh.freezePanes.freezeColumns(1);
65 }
66
67 if(mode==='q2chunk') {
68   const dir=args.inputDir;
69   const meta=JSON.parse(await fs.readFile(path.join(dir,'q2_meta.json'),'utf8'));
70   const times=await fs.readFile(path.join(dir,'q2_time.f64'));
71   const profiles=await fs.readFile(path.join(dir,'q2_profile.f64'));
72   const lo=Number(args.lo),hi=Number(args.hi);
73   if(!(lo>=1&&hi<=meta.shape[0]&&hi>lo)) throw new Error('Invalid source index interval');
74   const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(await
75   ↪ templateFile('result2.xlsx')));
76   for(const [sn,component] of [['温度',0],['水分浓度',1]]) {
77     const sh=wb.worksheets.getItem(sn);
78     formatSheet(sh,lo+1,hi,22,['时间\\到药材中心的距离',...meta.radius_cm]);
79     for(let b=lo;b<hi;b+=1000) {
80       const e=Math.min(hi,b+1000),rows=[];
81       for(let i=b;i<e;i++) {
82         const row=[times.readDoubleLE(i*8)];
83         for(let j=0;j<21;j++)
84         ↪ row.push(halfUp4(profiles.readDoubleLE(((i*21+j)*2+component)*8)));
85       rows.push(row);
86     }
87     sh.getRangeByIndexes(b,0,e-b,22).values=rows;
88   }
89 }
90
91 report('q2_chunk_populated',{lo,hi});
92 if(args.preview==='true') for(const sn of ['温度','水分浓度']) await
93 ↪ inspectAndRender(wb,sn,'result2_${sn}_head','A1:V8');
94 await saveCheck(wb,args.out,'q2_${lo}_${hi}',[['温度','A${lo+1}:V${Math.min(lo+2,hi)}'],
95 ↪ ['水分浓度','A${hi}:V${hi}']]);
96 }
97

```

```

90 if(mode==='q4') {
91   const data=JSON.parse(await fs.readFile(args.input,'utf8'));
92   const indexes=data.time_s.map((t,i)=>t>0?i:-1).filter(i=>i>=0);
93   const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(await
    ↪   templateFile('result4.xlsx')));
94   const sh=wb.worksheets.getItem('Sheet1');
95   formatSheet(sh,2,indexes.length+1,23,['时间\\到药材中心的距离',...Array.from({length:21},
    ↪   (_,j)=>j/10),'药材表面']);
96   const rows=indexes.map(i=>[data.time_s[i],...data.C_fixed[i].map(halfUp4),halfUp4(data.
    ↪   C_surface[i])]);
97   sh.getRangeByIndexes(1,0,rows.length,23).values=rows;
98   sh.getRange('W1').format.columnWidth=13;
99   sh.getRange('Y1').values=[[ '实际半径 R(t) / cm' ]];
100  sh.getRange('Y1').format={font:{name:font,size:10,bold:true},wrapText:true,columnWidth:
    ↪   18,verticalAlignment:'center',horizontalAlignment:'center'};
101  sh.getRangeByIndexes(1,24,rows.length,1).values=indexes.map(i=>[data.radius_m[i]*100]);
102  sh.getRangeByIndexes(1,24,rows.length,1).setNumberFormat('0.000000');
103  sh.getRange('AA1').values=[[ '输出说明' ]];
104  sh.getRange('AA2:AA6').values=[[ '时间单位: 秒; 固定径向坐标单位: 厘米。
    ↪   ',[ '水分浓度为干基含水率, 单位kg水/kg干物质。'],[ '固定径向点超出当前半径时留空, 不代表零。
    ↪   ',[ '表面值取r=R(t); 与固定点重合时两列数值一致。'],[ '含水率按十进制四位ROUND_HALF_UP舍入;
    ↪   未舍入轨迹另附, 严格终判使用未舍入全域最大值。' ]]];
105  sh.getRange('AA1:AA6').format={font:{name:font,size:10},columnWidth:94};
106  await inspectAndRender(wb,'Sheet1','result4_head','A1:Y8');
107  await inspectAndRender(wb,'Sheet1','result4_tail','A${Math.max(2,rows.length-5)}:Y${row
    ↪   s.length+1}`');
108  await saveCheck(wb,args.out,'q4',[['Sheet1','A1:Y3'],[ 'Sheet1','A${rows.length+1}:Y${ro
    ↪   ws.length+1}` ]]);
109 }
110
111 async function inspectAndRender(wb,sheetName,tag,range='A1:W8') {
112   const inspect=await wb.inspect({kind:'table',range:`_${sheetName}_${range}`,include:'v
    ↪   alues,formulas',tableMaxRows:8,tableMaxCols:24,maxChars:5000});
113   console.log(inspect.ndjson);
114   const img=await wb.render({sheetName,range,scale:1.3,format:'png'});
115   await fs.writeFile(path.join(args.previewDir|process.env.DRYING_WORKBOOK_TEMP|path.jo
    ↪   in(process.cwd(),'workbook_temp'),'_${tag}.png'),new Uint8Array(await
    ↪   img.arrayBuffer()));
116 }
117
118 if(mode==='preview') {
119   const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(args.input));
120   for(const sheetName of (args.sheets||'Sheet1').split(',')) {
121     await inspectAndRender(wb,sheetName,`${args.tag}||'preview'}_${sheetName}`,args.range|
    ↪   'A1:W8');
122   }
123 }
124
125 if(mode==='probe') {
126   const n=Number(args.rows||2000);
127   for(const name of ['result2.xlsx','result4.xlsx']) {
128     const original=await SpreadsheetFile.importXlsx(await FileBlob.load(await
    ↪   templateFile(name)));
129     report('template_import',{name});
130     const names=name==='result2.xlsx'?['温度','水分浓度']:['Sheet1'];
131     for(const sn of names) await
    ↪     inspectAndRender(original,sn,`template_${name}_${sn}`, 'A1:F5');
132   }
133   const wb=Workbook.create();
134   for(const sn of ['capacity_probe_T','capacity_probe_C']) {
135     const sh=wb.worksheets.add(sn);
136     sh.getRange('A1:V1').values=[[ 'BENCHMARK ONLY — synthetic
    ↪     data',...Array.from({length:21},(_,i)=>i/10)]];
137     for(let i=0;i<n;i+=2000) {

```

```

138     const count=Math.min(2000,n-i);
139     const rows=Array.from({length:count},(_,j)=>[i+j+1,...Array.from({length:21},(_,c)=
↵ >(i+j+c)/10000)]);
140     sh.getRangeByIndexes(i+1,0,count,22).values=rows;
141 }
142 sh.getRange(`B2:V${n+1}`).setNumberFormat('0.0000');
143 sh.freezePanes.freezeRows(1);
144 report('probe_populated',{sheet:sn,rows:n});
145 }
146 wb.recalculate(); report('probe_recalculate');
147 const out=await SpreadsheetFile.exportXlsx(wb); report('probe_export');
148 await out.save(path.join(process.env.DRYING_WORKBOOK_TEMP||path.join(process.cwd(),'wor
↵ kbook_temp'),'capacity_probe_${n}.xlsx'));
149 report('probe_saved');
150 }

```

A.40 geometry_solver_executed

support/project/q4_complete_delivery/v1/source/geometry_solver_executed.py

```

1  """Independent vertex/dual-volume axisymmetric validation of Q1/Q23/Q4.
2
3  All states remain coupled from t=0. No isothermal projection is used. The
4  reference grid follows homogeneous radial material contraction, while length
5  is fixed. Reference weights are constant; radial diffusion scales as q^-2,
6  side exchange as q^-1, axial diffusion/end exchange remain unchanged.
7  """
8  from __future__ import annotations
9  import argparse, hashlib, json, time
10 from pathlib import Path
11 import numpy as np
12 from scipy.integrate import solve_ivp
13 from scipy.interpolate import PchipInterpolator
14 from scipy.sparse import coo_matrix
15 from scipy.special import exp1
16 from openpyxl import load_workbook
17
18 ROOT=Path(__file__).resolve().parents[3]
19 DEST=Path(__file__).resolve().parents[1]/'validation'/ 'geometry'
20
21 def data_xlsx(path, width):
22     wb=load_workbook(path,read_only=True,data_only=True)
23     rows=list(wb.active.values); wb.close()
24     a=np.asarray([row[:width] for row in rows[1:] if row[0] is not None],float)
25     if not np.isfinite(a).all() or not np.all(np.diff(a[:,:])>0):
26         raise ValueError('Invalid input records')
27     return a
28
29 class Inputs:
30     def __init__(self):
31         self.env=data_xlsx(ROOT/'A 题/附件/附件 1.xlsx',3)
32         self.rad=data_xlsx(ROOT/'A 题/附件/附件 2.xlsx',2)
33         self.rad[:,1]*=.01
34         self.future=self.env[self.env[:,0]>=10800,1:].mean(axis=0)
35     def ambient(self,t,future=False):
36         if future:return self.future
37         return np.array([np.interp(t,self.env[:,0],self.env[:,j]) for j in (1,2)])
38     def radius(self,t):
39         if t>self.rad[-1,0]+1e-6:raise ValueError('Radius input ends at 72 h')
40         return float(np.interp(t,self.rad[:,0],self.rad[:,1]))
41
42 def properties(T,C,case):

```

```

43     if np.any(C<=0) or np.any(T<=-273.15):raise FloatingPointError('Nonpositive material
↪ state')
44     if case=='q1':
45         S=np.full_like(C,820.*2600.); k=np.full_like(C,.36)
46         D=7e-9*np.exp(-.89/C)
47         return S,k,D,np.zeros_like(C),np.zeros_like(C),np.zeros_like(C),D*.89/C**2
48     if case=='q23':a,b,c,d,e,f,A,B=650.,128.,1450.,2736.,.21,.38,.0024,.45
49     elif case=='q4':a,b,c,d,e,f,A,B=760.,90.,1850.,2150.,.12,.20,.00042,.30
50     else:raise ValueError(case)
51     rho=a+b*C;cp=c+d*C/(1+C);S=rho*cp;k=e+f*C/(1+C)
52     D=A*np.exp(-B/C-3850/(T+273.15))
53     return S,k,D,b*cp+rho*d/(1+C)**2,f/(1+C)**2,D*3850/(T+273.15)**2,D*B/C**2
54
55 def primitive_difference(a,b,B):
56     mid=(a+b)/2;delta=a-b;close=np.abs(delta)<.002*mid;value=np.empty_like(a)
57     if np.any(~close):
58         x=a[~close];y=b[~close]
59         value[~close]=x*np.exp(-B/x)-B*exp1(B/x)-y*np.exp(-B/y)+B*exp1(B/y)
60     if np.any(close):
61         m=mid[close];d=delta[close];v=np.zeros_like(m)
62         for x,w in
↪ ((-.8611363115940526,.3478548451374539),(-.3399810435848563,.6521451548625461))
↪ (,.3399810435848563,.6521451548625461),(.8611363115940526,.3478548451374539)):
63             v+=w*np.exp(-B/(m+x*d/2))
64         value[close]=v*d/2
65     return value
66
67 class GeometryFV:
68     def __init__(self,nr,nz,case,inputs,grading=2.,zgrading=2.,end_transfer=True,shrink=T)
↪ rue,flux='integral'):
69         self.nr,self.nz,self.case,self.inputs=nr,nz,case,inputs
70         self.shrink=shrink and case=='q4'
71         self.flux=flux
72         self.r=.02*(1-(1-np.linspace(0,1,nr+1))**grading)
73         rf=np.r_[0,(self.r[:-1]+self.r[1:])/2,.02];wr=np.diff(rf**2)/2
74         if nz:
75             self.z=.125*(1-(1-np.linspace(0,1,nz+1))**zgrading)
76             zf=np.r_[0,(self.z[:-1]+self.z[1:])/2,.125];wz=np.diff(zf)
77         else:self.z=np.array([0.]);wz=np.ones(1)
78         self.shape=(nz+1,nr+1);self.m=(nr+1)*(nz+1)
79         self.w=(wz[:,None]*wr).ravel();idx=np.arange(self.m).reshape(self.shape)
80         i=[idx[:,:-1].ravel()];j=[idx[:,1:].ravel()]
81         g=[(wz[:,None]*rf[None,1:-1]/np.diff(self.r)).ravel()]
82         self.radial_edges=len(i[0])
83         if nz:
84             i.append(idx[:-1].ravel());j.append(idx[1:].ravel())
85             g.append((wr[None,:]/np.diff(self.z[:,None])).ravel())
86         self.i=np.concatenate(i);self.j=np.concatenate(j);self.g=np.concatenate(g)
87         side=np.zeros(self.shape);end=np.zeros(self.shape)
88         side[:,-1]=.02*wz
89         if nz and end_transfer:end[-1,:]=wr
90         self.side=side.ravel();self.end=end.ravel()
91         rows=[];cols=[]
92         for dest in (self.i,self.j):
93             for src in (self.i,self.j):
94                 for a in range(2):
95                     for b in range(2):rows.append(2*dest+a);cols.append(2*src+b)
96         nodes=np.arange(self.m)
97         self.jrows=np.r_[np.concatenate(rows),2*nodes,2*nodes,2*nodes+1]
98         self.jcols=np.r_[np.concatenate(cols),2*nodes+1,2*nodes,2*nodes+1]
99     def rhs(self,t,y,ambient,jac=False):
100         q=self.inputs.radius(t)/.02 if self.shrink else 1.
101         g=self.g.copy();g[:self.radial_edges]/=q*q
102         area=self.side/q+self.end

```

```

103     U=y.reshape(self.m,2);T,C=U.T;i,j=self.i,self.j
104     S,k,D,Sc,kc,Dt,Dc=properties(T,C,self.case)
105     ks=k[i]+k[j];kf=2*k[i]*k[j]/ks;k1=2*(k[j]/ks)**2;kr=2*(k[i]/ks)**2
106     ds=D[i]+D[j];df=2*D[i]*D[j]/ds;d1=2*(D[j]/ds)**2;dr=2*(D[i]/ds)**2
107     dT=T[i]-T[j];dC=C[i]-C[j];fT=g*kf*dT
108     if self.flux=='integral':
109         B=.89 if self.case=='q1' else .45 if self.case=='q23' else .30
110         A=7e-9 if self.case=='q1' else (.0024 if self.case=='q23' else
111             ↪ .00042)*np.exp(-3850/((T[i]+T[j])/2+273.15))
112         fC=g*A*primitive_difference(C[i],C[j],B)
113         mci=g*A*np.exp(-B/C[i]);mcj=-g*A*np.exp(-B/C[j])
114         mti=np.zeros_like(fC) if self.case=='q1' else fC*1925/((T[i]+T[j])/2+273.15)**2
115         mtj=mti
116     else:
117         fC=g*df*dC
118         mci=g*(df+d1*Dc[i]*dC);mcj=g*(-df+dr*Dc[j]*dC)
119         mti=g*d1*Dt[i]*dC;mtj=g*dr*Dt[j]*dC
120     f=np.column_stack([np.bincount(j,weights=fT,minlength=self.m)-np.bincount(i,weigh
121         ↪ ts=fT,minlength=self.m),
122             np.bincount(j,weights=fC,minlength=self.m)-np.bincount(i,weigh
123         ↪ ts=fC,minlength=self.m)])
124     et,ec=ambient(t);f[:,0]=-area*25.*(T-et);f[:,1]=-area*8e-7*(C-ec)
125     storage=self.w[:,None]*np.column_stack([S,np.ones(self.m)]);f/=storage
126     if not jac:return f.ravel()
127     left=np.empty((len(i),2,2));right=np.empty_like(left)
128     left[:,0,0]=g*kf;right[:,0,0]=-g*kf
129     left[:,0,1]=g*k1*kC[i]*dT;right[:,0,1]=g*kr*kC[j]*dT
130     left[:,1,0]=mti;right[:,1,0]=mtj
131     left[:,1,1]=mci;right[:,1,1]=mcj
132     vals=[]
133     for dest,sgn in ((i,-1.),(j,1.)):
134         for v in (left,right):
135             for a in range(2):
136                 for b in range(2):vals.append(sgn*v[:,a,b]/storage[dest,a])
137     vals.extend([-f[:,0]*Sc/S,-area*25./storage[:,0],-area*8e-7/self.w])
138     return coo_matrix((np.concatenate(vals),(self.jrows,self.jcols)),shape=(2*self.m,
139         ↪ 2*self.m)).tocsc()
140
141 def observe(self,t,y):
142     u=y.reshape(self.shape+(2,));C=u[:,1];ix=np.unravel_index(np.argmax(C),C.shape)
143     radius=self.inputs.radius(t) if self.shrink else .02
144     # Material-coordinate samples, exactly same locations for paired 1D/2D.
145     target=np.linspace(0,.02,21)
146     mid=PchipInterpolator(self.r**2,u[0],axis=0)(target**2)
147     end=PchipInterpolator(self.r**2,u[-1],axis=0)(target**2)
148     return
149     ↪ mid,end,float(C[ix]),np.array([self.r[ix[1]]*radius/.02,self.z[ix[0]]]),radius
150
151 def run(case,nr,nz,end=None,rtol=2e-9,max_step=240.,shrink=True,end_transfer=True,flux='i
152     ↪ ntegral'):
153     inp=Inputs();op=GeometryFV(nr,nz,case,inp,shrink=shrink,end_transfer=end_transfer,flu
154     ↪ x=flux)
155     end=float(end if end is not None else (1800 if case=='q1' else 220000 if case=='q23'
156     ↪ else 259200))
157     label=f'{case}_{nr}x{nz}'+(' _fixed' if case=='q4' and not shrink else
158     ↪ '')+(' _sealed_ends' if nz and not end_transfer else '')+(' _integral' if
159     ↪ flux=='integral' else '')
160     if rtol!=2e-9:label+=f'_rtol{rtol:g}'
161     DEST.mkdir(parents=True,exist_ok=True)
162     if (DEST/(label+'.json')).exists():raise FileExistsError(label)
163     start=time.perf_counter();y=np.tile([28.,2.55],op.m)
164     sample_times=np.unique(np.r_[np.arange(0,end+1,60.),end,[100,300,900,1200,1500]])
165     sample_times=sample_times[(sample_times>0)&(sample_times<=end)]
166     snapshots_at=np.unique(np.r_[0,100,300,600,900,1200,1500,1800,3600,5400,7200,9000,108
167     ↪ 00,14400,np.arange(21600,end,21600)])

```

```

156 cuts=np.unique(np.r_[0,inp.env[:,0][inp.env[:,0]<end],inp.rad[:,0][inp.rad[:,0]<end]
    ↪ if op.shrink else [],end])
157 recorded=[];mid=[];edge=[];cmax=[];locations=[];radii=[];snapshots=[y.reshape(op.shap
    ↪ e+(2,)).copy()];st=[0.]
158 stats={'case':case,'nr':nr,'nz':nz,'grading':2.,'zgrading':2.,'shrink':op.shrink,'end'
    ↪ _transfer':end_transfer,
159         'rtol':rtol,'atol_T':1e-9,'atol_C':1e-11,'max_step':max_step,'nfev':0,'njev':0
    ↪ , 'nlu':0,
160         'flux':flux+' local coefficients, vertex dual finite
    ↪ volumes','temperature':'coupled throughout; no isothermal projection',
161         'environment_future':inp.future.tolist(),'source_sha':'anonymous-source'})
162 def record(t,Y):
163     m,e,c,l,R=op.observe(t,Y);recorded.append(t);mid.append(m);edge.append(e);cmax.ap
    ↪ pend(c);locations.append(l);radii.append(R)
164 record(0.,y)
165 def event(t,Y):return np.max(Y[1::2])-.15
166 event.direction=-1;event.terminal=True
167 root=None;event_states=[];event_times=[]
168 for a,b in zip(cuts[:-1],cuts[1:]):
169     ambient=(lambda t: inp.ambient(t,True)) if a>=14400 else inp.ambient
170     sol=solve_ivp(lambda t,Y:op.rhs(t,Y,ambient),(a,b),y,method='BDF',
171                 jac=lambda
    ↪ t,Y:op.rhs(t,Y,ambient,True),rtol=rtol,atol=np.tile([1e-9,1e-11],op.m),
172                 max_step=min(max_step,30.) if a<14400 else max_step,dense_output=True,
173                 events=event if case!='q1' else None)
174     if not sol.success:raise RuntimeError(sol.message)
175     for key in ('nfev','njev','nlu'):stats[key]+=int(getattr(sol,key))
176     stop=float(sol.t[-1]);ts=sample_times[(sample_times>a)&(sample_times<=stop)]
177     if len(ts):
178         vals=sol.sol(ts)
179         for t,Y in zip(ts,vals.T):record(float(t),Y)
180     sts=snapshots_at[(snapshots_at>a)&(snapshots_at<=stop)]
181     if len(sts):
182         for t,Y in zip(sts,sol.sol(sts).T):st.append(float(t));snapshots.append(Y.res
    ↪ hape(op.shape+(2,)).copy())
183     y=sol.y[:,-1]
184     if sol.t_events is not None and len(sol.t_events[0]):
185         root=float(sol.t_events[0][0]);record(root,y);st.append(root);snapshots.appen
    ↪ d(y.reshape(op.shape+(2,)).copy())
186         before=max(float(a),root-1.);after=root+1.
187         ext=solve_ivp(lambda t,Y:op.rhs(t,Y,ambient),(root,after),y,method='BDF',
188                     jac=lambda t,Y:op.rhs(t,Y,ambient,True),rtol=rtol,atol=np.tile([1e-9,1e-1
    ↪ 1],op.m),dense_output=True)
189         if not ext.success:raise RuntimeError(ext.message)
190         event_times=[before,root,after];event_states=[sol.sol(before).reshape(op.shap
    ↪ e+(2,)),y.reshape(op.shape+(2,)),ext.y[:,-1].reshape(op.shape+(2,))]
191         break
192     if b%21600==0 or b in (1800,10800,14400):print(label,'t=',b,'max C=',float(np.max
    ↪ (y[1::2])),',elapsed=',round(time.perf_counter()-start,2),flush=True)
193 stats['elapsed_s']=time.perf_counter()-start;stats['event_root_s']=root
194 stats['end_s']=float(recorded[-1]);stats['end_max_C']=float(cmax[-1]);stats['end_max_
    ↪ location_r_z_m']=locations[-1].tolist()
195 if root is not None:stats['event_bracket_Cmax']=[float(np.max(v[:,1])) for v in
    ↪ event_states]
196 stats['source_sha256']=hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
197 np.savez_compressed(DEST/(label+'.npz'),time_s=recorded,mid=mid,end=edge,Cmax=cmax,Cm
    ↪ ax_location_r_z_m=locations,
198         radius_m=radii,reference_r_m=op.r,z_m=op.z,weights=op.w,snapshot_times_s=st,snap
    ↪ shots=snapshots,
199         event_times_s=event_times,event_states=event_states)
200 (DEST/(label+'.json')).write_text(json.dumps(stats,indent=2,ensure_ascii=False)+'\n')
201 print(json.dumps(stats,ensure_ascii=False),flush=True)
202 return stats
203

```

```

204 if __name__=='__main__':
205     p=argparse.ArgumentParser();p.add_argument('--case',choices=['q1','q23','q4'],require_
↪     d=True)
206     p.add_argument('--nr',type=int,required=True);p.add_argument('--nz',type=int,default=0)
207     p.add_argument('--end',type=float);p.add_argument('--rtol',type=float,default=2e-9)
208     p.add_argument('--max-step',type=float,default=240.);p.add_argument('--fixed',action=
↪     'store_true');p.add_argument('--sealed-ends',action='store_true')
209     p.add_argument('--flux',choices=['integral','harmonic'],default='integral')
210     a=p.parse_args();run(a.case,a.nr,a.nz,a.end,a.rtol,a.max_step,not a.fixed,not
↪     a.sealed_ends,a.flux)

```

A.41 geometry_solver

support/project/q4_complete_delivery/v1/source/geometry_solver.py

```

1  """Independent vertex/dual-volume axisymmetric validation of Q1/Q23/Q4.
2
3  All states remain coupled from t=0. No isothermal projection is used. The
4  reference grid follows homogeneous radial material contraction, while length
5  is fixed. Reference weights are constant; radial diffusion scales as q^-2,
6  side exchange as q^-1, axial diffusion/end exchange remain unchanged.
7  """
8  from __future__ import annotations
9  import argparse, hashlib, json, time
10 from pathlib import Path
11 import numpy as np
12 from scipy.integrate import solve_ivp
13 from scipy.interpolate import PchipInterpolator
14 from scipy.sparse import coo_matrix
15 from scipy.special import exp1
16 from openpyxl import load_workbook
17
18 ROOT=Path(__file__).resolve().parents[3]
19 DEST=Path(__file__).resolve().parents[1]/'validation'/ 'geometry'
20
21 def data_xlsx(path, width):
22     wb=load_workbook(path,read_only=True,data_only=True)
23     rows=list(wb.active.values); wb.close()
24     a=np.asarray([row[:width] for row in rows[1:] if row[0] is not None],float)
25     if not np.isfinite(a).all() or not np.all(np.diff(a[:,0])>0):
26         raise ValueError('Invalid input records')
27     return a
28
29 class Inputs:
30     def __init__(self,input_dir=None):
31         packaged=Path(input_dir).resolve() if input_dir else
↪         Path(__file__).resolve().parents[1]/'inputs'
32         if input_dir is not None or (packaged/'attachment1.xlsx').exists():
33             env_path=packaged/'attachment1.xlsx';radius_path=packaged/'attachment2.xlsx'
34         else:
35             env_path=ROOT/'A 题/附件/附件 1.xlsx';radius_path=ROOT/'A 题/附件/附件 2.xlsx'
36         self.env=data_xlsx(env_path,3)
37         self.rad=data_xlsx(radius_path,2)
38         self.rad[:,1]*=.01
39         self.future=self.env[self.env[:,0]>=10800,1:].mean(axis=0)
40     def ambient(self,t,future=False):
41         if future:return self.future
42         return np.array([np.interp(t,self.env[:,0],self.env[:,j]) for j in (1,2)])
43     def radius(self,t):
44         if t>self.rad[-1,0]+1e-6:raise ValueError('Radius input ends at 72 h')
45         return float(np.interp(t,self.rad[:,0],self.rad[:,1]))
46

```

```

47 def properties(T,C,case):
48     if np.any(C<=0) or np.any(T<=-273.15):raise FloatingPointError('Nonpositive material
↪ state')
49     if case=='q1':
50         S=np.full_like(C,820.*2600.); k=np.full_like(C,.36)
51         D=7e-9*np.exp(-.89/C)
52         return S,k,D,np.zeros_like(C),np.zeros_like(C),np.zeros_like(C),D*.89/C**2
53     if case=='q23':a,b,c,d,e,f,A,B=650.,128.,1450.,2736.,.21,.38,.0024,.45
54     elif case=='q4':a,b,c,d,e,f,A,B=760.,90.,1850.,2150.,.12,.20,.00042,.30
55     else:raise ValueError(case)
56     rho=a+b*C;cp=c+d*C/(1+C);S=rho*cp;k=e+f*C/(1+C)
57     D=A*np.exp(-B/C-3850/(T+273.15))
58     return S,k,D,b*cp+rho*d/(1+C)**2,f/(1+C)**2,D*3850/(T+273.15)**2,D*B/C**2
59
60 def primitive_difference(a,b,B):
61     mid=(a+b)/2;delta=a-b;close=np.abs(delta)<.002*mid;value=np.empty_like(a)
62     if np.any(~close):
63         x=a[~close];y=b[~close]
64         value[~close]=x*np.exp(-B/x)-B*exp1(B/x)-y*np.exp(-B/y)+B*exp1(B/y)
65     if np.any(close):
66         m=mid[close];d=delta[close];v=np.zeros_like(m)
67         for x,w in
↪ ((-.8611363115940526,.3478548451374539),(-.3399810435848563,.6521451548625461))
↪ (,.3399810435848563,.6521451548625461),(.8611363115940526,.3478548451374539)):
68             v+=w*np.exp(-B/(m+x*d/2))
69         value[close]=v*d/2
70     return value
71
72 class GeometryFV:
73     def __init__(self,nr,nz,case,inputs,grading=2.,zgrading=2.,end_transfer=True,shrink=T)
↪ rue,flux='integral'):
74         self.nr,self.nz,self.case,self.inputs=nr,nz,case,inputs
75         self.shrink=shrink and case=='q4'
76         self.flux=flux
77         self.r=.02*(1-(1-np.linspace(0,1,nr+1))**grading)
78         rf=np.r_[0,(self.r[:-1]+self.r[1:])/2,.02];wr=np.diff(rf**2)/2
79         if nz:
80             self.z=.125*(1-(1-np.linspace(0,1,nz+1))**zgrading)
81             zf=np.r_[0,(self.z[:-1]+self.z[1:])/2,.125];wz=np.diff(zf)
82         else:self.z=np.array([0.]);wz=np.ones(1)
83         self.shape=(nz+1,nr+1);self.m=(nr+1)*(nz+1)
84         self.w=(wz[:,None]*wr).ravel();idx=np.arange(self.m).reshape(self.shape)
85         i=[idx[:,:-1].ravel()];j=[idx[:,1:].ravel()]
86         g=[(wz[:,None]*rf[None,1:-1]/np.diff(self.r)).ravel()]
87         self.radial_edges=len(i[0])
88         if nz:
89             i.append(idx[:,-1].ravel());j.append(idx[1:].ravel())
90             g.append((wr[None,:]/np.diff(self.z[:,None])).ravel())
91         self.i=np.concatenate(i);self.j=np.concatenate(j);self.g=np.concatenate(g)
92         side=np.zeros(self.shape);end=np.zeros(self.shape)
93         side[:,-1]=.02*wz
94         if nz and end_transfer:end[-1,:]=wr
95         self.side=side.ravel();self.end=end.ravel()
96         rows=[];cols=[]
97         for dest in (self.i,self.j):
98             for src in (self.i,self.j):
99                 for a in range(2):
100                     for b in range(2):rows.append(2*dest+a);cols.append(2*src+b)
101         nodes=np.arange(self.m)
102         self.jrows=np.r_[np.concatenate(rows),2*nodes,2*nodes,2*nodes+1]
103         self.jcols=np.r_[np.concatenate(cols),2*nodes+1,2*nodes,2*nodes+1]
104     def rhs(self,t,y,ambient,jac=False):
105         q=self.inputs.radius(t)/.02 if self.shrink else 1.
106         g=self.g.copy();g[:self.radial_edges]/=q*q

```

```

107     area=self.side/q+self.end
108     U=y.reshape(self.m,2);T,C=U.T;i,j=self.i,self.j
109     S,k,D,Sc,kc,Dt,Dc=properties(T,C,self.case)
110     ks=k[i]+k[j];kf=2*k[i]*k[j]/ks;kl=2*(k[j]/ks)**2;kr=2*(k[i]/ks)**2
111     ds=D[i]+D[j];df=2*D[i]*D[j]/ds;dl=2*(D[j]/ds)**2;dr=2*(D[i]/ds)**2
112     dT=T[i]-T[j];dC=C[i]-C[j];fT=g*kf*dT
113     if self.flux=='integral':
114         B=.89 if self.case=='q1' else .45 if self.case=='q23' else .30
115         A=7e-9 if self.case=='q1' else (.0024 if self.case=='q23' else
116             ↪ .00042)*np.exp(-3850/((T[i]+T[j])/2+273.15))
117         fC=g*A*primitive_difference(C[i],C[j],B)
118         mci=g*A*np.exp(-B/C[i]);mcj=-g*A*np.exp(-B/C[j])
119         mti=np.zeros_like(fC) if self.case=='q1' else fC*1925/((T[i]+T[j])/2+273.15)**2
120         mtj=mti
121     else:
122         fC=g*df*dC
123         mci=g*(df+dl*Dc[i]*dC);mcj=g*(-df+dr*Dc[j]*dC)
124         mti=g*dl*Dt[i]*dC;mtj=g*dr*Dt[j]*dC
125         f=np.column_stack([np.bincount(j,weights=fT,minlength=self.m)-np.bincount(i,weigh_
126             ↪ ts=fT,minlength=self.m),
127                 np.bincount(j,weights=fC,minlength=self.m)-np.bincount(i,weigh_
128             ↪ ts=fC,minlength=self.m)])
129         et,ec=ambient(t);f[:,0]=-area*25.*(T-et);f[:,1]=-area*8e-7*(C-ec)
130         storage=self.w[:,None]*np.column_stack([S,np.ones(self.m)]);f/=storage
131         if not jac:return f.ravel()
132         left=np.empty((len(i),2,2));right=np.empty_like(left)
133         left[:,0,0]=g*kf;right[:,0,0]=-g*kf
134         left[:,0,1]=g*kl*kC[i]*dT;right[:,0,1]=g*kr*kC[j]*dT
135         left[:,1,0]=mti;right[:,1,0]=mtj
136         left[:,1,1]=mci;right[:,1,1]=mcj
137         vals=[]
138         for dest,sgn in ((i,-1.),(j,1.)):
139             for v in (left,right):
140                 for a in range(2):
141                     for b in range(2):vals.append(sgn*v[:,a,b]/storage[dest,a])
142         vals.extend([-f[:,0]*Sc/S,-area*25./storage[:,0],-area*8e-7/self.w])
143         return coo_matrix((np.concatenate(vals),(self.jrows,self.jcols)),shape=(2*self.m,
144             ↪ 2*self.m)).tocsc()
145
146     def observe(self,t,y):
147         u=y.reshape(self.shape+(2,));C=u[:,1];ix=np.unravel_index(np.argmax(C),C.shape)
148         radius=self.inputs.radius(t) if self.shrink else .02
149         # Material-coordinate samples, exactly same locations for paired 1D/2D.
150         target=np.linspace(0,.02,21)
151         mid=PchipInterpolator(self.r**2,u[0],axis=0)(target**2)
152         end=PchipInterpolator(self.r**2,u[-1],axis=0)(target**2)
153         return
154         ↪ mid,end,float(C[ix]),np.array([self.r[ix[1]]*radius/.02,self.z[ix[0]]]),radius
155
156     def run(case,nr,nz,end=None,rtol=2e-9,max_step=240.,shrink=True,end_transfer=True,flux='i
157         ↪ ntegral',out_dir=None,input_dir=None):
158         source_hash_at_start=hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
159         dest=Path(out_dir).resolve() if out_dir else DEST
160         inp=Inputs(input_dir);op=GeometryFV(nr,nz,case,inp,shrink=shrink,end_transfer=end_tra
161             ↪ nsfer,flux=flux)
162         end=float(end if end is not None else (1800 if case=='q1' else 220000 if case=='q23'
163             ↪ else 259200))
164         label=f'{case}_{nr}x{nz}'+('_fixed' if case=='q4' and not shrink else
165             ↪ '')+('_sealed_ends' if nz and not end_transfer else '')+('_integral' if
166             ↪ flux=='integral' else '')
167         if rtol!=2e-9:label+=f'_rtol{rtol:g}'
168         dest.mkdir(parents=True,exist_ok=True)
169         if any((dest/(label+suffix)).exists() for suffix in ('.json','.npz')):raise
170             ↪ FileExistsError(label)
171         start=time.perf_counter();y=np.tile([28.,2.55],op.m)

```

```

160 sample_times=np.unique(np.r_[np.arange(0,end+1,60.),end,[100,300,900,1200,1500]])
161 sample_times=sample_times[(sample_times>0)&(sample_times<=end)]
162 snapshots_at=np.unique(np.r_[0,100,300,600,900,1200,1500,1800,3600,5400,7200,9000,108
↪ 00,14400,np.arange(21600,end,21600)])
163 cuts=np.unique(np.r_[0,inp.env[:,0][inp.env[:,0]<end],inp.rad[:,0][inp.rad[:,0]<end]
↪ if op.shrink else [],end])
164 recorded=[];mid=[];edge=[];cmax=[];locations=[];radii=[];snapshots=[y.reshape(op.shap
↪ e+(2,)).copy()];st=[0.]
165 stats={'case':case,'nr':nr,'nz':nz,'grading':2.,'zgrading':2.,'shrink':op.shrink,'end'
↪ _transfer':end_transfer,
166         'rtol':rtol,'atol_T':1e-9,'atol_C':1e-11,'max_step':max_step,'nfev':0,'njev':0
↪ , 'nlu':0,
167         'flux':flux+' local coefficients, vertex dual finite
↪ volumes','temperature':'coupled throughout; no isothermal projection',
168         'environment_future':inp.future.tolist(),'source_sha':'anonymous-source'})
169 def record(t,Y):
170     m,e,c,l,R=op.observe(t,Y);recorded.append(t);mid.append(m);edge.append(e);cmax.ap
↪ pend(c);locations.append(l);radii.append(R)
171 record(0.,y)
172 def event(t,Y):return np.max(Y[1::2])-.15
173 event.direction=-1;event.terminal=True
174 root=None;event_states=[];event_times=[]
175 for a,b in zip(cuts[:-1],cuts[1:]):
176     ambient=(lambda t: inp.ambient(t,True)) if a>=14400 else inp.ambient
177     sol=solve_ivp(lambda t,Y:op.rhs(t,Y,ambient),(a,b),y,method='BDF',
178                 jac=lambda
↪ t,Y:op.rhs(t,Y,ambient,True),rtol=rtol,atol=np.tile([1e-9,1e-11],op.m),
179                 max_step=min(max_step,30.) if a<14400 else max_step,dense_output=True,
180                 events=event if case!='q1' else None)
181     if not sol.success:raise RuntimeError(sol.message)
182     for key in ('nfev','njev','nlu'):stats[key]+=int(getattr(sol,key))
183     stop=float(sol.t[-1]);ts=sample_times[(sample_times>a)&(sample_times<=stop)]
184     if len(ts):
185         vals=sol.sol(ts)
186         for t,Y in zip(ts,vals.T):record(float(t),Y)
187     sts=snapshots_at[(snapshots_at>a)&(snapshots_at<=stop)]
188     if len(sts):
189         for t,Y in zip(sts,sol.sol(sts).T):st.append(float(t));snapshots.append(Y.res
↪ hape(op.shape+(2,)).copy())
190     y=sol.y[:, -1]
191     if sol.t_events is not None and len(sol.t_events[0]):
192         root=float(sol.t_events[0][0]);record(root,y);st.append(root);snapshots.append
↪ d(y.reshape(op.shape+(2,)).copy())
193         before=max(float(a),root-1.);after=root+1.
194         ext=solve_ivp(lambda t,Y:op.rhs(t,Y,ambient),(root,after),y,method='BDF',
195                     jac=lambda t,Y:op.rhs(t,Y,ambient,True),rtol=rtol,atol=np.tile([1e-9,1e-1
↪ 1],op.m),dense_output=True)
196         if not ext.success:raise RuntimeError(ext.message)
197         event_times=[before,root,after];event_states=[sol.sol(before).reshape(op.shap
↪ e+(2,)),y.reshape(op.shape+(2,)),ext.y[:, -1].reshape(op.shape+(2,))]
198         break
199         if b%21600==0 or b in (1800,10800,14400):print(label,'t=',b,'max C=',float(np.max
↪ (y[1::2])), 'elapsed=',round(time.perf_counter()-start,2),flush=True)
200     stats['elapsed_s']=time.perf_counter()-start;stats['event_root_s']=root
201     stats['end_s']=float(recorded[-1]);stats['end_max_C']=float(cmax[-1]);stats['end_max_
↪ location_r_z_m']=locations[-1].tolist()
202     if root is not None:stats['event_bracket_Cmax']=[float(np.max(v[:, :1])) for v in
↪ event_states]
203     stats['source_sha256']=source_hash_at_start
204     np.savez_compressed(dest/(label+'.npz'),time_s=recorded,mid=mid,end=edge,Cmax=cmax,Cm
↪ ax_location_r_z_m=locations,
205     radius_m=radii,reference_r_m=op.r,z_m=op.z,weights=op.w,snapshot_times_s=st,snap
↪ shots=snapshots,
206     event_times_s=event_times,event_states=event_states)

```

```

207 (dest/(label+'.json')).write_text(json.dumps(stats,indent=2,ensure_ascii=False)+'\n')
208 print(json.dumps(stats,ensure_ascii=False),flush=True)
209 return stats
210
211 if __name__=='__main__':
212     p=argparse.ArgumentParser();p.add_argument('--case',choices=['q1','q23','q4'],require_
↪ d=True)
213     p.add_argument('--nr',type=int,required=True);p.add_argument('--nz',type=int,default=0)
214     p.add_argument('--end',type=float);p.add_argument('--rtol',type=float,default=2e-9)
215     p.add_argument('--max-step',type=float,default=240.);p.add_argument('--fixed',action=
↪ 'store_true');p.add_argument('--sealed-ends',action='store_true')
216     p.add_argument('--flux',choices=['integral','harmonic'],default='integral')
217     p.add_argument('--out-dir',type=Path,help='New output location; existing run files are
↪ never overwritten.')
218     p.add_argument('--input-dir',type=Path,help='Directory containing attachment1.xlsx and
↪ attachment2.xlsx.')
219     a=p.parse_args();run(a.case,a.nr,a.nz,a.end,a.rtol,a.max_step,not a.fixed,not
↪ a.sealed_ends,a.flux,a.out_dir,a.input_dir)

```

A.42 geometry_checks

support/project/q4_complete_delivery/v1/source/geometry_checks.py

```

1  """Reproduce the executed Jacobian and dimension-reduction checks in a new folder."""
2  import argparse,json
3  from pathlib import Path
4  import numpy as np
5  from geometry_solver import GeometryFV,Inputs
6
7  def checks(out):
8      out=Path(out);out.mkdir(parents=True,exist_ok=True)
9      names=['jacobian_flux_checks.json','dimension_reduction_check.json']
10     if any((out/n).exists() for n in names):raise FileExistsError('Use a new validation
↪ output directory')
11     inp=Inputs();records={}
12     for flux in ('integral','harmonic'):
13         for case in ('q1','q23','q4'):
14             op=GeometryFV(8,6,case,inp,flux=flux);rng=np.random.default_rng(3402)
15             y=np.column_stack([28+10*rng.random(op.m),1.5+.5*rng.random(op.m)]).ravel()
16             v=rng.standard_normal(2*op.m);eps=1e-5
17             analytic=op.rhs(4000,y,inp.ambient,True)@v
18             numeric=(op.rhs(4000,y+eps*v,inp.ambient)-op.rhs(4000,y-eps*v,inp.ambient))/(
↪ 2*eps)
19             r={'jacobian_direction_max_abs':float(np.max(abs(analytic-numeric))),
20              'jacobian_direction_relative_norm':float(np.linalg.norm(analytic-numeric)/
↪ np.linalg.norm(analytic))}
21             assert r['jacobian_direction_relative_norm']<1e-8
22             records[case+'_'+flux]=r
23             (out/names[0]).write_text(json.dumps(records,indent=2)+'\n')
24             op=GeometryFV(20,16,'q4',inp,end_transfer=False);one=GeometryFV(20,0,'q4',inp)
25             r=one.r;profile=np.column_stack([28+8*(r/.02)**2,2.55-.5*(r/.02)**2])
26             y=np.broadcast_to(profile,(17,21,2)).copy().ravel()
27             a=op.rhs(9000,y,inp.ambient).reshape(17,21,2)
28             b=one.rhs(9000,profile.ravel(),inp.ambient).reshape(21,2)
29             err=float(np.max(abs(a-b)));assert err<1e-12
30             (out/names[1]).write_text(json.dumps({'no_end_exchange_radially_identical_rhs_max_abs'
↪ ':err','time_s':9000,'case':'q4 moving material reference'},indent=2)+'\n')
31
32 if __name__=='__main__':
33     p=argparse.ArgumentParser();p.add_argument('--out-dir',required=True,type=Path)
34     checks(p.parse_args().out_dir)

```

A.43 geometry_analysis

support/project/q4_complete_delivery/v1/source/geometry_analysis.py

```

1  """Summarize completed geometry runs; never treats absent runs as passed."""
2  from pathlib import Path
3  import csv,hashlib,json
4  import numpy as np
5  from geometry_solver import DEST
6
7  def summarize():
8      rows=[];details={};files=[];event_checks={}
9      for f in sorted(DEST.glob('q*x*.json')):
10         s=json.loads(f.read_text())
11         if not s.get('nz'):continue
12         suffix='_integral' if s['flux'].startswith('integral') else ''
13         base=f.stem.replace(f"{s['case']}_[{s['nr']}]x[{s['nz']}]","f"{s['case']}_[{s['nr']}]x0"
14         ↪ ,1)
15         if not (DEST/(base+'.json')).exists():continue
16         one=json.loads((DEST/(base+'.json')).read_text())
17         a=np.load(DEST/(base+'.npz'));b=np.load(f.with_suffix('.npz'))
18         ts,ia,ib=np.intersect1d(a['time_s'],b['time_s'],return_indices=True)
19         mid=b['mid'][ib]-a['mid'][ia];end=b['end'][ib]-a['mid'][ia]
20         row={'case':s['case'],'nr':s['nr'],'nz':s['nz'],'flux':suffix.strip('_') or
21         ↪ 'harmonic','rtol':s['rtol'],'max_step_s':s['max_step'],
22         ↪ 'event_1d_s':one['event_root_s'],'event_2d_s':s['event_root_s'],
23         ↪ 'end_effect_s':None if one['event_root_s'] is None or s['event_root_s'] is
24         ↪ None else s['event_root_s']-one['event_root_s'],
25         ↪ 'max_mid_T_difference_all_samples_K':float(abs(mid[:,:,:0]).max()),
26         ↪ 'max_mid_C_difference_all_samples':float(abs(mid[:,:,:1]).max()),
27         ↪ 'max_end_T_difference_all_samples_K':float(abs(end[:,:,:0]).max()),
28         ↪ 'max_end_C_difference_all_samples':float(abs(end[:,:,:1]).max()),
29         ↪ 'paired_sample_count':len(ts),'elapsed_s':s['elapsed_s']}
30         masks={'q1_table_range':ts<=1800,'q2_table_range':ts<=10800,'full_common_range':n
31         ↪ p.ones(len(ts),bool)}
32         d={'run':f.stem,'paired_1d':base,'summary':row,'intervals':{},'end_location_r_z_m'
33         ↪ ':s['end_max_location_r_z_m']}
34         for label,mask in masks.items():
35             d['intervals'][label]={ 'last_sample_s':float(ts[mask][-1]),'max_mid_differenc
36             ↪ e_T_C':np.max(abs(mid[mask]),axis=(0,1)).tolist(),
37             ↪ 'max_end_difference_T_C':np.max(abs(end[mask]),axis=(0,1)).tolist()}
38         if s['event_root_s'] is not None:
39             d['event']={'time_s':b['event_times_s'].tolist(),'Cmax':[float(v[:,:,:1].max())
40             ↪ for v in b['event_states']],
41             ↪ 'root_max_r_z_m':s['end_max_location_r_z_m']}
42             states=b['event_states'];c=states[:,:,:1]
43             ec={'all_event_fields_finite':bool(np.isfinite(states).all()),'minimum_C':flo
44             ↪ at(c.min()),
45             ↪ 'largest_positive_radial_C_increment':float(np.max(np.diff(c,axis=2))),
46             ↪ 'largest_positive_axial_C_increment':float(np.max(np.diff(c,axis=1))),
47             ↪ 'max_locations_zi_ri':[list(map(int,np.unravel_index(np.argmax(v),v.shape
48             ↪ ))) for v in c]}
49             assert ec['all_event_fields_finite'] and ec['minimum_C']>0
50             assert ec['largest_positive_radial_C_increment']<1e-10 and
51             ↪ ec['largest_positive_axial_C_increment']<1e-10
52             event_checks[f.stem]=ec
53         profiles=[]
54         for t in [100,300,600,900,1200,1500,1800,3600,5400,7200,9000,10800,21600,43200,86
55         ↪ 400,129600,172800]:
56             hit=np.flatnonzero(ts==t)
57             if len(hit):
58                 j=int(hit[0]);profiles.append({'time_s':t,'reference_radius_fraction':np.
59                 ↪ linspace(0,1,21).tolist(),

```

```

48         'actual_R_m':float(b['radius_m'][ib[j]]), 'mid_2d_T_C':b['mid'][ib[j]
↪      ]].tolist(),
49         'end_2d_T_C':b['end'][ib[j]].tolist(), 'radial_1d_T_C':a['mid'][ia[j]
↪      ]].tolist())}
50     d['profiles']=profiles
51     rows.append(row);details[f.stem]=d
52     for p in [f,f.with_suffix('.npz'),DEST/(base+'.json'),DEST/(base+'.npz')]:
53         files.append({'path':p.name, 'bytes':p.stat().st_size, 'sha256':hashlib.sha256(
↪      p.read_bytes()).hexdigest()})
54     (DEST/'geometry_comparison.json').write_text(json.dumps(details,indent=2,ensure_ascii
↪      =False)+'\n')
55     (DEST/'event_field_shape_checks.json').write_text(json.dumps(event_checks,indent=2)+'
↪      \n')
56     if rows:
57         with (DEST/'geometry_comparison.csv').open('w',newline='') as stream:
58             w=csv.DictWriter(stream,fieldnames=list(rows[0]));w.writeheader();w.writerows
↪      (rows)
59     (DEST/'geometry_evidence_manifest.json').write_text(json.dumps({'completed_runs':len(
↪      rows), 'files':list([a['path']:a for a in files].values())},indent=2)+'\n')
60     print(json.dumps(rows,ensure_ascii=False,indent=2))
61     return rows,details
62
63 if __name__=='__main__':summarize()

```

A.44 geometry_report

support/project/q4_complete_delivery/v1/source/geometry_report.py

```

1  """Build geometry discussion from completed comparison JSON and endpoint metadata."""
2  from pathlib import Path
3  import json
4  import numpy as np
5  from geometry_solver import DEST
6  from geometry_analysis import summarize
7
8  rows,data=summarize();v1=DEST.parents[1]
9  q3=json.loads((v1/'q123_closeout/validation/q2_closeout_summary.json').read_text())
10 q4=json.loads((v1/'output/end_event.json').read_text())
11 main4=json.loads((v1/'output/main.json').read_text())
12 def val(key,group='full_common_range'):
13     return data[key]['intervals'][group]
14 def table(selected):
15     out=['| 模型/通量 | 径向 × 轴向区间 | 配对一维临界/s| 二维临界/s| 二维 - 配对一维/s|',
16         '|---|---|---|---|---|']
17     for key in selected:
18         if key not in data:continue
19         s=data[key]['summary'];tag=s['case']+'/'+'s['flux']+'(' + (紧配置) ' if s['rtol']!=2e-9
↪      else '')
20         out.append(f"|{tag}|{s['nr']}]×{s['nz']}]|{s['event_1d_s']:.9f}|{s['event_2d_s']:.9
↪      f}|{s['end_effect_s']:.9f}|")
21     return '\n'.join(out)
22
23 q3keys=['q23_40x64','q23_80x128_integral','q23_80x256_integral']
24 q4keys=['q4_40x64','q4_40x64_integral','q4_40x128_integral','q4_80x128_integral','q4_40x6
↪      4_integral_rtol2e-11']
25 q3d=[data[k]['summary']['end_effect_s'] for k in q3keys[1:] if k in data]
26 q4d=[data[k]['summary']['end_effect_s'] for k in q4keys[1:] if k in data]
27 q3est=[q3['critical_s']+min(q3d),q3['critical_s']+max(q3d)]
28 q4est=[q4['critical_s']+min(q4d),q4['critical_s']+max(q4d)]
29 # Explicit estimate: translate the one-dimensional local event slope by the
30 # matched geometry shift; these are not evaluated coarse 2D endpoint fields.

```

```

31 slope=main4['stats']['critical_slope'] if 'stats' in main4 and 'critical_slope' in
↪ main4['stats'] else None
32 if slope is None:
33     def find(x):
34         if isinstance(x,dict):
35             if 'critical_slope' in x:return x['critical_slope']
36             for v in x.values():
37                 a=find(v)
38                 if a is not None:return a
39     return None
40     slope=find(main4)
41 c4est=sorted(.15+slope*(q4['execution_s']-t) for t in q4est)
42 est={'method':'matched 2D-minus-1D event correction applied to converged 1D; Cmax uses
↪ local event linearization. Estimates, not full-field evaluations at official execution
↪ time.',
43     'q3_converged_1d_root_s':q3['critical_s'],'q3_corrected_2d_root_range_s':q3est,
44     'q4_converged_1d_root_s':q4['critical_s'],'q4_matched_geometry_shift_range_s':[min(q
↪ 4d),max(q4d)],
45     'q4_corrected_2d_root_range_s':q4est,'q4_execution_s':q4['execution_s'],
46     'q4_linearized_Cmax_estimate_at_execution':c4est,'q4_1d_critical_slope':slope,
47     'all_completed_q3_q4_event_wettest_locations_r_z_m':{k:data[k]['end_location_r_z_m']
↪ for k in q3keys+q4keys if k in data}}
48 (DEST/'corrected_global_event_estimates.json').write_text(json.dumps(est,indent=2,ensure_
↪ ascii=False)+'\n')
49
50 a=val('q1_80x128');b=val('q23_80x128_integral','q2_table_range')
51 finite_q3='q23_80x256_integral' if 'q23_80x256_integral' in data else 'q23_80x128_integral'
52 new_q3=data[finite_q3]['summary']
53 paragraph=f'''# 同一有效模型的一维/二维适用范围与第三问终点
54
55 本轮固定读取提交为 `anonymous-source`。比较均采用用户选择的题给物性、
↪ 有效热容量与等效空气含水率边界，不含显式潜热；没有把 F 或 R_L 的旧二维差混入当前模型的误差依据。
56
57 ## 方法与可比较性
58
59 二维区域为圆柱的一半轴向剖面，`0≤r≤R`、`0≤z≤0.125 m`；`z=0`为中截面。轴心及中截面施加对称条件，
↪ 侧面和端面均使用 `hT=25`、`hm=8e-7`，角部按真实相交面面积求和，没有新增端面倍率。
60
61 每个二维算例都从统一初态 `T=28 °C`、`C=2.55` 联立求温湿场，并配对完全相同径向节点、
↪ 径向通量和积分配置的一维算例。由“二维临界-同网格一维临界”提取端部效应，
↪ 避免把粗径向离散偏差误称为几何影响。
62
63 节点对偶有限体积采用二次加密网格，向侧面和端面加密；热通量使用局部导热系数的调和面值，主要水通量使用
↪ `D=A(T) exp(-B/C)` 的含水率积分。另保留调和和扩散系数的独立粗离散作为交叉证据。
64
65 4h 前在附件1每个60s节点分段，之后为末小时61点算术均值平台。全程温度都参与联立更新，没有后期等温投影。
↪ 以下最大差是每60s和题目指定时刻、21个径向输出点的采样差，不能冒充连续时空严格上界。
66
67 ## 第一、二问：中截面与端部应分别说明
68
69 |范围与本轮网格|中截面最大温差/K|中截面最大含水率差/(kg/kg)|端面最大温差/K|端面最大含水率差/(kg/
↪ kg)|
70 |---|---:|---:|---:|
71 |Q1, 0-1800s, 80x128|{a['max_mid_difference_T_C'][0]:.8g}|{a['max_mid_difference_T_C'][1]:
↪ .8g}|{a['max_end_difference_T_C'][0]:.8g}|{a['max_end_difference_T_C'][1]:.8g}|
72 |Q2, 0-10800s, 80x128|{b['max_mid_difference_T_C'][0]:.8g}|{b['max_mid_difference_T_C'][1]
↪ :.8g}|{b['max_end_difference_T_C'][0]:.8g}|{b['max_end_difference_T_C'][1]:.8g}|
73
74 Q1 的40x64与80x128配对中截面温差分别约 `8.46e-8` 与 `5.65e-8 K`，含水率差均约 `4e-9`，
↪ 已接近积分误差水平。其一维解足以代表题表指定的中截面径向输出，结论不推广到端面或整体平均值。
75
76 Q2 中截面温差约 `0.00169 K`，因此不能声称“一维与二维所有温度均四位小数一致”。含水率差约 `2.25e-5
↪ kg/kg`；接近舍入边界的单格仍可能变化。工程建模容差、四位显示精度、离散误差与几何差属于不同尺度。
77

```

```

78  ## 第三问：有限圆柱的全域临界
79
80  {table(q3keys)}
81
82  原无潜热一维模型的精细临界为 `{q3['critical_s']:.9f} s`；本轮第二问完整逐秒轨迹继续导出到
    ↳ `{q3['time_end_s']:.2f} s`，对应 `57.4741 h`。这一端点仍属该有效模型的保守执行口径，
    ↳ 不把它重新命名为精确二维最短时长。
83
84  最细已完成二维 `{new_q3['nr']}\{new_q3['nz']}` 的原始临界是 `{new_q3['event_2d_s']:.9f} s`，
    ↳ 最湿节点确为 `(r,z)=(0,0)`。同一原始场在等号根前后1s分别高于与低于阈值，完整场保存在相应
    ↳ `event_states` 数组。
85
86  去除匹配的一维径向偏差后，当前已完成积分网格给出的有限圆柱临界估计范围为 `{q3est[0]:.6f}-
    ↳ {q3est[1]:.6f} s`。这些数是相同模型的配对校正估计，不是另一次精细二维实跑，也不是严格误差界；
    ↳ 它们支持保留原一维执行端点的保守方向。
87
88  历史同模型40×64、80×64、40×128结果已检查：端部效应约 `-7.66`、`-7.66`、`-7.51 s`，
    ↳ 其中后两组含已量化的微小后期等温投影。本轮全程耦合80×128补齐交叉节点并保存了完整二维事件场；
    ↳ 历史投影结果只用于核对趋势。
89
90  本轮拟做的160×128可选交叉加密因输出工作簿时的内存压力主动中断，退出码130；中断记录与原日志保留，
    ↳ 没有完整轨迹，未将其列为通过。已完成网格以及同网格一维比较足以将秒级几何影响与数秒径向偏差区分。
91
92  ## 文件与复算
93
94  积分通量主验证的实际源码为 `../source/geometry_solver_executed.py`，SHA256 为
    ↳ `7f4fc1ecad068ff3f5af12f48374ec742b12bb5f85aa430d3c16c2cb8e24d80d`。
    ↳ `geometry_solver.py` 仅增加冷启动目录接口；AST核对表明物性与离散算子完全未改。
95
96  先完成的Q1与调和通量粗算使用添加积分通量分支前的代码，原执行哈希为
    ↳ `ae8887e12f10cf81375678ccdec66d155823f157673aa3f769c84a75713bc7eb`；当前程序的 `--flux
    ↳ harmonic` 保留该离散。具体执行版本和磁盘哈希记录时点见 `source_execution_audit.json`。
97
98  所有对比数来自 `../validation/geometry/geometry_comparison.json` 与原始 NPZ。每份 NPZ
    ↳ 包含温湿二维快照、实际半径、参考径向坐标、轴向坐标、中截面/端面轨迹、
    ↳ 全域最大值与位置以及事件前后完整状态。文件哈希见同目录清单。
99
100  从新目录复算的例子如下；若已有同名 JSON 或 NPZ，程序拒绝覆盖。
    ↳ 输出接口冒烟试验仅验证路径与拒覆盖行为，不充当任何物理精度试验。
101
102  ```bash
103  OPENBLAS_NUM_THREADS=1 python source/geometry_solver.py --case q23 --nr 80 --nz 128
    ↳ --out-dir reproduced/q23_geometry_fresh --input-dir inputs
104  OPENBLAS_NUM_THREADS=1 python source/geometry_solver.py --case q23 --nr 80 --nz 0
    ↳ --out-dir reproduced/q23_geometry_fresh --input-dir inputs
105  ```
106  '''
107  (v1/'q123_closeout/一维二维适用范围.md').write_text(paragraph)
108
109  g=val('q4_80x128_integral' if 'q4_80x128_integral' in data else 'q4_40x128_integral')
110  q4text=f'''# 第四问：独立有限体积与二维验证
111
112  本轮只改变几何维数，不改变附录4物性、换热换质系数、空气等效边界或未来平台。长度固定为0.25m，
    ↳ 材料均匀径向收缩，网格随材料运动。二维算例从初态完整联立求解温度与含水率，没有潜热项或等温投影。
113
114  ## 参考区域算子
115
116  令  $\xi=r/R(t)$ ，材料速度与网格速度均为  $r\ R'/R$ ，相对速度为零。热方程和干基含水率方程分别为：
117
118  \[
119  \rho(C)c_p(C)\partial_t T
120  =\frac{1}{R(t)^2}\partial_{\xi}\{\xi k(C)\partial_{\xi} T\}+\partial_z(k(C)\partial_z T),
121  \]
```

```

122 \\[
123 \\partial_t C
124 =\\frac{1}{R(t)^2}\\xi}\\partial_\\xi(\\xi D(T,C)\\partial_\\xi
    ↪ C)+\\partial_z(D(T,C)\\partial_zC).
125 \\]
126
127 以初始网格的对偶体积作权重，径向内部导通系数乘  $(R_0/R)^2$ ，侧面交换乘  $R_0/R$ ，
    ↪ 轴向内部项与端面交换不变。这与当前体积、界面面积同时更新完全等价；
    ↪ 没有重复加入收缩对流或干基C浓缩项。
128
129 ## 实际完成的配对二维证据
130
131 {table(q4keys)}
132
133 默认积分配置在40×64→40×128时，配对几何差改变约  $0.0004711\text{ s}$ ；40×128→80×128改变约  $-0.0004190$ 
    ↪  $\text{s}$ 。紧40×64配置采用  $\text{rtol}=2\text{e-}11$ 、 $\text{max\_step}=60\text{s}$ ，几何差相对默认配置变化约  $0.00564\text{ s}$ ，
    ↪ 需纳入小差异的解释。
134
135 综合这些实际配对，端面对最湿点临界的影响约为提前  $0.0286-0.0343\text{ s}$ ，
    ↪ 不宜把小数后所有位都解释为物理精度。最细80×128原场的全域最湿点为  $(r,z)=(0,0)$ ，
    ↪ 并保存了等号根与前后1s的完整二维状态。
136
137 最细已完成网格全程中截面采样最大差为  $\Delta T=\{g[\text{'max\_mid\_difference\_T\_C'}][0]:.8g\}\text{ K}$ 、
    ↪  $\Delta C=\{g[\text{'max\_mid\_difference\_T\_C'}][1]:.8g\}\text{ kg/kg}$ ；端面差为
    ↪  $\{g[\text{'max\_end\_difference\_T\_C'}][0]:.8g\}\text{ K}$ 、 $\{g[\text{'max\_end\_difference\_T\_C'}][1]:.8g\}\text{ kg/kg}$ 。
    ↪ 一维适用依据限定于中截面和最湿点。
138
139 ## 径向离散误差与全域估计的边界
140
141 二维80×128原临界  $\{data[\text{'q4\_80x128\_integral'}][\text{'summary'}][\text{'event\_2d\_s'}]:.9f\}\text{ s}$ 
    ↪ 晚于谱一维临界，是粗径向离散偏差；同网格一维也同样偏晚。不能宣称这个原始粗二维场在正式
    ↪  $\{q4[\text{'execution\_s'}]:.2f\}\text{ s}$  时已经实际过线。
142
143 将各同网格几何差加到精细一维临界  $\{q4[\text{'critical\_s'}]:.9f\}\text{ s}$  上，得到全域临界估计
    ↪  $\{q4est[0]:.6f\}-\{q4est[1]:.6f\}\text{ s}$ 。结合一维临界斜率  $\{slope:.12g\}\text{ (kg/kg)/s}$ ，
    ↪ 正式执行时刻的全域最大C估计为  $\{c4est[0]:.12f\}-\{c4est[1]:.12f\}$ 。
144
145 上一段是“配对径向校正+临界附近线性化”的数值估计，并非正式时刻的新二维场求值或严格上界。
    ↪ 它与一维实测执行最大值  $\{q4[\text{'execution\_max\_C'}]:.12f\}$ 
    ↪ 一起支持一维工程执行时刻的适用性和保守方向；不宣称得到无限精度的二维最短时长。
146
147 独立积分FV的n320/n640紧配置临界为  $183931.398320243 / 183931.248715026\text{ s}$ ，二阶外推
    ↪  $183931.198846621\text{ s}$ 。n640进一步减小步长与相对容差后为  $183931.259744118\text{ s}$ ；
    ↪ 加上同一空间修正约为  $183931.209876\text{ s}$ ，与谱主临界相差约0.00184s。
148
149 上述空间外推和误差变化是实证数值检查，不是严格数学界。完整单网格数据与容差保存在各JSON中；
    ↪ 没有把粗网格根直接替换为谱根，也没有为缩短时长调整物性、交换系数或观测收缩数据。
150
151 ## 验证与来源
152
153 `jacobian_flux_checks.json`保存两种通量、三组物性的解析Jacobian方向差分检查；相对范数差约1e-10。
    ↪ `dimension_reduction_check.json`保存关闭端面交换、轴向均匀状态下二维与一维RHS的一致性；
    ↪ 该结果只验证维数退化，不验证开放端部的几何误差。
154
155 `source_execution_audit.json`记录原始执行源与后续接口修订，
    ↪ `interface_revision_checks.json`记录AST不变及冷启动拒覆盖检查。
    ↪ 任何旧JSON中的源码哈希都应结合该执行记录解释，因为旧实现曾在运行结束时才读取磁盘上的文件哈希。
156
157 原始矩阵、未舍入轨迹、完整二维事件场、配对对比JSON和CSV均在本目录。
    ↪ `corrected_global_event_estimates.json`清楚标识校正估计；两张PNG直接由保存的二维场生成，
    ↪ 图中显示的事件场属于对应原始离散网格。
    ...
158
159 (DEST/'README.md').write_text(q4text)

```

```
160 print('Wrote closeout geometry discussion and independent geometry README.')
```

A.45 本次整体复审

A.46 diagnose_saved_fields

support/project/q1234_overall_review_delivery/v1/reviews/physics/diagnose_saved_fields.py

```
1  """Read frozen arrays; do algebra and spatial quadrature only. Never solve PDEs.
2
3  Run from the repository root with Python, NumPy, SciPy and openpyxl.
4  Writes only beside this script. All reported physical interpretations are conditional.
5  """
6  from pathlib import Path
7  import csv
8  import hashlib
9  import json
10 import platform
11 import subprocess
12
13 import numpy as np
14 import scipy
15 from scipy.fft import dct
16 from scipy.special import exp1
17 from openpyxl import load_workbook
18
19 OUT = Path(__file__).resolve().parent
20 ROOT = OUT.parents[3]
21 FILES = {
22     "q4_saved_field": "q4_complete_delivery/v1/output/main.npz",
23     "q1_saved_field": "q1_complete_delivery/q1_delivery/output/q1_unrounded.npz",
24     "environment": "q4_complete_delivery/v1/inputs/attachment1.xlsx",
25     "radius_input": "q4_complete_delivery/v1/inputs/attachment2.xlsx",
26     "q4_model": "q4_complete_delivery/v1/第四问模型与文献采用.md",
27     "q4_source": "q4_complete_delivery/v1/source/q4_spectral.py",
28     "problem_text": "readable/problem/fulltext.md",
29 }
30
31
32 def array_workbook(path):
33     wb = load_workbook(path, read_only=True, data_only=True)
34     rows = list(wb.worksheets[0].values)
35     wb.close()
36     return np.asarray(rows[1:], float)
37
38
39 def reconstructed_c_integral(c, gauss_order):
40     """Independent Gauss integration of density from the primitive polynomial.
41
42     Uses NumPy Chebyshev evaluation, not the production quadrature or PDE operator.
43     C interpolation follows the frozen definition: interpolate F(C), then invert it.
44     """
45     primitive = c * np.exp(-0.30 / c) - 0.30 * exp1(0.30 / c)
46     coeff = dct(primitive[::-1], type=1) / (len(c) - 1)
47     coeff[[0, -1]] *= 0.5
48     points, weights = np.polynomial.legendre.leggauss(gauss_order)
49     target = np.polynomial.chebyshev.chebval(points, coeff)
50     cc = np.interp((points + 1) / 2, (1 - np.cos(np.linspace(0, np.pi, len(c)))) / 2, c)
51     for _ in range(50):
52         step = (cc * np.exp(-0.30 / cc) - 0.30 * exp1(0.30 / cc) - target) / np.exp(-0.30 /
53 ↪ cc)
```

```

53     while np.any(cc - step <= 0):
54         bad = cc - step <= 0
55         step[bad] *= 0.5
56     cc -= step
57     if np.max(np.abs(step)) < 1e-13:
58         break
59 else:
60     raise RuntimeError("Primitive inverse did not converge")
61 assert np.max(np.abs(cc * np.exp(-0.30 / cc) - 0.30 * exp1(0.30 / cc) - target)) <
↪ 1e-12
62 return float(np.dot(weights, (760 + 90 * cc) / (1 + cc)) / 2)
63
64
65 def main():
66     base_commit = "anonymous-code-snapshot"
67     provenance = {
68         key: {"path": value, "sha256": hashlib.sha256((ROOT /
↪ value).read_bytes()).hexdigest()}
69         for key, value in FILES.items()
70     }
71     data = array_workbook(ROOT / FILES["environment"])
72     future = data[data[:, 0] >= 10800, 1:].mean(axis=0)
73     q4 = np.load(ROOT / FILES["q4_saved_field"])
74     t, radii, fields, stored_w = (q4[k] for k in ("time_s", "radius_m", "full_IC",
↪ "quadrature_weights"))
75     rho_d0 = (760 + 90 * 2.55) / 3.55 # initial matching convention, not a measurement
76     q_implied = (760 + 90 * fields[:, :, 1]) / (1 + fields[:, :, 1])
77     ratios = (radii / 0.02) ** 2 * (q_implied @ stored_w) / rho_d0
78     rho_d_material = rho_d0 * (0.02 / radii) ** 2
79     rows = []
80     for i in sorted({int(np.argmin(ratios)), int(np.searchsorted(t, 7 * 3600)), len(t) -
↪ 1}):
81         quad = {str(n): reconstructed_c_integral(fields[i, :, 1], n) for n in (192, 384)}
82         ratio_independent = (radii[i] / 0.02) ** 2 * quad["384"] / rho_d0
83         rows.append({
84             "time_s": float(t[i]), "radius_m": float(radii[i]),
85             "implied_dry_mass_ratio_nodal": float(ratios[i]),
86             "implied_dry_mass_ratio_gauss384": float(ratio_independent),
87             "gauss192_384_rho_d_integral_difference_kg_m3": quad["192"] - quad["384"],
88             "length_for_global_mass_only_cm": float(25 / ratio_independent),
89             "physical_rho_d_minmax_if_empirical_rho_real": [float(q_implied[i].min()),
↪ float(q_implied[i].max())],
90             "affine_material_rho_d_kg_m3": float(rho_d_material[i]),
91             "qualification": "Changing length alone preserves these saved C and R only
↪ diagnostically; not a recomputed model. Local consistency does not follow
↪ from global correction.",
92         })
93     energy = []
94     for hour in (6, 12, 24, 48):
95         i = int(np.searchsorted(t, hour * 3600))
96         ts, cs = fields[i, -1]
97         j = rho_d_material[i] * 8e-7 * (cs - future[1])
98         energy.append({
99             "time_s": float(t[i]), "surface_T_degC": float(ts), "surface_C": float(cs),
100             "convective_heat_input_on_saved_temperature_W_m2": float(25 * (future[0] -
↪ ts)),
101             "conditional_material_water_flux_kg_m2_s": float(j),
102             "conditional_latent_demand_at_2_45_MJ_kg_W_m2": float(2.45e6 * j),
103         })
104     q1 = np.load(ROOT / FILES["q1_saved_field"])
105     volume = np.pi * 0.02**2 * 0.25
106     mass_loss = 820 / 3.55 * volume * (2.55 - q1["average_C"][-1])
107     sensible = 820 * 2600 * volume * (q1["average_temperature_degC"][-1] - 28)

```

```

108 qconv = float(np.trapezoid(2 * np.pi * 0.02 * 0.25 * 25 * (q1["environment"][:, 0] -
109 ↪ q1["temperature_degC"][:, -1]), q1["time_s"]))
110 scale = (0.02 / radii[-1]) ** 2
111 result = {
112     "base_commit": base_commit,
113     "scope": "Algebra and saved-field quadrature only; no PDE integration, no new
114 ↪ temperature or drying-time solution, no calibration.",
115     "runtime": {"python": platform.python_version(), "numpy": np.__version__, "scipy":
116 ↪ scipy.__version__},
117     "sources": provenance,
118     "script_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
119     "q4_future_environment_T_C": future.tolist(),
120     "conditional_air_state_if_W_and_101_325kPa": {
121         "vapor_partial_pressure_kPa": float(101.325 * future[1] / (0.621945 +
122 ↪ future[1])),
123         "relative_humidity_using_FAO_eq11": float((101.325 * future[1] / (0.621945 +
124 ↪ future[1])) / (0.6108 * np.exp(17.27 * future[0] / (future[0] + 237.3)))),
125         "qualification": "Only if the workbook kg/kg means water per dry air, with
126 ↪ stated pressure. This is not a verified interpretation of the problem
127 ↪ input.",
128     },
129     "q4_density_contradiction_diagnostics": rows,
130     "q4_conditional_latent_flux": energy,
131     "q4_boundary_coefficient_scaling": {
132         "terminal_R_over_R0": float(radii[-1] / 0.02),
133         "rho_d_and_per_area_K_C_multiplier_at_fixed_effective_hm": float(scale),
134         "total_mass_rate_multiplier_at_fixed_surface_C_difference": float(0.02 /
135 ↪ radii[-1]),
136         "hypothetical_effective_hm_if_K_C_constant_m_s": float(8e-7 / scale),
137         "qualification": "Hypothetical hm is a sensitivity contract only; not a fitted
138 ↪ or recommended replacement parameter.",
139     },
140     "q1_conditional_energy_readback": {
141         "initial_dry_mass_kg": float(820 / 3.55 * volume),
142         "water_loss_kg": float(mass_loss), "latent_J": float(2.45e6 * mass_loss),
143         "baseline_sensible_J": float(sensible), "baseline_convective_integral_J":
144 ↪ qconv,
145         "latent_over_baseline_sensible": float(2.45e6 * mass_loss / sensible),
146         "qualification": "rho=820 interpreted as initial wet density only; all
147 ↪ effective moisture loss assumed evaporative; latent value is a scale
148 ↪ diagnostic.",
149     },
150     "assertions": {
151         "density_gauss192_384_close":
152 ↪ all(abs(x["gauss192_384_rho_d_integral_difference_kg_m3"]) < 1e-8 for x in
153 ↪ rows),
154         "q4_required_length_exceeds_initial_at_7h_and_end":
155 ↪ all(x["length_for_global_mass_only_cm"] > 25 for x in rows),
156         "rho_d_shape_not_spatially_uniform_at_7h":
157 ↪ bool(np.ptp(q_implied[int(np.searchsorted(t, 7*3600))]) > 1),
158     },
159 }
160 assert all(result["assertions"].values()), result["assertions"]
161 (OUT / "saved_field_physics_diagnostics.json").write_text(json.dumps(result,
162 ↪ ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
163 with (OUT / "q4_implied_dry_mass_curve.csv").open("w", encoding="utf-8", newline="")
164 ↪ as f:
165     writer = csv.writer(f)
166     writer.writerow(["time_s", "radius_m",
167 ↪ "implied_dry_mass_ratio_if_empirical_rho_real",
168 ↪ "length_cm_for_global_mass_only"])
169     writer.writerows(zip(t, radii, ratios, 25 / ratios))
170 print(json.dumps(result, ensure_ascii=False, indent=2))
171

```

```

152
153 if __name__ == "__main__":
154     main()

```

A.47 environment_probe

support/project/q1234_overall_review_delivery/v1/source/environment_probe.py

```

1  """Conditional future-environment probes from frozen 4 h states.
2
3  Reuses the audited effective-model operators; not an independent physical model
4  or a replacement for any official result. Q4 radius remains the observed input.
5  """
6  from pathlib import Path
7  import argparse
8  import hashlib
9  import json
10 import platform
11 import sys
12 import time
13 import numpy as np
14 import scipy
15 from scipy.integrate import solve_ivp
16
17 REPO = Path(__file__).resolve().parents[3]
18 DELIVERY = REPO / 'q4_complete_delivery/v1'
19 sys.path.insert(0, str(DELIVERY / 'source'))
20 sys.path.insert(0, str(DELIVERY / 'q123_closeout/source/legacy_q3'))
21 import q4_spectral
22 import q3_reference
23 from q4_common import Radius, Environment, load_input, read_numeric
24
25 SCENARIOS = {'baseline': (0., 0.), 'temperature_minus_1K': (-1., 0.),
26              'temperature_plus_1K': (1., 0.), 'boundary_minus_0p005': (0., -.005),
27              'boundary_plus_0p005': (0., .005)}
28
29
30 def sha(path):
31     return hashlib.sha256(path.read_bytes()).hexdigest()
32
33
34 def run(case, n, scenario, out):
35     stem = out / f'{case}_{scenario}_n{n}'
36     if stem.with_suffix('.json').exists() or stem.with_suffix('.npz').exists():
37         raise FileExistsError(stem)
38     out.mkdir(parents=True, exist_ok=True)
39     env_path = DELIVERY / 'inputs/attachment1.xlsx'
40     future = Environment(load_input(env_path)).future
41     boundary = future + np.array(SCENARIOS[scenario])
42     env = lambda t: boundary
43     if case == 'q3':
44         original = DELIVERY / 'q123_closeout/output/q23_unified.npz'
45         z = np.load(original)
46         idx = int(np.flatnonzero(z['snapshot_time_s'] == 14400.)[0])
47         full = z['snapshot_full_TC'][idx]
48         op = q3_reference.Reference(n)
49         official, official_root = 206906.76, 206906.38993232802
50         module = q3_reference
51     else:
52         original = DELIVERY / 'output/main.npz'
53         z = np.load(original)
54         idx = int(np.flatnonzero(z['time_s'] == 14400.)[0])

```

```

55     full = z['full_TC'][idx]
56     radius = Radius(read_numeric(DELIVERY / 'inputs/attachment2.xlsx', 2)[1])
57     op = q4_spectral.Reference(n, radius)
58     official, official_root = 183931.56, 183931.21171548913
59     module = q4_spectral
60     old_n = len(z['x']) - 1
61     # Use only exact nested Chebyshev nodes, avoiding an interpolation error.
62     assert old_n % n == 0
63     start = full[:old_n // n].copy()
64     assert np.max(abs(z['x'][:old_n // n] - op.x)) < 1e-14
65     y = start[:-1].ravel()
66     saved_t, saved_full = [14400.], [op.surface(14400., y.reshape(n, 2), env)]
67     critical = None
68     official_full = None
69     event_full = None
70     nfev = nlu = 0
71     tic = time.perf_counter()
72     # Q4 never extends the observed radius beyond 72 h in these probes.
73     cuts = np.unique(np.r_[np.arange(14400., 259201., 1800.), official])
74     for a, b in zip(cuts[:-1], cuts[1:]):
75         def event(t, state):
76             return float(np.max(state[1::2]) - .15)
77         event.direction = -1
78         event.terminal = False
79         sol = solve_ivp(lambda t, u: op.evaluate(t, u, env), (a, b), y,
80                        method='Radau', jac=lambda t, u: op.evaluate(t, u, env, True),
81                        rtol=2e-9, atol=np.tile([2e-10, 2e-12], n),
82                        max_step=300., dense_output=True, events=event)
83         if not sol.success:
84             raise RuntimeError(sol.message)
85         nfev += sol.nfev
86         nlu += sol.nlu
87         if critical is None and len(sol.t_events[0]):
88             critical = float(sol.t_events[0][0])
89             event_full = op.surface(critical, sol.sol(critical).reshape(n, 2), env)
90         y = sol.y[:, -1]
91         state = op.surface(b, y.reshape(n, 2), env)
92         saved_t.append(float(b))
93         saved_full.append(state)
94         if b == official:
95             official_full = state.copy()
96         if critical is not None and official_full is not None:
97             break
98     result = {'case': case, 'scenario': scenario, 'n': n,
99             'scope': 'Actual coupled 1D continuation from frozen 4 h; observed Q4 R(t)
100                 ↪ fixed across environment probes; not a physical confidence interval.',
101             'baseline_future_T_b': future.tolist(), 'future_T_b': boundary.tolist(),
102             'perturbation_T_b': list(SCENARIOS[scenario]),
103             'perturbation_status': 'Analyst-selected stress size, not a measured
104                 ↪ uncertainty or a confidence interval.',
105             'start_s': 14400., 'source_state_node_count': old_n + 1,
106             'initial_state_projection': 'Exact nested Chebyshev nodes; same source state
107                 ↪ for all scenarios.',
108             'source_npz': original.relative_to(REPO).as_posix(), 'source_npz_sha256':
109                 ↪ sha(original),
110             'operator_source': Path(module.__file__).relative_to(REPO).as_posix(),
111             'operator_sha256': sha(Path(module.__file__)),
112             'environment_input_sha256': sha(env_path), 'script_sha256':
113                 ↪ sha(Path(__file__)),
114             'method': 'Radau', 'rtol': 2e-9, 'atol_TC': [2e-10, 2e-12], 'max_step_s':
115                 ↪ 300.,
116             'critical_s': critical, 'critical_h': critical / 3600 if critical else None,
117             'difference_from_frozen_official_root_s': critical - official_root if
118                 ↪ critical else None,

```

```

112         'polynomial_extrema_at_event': op.extrema(event_full) if event_full is not
113         ↪ None else None,
114         'official_execution_s': official,
115         'Cmax_at_official_execution': op.extrema(official_full)['max_C'] if
116         ↪ official_full is not None else None,
117         'strict_at_official_execution_in_this_probe':
118         ↪ bool(op.extrema(official_full)['max_C'] < .15) if official_full is not
119         ↪ None else None,
120         'max_boundary_residual': op.max_bc_residual,
121         'nfev': nfev, 'nlu': nlu, 'elapsed_s': time.perf_counter() - tic,
122         'runtime': {'python': platform.python_version(), 'numpy': np.__version__,
123         ↪ 'scipy': scipy.__version__}}
124     assert critical is not None and official_full is not None, 'No event within observed
125     ↪ radius span; report requires explicit unresolved state.'
126     states = np.asarray(saved_full)
127     assert np.isfinite(states).all() and states[:, :, 1].min() > 0
128     np.savez_compressed(stem.with_suffix('.npz'), time_s=np.asarray(saved_t),
129     ↪ full_TC=states,
130                        x=op.x, source_initial_full_TC=start, event_full_TC=event_full,
131                        official_full_TC=official_full)
132     stem.with_suffix('.json').write_text(json.dumps(result, ensure_ascii=False, indent=2) +
133     ↪ '\n', encoding='utf-8')
134     print(json.dumps(result, ensure_ascii=False), flush=True)
135
136
137 if __name__ == '__main__':
138     p = argparse.ArgumentParser()
139     p.add_argument('--case', choices=['q3', 'q4'], required=True)
140     p.add_argument('--n', type=int, default=40)
141     p.add_argument('--scenario', choices=list(SCENARIOS) + ['all'], default='all')
142     p.add_argument('--out', type=Path, required=True)
143     a = p.parse_args()
144     for scenario in SCENARIOS if a.scenario == 'all' else [a.scenario]:
145         run(a.case, a.n, scenario, a.out)

```

A.48 audit_environment_evidence

support/project/q1234_overall_review_delivery/v1/reviews/environment_check/audit_environment_evidence.py

```

1  """Independent postprocessing of 14 future-environment probe result pairs.
2
3  Does not import the probe or solver operators and does not run any PDE.
4  Reconstructs primitive polynomials with least-squares Chebyshev interpolation
5  (the solver used a DCT) and independently checks Robin residuals and maxima.
6  """
7  from pathlib import Path
8  import hashlib
9  import json
10 import subprocess
11 import numpy as np
12 from numpy.polynomial import Chebyshev
13 from scipy.optimize import brentq
14 from scipy.special import exp1
15 from openpyxl import load_workbook
16
17 HERE = Path(__file__).resolve().parent
18 ROOT = HERE.parents[3]
19 BASE = ROOT / "q1234_overall_review_delivery/v1"
20 DATA = BASE / "evidence/environment"
21 SCRIPT = BASE / "source/environment_probe.py"

```

```

22 OUT = HERE / "environment_evidence_audit.json"
23
24
25 def sha(path):
26     return hashlib.sha256(path.read_bytes()).hexdigest()
27
28
29 def read_xlsx(path, columns):
30     book = load_workbook(path, read_only=True, data_only=True)
31     rows = [r[:columns] for r in list(book.active.values)[1:] if r[0] is not None]
32     book.close()
33     return np.asarray(rows, float)
34
35
36 def primitive(c, b):
37     return c*np.exp(-b/c)-b*exp1(b/c)
38
39
40 def check_field(case, x, state, time, ambient, radius_data):
41     n = len(x)-1
42     b = .45 if case == "q3" else .30
43     t, c = state.T
44     u = primitive(c, b)
45     # Fit around a constant reference to reduce cancellation in derivatives.
46     p = Chebyshev.fit(x, u-u[-1], n, domain=[0, 1])
47     dt = Chebyshev.fit(x, t-t[-1], n, domain=[0, 1]).deriv()(1)
48     du = p.deriv()(1)
49     roots = p.deriv().roots()
50     roots = roots[np.abs(roots.imag) < 1e-9].real
51     roots = roots[(roots > 1e-12) & (roots < 1-1e-12)]
52     candidates = np.r_[0, roots, 1]
53     values = p(candidates)+u[-1]
54     index = np.argmax(values)
55     maximum = brentq(lambda value: primitive(value, b)-values[index], 1e-7, 10.,
60 ↪ xtol=5e-15)
56     radius = .02 if case == "q3" else np.interp(time, radius_data[:, 0], radius_data[:,
60 ↪ 1])*0.01
57     if case == "q3":
58         k = .21+.38*c[-1]/(1+c[-1]); prefactor=.0024
59     else:
60         k = .12+.20*c[-1]/(1+c[-1]); prefactor=.00042
61     thermal = 2/radius*k*dt+25*(t[-1]-ambient[0])
62     mass = 2/radius*prefactor*np.exp(-3850/(t[-1]+273.15))*du+8e-7*(c[-1]-ambient[1])
63     return {"max_C": float(maximum), "x_at_max": float(candidates[index]),
64           "stationary_points": int(len(roots)), "node_max_C": float(c.max()),
65           "heat_boundary_residual_W_m2": float(thermal),
66           "moisture_boundary_residual_m_s": float(mass), "radius_m": float(radius)}
67
68
69 def main():
70     if OUT.exists():
71         raise FileExistsError(OUT)
72     env_path = ROOT / "q4_complete_delivery/v1/inputs/attachment1.xlsx"
73     rad_path = ROOT / "q4_complete_delivery/v1/inputs/attachment2.xlsx"
74     env = read_xlsx(env_path, 3)
75     rad = read_xlsx(rad_path, 2)
76     future = env[env[:, 0]>=10800, 1:].mean(axis=0)
77     paths = sorted(DATA.glob("*.json"))
78     assert len(paths) == 14 and len(list(DATA.glob("*.npz"))) == 14
79     scenarios = {"baseline": [0.,0.], "temperature_minus_1K": [-1.,0.],
80               "temperature_plus_1K": [1.,0.], "boundary_minus_0p005": [0.,-.005],
81               "boundary_plus_0p005": [0.,.005]}
82     rows=[]; hash_records={}; originals={}; starts={}
83     for path in paths:

```

```

84     info=json.loads(path.read_text(encoding="utf-8"))
85     raw=np.load(path.with_suffix(".npz"))
86     case,n=info["case"],info["n"]
87     original=ROOT/info["source_npz"]
88     for file in (path,path.with_suffix(".npz"), original,
89     ↪ ROOT/info["operator_source"], SCRIPT,env_path,rad_path):
90         hash_records[str(file.relative_to(ROOT))]=sha(file)
91     assert sha(SCRIPT)==info["script_sha256"]
92     assert sha(original)==info["source_npz_sha256"]
93     assert sha(ROOT/info["operator_source"])==info["operator_sha256"]
94     assert sha(env_path)==info["environment_input_sha256"]
95     assert np.array_equal(future,np.asarray(info["baseline_future_I_b"]))
96     ambient=future+np.asarray(scenarios[info["scenario"]])
97     assert np.array_equal(ambient,np.asarray(info["future_I_b"]))
98     if original not in originals:
99         source=np.load(original)
100         key="snapshot_time_s" if case=="q3" else "time_s"
101         fieldkey="snapshot_full_TC" if case=="q3" else "full_TC"
102         idx=int(np.flatnonzero(source[key]==14400)[0])
103         originals[original]=(source["x"],source[fieldkey][idx])
104     old_x,old_field=originals[original]
105     step=(len(old_x)-1)//n
106     expected=old_field[::step]
107     assert expected.shape==(n+1,2)
108     assert np.array_equal(expected,raw["source_initial_full_TC"])
109     assert np.max(np.abs(old_x[::step]-raw["x"]))<1e-14
110     assert np.array_equal(raw["full_TC"][0,-1],expected[-1])
111     pair=(case,n)
112     if pair in starts:assert np.array_equal(starts[pair],raw["source_initial_full_TC"])
113     starts[pair]=raw["source_initial_full_TC"].copy()
114     assert raw["time_s"][0]==14400 and np.all(np.diff(raw["time_s"])>0)
115     ix=int(np.flatnonzero(raw["time_s"]==info["official_execution_s"])[0])
116     assert np.array_equal(raw["full_TC"][ix],raw["official_full_TC"])
117     assert 14400 < info["critical_s"] <= raw["time_s"][-1] <= 259200
118     event=check_field(case,raw["x"],raw["event_full_TC"],info["critical_s"],ambient,r)
119     ↪ ad)
120     official=check_field(case,raw["x"],raw["official_full_TC"],info["official_execution_s"],ambient,rad)
121     ↪ on_s"],ambient,rad)
122     initial=check_field(case,raw["x"],raw["full_TC"][0],14400,ambient,rad)
123     assert abs(event["max_C"]-.15)<1e-10
124     assert abs(event["max_C"]-info["polynomial_extrema_at_event"]["max_C"])<1e-10
125     assert abs(official["max_C"]-info["Cmax_at_official_execution"])<1e-10
126     assert (official["max_C"]<.15)==info["strict_at_official_execution_in_this_probe"]
127     for diagnostic in (event,official,initial):
128         assert abs(diagnostic["heat_boundary_residual_W_m2"])<1e-6
129         assert abs(diagnostic["moisture_boundary_residual_m_s"])<1e-14
130     rows.append({"name":path.stem,"case":case,"scenario":info["scenario"],"n":n,
131     "critical_s":info["critical_s"],"critical_h":info["critical_h"],
132     "event_independent":event,"official_independent":official,
133     "initial_independent":initial,
134     "surface_TC_reset_from_source_at_4h":(raw["full_TC"][0,-1]-expected[-1]).tolist(),
135     ↪ st(),
136     "source_initial_interior_max_difference":float(np.max(np.abs(raw["full_TC"][0]
137     ↪ ,:-1]-expected[: -1]))),
138     "baseline_error_vs_frozen_root_s":info["difference_from_frozen_official_root_
139     ↪ s"] if info["scenario"]=="baseline" else None})
140     by={r["case"],r["scenario"],r["n"]}:r for r in rows}
141     for r in rows:
142         base=by.get((r["case"],"baseline",r["n"]))
143         r["difference_from_same_n_baseline_s"]=r["critical_s"]-base["critical_s"]
144         r["difference_from_same_n_baseline_h"]=r["difference_from_same_n_baseline_s"]/3600
145     refinement={}
146     for case,n in (("q3",80),("q4",60)):
147         base40=by[case,"baseline",40];basefine=by[case,"baseline",n]

```

```

142     low40=by[case,"temperature_minus_1K",40];lowfine=by[case,"temperature_minus_1K",n]
143     shift40=low40["critical_s"]-base40["critical_s"]
144     shiftfine=lowfine["critical_s"]-basefine["critical_s"]
145     refinement[case]={ "coarse_n":40,"fine_n":n,
146         "baseline_refinement_change_s":basefine["critical_s"]-base40["critical_s"],
147         "minus1K_refinement_change_s":lowfine["critical_s"]-low40["critical_s"],
148         "minus1K_effect_coarse_s":shift40,"minus1K_effect_fine_s":shiftfine,
149         "effect_refinement_change_s":shiftfine-shift40,"effect_fine_h":shiftfine/3600,
150         "fine_baseline_error_vs_frozen_root_s":basefine["baseline_error_vs_frozen_roo
        ↳ t_s"]}}
151     report={"status":"PASS","git_commit_read":"anonymous-code-snapshot",
152         "scope":"Independent artifact and polynomial/boundary postprocessing only. No PDE
        ↳ solve, no independent physical calibration.",
153         "pairs_checked":14,"scenarios_n40":10,"refinement_runs":4,
154         "records":rows,"refinement":refinement,"sha256":hash_records,
155         "limitations":["Nested node restriction is exact at retained nodes, not an
        ↳ error-free lower-degree representation.",
156         "Only baseline and minus-1 K were spatially refined; no temporal refinement
        ↳ was added in this audit.",
157         "Events were detected at interior nodes; independent whole-radial
        ↳ primitive-polynomial extrema validate the saved event states, not
        ↳ continuum maxima.",
158         "4 h future-platform changes are step interventions. Surface is algebraically
        ↳ reset; bulk state is unchanged.",
159         "Q4 radius is held to observed R(t) under each changed environment, so results
        ↳ are conditional, not feedback predictions."]}
160     OUT.write_text(json.dumps(report,ensure_ascii=False,indent=2)+"\n",encoding="utf-8")
161     print(json.dumps({"status":"PASS","pairs_checked":14,"refinement":refinement},ensure_
        ↳ ascii=False,indent=2))
162     for r in rows: print(f'{r["name"]}: {r["critical_h"]:.10f} h; delta same-grid baseline
        ↳ {r["difference_from_same_n_baseline_s"]:.9f} s; official Cmax
        ↳ {r["official_independent"] ["max_C"]:.12f}')
163
164
165 if __name__=="__main__":main()

```

A.49 audit_saved_geometry_scope

support/project/q1234_overall_review_delivery/v1/reviews/geometry/audit_saved_geomet
ry_scope.py

```

1  """Read saved same-grid 1D/2D fields and independently integrate stored weights."""
2  from pathlib import Path
3  import hashlib
4  import json
5  import numpy as np
6
7  HERE = Path(__file__).resolve().parent
8  ROOT = HERE.parents[3]
9  BASE = ROOT / "q4_complete_delivery/v1/validation/geometry"
10 target = HERE / "saved_geometry_scope.json"
11 if target.exists():
12     raise FileExistsError(target)
13 results = []
14 for two_name, one_name, when in (("q1_80x128", "q1_80x0", 1800),
15     ("q23_80x256_integral", "q23_80x0_integral", 10800),
16     ("q4_80x128_integral", "q4_80x0_integral", 21600)):
17     items = []
18     sources = {}
19     for name in (two_name, one_name):
20         path = BASE / (name + ".npz")

```

```

21     sources[str(path.relative_to(ROOT))] =
22         ↪ hashlib.sha256(path.read_bytes()).hexdigest()
23     field = np.load(path)
24     ix = int(np.flatnonzero(field["snapshot_times_s"] == when)[0])
25     state = field["snapshots"][ix].reshape(-1, 2)
26     weights = field["weights"]
27     assert len(state) == len(weights) and np.all(weights > 0)
28     # Common shrinkage Jacobian cancels from the normalized average.
29     items.append((state * weights[:, None]).sum(axis=0) / weights.sum())
30     two, one = items
31     item = {"two_dimensional": two_name, "paired_one_dimensional": one_name,
32            "saved_time_s": when, "mean_T_C_2d": two.tolist(), "mean_T_C_1d": one.tolist(),
33            "mean_T_C_2d_minus_1d": (two-one).tolist(),
34            "normalized_water_loss_2d_relative_increase": float((2.55-two[1])/(2.55-one[1])-1),
35            "source_sha256": sources,
36            "interpretation": "Postprocessing saved states, not new PDE runs. Volume means
37            ↪ equal dry-mass means only for the current spatially uniform dry density
38            ↪ assumption. Thermal mean is arithmetic volume mean, not enthalpy."}
39     results.append(item)
40     print(json.dumps(item, ensure_ascii=False), flush=True)
41     target.write_text(json.dumps(results, ensure_ascii=False, indent=2)+"\n", encoding="utf-8")

```

A.50 audit_geometry_endpoints

support/project/q1234_overall_review_delivery/v1/reviews/geometry/audit_geometry_endpoints.py

```

1  """Read frozen geometry fields; targeted continuations, never full cold starts.
2
3  Uses the frozen FV operator, so this is not an independent discretization.
4  All generated files remain next to this script. No original data is modified.
5  """
6  from pathlib import Path
7  import hashlib
8  import importlib.util
9  import json
10 import platform
11 import subprocess
12 import time
13 import numpy as np
14 import scipy
15 from scipy.integrate import solve_ivp
16
17 HERE = Path(__file__).resolve().parent
18 ROOT = HERE.parents[3]
19 SOURCE = ROOT / "q4_complete_delivery/v1/source/geometry_solver_executed.py"
20 GEOM = ROOT / "q4_complete_delivery/v1/validation/geometry"
21
22
23 def sha(path):
24     return hashlib.sha256(path.read_bytes()).hexdigest()
25
26
27 def load_operator():
28     spec = importlib.util.spec_from_file_location("frozen_geometry", SOURCE)
29     mod = importlib.util.module_from_spec(spec)
30     spec.loader.exec_module(mod)
31     return mod
32
33
34 def main():

```

```

35 if (HERE / "endpoint_audit.json").exists():
36     raise FileExistsError("Refusing to overwrite completed endpoint audit")
37 mod = load_operator()
38 inputs = mod.Inputs()
39 ambient = lambda t: inputs.ambient(t, True)
40 report = {
41     "git_commit_read": "anonymous-code-snapshot",
42     "scope": "Targeted continuations from previously saved 2D states. Shared frozen FV
↳ operator. No full PDE cold start or continuum error bound.",
43     "runtime": {"python": platform.python_version(), "numpy": np.__version__, "scipy":
↳ scipy.__version__},
44     "source_sha256": sha(SOURCE), "cases": {}, "sources": {},
45 }
46 for case, nr, nz, official in (("q23", 80, 256, 206906.76), ("q4", 80, 128,
↳ 183931.56)):
47     source_npz = GEOM / f"{case}_{nr}x{nz}_integral.npz"
48     source_json = source_npz.with_suffix(".json")
49     report["sources"][str(source_npz.relative_to(ROOT))] = sha(source_npz)
50     report["sources"][str(source_json.relative_to(ROOT))] = sha(source_json)
51     saved = np.load(source_npz)
52     op = mod.GeometryFV(nr, nz, case, inputs, flux="integral")
53     assert saved["event_states"].shape[1:] == op.shape + (2,)
54     if case == "q23":
55         initial_t = float(saved["event_times_s"][0])
56         initial = saved["event_states"][0].ravel()
57         evaluation_times = np.unique(np.r_[saved["event_times_s"], official])
58         max_step = 0.5
59     else:
60         prior = np.flatnonzero(saved["snapshot_times_s"] < official)[-1]
61         initial_t = float(saved["snapshot_times_s"][prior])
62         initial = saved["snapshots"][prior].ravel()
63         evaluation_times = np.array([official, *saved["event_times_s"]])
64         evaluation_times = np.unique(evaluation_times[evaluation_times >= initial_t])
65         max_step = 30.0
66         # The existing radius is a constant 0.012 m across this short window.
67         checks = np.r_[initial_t, evaluation_times[-1], inputs.rad[inputs.rad[:, 0]
↳ >= initial_t] & (inputs.rad[:, 0] <= evaluation_times[-1]), 0]
68         assert np.ptp([inputs.radius(t) for t in checks]) == 0
69     started = time.perf_counter()
70     print(f"START {case}: saved {initial_t:.9f} s -> {evaluation_times[-1]:.9f} s",
↳ flush=True)
71     solution = solve_ivp(lambda t, y: op.rhs(t, y, ambient),
72         (initial_t, float(evaluation_times[-1])), initial, method="BDF",
73         jac=lambda t, y: op.rhs(t, y, ambient, True),
74         rtol=2e-11, atol=np.tile([1e-10, 1e-13], op.m),
75         max_step=max_step, t_eval=evaluation_times)
76     assert solution.success, solution.message
77     states = solution.y.T.reshape((-1,) + op.shape + (2,))
78     official_idx = np.flatnonzero(solution.t == official)[0]
79     final = states[official_idx]
80     c = final[:, :, 1]
81     ix = np.unravel_index(np.argmax(c), c.shape)
82     item = {
83         "initial_saved_time_s": initial_t, "execution_s": official,
84         "rtol": 2e-11, "atol_T": 1e-10, "atol_C": 1e-13,
85         "max_step_s": max_step, "nfev": solution.nfev, "nlu": solution.nlu,
86         "elapsed_s": time.perf_counter() - started,
87         "direct_FV_Cmax_at_execution": float(c[ix]),
88         "threshold_margin_0p15_minus_Cmax": float(.15-c[ix]),
89         "wettest_r_z_m": [float(op.r[ix][1]) * (inputs.radius(official) / .02 if case
↳ == "q4" else 1)], float(op.z[ix[0]])],
90         "all_nodes_strictly_pass": bool(np.all(c < .15)),
91         "radial_max_positive_difference": float(np.max(np.diff(c, axis=1))),
92         "axial_max_positive_difference": float(np.max(np.diff(c, axis=0))),

```

```

93         "upstream_error": "Unchanged saved coarse-grid history; local tight solve does
          ↪ not eliminate upstream spatial/temporal errors.",
94     }
95     if case in ("q23", "q4"):
96         repeated = []
97         for t, y in zip(saved["event_times_s"], saved["event_states"]):
98             idx = np.flatnonzero(solution.t == t)[0]
99             repeated.append({"time_s": float(t), "Cmax_new": float(np.max(states[idx,
          ↪ :, :, 1])),
100                             "Cmax_saved": float(np.max(y[:, :, 1])),
101                             "max_abs_C_difference": float(np.max(np.abs(states[idx, :, :, 1]-y[:,
          ↪ :, 1])),
102                             "max_abs_T_difference": float(np.max(np.abs(states[idx, :, :, 0]-y[:,
          ↪ :, 0]))))})
103         item["continuation_crosscheck_with_frozen_events"] = repeated
104         np.savez_compressed(HERE / f"{case}_direct_execution_field.npz",
          ↪ time_s=solution.t, states=states,
105                             reference_r_m=op.r, z_m=op.z, source_initial_time_s=initial_t)
106         report["cases"][case] = item
107         print(json.dumps(item, ensure_ascii=False), flush=True)
108
109     # A deterministic observed nonuniform Q2 field demonstrates mixed-sign
110     # cross-temperature effects. This refutes naive cooperative-system claims,
111     # not the observed direction of the actual 1D/2D event shift.
112     saved = np.load(GEOM / "q23_80x256_integral.npz")
113     ix = int(np.flatnonzero(saved["snapshot_times_s"] == 10800)[0])
114     op = mod.GeometryFV(80, 256, "q23", inputs, flux="integral")
115     matrix = op.rhs(10800, saved["snapshots"][ix].ravel(), inputs.ambient, True).tocoo()
116     select = (matrix.row % 2 == 1) & (matrix.col % 2 == 0)
117     values = matrix.data[select]
118     report["comparison_principle_diagnostic"] = {
119         "state": "saved Q23 80x256 field at 10800 s", "block": "d(Cdot)/d(T)",
120         "minimum": float(values.min()), "maximum": float(values.max()),
121         "negative_entries": int(np.sum(values < 0)), "positive_entries": int(np.sum(values
          ↪ > 0)),
122         "meaning": "The coupled semidiscrete operator is not cooperative in componentwise
          ↪ (T,C) order. A scalar D>0 comparison argument alone does not establish 1D >= 2D
          ↪ moisture for the coupled model.",
123     }
124     (HERE / "endpoint_audit.json").write_text(json.dumps(report, ensure_ascii=False,
          ↪ indent=2)+"\n", encoding="utf-8")
125     print("COMPLETE", flush=True)
126
127
128 if __name__ == "__main__":
129     main()

```

A.51 本稿图表后处理

A.52 build_assets

support/project/paper/q1234_draft_v1/build_assets.py

```

1  """Build six validated tables and five figures from frozen evidence only.
2
3  No PDE solver is imported or executed. Writes are restricted to the asset
4  directories and three named evidence files in this draft.
5  """
6  from __future__ import annotations
7  import csv
8  import hashlib

```

```

9 import json
10 from pathlib import Path
11 import platform
12 import shutil
13 import subprocess
14 import sys
15 import numpy as np
16 import scipy
17 import matplotlib
18 matplotlib.use("Agg")
19 import matplotlib.pyplot as plt
20 from matplotlib import font_manager
21 from matplotlib.ticker import FixedLocator, FixedFormatter, MultipleLocator
22 from openpyxl import load_workbook
23
24 HERE=Path(__file__).resolve().parent
25 ROOT=HERE.parents[1]
26 TABLES=HERE/"tables"
27 FIGURES=HERE/"figures"
28 EVIDENCE=HERE/"evidence"
29 COLORS=["#0072B2", "#D55E00", "#009E73", "#CC79A7", "#666666"]
30 STYLES=["-", "--", "-.", ":", (0, (5, 1, 1, 1))]
31 SOURCES={}
32 OUTPUTS=[]
33 MESSAGES=[]
34 EXPECTED={}
35 FIGURE_RECORDS=[]
36
37
38 def log(message):
39     print(message, flush=True)
40     MESSAGES.append(message)
41
42
43 def sha(path):
44     return hashlib.sha256(path.read_bytes()).hexdigest()
45
46
47 def rel(path):
48     return path.relative_to(ROOT).as_posix()
49
50
51 def record_source(path, purpose):
52     key=rel(path)
53     if key not in SOURCES: SOURCES[key]={"sha256":sha(path), "purposes":[]}
54     if purpose not in SOURCES[key]["purposes"]: SOURCES[key]["purposes"].append(purpose)
55     return path
56
57
58 def load_npz(path, purpose):
59     record_source(path, purpose)
60     with np.load(path, allow_pickle=False) as z: return {k:z[k] for k in z.files}
61
62
63 def csv_rows(path, purpose):
64     record_source(path, purpose)
65     with path.open(encoding="utf-8-sig", newline="") as f: return list(csv.reader(f))
66
67
68 def match_time(times, wanted):
69     i=int(np.argmin(abs(times-wanted)))
70     assert abs(float(times[i])-float(wanted))<1e-6, (wanted, times[i])
71     return i
72

```

```

73
74 def fmt(value):
75     return "--" if value is None or not np.isfinite(value) else f"{float(value):.4f}"
76
77
78 def make_table(name,times,values,time_unit,source_files,official_rows):
79     body=[]
80     for t,values_row in zip(times,values):
81         time_text=str(int(t)) if time_unit=="s" else fmt(t)
82         body.append([time_text,*[fmt(v) for v in values_row]])
83     # Official CSVs are independent frozen table artifacts. Compare display
84     # values rather than relying on agreement of underlying float encodings.
85     assert len(body)==len(official_rows)
86     for actual,official in zip(body,official_rows):
87         expected_time=str(int(float(official[0]))) if time_unit=="s" else
88         ↪ fmt(float(official[0]))
89         expected=[expected_time,*[fmt(float(v)) if v.strip() else "--" for v in
90         ↪ official[1:1+len(actual)-1]]]
91         assert actual==expected,(name,actual,expected)
92     q4=name=="q4_moisture"
93     columns=7 if q4 else 6
94     lines=[r"\begin{tabular}{"+r"*columns+"}",r"\toprule"]
95     if q4:
96         lines += [r" 时间 / h & \multicolumn{5}{c}{径向距离 / cm} & 当前表面 \\",
97         ↪ r"\cmidrule(lr){2-6}",r" & 0 & 0.5 & 1 & 1.5 & 2 & \\\"]
98     else:
99         lines += [f" 时间 / {time_unit}"+r" & \multicolumn{5}{c}{径向距离 / cm} \\",
100        ↪ r"\cmidrule(lr){2-6}",r" & 0 & 0.5 & 1 & 1.5 & 2 & \\\"]
101     lines += [r"\midrule",*[" & ".join(row)+r" \\" for row in
102     ↪ body],r"\bottomrule",r"\end{tabular}"]
103     path=TABLES/(name+".tex")
104     path.write_text("\n".join(lines)+"\n",encoding="utf-8")
105     OUTPUTS.append(path)
106     EXPECTED[name]={"latex_file":rel(path),"column_count":columns,
107     ↪ "headers":["时间/"+time_unit,"0 cm","0.5 cm","1 cm","1.5 cm","2 cm"]+(["当前表面"]
108     ↪ if q4 else []),
109     ↪ "time_unit":time_unit,"rows":body,
110     ↪ "flat_numeric_cells":[value for row in body for value in row if value!="--"],
111     ↪ "body_numeric_cell_count":sum(value!="--" for row in body for value in row),
112     ↪ "moisture_or_temperature_numeric_cell_count":sum(value!="--" for row in body for
113     ↪ value in row[1:]),
114     ↪ "domain_empty_cell_count":sum(value=="--" for row in body for value in row),
115     ↪ "sources":[rel(p) for p in
116     ↪ source_files],"all_frozen_csv_four_decimal_cells_match":True}
117     log(f"TABLE PASS {name}: {len(body)} rows, {columns} columns,
118     ↪ {EXPECTED[name]['domain_empty_cell_count']} domain-empty cells")
119
120
121 def style():
122     available={item.name for item in font_manager.fontManager.ttflist}
123     selected=next((s for s in ("Microsoft YaHei","SimSun") if s in available),None)
124     if selected is None:raise RuntimeError("Chinese plotting font not found")
125     plt.rcParams.update({"font.family":selected,"font.size":8.5,"axes.labelsize":8.5,
126     ↪ "axes.titlesize":9,"xtick.labelsize":8,"ytick.labelsize":8,"legend.fontsize":7.5,
127     ↪ "axes.linewidth":.65,"lines.linewidth":1.3,"axes.unicode_minus":False,
128     ↪ "pdf.fonttype":42,"ps.fonttype":42,"figure.facecolor":"white","savefig.dpi":300})
129     return selected
130
131
132 def panels(width=16.,height=7.4):
133     fig,axes=plt.subplots(1,2,figsize=(width/2.54,height/2.54))
134     fig.subplots_adjust(left=.085,right=.975,bottom=.19,top=.87,wspace=.32)
135     for ax in axes:
136         ax.spines[["top","right"]].set_visible(False)

```

```

130         ax.grid(axis="y",color="#D7DCE0",lw=.5,alpha=.8)
131         ax.set_axisbelow(True)
132         ax.tick_params(direction="out",pad=2)
133     return fig,axes
134
135
136 def save_figure(fig,name,record):
137     for suffix in ("pdf","png"):
138         path=FIGURES/(name+"."+suffix)
139         if suffix=="pdf":fig.savefig(path,metadata={"CreationDate":None,"ModDate":None})
140         else:fig.savefig(path,dpi=300)
141         OUTPUTS.append(path)
142     plt.close(fig)
143     FIGURE_RECORDS.append({"name":name,**record})
144     log("FIGURE BUILT "+name)
145
146
147 def read_radius():
148     path=record_source(ROOT/"q4_complete_delivery/v1/inputs/attachment2.xlsx","q4_shrinkage")
149     wb=load_workbook(path,read_only=True,data_only=True)
150     data=np.asarray([row[:2] for row in list(wb.active.values)[1:] if row[0] is not
151     ↪ None],float)
152     wb.close()
153     assert data.shape==(145,2) and np.all(np.diff(data[:,0])>0)
154     return path,data
155
156 def main():
157     for directory in (TABLES,FIGURES,EVIDENCE):directory.mkdir(parents=True,exist_ok=True)
158     font=style()
159     log("POSTPROCESS ONLY: no PDE solver imported or run")
160     log(f"RUNTIME Python={platform.python_version()} numpy={np.__version__}
161     ↪ scipy={scipy.__version__} matplotlib={matplotlib.__version__}; font={font};
162     ↪ executable={sys.executable}")
163     q1path=ROOT/"q1_complete_delivery/q1_delivery/output/q1_unrounded.npz"
164     q23path=ROOT/"q4_complete_delivery/v1/q123_closeout/output/q23_unified.npz"
165     q3path=ROOT/"q3_refinement_delivery/output/solution.npz"
166     q4path=ROOT/"q4_complete_delivery/v1/output/main.npz"
167     q1=load_npz(q1path,"Q1 actual table states")
168     q23=load_npz(q23path,"Current Q2 table states and Q3 full-process plot")
169     q3=load_npz(q3path,"Current formal Q3 table 5 and endpoint crosscheck")
170     q4=load_npz(q4path,"Current Q4 table 6, full-process and radial plots")
171     positions=[0,5,10,15,20]
172     assert np.array_equal(q1["radius_cm"][positions],np.array([0,.5,1,1.5,2]))
173     assert np.array_equal(q23["radius_cm"][positions],np.array([0,.5,1,1.5,2]))
174     for suffix,key in
175     ↪ (("temperature","temperature_degC"),("moisture","moisture_dry_basis")):
176         official_path=ROOT/f"q1_complete_delivery/q1_delivery/output/table_{suffix}.csv"
177         official=csv_rows(official_path,"Q1 frozen official four-decimal crosscheck")[1:]
178         times=np.array([100,300,600,900,1200,1500,1800])
179         values=q1[key][[match_time(q1["time_s"],t) for t in times]][:,positions]
180         make_table("q1_"+suffix,times,values,"s",[q1path,official_path],official)
181     for suffix,channel in (("temperature",0),("moisture",1)):
182         official_path=ROOT/f"q2_final_delivery/output/table_{suffix}.csv"
183         official=csv_rows(official_path,"Q2 frozen official four-decimal crosscheck")[1:]
184         times=np.arange(.5,3.01,.5)
185         values=q23["profile_TC"][[match_time(q23["time_s"],t*3600) for t in
186         ↪ times]][:,positions,channel]
187         make_table("q2_"+suffix,times,values,"h",[q23path,official_path],official)
188     q3csv=ROOT/"q3_refinement_delivery/output/table5.csv"
189     official=csv_rows(q3csv,"Current formal Q3 table 5 display crosscheck")[1:]
190     times=np.array([float(row[0]) for row in official])

```

```

187 values=q3["profile_TC"][[match_time(q3["time_s"],t*3600) for t in
    ↳ times]][[:,positions,1]
188 make_table("q3_moisture",times,values,"h",[q3path,q3csv],official)
189 # The new unified solution gives exactly the same four-decimal Q3 rows.
190 common=q23["profile_TC"][[match_time(q23["time_s"],t*3600) for t in
    ↳ times]][[:,positions,1]
191 assert [[fmt(v) for v in row] for row in common]==[[fmt(v) for v in row] for row in
    ↳ values]
192 log("Q3 current formal table and new Q23 unified trajectory: all four-decimal cells
    ↳ agree")
193 q4csv=ROOT/"q4_complete_delivery/v1/output/table6_unrounded.csv"
194 official=csv_rows(q4csv,"Current formal Q4 table 6 display crosscheck")[1:]
195 times=np.array([float(row[0]) for row in official])
196 indices=[match_time(q4["time_s"],t*3600) for t in times]
197 values=np.c_[q4["sample_TC"][indices][[:,positions,1],q4["surface_TC"][indices,1]]
198 assert np.array_equal(np.isnan(values[:,5]),q4["fixed_radius_m"][positions][None,:]>
    ↳ q4["radius_m"][indices,None]+1e-12)
199 make_table("q4_moisture",times,values,"h",[q4path,q4csv],official)
200 EXPECTED["q4_moisture"]["row_actual_radius_cm"]=[fmt(v*100) for v in
    ↳ q4["radius_m"][indices]]
201
202 for name in ("q1_profiles","q2_profiles"):
203     copied=[]
204     for suffix in ("pdf","png"):
205         original=record_source(ROOT/f"paper/q1_q2_stage/figures/{name}.{suffix}","Acc
            ↳ epted profile figure copied byte-for-byte")
206         target=FIGURES/original.name
207         shutil.copyfile(original,target)
208         assert sha(original)==sha(target)
209         OUTPUTS.append(target)
210         copied.append({"from":rel(original),"to":rel(target),"sha256":sha(original)})
211     FIGURE_RECORDS.append({"name":name,"operation":"byte-for-byte copy of accepted
        ↳ figure","copied":copied})
212     log("FIGURE COPIED "+name)
213
214 fig,axes=panels(height=7.5)
215 qt=np.unique(np.r_[np.arange(0,206907,60),206906.76])
216 qi=np.array([match_time(q23["time_s"],t) for t in qt])
217 series=["Q3:
    ↳ 固定半径",qt/3600,q23["profile_TC"][qi,0,1],q23["profile_TC"][qi,-1,1],57.4741),
218     ("Q4: 规定收缩",q4["time_s"]/3600,q4["full_TC"][[:,0,1],q4["surface_TC"][[:,1],5]
        ↳ 1.0921)]
219 for ax,(title,t,center,surface,execution) in zip(axes,series):
220     ax.plot(t,center,color=COLORS[0],label=" 轴心")
221     ax.plot(t,surface,color=COLORS[1],ls="--",label=" 当前表面")
222     ax.axhline(.15,color="#666666",lw=.9,ls=":",label=" 达标阈值 0.15")
223     ax.plot(execution,center[-1],"o",color=COLORS[0],ms=3.5)
224     ax.annotate(f"{execution:.4f}
        ↳ h",xy=(execution,center[-1]),xytext=(execution-14,.245),
225         fontsize=7.5,arrowprops={"arrowstyle":"-", "lw":.6, "color":"#555555"})
226     ax.set(xlabel="时间 / h",ylabel="干基含水率 /
        ↳ (kg/kg)",title=title,xlim=(0,60),ylim=(.045,2.9),yscale="log")
227     ticks=[.05,.1,.15,.5,1,2.5]
228     ax.yaxis.set_major_locator(FixedLocator(ticks));ax.yaxis.set_major_formatter(Fixe
        ↳ dFormatter(["0.05","0.1","0.15","0.5","1","2.5"]))
229     ax.minorticks_off();ax.xaxis.set_major_locator(MultipleLocator(12))
230     ax.legend(loc="upper right",frameon=False,handlelength=2.2)
231 save_figure(fig,"q3_q4_drying",{sources:[rel(q23path),rel(q4path)],
232     "panels":"Q3 left; Q4 right; shared x 0-60 h; logarithmic C axis;
        ↳ axis/surface/0.15 threshold; curves stop at official execution",
233     "time_sampling":"Q3 every 60 s plus execution; Q4 all stored 60 s outputs plus
        ↳ execution"})
234
235 radius_path,observed=read_radius()

```

```

236 fig,axes=panels(height=7.5)
237 axes[0].plot(observed[:,0]/3600,observed[:,1],color=COLORS[0],lw=1.2,label="附件2观测")
238 axes[0].scatter(observed[:,4,0]/3600,observed[:,4,1],s=5,color=COLORS[0],zorder=3)
239 axes[0].axvline(51.0921,color=COLORS[1],ls="--",lw=1,label=" 执行 51.0921 h")
240 axes[0].set(xlabel="时间 / h",ylabel="外半径 / cm",title="(a)
↳ 观测半径历程",xlim=(0,72),ylim=(1.15,2.04))
241 axes[0].xaxis.set_major_locator(MultipleLocator(12));axes[0].legend(frameon=False,loc_
↳ ="upper right")
242 profile_hours=[6,12,24,51.0921]
243 for h,color,ls in zip(profile_hours,COLORS,STYLES):
244     i=match_time(q4["time_s"],h*3600)
245     r_cm=q4["radius_m"][i]*100*np.sqrt(q4["x"])
246     c=q4["full_TC"][i,:,1]
247     axes[1].plot(r_cm,c,color=color,ls=ls,label=(f"{h:g} h" if h!=51.0921 else "结束
↳ 51.0921 h"))
248     axes[1].plot(r_cm[-1],c[-1],"o",ms=2.5,color=color)
249     assert np.all(r_cm<=q4["radius_m"][i]*100+1e-12)
250 axes[1].axhline(.15,color="#aaaaaa",lw=.7,ls=":")
251 axes[1].set(xlabel="当前径向距离 / cm",ylabel="干基含水率 / (kg/kg)",title="(b)
↳ 当前材料域内剖面",xlim=(0,1.45),ylim=(0,1.84))
252 axes[1].xaxis.set_major_locator(MultipleLocator(.5));axes[1].legend(frameon=False,loc_
↳ ="upper right",fontsize=7.)
253 save_figure(fig,"q4_shrinkage",{ "sources":[rel(radius_path),rel(q4path)],"profile_hou_
↳ rs":profile_hours,
254     "panels":["left observed R over 0-72 h; right actual-r profiles, points terminate
↳ at current surface; no extrapolation outside material"]})
255
256 fig,axes=panels(height=7.5)
257 scenarios=["temperature_minus_1K","temperature_plus_1K","boundary_minus_0p005","bound_
↳ ary_plus_0p005"]
258 labels=[" 温度 -1 K"," 温度 +1 K"," 边界 b -0.005"," 边界 b +0.005"]
259 plotdata=[]
260 for ax,case,title in zip(axes,("q3","q4"),("Q3: 固定半径","Q4: 规定收缩")):
261     base_path=record_source(ROOT/f"q1234_overall_review_delivery/v1/evidence/environtm_
↳ ent/{case}_baseline_n40.json","Same-n40 environment baseline")
262     baseline=json.loads(base_path.read_text(encoding="utf-8"))
263     changes=[]
264     for scenario in scenarios:
265         path=record_source(ROOT/f"q1234_overall_review_delivery/v1/evidence/environtm_
↳ nt/{case}_{scenario}_n40.json","Executed environment root difference")
266         data=json.loads(path.read_text(encoding="utf-8"))
267         assert data["case"]==case and data["n"]==40
268         change=(data["critical_s"]-baseline["critical_s"])/3600
269         changes.append(change);plotdata.append({"case":case,"scenario":scenario,"delt_
↳ a_root_h":change,"source":rel(path)})
270 y=np.arange(4)
271 ax.barh(y,changes,color=[COLORS[1] if d>0 else COLORS[0] for d in
↳ changes],height=.56)
272 ax.axvline(0,color="#666666",lw=.8)
273 for yy,value in zip(y,changes):
274     if abs(value)>.3:
275         # Large-bar labels sit inside bars, avoiding overlap with the
276         # Chinese y tick labels outside the plotting rectangle.
277         ax.text(value-(.07 if value>=0 else
↳ -.07),yy,f"{value:+.3f}",va="center",ha="right" if value>=0 else
↳ "left",fontsize=7.3,color="white")
278     else:
279         ax.text(value+(.06 if value>=0 else
↳ -.06),yy,f"{value:+.3f}",va="center",ha="left" if value>=0 else
↳ "right",fontsize=7.3)
280 ax.set_yticks(y,labels,fontsize=7.3);ax.invert_yaxis()
281 ax.set(xlabel=" 临界根变化 / h",title=title,xlim=(-2.45,2.45))
282 ax.xaxis.set_major_locator(MultipleLocator(1))
283 ax.grid(axis="y",visible=False);ax.grid(axis="x",color="#D7DCE0",lw=.5)

```

```

284 fig.subplots_adjust(left=.16,right=.975,wspace=.60)
285 save_figure(fig,"environment_sensitivity",{ "panels":"Both are analyst-selected changes
↳ after 4 h, not error bars or confidence intervals; Q4 R(t) unchanged",
↳ "data":plotdata})
286
287 expected_path=EVIDENCE/"table_expected.json"
288 expected_document={"schema_version":1,"scope":"Body cells, including times; all
↳ nonblank values are exact desired PDF strings.",
289 "tables":EXPECTED,"total_numeric_body_cells":sum(v["body_numeric_cell_count"] for v
↳ in EXPECTED.values()),
290 "total_temperature_moisture_cells":sum(v["moisture_or_temperature_numeric_cell_co
↳ unt"] for v in EXPECTED.values()),
291 "total_domain_empty_cells":sum(v["domain_empty_cell_count"] for v in
↳ EXPECTED.values())}
292 expected_path.write_text(json.dumps(expected_document,ensure_ascii=False,indent=2)+"\
↳ n",encoding="utf-8")
293 OUTPUTS.append(expected_path)
294 for source,description in SOURCES.items():assert
↳ sha(ROOT/source)==description["sha256"],source
295 assert len(EXPECTED)==6 and len(FIGURE_RECORDS)==5
296 log(f"FINAL PASS: 6 tables; {expected_document['total_temperature_moisture_cells']}
↳ temperature/moisture cells and {expected_document['total_domain_empty_cells']}
↳ domain-empty cells; 5 figure PDFs + PNG previews; all frozen source hashes
↳ unchanged")
297 logfile=EVIDENCE/"assets_build.log"
298 logfile.write_text("\n".join(MESSAGES)+"\n",encoding="utf-8")
299 OUTPUTS.append(logfile)
300 manifest={"git_commit_read":"anonymous-code-snapshot",
301 "script":rel(Path(__file__)), "script_sha256":sha(Path(__file__)),
302 "runtime":{"python":platform.python_version(),"numpy":np.__version__,"scipy":scip
↳ y.__version__,"matplotlib":matplotlib.__version__,"executable":sys.executable}
↳ ,"font":font},
303 "scope":"Frozen evidence postprocessing only; no PDE runs; generated files replace
↳ only named draft asset outputs.",
304 "sources":SOURCES,"figures":FIGURE_RECORDS,
305 "outputs":{"rel(path):{"sha256":sha(path),"bytes":path.stat().st_size} for path in
↳ OUTPUTS}}
306 (EVIDENCE/"assets_manifest.json").write_text(json.dumps(manifest,ensure_ascii=False,i
↳ ndent=2)+"\n",encoding="utf-8")
307
308
309 if __name__=="__main__":main()

```

A.53 新增完整复现与常规导出代码

A.54 reproduce

reproduce.py

```

1 """Recompute the paper from packaged source and inputs in a new directory.
2
3 Usage: python -X utf8 reproduce.py --out ../reproduced --jobs 3
4 No saved answer is used as a numerical initial state. The results/reference
5 directories are read only by the final comparison, never by PDE solvers.
6 """
7 from __future__ import annotations
8 import argparse
9 from concurrent.futures import ThreadPoolExecutor, as_completed
10 import hashlib
11 import json
12 import os

```

```

13 from pathlib import Path
14 import shutil
15 import subprocess
16 import sys
17 import time
18
19 HERE = Path(__file__).resolve().parent
20
21
22 def sha(path):
23     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
24
25
26 def save(path, value):
27     path.parent.mkdir(parents=True, exist_ok=True)
28     path.write_text(json.dumps(value, ensure_ascii=False, indent=2) + "\n",
29                     ↪ encoding="utf-8")
30
31
32 def prepare(out):
33     source = HERE / "support/project"
34     manifest = {p.relative_to(source).as_posix(): sha(p) for p in source.rglob("*") if
35                 ↪ p.is_file()}
36     record = out / "source_manifest.json"
37     if record.exists():
38         if json.loads(record.read_text(encoding="utf-8")) != manifest:
39             raise RuntimeError("Packaged source/input changed; choose a new output
40                               ↪ directory")
41         for name, digest in manifest.items():
42             if sha(out / "project" / name) != digest:
43                 raise RuntimeError("Working source/input changed: " + name)
44     else:
45         if out.exists() and any(out.iterdir()):
46             raise FileExistsError("Choose an empty output directory")
47         out.mkdir(parents=True, exist_ok=True)
48         shutil.copytree(source, out / "project")
49         save(record, manifest)
50
51     # The preserved executed geometry snapshot uses the original attachment
52     # layout. Create byte-identical aliases from packaged inputs, never read
53     # the author's external working directory.
54     aliases = out / "project/A 题/附件"
55     aliases.mkdir(parents=True, exist_ok=True)
56     for original, target in [("attachment1.xlsx", "附件1.xlsx"), ("attachment2.xlsx",
57                             ↪ "附件2.xlsx")]:
58         src = source / "q4_complete_delivery/v1/inputs" / original
59         dst = aliases / target
60         if dst.exists():
61             if sha(dst) != sha(src):
62                 raise RuntimeError("Input alias differs from packaged attachment")
63         else:
64             shutil.copyfile(src, dst)
65     return out / "project", manifest
66
67
68 def commands(root):
69     q1 = root / "q1_complete_delivery/q1_delivery"
70     q3 = root / "q3_refinement_delivery"
71     q4 = root / "q4_complete_delivery/v1"
72     q23 = q4 / "q123_closeout"
73     geom = q4 / "validation/geometry"
74     tasks = {}
75     def add(name, args, products):
76         tasks[name] = ([str(a) for a in args], [Path(p) for p in products])
77     def geometry(name, case, nr, nz, flux="integral", rtol="2e-9", step="240"):

```

```

73     label = f"{case}_{nr}x{nz}" + ("integral" if flux == "integral" else "")
74     if float(rtol) != 2e-9:
75         label += "_rtol" + format(float(rtol), "g")
76     add(name, [q4 / "source/geometry_solver.py", "--case", case, "--nr", nr, "--nz",
77             ↪ nz,
78             "--flux", flux, "--rtol", rtol, "--max-step", step,
79             "--input-dir", q4 / "inputs", "--out-dir", geom],
80         [geom / (label + ext) for ext in (".json", ".npz")])
81     # Start the two longest independent calculations before the small jobs.
82     geometry("geometry-q23-2d", "q23", 80, 256)
83     geometry("geometry-q4-2d", "q4", 80, 128)
84     add("q1-main", [q1 / "source/q1_solver.py", "--input", q1 / "input/附件 1.xlsx",
85             ↪ "--out", q1 / "output", "--n", 10240, "--quadrature"],
86         [q1 / "output" / x for x in ("q1_unrounded.npz", "run.json",
87             ↪ "table_temperature.csv", "table_moisture.csv")])
88     add("q23-main", [q23 / "source/solve_unified_q23.py", "--input", q23 /
89             ↪ "inputs/attachment1.xlsx",
90             "--out", q23 / "output/q23_unified", "--n", 80, "--method", "Radau",
91             "--rtol", "2e-13", "--max-step", 30, "--step", 1, "--end",
92             ↪ "206906.76"],
93         [q23 / "output" / ("q23_unified" + ext) for ext in (".json", ".npz")])
94     add("q3-main", [q3 / "source/q3_refined_reference.py", "--input", q3 /
95             ↪ "inputs/attachment1.xlsx",
96             "--out", q3 / "output/solution", "--n", 80, "--scenario", "mean",
97             ↪ "--method", "Radau",
98             "--rtol", "2e-11", "--atol-c", "2e-13", "--max-step", 120,
99             ↪ "--budget-s", "0.03", "--quantum-s", "0.36"],
100         [q3 / "output" / ("solution" + ext) for ext in (".json", ".npz")])
101     for name, stem, n, kind, method, rtol, step, audit in [
102         ("q4-main", "output/main", 120, "linear", "Radau", "2e-11", 90, True),
103         ("q4-spectral80", "validation/spectral80", 80, "linear", "Radau", "2e-10", 180,
104             ↪ False),
105         ("q4-spectral160", "validation/spectral160", 160, "linear", "Radau", "2e-11", 90,
106             ↪ False),
107         ("q4-time120-bdf", "validation/time120_bdf", 120, "linear", "BDF", "1e-11", 45,
108             ↪ False),
109         ("q4-fixed80", "validation/fixed80", 80, "fixed", "Radau", "2e-10", 180, False),
110         ("q4-pchip80", "validation/pchip80", 80, "pchip", "Radau", "2e-10", 180, False),
111     ]:
112         add(name, [q4 / "source/run_q4.py", "--out", q4 / stem, "--n", n, "--kind", kind,
113             ↪ "--method", method, "--rtol", rtol, "--max-step", step] + (["--audit"]
114             ↪ if audit else []),
115             [q4 / (stem + ext) for ext in (".json", ".npz")])
116     geometry("q4-fv320", "q4", 320, 0)
117     geometry("q4-fv320-tight", "q4", 320, 0, rtol="2e-10", step="120")
118     geometry("q4-fv640-tight", "q4", 640, 0, rtol="2e-10", step="120")
119     geometry("geometry-q1-1d", "q1", 80, 0, "harmonic")
120     geometry("geometry-q1-2d", "q1", 80, 128, "harmonic")
121     geometry("geometry-q23-1d", "q23", 80, 0)
122     geometry("geometry-q4-1d", "q4", 80, 0)
123     return tasks
124
125 def execute(name, args, products, root, out, source_signature):
126     record = out / "logs" / (name + ".json")
127     command = [sys.executable, "-X", "utf8", "-B", *args]
128     identity = {"command": [a.replace(str(out), "<OUTPUT>") for a in command],
129             ↪ "source_signature": source_signature}
130     if record.exists():
131         old = json.loads(record.read_text(encoding="utf-8"))
132         if old.get("returncode") == 0 and all(old.get(k) == v for k, v in
133             ↪ identity.items()):
134             if all(p.is_file() and sha(p) ==
135                 ↪ old["outputs"].get(p.relative_to(out).as_posix()) for p in products):

```

```

125         print("VERIFIED RESUME " + name, flush=True)
126         return old
127         raise RuntimeError("Incomplete or changed run: " + name + "; use a new output
↳ directory")
128     env = os.environ.copy()
129     env.update(PYTHONUTF8="1", PYTHONDONTWRITEBYTECODE="1", OPENBLAS_NUM_THREADS="1",
130               OMP_NUM_THREADS="1", MKL_NUM_THREADS="1", MPLBACKEND="Agg")
131     record.parent.mkdir(parents=True, exist_ok=True)
132     start = time.time()
133     print("START " + name, flush=True)
134     with (record.with_suffix(".log")).open("wb") as log:
135         process = subprocess.run(command, cwd=root, env=env, stdout=log,
↳ stderr=subprocess.STDOUT)
136     result = dict(identity, returncode=process.returncode, elapsed_s=time.time() - start,
137                  outputs={p.relative_to(out).as_posix(): sha(p) for p in products if
↳ p.is_file()})
138     save(record, result)
139     if process.returncode or len(result["outputs"]) != len(products):
140         raise RuntimeError("Run failed: " + name + "; see " +
↳ str(record.with_suffix(".log")))
141     print("PASS " + name + " (" + str(round(result["elapsed_s"], 1)) + " s)", flush=True)
142     return result
143
144
145 def main():
146     parser = argparse.ArgumentParser(description=__doc__)
147     parser.add_argument("--out", type=Path, required=True)
148     parser.add_argument("--jobs", type=int, default=3)
149     parser.add_argument("--stage", choices=["solve", "postprocess", "all"], default="all")
150     args = parser.parse_args()
151     out = args.out.resolve()
152     if out == HERE or HERE in out.parents:
153         raise ValueError("Output must be outside the submission package")
154     root, manifest = prepare(out)
155     signature = hashlib.sha256(json.dumps(manifest, sort_keys=True).encode()).hexdigest()
156     if args.stage in ("solve", "all"):
157         tasks = commands(root)
158         errors = []
159         with ThreadPoolExecutor(max_workers=max(1, min(args.jobs, 4))) as pool:
160             futures = {pool.submit(execute, name, cmd, products, root, out, signature):
↳ name
161
162                 for name, (cmd, products) in tasks.items()}
162             for future in as_completed(futures):
163                 try:
164                     future.result()
165                 except Exception as error:
166                     errors.append(str(error))
167                     print("FAIL " + str(error), flush=True)
168             if errors:
169                 raise RuntimeError("\n".join(errors))
170     if args.stage in ("postprocess", "all"):
171         from reproduction.postprocess import finish
172         finish(HERE, out, root, signature)
173
174
175 if __name__ == "__main__":
176     main()

```

A.55 figures

reproduction/figures.py

```
1 """Regenerate seven numerical figures from the new calculations.
2
3 The original publication PDFs remain in figures/. These review figures use
4 portable fonts and expose the same physical quantities, not pixel identity.
5 Five schematic/workflow diagrams have no numerical solver output.
6 """
7 import json
8 from pathlib import Path
9 import numpy as np
10 import matplotlib
11 matplotlib.use("Agg")
12 import matplotlib.pyplot as plt
13 from reproduction.verify import at, data, load
14 from reproduce import sha, save
15
16
17 def build(root, out):
18     dest = out / "figures"
19     dest.mkdir(parents=True, exist_ok=True)
20     q1, q2, q3, q4 = data(root)
21     plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 10,
22                          "axes.spines.top": False, "axes.spines.right": False,
23                          "figure.constrained_layout.use": True, "pdf.fonttype": 42})
24     outputs = []
25     def finish(fig, name, sources):
26         for ext in ("png", "pdf"):
27             path = dest / (name + "." + ext)
28             fig.savefig(path, dpi=170)
29             outputs.append({"path": path.relative_to(out).as_posix(), "sha256": sha(path),
30                            ↪ "sources": sources})
31     plt.close(fig)
32     fig, axes = plt.subplots(1, 2, figsize=(11, 4))
33     for time in [100, 300, 600, 900, 1200, 1500, 1800]:
34         idx = at(q1["time_s"], time)
35         for ax, key in zip(axes, ["temperature_degC", "moisture_dry_basis"]):
36             ax.plot(q1["radius_cm"], q1[key][idx], label=f"{time} s")
37     for ax, ylabel in zip(axes, ["Temperature / deg C", "Dry-basis moisture / (kg/kg)"]):
38         ax.set(xlabel="Radius / cm", ylabel=ylabel)
39         ax.grid(alpha=.2)
40     axes[0].legend(ncols=2, fontsize=8)
41     finish(fig, "q1_profiles", ["q1_unrounded.npz"])
42     fig, axes = plt.subplots(1, 2, figsize=(11, 4))
43     sel = (q2["time_s"] <= 10800) & (q2["time_s"] % 30 == 0)
44     for ax, channel, title in zip(axes, [0, 1], ["Temperature / deg C", "Dry-basis
45     ↪ moisture / (kg/kg)"]):
46         mesh = ax.pcolormesh(q2["radius_cm"], q2["time_s"][sel] / 3600,
47                              q2["profile_TC"][sel, :, channel], shading="auto",
48                              ↪ cmap="viridis")
49         ax.set(xlabel="Radius / cm", ylabel="Time / h", title=title)
50         fig.colorbar(mesh, ax=ax)
51     finish(fig, "q2_fields", ["q23_unified.npz", "first 3h"])
52     sel = q2["time_s"] <= 10800
53     fields = q2["profile_TC"][sel]
54     times = q2["time_s"][sel] / 3600
55     fig, axes = plt.subplots(1, 2, figsize=(11, 4))
56     mechanism = {}
57     for ax, i, location in zip(axes, [0, -1], ["Center", "Surface"]):
58         T, C = fields[:, i, 0], fields[:, i, 1]
59         H = 3850 * (1 / 301.15 - 1 / (T + 273.15))
60         M = .45 * (1 / 2.55 - 1 / C)
```

```

58     ax.plot(times, H, label="Temperature term H")
59     ax.plot(times, M, label="Moisture term M")
60     ax.plot(times, H + M, label="ln(D/D0)", lw=2)
61     ax.set(xlabel="Time / h", ylabel="Log ratio", title=location)
62     ax.grid(alpha=.2)
63     ratio = np.exp(H + M)
64     mechanism[location] = {"ratio_end": float(ratio[-1]), "peak_time_h":
        ↳ float(times[np.argmax(ratio)]),
65                           "peak_ratio": float(ratio.max())}
66 axes[0].legend(fontsize=8)
67 finish(fig, "q2_mechanism", ["q23_unified.npz; analytical log decomposition"])
68 save(out / "mechanism.json", mechanism)
69 fig, axes = plt.subplots(1, 2, figsize=(11, 4))
70 for ax in axes:
71     ax.plot(q2["time_s"] / 3600, q2["profile_TC"][:, 0, 1], label="Q3 center")
72     ax.plot(q4["time_s"] / 3600, q4["max_C"], label="Q4 radial maximum")
73     ax.axhline(.15, ls="--", color="black", lw=.8)
74     ax.set(xlabel="Time / h", ylabel="Dry-basis moisture / (kg/kg)")
75     ax.grid(alpha=.2)
76 axes[0].legend()
77 axes[1].set(xlim=(45, 59), ylim=(.145, .19), title="Endpoint region")
78 finish(fig, "q3_q4_drying", ["q23_unified.npz", "main.npz"])
79 fig, axes = plt.subplots(1, 2, figsize=(11, 4))
80 axes[0].plot(q4["time_s"] / 3600, q4["radius_m"] * 100)
81 axes[0].set(xlabel="Time / h", ylabel="Observed radius / cm")
82 fixed = load(root /
        ↳ "q4_complete_delivery/v1/validation/fixed80.json")["event"]["critical_h"]
83 shrinking = load(root / "q4_complete_delivery/v1/output/end_event.json")["critical_h"]
84 axes[1].bar(["Fixed radius", "Observed shrinkage"], [fixed, shrinking],
        ↳ color=["#D55E00", "#0072B2"])
85 axes[1].set(ylabel="Critical time / h")
86 for i, value in enumerate([fixed, shrinking]):
87     axes[1].text(i, value + 2, f"{value:.4f}", ha="center")
88 finish(fig, "q4_shrinkage", ["main.npz", "fixed80.json", "end_event.json"])
89 fig, axes = plt.subplots(2, 3, figsize=(12, 6))
90 geometry = root / "q4_complete_delivery/v1/validation/geometry"
91 for col, (case, pair, time) in enumerate([
92     ("Q1, 1800 s", ["q1_80x128", "q1_80x0"], 1800),
93     ("Q2, 3 h", ["q23_80x256_integral", "q23_80x0_integral"], 10800),
94     ("Q4, 6 h", ["q4_80x128_integral", "q4_80x0_integral"], 21600)]):
95     values = [np.load(geometry / (p + ".npz")) for p in pair]
96     fields = [z["snapshots"][at(z["snapshot_times_s"], time), ..., 1] for z in values]
97     radius = values[0]["reference_r_m"] *
        ↳ values[0]["radius_m"][at(values[0]["time_s"], time)] / .02 * 100
98     axes[0, col].plot(radius, fields[0][0] - fields[1][0])
99     axes[0, col].set(title=case, xlabel="Radius / cm", ylabel="C(2D)-C(1D) / (kg/kg)")
100     means = [float(f.ravel() @ z["weights"] / z["weights"].sum()) for f, z in
        ↳ zip(fields, values)]
101     axes[1, col].bar(["2D", "1D"], means, color=["#0072B2", "#D55E00"])
102     axes[1, col].set(ylabel="Whole-cylinder mean / (kg/kg)")
103     for i, value in enumerate(means):
104         axes[1, col].text(i, value, f"{value:.6f}", ha="center", va="bottom",
            ↳ fontsize=8)
105 finish(fig, "dimension_reduction", ["six newly generated matched 1D/2D fields"])
106 fig, ax = plt.subplots(figsize=(9, 4.5))
107 envdir = root / "q1234_overall_review_delivery/v1/evidence/environment"
108 scenarios = ["temperature_minus_1K", "temperature_plus_1K", "boundary_minus_0p005",
        ↳ "boundary_plus_0p005"]
109 for case, offset, marker in [("q3", -.12, "o"), ("q4", .12, "s")]:
110     base = load(envdir / f"{case}_baseline_n40.json")["critical_s"]
111     deltas = [(load(envdir / f"{case}_{s}_n40.json")["critical_s"] - base) / 3600 for s
        ↳ in scenarios]
112     ax.scatter(deltas, np.arange(4) + offset, marker=marker, label=case.upper())
113     for i, value in enumerate(deltas):

```

```

114         ax.annotate(f"{value:+.6f}", (value, i + offset), xytext=(6, 0),
115             ↪ textcoords="offset points", fontsize=8)
116 ax.axvline(0, color="gray", lw=.8)
117 ax.set_yticks(range(4), ["Temperature -1 K", "Temperature +1 K", "Boundary b -0.005",
118     ↪ "Boundary b +0.005"])
119 ax.set(xlabel="Change of critical time / h", xlim=(-2, 2.45))
120 ax.legend()
121 finish(fig, "environment_sensitivity", ["ten newly generated n40 environmental
122     ↪ scenarios"])
123 save(out / "figure_manifest.json", {"numerical_figures": 7, "layout": "Portable
124     ↪ numerical review plots; not pixel-identical publication layout", "files":
125     ↪ outputs})

```

A.56 material_motion

reproduction/material_motion.py

```

1  """Cold-start sealed-boundary pure-shrinkage check over observed 72 hours."""
2  import argparse
3  import json
4  from pathlib import Path
5  import sys
6  import numpy as np
7  from scipy.integrate import solve_ivp
8
9
10 def main():
11     p = argparse.ArgumentParser(description=__doc__)
12     p.add_argument("--root", type=Path, required=True)
13     p.add_argument("--out", type=Path, required=True)
14     a = p.parse_args()
15     sys.path.insert(0, str(a.root / "q4_complete_delivery/v1/source"))
16     from geometry_solver import GeometryFV, Inputs
17     inputs = Inputs(a.root / "q4_complete_delivery/v1/inputs")
18     op = GeometryFV(40, 0, "q4", inputs)
19     op.side[:] = 0.0
20     op.end[:] = 0.0
21     initial = np.tile([28., 2.55], op.m)
22     times = np.linspace(0, 259200, 145)
23     solution = solve_ivp(lambda t, y: op.rhs(t, y, inputs.ambient), (0, times[-1]),
24         ↪ initial,
25         method="BDF", jac=lambda t, y: op.rhs(t, y, inputs.ambient, True),
26         rtol=2e-11, atol=1e-13, t_eval=times, max_step=1800)
27     assert solution.success
28     change = float(np.max(abs(solution.y - initial[:, None])))
29     ratios = np.array([inputs.radius(t) / .02 for t in times])
30     rho0 = (760 + 90 * 2.55) / 3.55
31     dry_mass = rho0 / ratios**2 * ratios**2
32     water_mass = dry_mass * solution.y[1:2].mean(axis=0)
33     dry_error = float(np.max(abs(dry_mass / dry_mass[0] - 1)))
34     water_error = float(np.max(abs(water_mass / water_mass[0] - 1)))
35     assert change < 1e-12 and max(dry_error, water_error) < 1e-14
36     a.out.parent.mkdir(parents=True, exist_ok=True)
37     a.out.write_text(json.dumps({"scope": "zero heat/moisture exchange, uniform initial
38         ↪ state, prescribed observed radius",
39         "end_h": 72, "temperature_moisture_max_change": change,
40         "relative_dry_mass_change": dry_error, "relative_water_mass_change": water_error,
41         "pass": True}, indent=2) + "\n", encoding="utf-8")
42
43 if __name__ == "__main__":
44     main()

```

A.57 portable_workbooks

reproduction/portable_workbooks.py

```
1 """Stream standard OOXML workbooks; read them back with openpyxl."""
2 from pathlib import Path
3 import json
4 import zipfile
5 from xml.sax.saxutils import escape
6 import numpy as np
7 from openpyxl import load_workbook
8 from reproduce import sha, save
9
10
11 def write(path, sheets):
12     path = Path(path)
13     if path.exists():
14         raise FileExistsError(path)
15     path.parent.mkdir(parents=True, exist_ok=True)
16     ns = "http://schemas.openxmlformats.org/spreadsheetml/2006/main"
17     rel = "http://schemas.openxmlformats.org/officeDocument/2006/relationships"
18     def column(n):
19         name = ""
20         while n:
21             n, remainder = divmod(n - 1, 26)
22             name = chr(65 + remainder) + name
23         return name
24     columns = [column(i) for i in range(1, 40)]
25     styles = f'''<styleSheet xmlns="{ns}"><numFmts count="2"><numFmt numFmtId="164"
    ↪ formatCode="0.0000"/><numFmt numFmtId="165" formatCode="0.00"/></numFmts><font
    ↪ count="2"><font><sz val="10"/><name val="Arial"/></font><font><b/><sz
    ↪ val="10"/><name val="Arial"/></font></fonts><fills count="3"><fill><patternFill
    ↪ patternType="none"/></fill><fill><patternFill
    ↪ patternType="gray125"/></fill><fill><patternFill patternType="solid"><fgColor
    ↪ rgb="FFD9EAF7"/></patternFill></fill></fills><borders count="1"><border><left/><r
    ↪ ight/><top/><bottom/><diagonal/></border></borders><cellStyleXfs count="1"><xf
    ↪ numFmtId="0" fontId="0" fillId="0" borderId="0"/></cellStyleXfs><cellXfs
    ↪ count="4"><xf numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0"/><xf
    ↪ numFmtId="164" fontId="0" fillId="0" borderId="0" xfId="0"/><xf numFmtId="165"
    ↪ fontId="0" fillId="0" borderId="0" xfId="0"/><xf numFmtId="0" fontId="1"
    ↪ fillId="2" borderId="0" xfId="0" applyAlignment="1"><alignment horizontal="center"
    ↪ vertical="center" wrapText="1"/></xf></cellXfs><cellStyles count="1"><cellStyle
    ↪ name="Normal" xfId="0" builtinId="0"/></cellStyles></styleSheet>'''
26 with zipfile.ZipFile(path, "w", zipfile.ZIP_DEFLATED, compresslevel=6) as archive:
27     archive.writestr("[Content_Types].xml", '<Types
    ↪ xmlns="http://schemas.openxmlformats.org/package/2006/content-types"><Default
    ↪ Extension="rels" ContentType="application/vnd.openxmlformats-package.relation
    ↪ ships+xml"/><Default Extension="xml" ContentType="application/xml"/><Override
    ↪ PartName="/xl/workbook.xml" ContentType="application/vnd.openxmlformats-offic
    ↪ edocument.spreadsheetml.sheet.main+xml"/><Override PartName="/xl/styles.xml"
    ↪ ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.styl
    ↪ es+xml"/>' + ''.join(f'<Override PartName="/xl/worksheets/sheet{i}.xml"
    ↪ ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.work
    ↪ sheet+xml"/>' for i in range(1, len(sheets) + 1)) + '</Types>')
28     archive.writestr("_rels/.rels", f'<Relationships xmlns="http://schemas.openxmlfor
    ↪ mats.org/package/2006/relationships"><Relationship Id="rId1"
    ↪ Type="{rel}/officeDocument" Target="xl/workbook.xml"/></Relationships>')
29     archive.writestr("xl/workbook.xml", f'<workbook xmlns="{ns}"
    ↪ xmlns:r="{rel}"><sheets>' + ''.join(f'<sheet name="{escape(s[0])}"
    ↪ sheetId="{i}" r:id="rId{i}"/>' for i, s in enumerate(sheets, 1)) +
    ↪ '</sheets></workbook>')
```

```

30     archive.writestr("xl/_rels/workbook.xml.rels", '<Relationships
    ↪ xmlns="http://schemas.openxmlformats.org/package/2006/relationships">' +
    ↪ ''.join(f'<Relationship Id="rId{i}" Type="{rel}/worksheet"
    ↪ Target="worksheets/sheet{i}.xml"/>' for i in range(1, len(sheets) + 1)) +
    ↪ f'<Relationship Id="styles" Type="{rel}/styles"
    ↪ Target="styles.xml"/></Relationships>')
31 archive.writestr("xl/styles.xml", styles)
32 for sheet_id, (name, header, rows, raw) in enumerate(sheets, 1):
33     with archive.open(f"xl/worksheets/sheet{sheet_id}.xml", "w") as stream:
34         stream.write((f'<worksheet xmlns="{ns}"><sheetViews><sheetView
    ↪ workbookViewId="0"><pane xSplit="1" ySplit="1" topLeftCell="B2"
    ↪ activePane="bottomRight"
    ↪ state="frozen"/></sheetView></sheetViews><cols><col min="1" max="1"
    ↪ width="27" customWidth="1"/><col min="2" max="27" width="12"
    ↪ customWidth="1"/></cols><sheetData><row r="1" ht="36"
    ↪ customHeight="1">').encode())
35         for j, value in enumerate(header):
36             if value is None:
37                 continue
38             content = f'<is><t>{escape(value)}</t></is>' if isinstance(value, str)
    ↪ else f'<v>{value}</v>'
39             kind = 't="inlineStr"' if isinstance(value, str) else ''
40             stream.write(f'<c r="{columns[j]}1"
    ↪ s="3"{kind}>{content}</c>'.encode("utf-8"))
41         stream.write(b'</row>')
42         for i, values in enumerate(rows, 2):
43             cells = [f'<row r="{i}">']
44             for j, value in enumerate(values):
45                 if value is None or not np.isfinite(value):
46                     continue
47                 number = f"{float(value):.4f}" if j > 0 and not raw else
    ↪ f"{float(value):.17g}"
48                 cells.append(f'<c r="{columns[j]}{i}" s="{2 if j == 0 else
    ↪ 1}"><v>{number}</v></c>')
49             cells.append('</row>')
50             stream.write(''.join(cells).encode())
51         stream.write(b'</sheetData></worksheet>')
52
53
54 def field_rows(times, values):
55     for t, row in zip(times, values):
56         yield [float(t), *row]
57
58
59 def export(root, dest):
60     dest.mkdir(parents=True, exist_ok=True)
61     input_paths = [root / name for name in [
62         "q1_complete_delivery/q1_delivery/output/q1_unrounded.npz",
63         "q4_complete_delivery/v1/q123_closeout/output/q23_unified.npz",
64         "q3_refinement_delivery/output/solution.npz",
65         "q4_complete_delivery/v1/output/main.npz"]]
66     input_hashes = {p.relative_to(root).as_posix(): sha(p) for p in input_paths}
67     checkpoint = dest / "export_manifest.json"
68     if checkpoint.exists():
69         old = json.loads(checkpoint.read_text(encoding="utf-8"))
70         assert old["inputs"] == input_hashes and old["builder_sha256"] ==
    ↪ sha(Path(__file__))
71         assert all(sha(dest / name) == digest for name, digest in old["outputs"].items())
72         print("VERIFIED RESUME portable workbook export", flush=True)
73         return
74     q1 = np.load(root / "q1_complete_delivery/q1_delivery/output/q1_unrounded.npz")
75     q2 = np.load(root / "q4_complete_delivery/v1/q123_closeout/output/q23_unified.npz")
76     q3 = np.load(root / "q3_refinement_delivery/output/solution.npz")
77     q4 = np.load(root / "q4_complete_delivery/v1/output/main.npz")

```

```

78 radii = [j / 10 for j in range(21)]
79 write(dest / "result1.xlsx", [
80     ("温度", ["时间/s; 径向距离/cm; 温度/°C", *radii], field_rows(q1["time_s"][1:],
81         ↪ q1["temperature_degC"][1:]), False),
82     ("水分浓度", ["时间/s; 径向距离/cm; 干基含水率/(kg/kg)", *radii],
83         ↪ field_rows(q1["time_s"][1:], q1["moisture_dry_basis"][1:]), False)])
84 print("EXPORTED result1.xlsx", flush=True)
85 for filename, mask in [("result2.xlsx", q2["time_s"] > 0),
86     ("result2_前3小时.xlsx", (q2["time_s"] > 0) & (q2["time_s"] <=
87         ↪ 10800))]:
88     write(dest / filename, [(name, [" 时间/s; 径向距离/cm", *radii],
89         field_rows(q2["time_s"][mask], q2["profile_TC"][mask, :, ch]), False)
90         for name, ch in [(" 温度", 0), (" 水分浓度", 1)])])
91     print("EXPORTED " + filename, flush=True)
92 mask = q2["time_s"] > 0
93 np.savez_compressed(dest / "result2_ 全程轨迹_206907x21.npz",
94     time_s=q2["time_s"][mask], radius_cm=q2["radius_cm"],
95     temperature_TC=np.round(q2["profile_TC"][mask, :, 0], 4),
96     moisture_C=np.round(q2["profile_TC"][mask, :, 1], 4))
97 write(dest / "result3.xlsx", [("Sheet1", [" 时间/s\\到药材中心的距离/cm", *radii],
98     field_rows(q3["time_s"][1:], q3["profile_TC"][1:, :, 1]),
99     ↪ True)])
100
101 def q4_rows():
102     for i in range(1, len(q4["time_s"])):
103         yield [q4["time_s"][i], *q4["sample_TC"][i, :, 1], q4["surface_TC"][i, 1],
104             None, q4["radius_m"][i] * 100]
105 write(dest / "result4.xlsx", [("Sheet1", [" 时间/s\\到药材中心的距离/cm", *radii,
106     "药材表面", None, "实际半径R(t) / cm"], q4_rows(),
107     ↪ False)])
108 print("EXPORTED result3.xlsx and result4.xlsx", flush=True)
109 save(checkpoint, {"inputs": input_hashes, "builder_sha256": sha(Path(__file__)),
110     "outputs": {p.name: sha(p) for p in sorted(dest.iterdir()) if
111     ↪ p.is_file()}})
112
113 def compare_workbook_to_arrays(path, arrays):
114     """Read every exported numerical cell independently using openpyxl."""
115     book = load_workbook(path, read_only=True, data_only=True)
116     assert len(book.worksheets) == len(arrays), "Missing or extra workbook sheet"
117     report = []
118     for sheet, (times, values, rounded) in zip(book, arrays):
119         rows = sheet.iter_rows(values_only=True)
120         header = next(rows)
121         count = mismatches = 0
122         for i, row in enumerate(rows):
123             assert i < len(times), "Extra workbook row"
124             assert abs(float(row[0]) - times[i]) < 1e-8, (sheet.title, i, "time")
125             for j, expected in enumerate(values[i], 1):
126                 actual = row[j] if j < len(row) else None
127                 if expected is None or not np.isfinite(expected):
128                     assert actual is None, (sheet.title, i, j, "outside-domain cell")
129                 else:
130                     value = float(f"{float(expected):.4f}") if rounded else float(expected)
131                     if actual is None or not np.isfinite(float(actual)) or
132                     ↪ abs(float(actual) - value) > (1e-10 if rounded else 1e-13):
133                         mismatches += 1
134             count += 1
135         assert i + 1 == len(times), "Missing workbook rows"
136         report.append({"sheet": sheet.title, "rows": len(times), "cells": count,
137             ↪ "mismatches": mismatches})
138     book.close()
139     assert not any(r["mismatches"] for r in report), (str(path), report)
140     return report

```

A.58 postprocess

reproduction/postprocess.py

```
1 """Complete dependent calculations, export and compare without external data."""
2 from concurrent.futures import ThreadPoolExecutor, as_completed
3 import hashlib
4 import json
5 import sys
6 from pathlib import Path
7 import numpy as np
8
9 from reproduce import execute, save, sha
10
11
12 def extra_tasks(package, root):
13     q4 = root / "q4_complete_delivery/v1"
14     q3 = root / "q3_refinement_delivery"
15     review = root / "q1234_overall_review_delivery/v1"
16     tasks = {}
17     def add(name, cmd, products):
18         tasks[name] = ([str(v) for v in cmd], list(products))
19     destination = root / "q2_refinement_delivery/reproduced_sensitivity"
20     add("q2-sensitivity", [package / "reproduction/supplement.py", "--root", root,
21                           "--out", destination, "--case", "q2-sensitivity"],
22        [destination / "sensitivity.json", destination / "sensitivity_baseline.npz"])
23     for case, refined_n in [("q3", 80), ("q4", 60)]:
24         for n, scenarios in [(40, ["all"]), (refined_n, ["baseline",
25                  ↪ "temperature_minus_1K"])]):
26             for scenario in scenarios:
27                 names = ["baseline", "temperature_minus_1K", "temperature_plus_1K",
28                  ↪ "boundary_minus_0p005", "boundary_plus_0p005"] if scenario == "all"
29                  ↪ else [scenario]
30                 add(f"environment-{case}-{n}-{scenario}",
31                    [review / "source/environment_probe.py", "--case", case, "--n", n,
32                      "--scenario", scenario, "--out", review / "evidence/environment"],
33                    [review / "evidence/environment" / f"{case}_{s}_{n}{ext}" for s in
34                     ↪ names for ext in (".json", ".npz")])
35     add("q3-time-mean", [q3 / "source/q3_refined_reference.py", "--input", q3 /
36                  ↪ "inputs/attachment1.xlsx",
37                      "--out", q3 / "validation/time_mean", "--n", 80, "--scenario",
38                      ↪ "time_mean",
39                      "--rtol", "2e-11", "--max-step", 120],
40        [q3 / "validation" / ("time_mean" + ext) for ext in (".json", ".npz")])
41     add("q4-fv640-extra-tight", [q4 / "source/geometry_solver.py", "--case", "q4", "--nr",
42                  ↪ 640,
43                      "--nz", 0, "--flux", "integral", "--rtol", "2e-11",
44                      ↪ "--max-step", 120,
45                      "--input-dir", q4 / "inputs", "--out-dir", q4 /
46                      ↪ "validation/geometry"],
47        [q4 / "validation/geometry" / ("q4_640x0_integral_rtol2e-11" + ext) for ext in
48         ↪ (".json", ".npz")])
49     add("geometry-algebra-checks", [q4 / "source/geometry_checks.py", "--out-dir", q4 /
50                  ↪ "validation/algebra"],
51        [q4 / "validation/algebra" / s for s in ("jacobian_flux_checks.json",
52         ↪ "dimension_reduction_check.json")])
53     add("material-motion-72h", [package / "reproduction/material_motion.py", "--root",
54                  ↪ root,
55                      "--out", q4 / "validation/material_motion.json"],
56        [q4 / "validation/material_motion.json"])
57     add("geometry-endpoint-continuations", [review /
58                  ↪ "reviews/geometry/audit_geometry_endpoints.py"],
59        [review / "reviews/geometry/endpoint_audit.json", review /
60         ↪ "reviews/geometry/q23_direct_execution_field.npz",
```

```

46         review / "reviews/geometry/q4_direct_execution_field.npz"])
47     return tasks
48
49
50 def finish(package, out, root, signature):
51     q4 = root / "q4_complete_delivery/v1"
52     # All budget terms are regenerated; finalize refuses a state not actually evaluated.
53     execute("q4-finalize", [str(q4 / "source/finalize_q4.py"), "--base", str(q4)],
54             [q4 / "output/end_event.json", q4 / "output/table6_unrounded.csv"], root, out,
55             ↪ signature)
56     tasks = extra_tasks(package, root)
57     errors = []
58     with ThreadPoolExecutor(max_workers=2) as pool:
59         futures = {pool.submit(execute, name, cmd, products, root, out, signature): name
60                    for name, (cmd, products) in tasks.items()}
61         for future in as_completed(futures):
62             try:
63                 future.result()
64             except Exception as error:
65                 errors.append(str(error))
66                 print("FAIL " + str(error), flush=True)
67     if errors:
68         raise RuntimeError("\n".join(errors))
69     execute("q4-numeric-comparison", [str(q4 / "source/check_numerics.py"), "--base",
70     ↪ str(q4)],
71           [q4 / "validation/numeric_comparison.json"], root, out, signature)
72     physics = root / "q1234_overall_review_delivery/v1/reviews/physics"
73     # This diagnostic computes quantities from newly generated fields, not historical
74     ↪ arrays.
75     execute("physics-diagnostics", [str(physics / "diagnose_saved_fields.py")],
76           [physics / "saved_field_physics_diagnostics.json"], root, out, signature)
77     from reproduction.verify import verify
78     result = verify(package, root, out)
79     save(out / "comparison.json", result)
80     if not result["pass"]:
81         raise AssertionError("Paper comparison failed; inspect comparison.json")
82     from reproduction.portable_workbooks import export
83     export(root, out / "results")
84     from reproduction.verify import verify_exports
85     book_record = out / "workbook_verification.json"
86     if book_record.exists():
87         prior = json.loads(book_record.read_text(encoding="utf-8"))
88         assert prior["pass"] and prior.get("export_manifest_sha256") == sha(out /
89         ↪ "results/export_manifest.json")
90     else:
91         save(book_record, verify_exports(root, out))
92     from reproduction.figures import build
93     build(root, out)
94     print("COMPLETE: paper comparisons and portable workbooks passed", flush=True)

```

A.59 supplement

reproduction/supplement.py

```

1  """Additional paper scenarios using only the newly calculated workspace."""
2  import argparse
3  import sys
4  from pathlib import Path
5
6
7  def main():
8      p = argparse.ArgumentParser(description=__doc__)

```

```

9     p.add_argument("--root", type=Path, required=True)
10    p.add_argument("--out", type=Path, required=True)
11    p.add_argument("--case", choices=["q2-sensitivity"], required=True)
12    a = p.parse_args()
13    path = a.root / "q2_refinement_delivery/runtime/source"
14    sys.path.insert(0, str(path))
15    from q2_core import Environment, load_environment
16    from q2_tests import run_sensitivity
17    data, _ = load_environment(a.root / "q2_refinement_delivery/runtime/inputs/附件1.xlsx")
18    run_sensitivity(Environment(data), a.out)
19
20
21 if __name__ == "__main__":
22     main()

```

A.60 verify

reproduction/verify.py

```

1  """Compare regenerated numbers with paper/reference data; never feed solvers."""
2  import json
3  from pathlib import Path
4  import numpy as np
5  from openpyxl import load_workbook
6  from reproduction.portable_workbooks import compare_workbook_to_arrays
7  from reproduce import sha
8
9
10 def load(path):
11     return json.loads(Path(path).read_text(encoding="utf-8"))
12
13
14 def at(times, wanted):
15     index = int(np.argmax(abs(np.asarray(times) - wanted)))
16     assert abs(float(times[index]) - wanted) < 1e-6, (wanted, times[index])
17     return index
18
19
20 def fmt(x, decimals=4):
21     return "--" if x is None or not np.isfinite(x) else f"{float(x):.{decimals}f}"
22
23
24 def data(root):
25     return [np.load(root / path) for path in [
26         "q1_complete_delivery/q1_delivery/output/q1_unrounded.npz",
27         "q4_complete_delivery/v1/q123_closeout/output/q23_unified.npz",
28         "q3_refinement_delivery/output/solution.npz",
29         "q4_complete_delivery/v1/output/main.npz"]]
30
31
32 def verify(package, root, out):
33     q1, q2, q3, q4 = data(root)
34     expected = load(package / "reference/paper_tables.json")["tables"]
35     result = {"scope": "New PDE solves versus the saved paper display and submitted result",
36             "values":
37                 "tables": {}, "claims": [], "full_outputs": {}}
38     def claim(name, actual, expected_value, tolerance):
39         ok = abs(actual - expected_value) <= tolerance
40         result["claims"].append({"name": name, "actual": float(actual), "reference":
41             expected_value,
42             "absolute_tolerance": tolerance, "pass": bool(ok)})
43     positions = [0, 5, 10, 15, 20]

```

```

42 for name, table in expected.items():
43     rows = []
44     for old in table["rows"]:
45         t = float(old[0]) * (1 if table["time_unit"] == "s" else 3600)
46         if name.startswith("q1"):
47             key = "temperature_degC" if name.endswith("temperature") else
48                 ↪ "moisture_dry_basis"
49             values = q1[key][at(q1["time_s"], t), positions]
50         elif name.startswith("q2"):
51             channel = 0 if name.endswith("temperature") else 1
52             values = q2["profile_TC"][at(q2["time_s"], t), positions, channel]
53         elif name.startswith("q3"):
54             values = q3["profile_TC"][at(q3["time_s"], t), positions, 1]
55         else:
56             i = at(q4["time_s"], t)
57             values = np.r_[q4["sample_TC"][i, positions, 1], q4["surface_TC"][i, 1]]
58             rows.append([str(int(t)) if table["time_unit"] == "s" else fmt(t / 3600),
59                 ↪ *[fmt(v) for v in values]])
60         mismatch = [{"row": i + 1, "column": j + 1, "actual": a, "reference": b}
61             for i, (actual, target) in enumerate(zip(rows, table["rows"]))
62             for j, (a, b) in enumerate(zip(actual, target)) if a != b]
63         result["tables"][name] = {"pass": not mismatch, "rows": rows,
64             ↪ "numeric_cells": sum(v != "--" for r in rows for v in
65                 ↪ r), "mismatches": mismatch}
66     q3j = load(root / "q3_refinement_delivery/output/solution.json")["event"]
67     q4j = load(root / "q4_complete_delivery/v1/output/end_event.json")
68     claim("Q3 critical seconds", q3j["critical_s"], 206906.389932328, 0.0005)
69     claim("Q3 execution hours", q3j["execution_h"], 57.4741, 1e-10)
70     claim("Q3 execution maximum C", q3j["execution_max"], 0.149999892208, 1e-9)
71     claim("Q4 critical seconds", q4j["critical_s"], 183931.211715489, 0.0005)
72     claim("Q4 execution hours", q4j["execution_h"], 51.0921, 1e-10)
73     assert q3j["execution_max"] < .15 and q4j["execution_max_C"] < .15
74     fixed = load(root /
75         ↪ "q4_complete_delivery/v1/validation/fixed80.json")["event"]["critical_h"]
76     claim("Q4 same-property fixed-radius critical hours", fixed, 129.8484, 0.00005)
77     claim("Q4 time reduction percent", 100 * (1 - q4j["critical_h"] / fixed), 60.6526,
78         ↪ 0.00005)
79     # Verify every saved Q2 value, without equating binary files with numerical equality.
80     old = np.load(package / "results/result2_ 全程轨迹 _206907x21.npz")
81     mask = q2["time_s"] > 0
82     np.testing.assert_allclose(q2["time_s"][mask], old["time_s"], rtol=0, atol=1e-8)
83     for key, ch in [("temperature_TC", 0), ("moisture_C", 1)]:
84         actual = q2["profile_TC"][mask, :, ch]
85         target = old[key]
86         assert actual.shape == target.shape
87         delta = abs(actual - target)
88         rounding_difference = np.round(actual, 4) != target
89         ii = np.argwhere(rounding_difference)
90         result["full_outputs"]["q2_" + key] = {
91             "cells": int(target.size), "max_abs_unrounded_to_submitted_rounded":
92                 ↪ float(delta.max()),
93             "four_decimal_different_cells": int(rounding_difference.sum()),
94             "first_differences": [{"time_s": float(old["time_s"][i]), "radius_cm":
95                 ↪ float(old["radius_cm"][j])},
96                 ↪ {"time_s": float(actual[i, j]), "radius_cm":
97                     ↪ float(target[i, j])} for i, j in ii[:12]],
98             "pass": bool(np.all(delta <= 0.00005002)),
99             "tolerance_reason": "Submitted values are rounded to 4 decimals; half a
100                 ↪ display unit plus 2e-8 numerical tolerance"}
101     # Compare all workbook values from the other three questions at their displayed
102     ↪ precision.
103     for case, times, values in [(1, q1["time_s"][1:], [q1["temperature_degC"][1:],
104         ↪ q1["moisture_dry_basis"][1:]]),
105         (3, q3["time_s"][1:], [q3["profile_TC"][1:, :, 1]]),

```

```

95         (4, q4["time_s"][1:], [np.c_[q4["sample_TC"][1:, :, 1],
96             ↪ q4["surface_TC"][1:, 1]]]):
97     workbook = load_workbook(package / f"results/result{case}.xlsx", read_only=True,
98     ↪ data_only=True)
99     checked = differences = 0
100     largest = 0.0
101     for sheet, actual in zip(workbook, values):
102         rows = sheet.iter_rows(values_only=True)
103         next(rows)
104         for i, row in enumerate(rows):
105             assert i < len(times) and abs(row[0] - times[i]) < 1e-7
106             for j, x in enumerate(actual[i], 1):
107                 old_value = row[j] if j < len(row) else None
108                 if not np.isfinite(x):
109                     assert old_value is None
110                 else:
111                     assert old_value is not None
112                     largest = max(largest, abs(float(x) - old_value))
113                     differences += fmt(x) != fmt(old_value)
114                 checked += 1
115             assert i + 1 == len(times)
116         workbook.close()
117     result["full_outputs"][f"q{case}_workbook"] = {"cells": checked,
118     ↪ "four_decimal_different_cells": int(differences),
119     ↪ "max_abs_difference": largest, "pass": largest <= (2e-8 if case == 3 else
120     ↪ 0.00005002)}
121 # Recompute the additional table of whole-cylinder averages.
122 geometry = root / "q4_complete_delivery/v1/validation/geometry"
123 result["geometry_table"] = []
124 for case, pair, t, means, percentage in [
125     ("q1", ["q1_80x128", "q1_80x0"], 1800, [2.274929, 2.293532], 7.2533),
126     ("q2", ["q23_80x256_integral", "q23_80x0_integral"], 10800, [1.327482, 1.382492],
127     ↪ 4.7118),
128     ("q4", ["q4_80x128_integral", "q4_80x0_integral"], 21600, [1.024545, 1.065198],
129     ↪ 2.7379)]:
130     actual = []
131     for stem in pair:
132         z = np.load(geometry / (stem + ".npz"))
133         field = z["snapshots"][at(z["snapshot_times_s"], t), ..., 1]
134         actual.append(float(field.ravel() @ z["weights"] / z["weights"].sum()))
135     pct = 100 * ((2.55 - actual[0]) / (2.55 - actual[1]) - 1)
136     claim(case + " whole 2D mean", actual[0], means[0], 0.000005)
137     claim(case + " whole 1D mean", actual[1], means[1], 0.000005)
138     claim(case + " loss increase percent", pct, percentage, 0.0005)
139     result["geometry_table"].append({"case": case, "time_s": t, "means_2d_1d": actual,
140     ↪ "loss_increase_percent": pct})
141 # Every displayed six-decimal environmental root and root difference.
142 envdir = root / "q1234_overall_review_delivery/v1/evidence/environment"
143 result["environment_table"] = []
144 scenarios = ["baseline", "temperature_minus_1K", "temperature_plus_1K",
145     ↪ "boundary_minus_0p005", "boundary_plus_0p005"]
146 targets = {"q3": ([57.473996, 59.341275, 55.688357, 57.382748, 57.584113], [0,
147     ↪ 1.867279, -1.785639, -0.091249, 0.110117]),
148     "q4": ([51.092003, 52.729346, 49.525343, 50.993818, 51.218753], [0,
149     ↪ 1.637343, -1.566660, -0.098185, 0.126750])}
150 for case in ("q3", "q4"):
151     baseline = load(envdir / f"{case}_baseline_n40.json")["critical_s"]
152     for i, scenario in enumerate(scenarios):
153         value = load(envdir / f"{case}_{scenario}_n40.json")["critical_s"]
154         delta = (value - baseline) / 3600
155         claim(f"{case} {scenario} root hours", value / 3600, targets[case][0][i],
156         ↪ 0.000005)
157         claim(f"{case} {scenario} change hours", delta, targets[case][1][i], 0.000005)

```

```

147         result["environment_table"].append({"case": case, "scenario": scenario,
148             ↳ "root_h": value / 3600, "delta_h": delta})
149 sensitivity = load(root /
150     ↳ "q2_refinement_delivery/reproduced_sensitivity/sensitivity.json")
149 for name, center, surface in [("hm_0.8", 1.8581, 1.1775), ("hm_1.2", 1.6921, 0.8737)]:
150     claim("Q2 " + name + " center at 3h", sensitivity[name]["C_center_end"], center,
151         ↳ 0.00005)
151     claim("Q2 " + name + " surface at 3h", sensitivity[name]["C_surface_end"],
152         ↳ surface, 0.00005)
152 endpoint = load(root /
153     ↳ "q1234_overall_review_delivery/v1/reviews/geometry/endpoint_audit.json")
153 previous_endpoint = load(package /
154     ↳ "support/evidence/q4_endpoint_20260912/endpoint_audit.json")
154 result["geometry_endpoint_reproduction"] = endpoint["cases"]
155 for case in ["q23", "q4"]:
156     claim(case + " coarse 2D Cmax at the stated time",
157         endpoint["cases"][case]["direct_FV_Cmax_at_execution"],
158         previous_endpoint["cases"][case]["direct_FV_Cmax_at_execution"], 1e-8)
159 assert endpoint["cases"][case]["all_nodes_strictly_pass"]
160 assert not endpoint["cases"][case]["all_nodes_strictly_pass"]
161 result["paper_numeric_cells"] = sum(t["numeric_cells"] for t in
162     ↳ result["tables"].values())
162 result["pass"] = all(t["pass"] for t in result["tables"].values()) and all(c["pass"]
163     ↳ for c in result["claims"]) and all(c["pass"] for c in
164     ↳ result["full_outputs"].values())
163 return result
164
165
166 def verify_exports(root, out):
167     q1, q2, q3, q4 = data(root)
168     report = {}
169     specs = {
170         "result1.xlsx": [(q1["time_s"][1:], q1[key][1:], True) for key in
171             ↳ ["temperature_degC", "moisture_dry_basis"]],
171         "result2.xlsx": [(q2["time_s"][1:], q2["profile_TC"][1:, :, c], True) for c in [0,
172             ↳ 1]],
172         "result2_前3小时.xlsx": [(q2["time_s"][1:10801], q2["profile_TC"][1:10801, :, c],
173             ↳ True) for c in [0, 1]],
173         "result3.xlsx": [(q3["time_s"][1:], q3["profile_TC"][1:, :, 1], False)],
174         "result4.xlsx": [(q4["time_s"][1:], np.c_[q4["sample_TC"][1:, :, 1],
175             ↳ q4["surface_TC"][1:, 1],
176             ↳ np.full(len(q4["time_s"]) - 1, np.nan),
177             ↳ q4["radius_m"][1:] * 100], True)]]
176 for filename, arrays in specs.items():
177     report[filename] = compare_workbook_to_arrays(out / "results" / filename, arrays)
178     print("READ-BACK PASS " + filename, flush=True)
179 return {"pass": True, "workbooks": report, "total_cells": sum(r["cells"] for rows in
180     ↳ report.values()) for r in rows),
181     "export_manifest_sha256": sha(out / "results/export_manifest.json")}

```